Lecture 16:

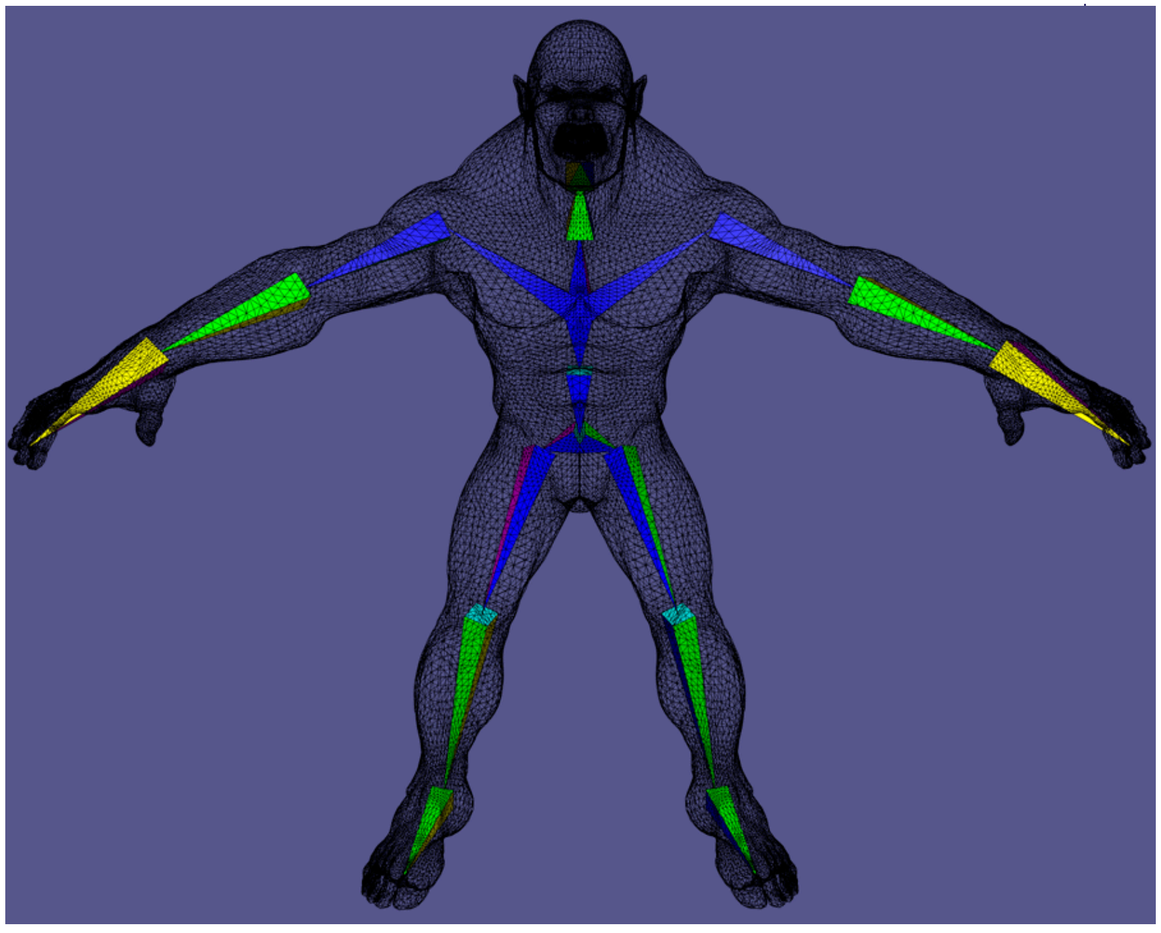
Intro to Animation and Inverse Kinematics

Computer Graphics and Imaging
UC Berkeley CS184/284A

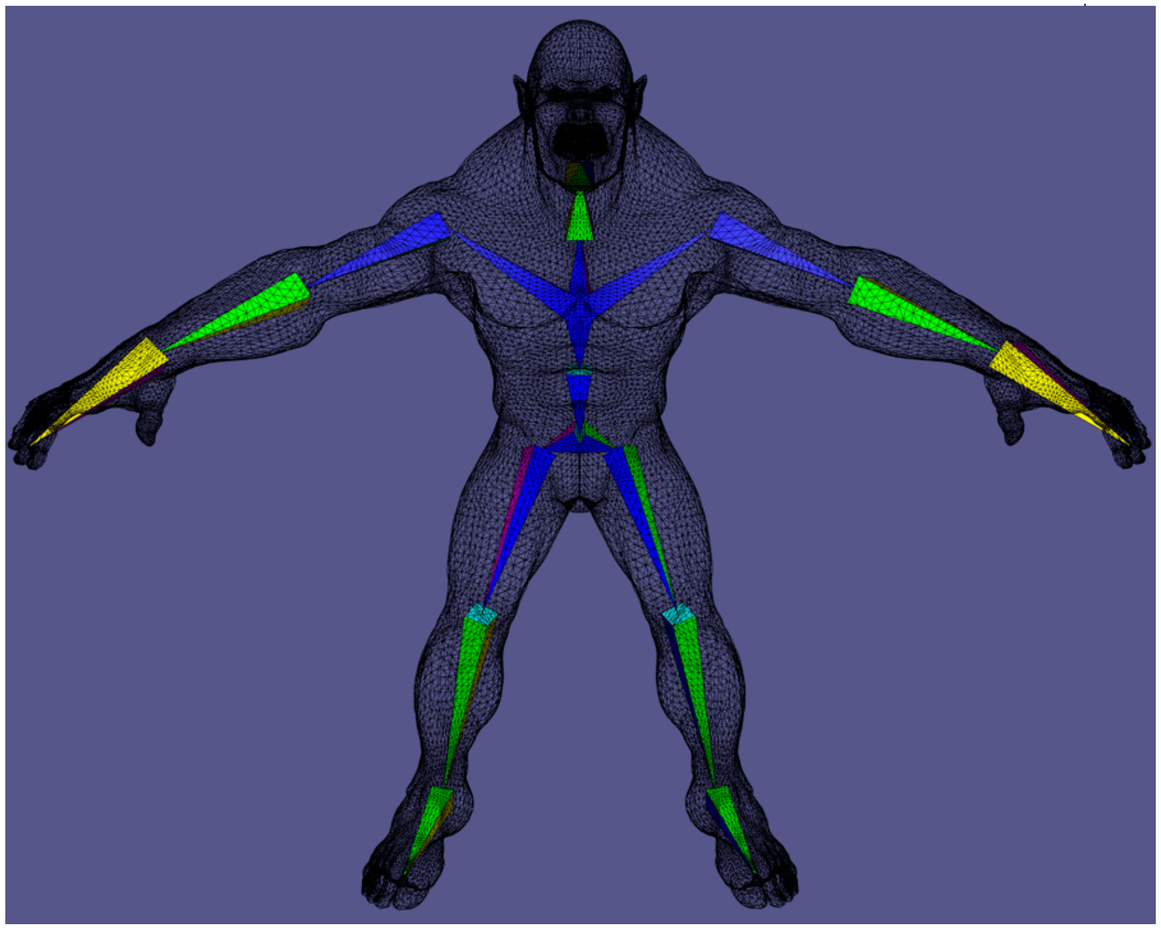
Principles of Animation

Principles of Animation

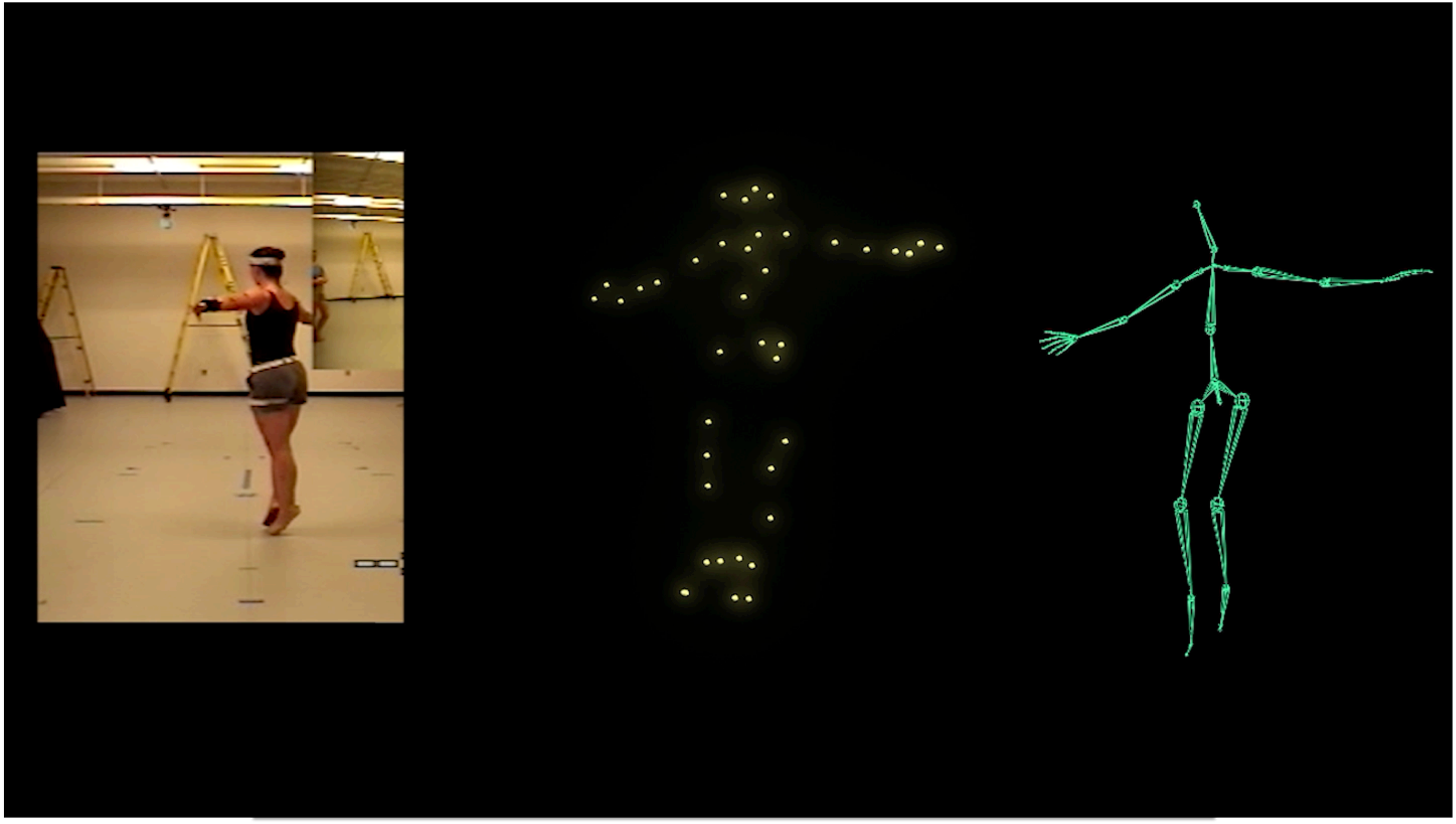
Rigging & Skinning



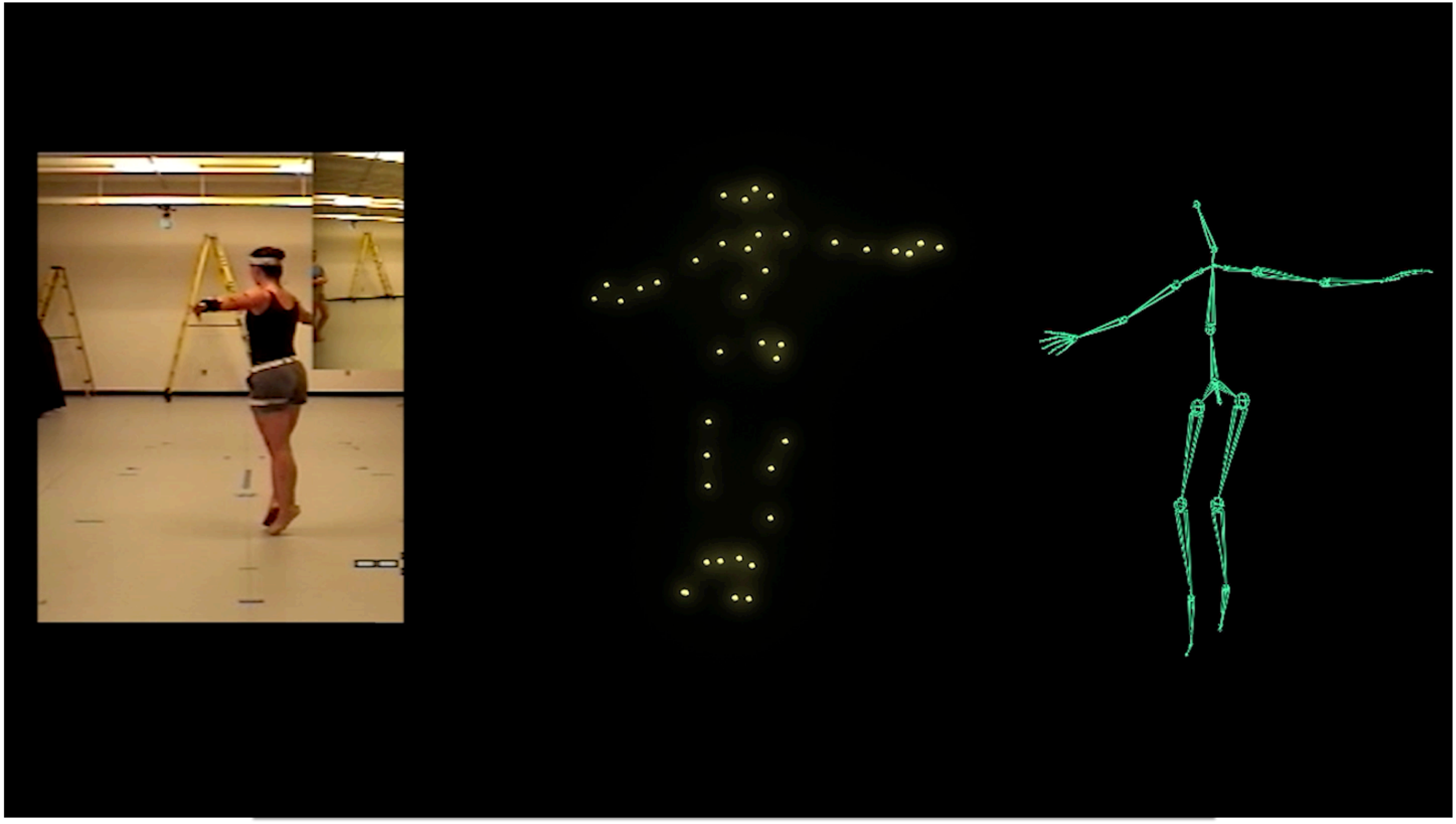
Rigging & Skinning



Motion Capture



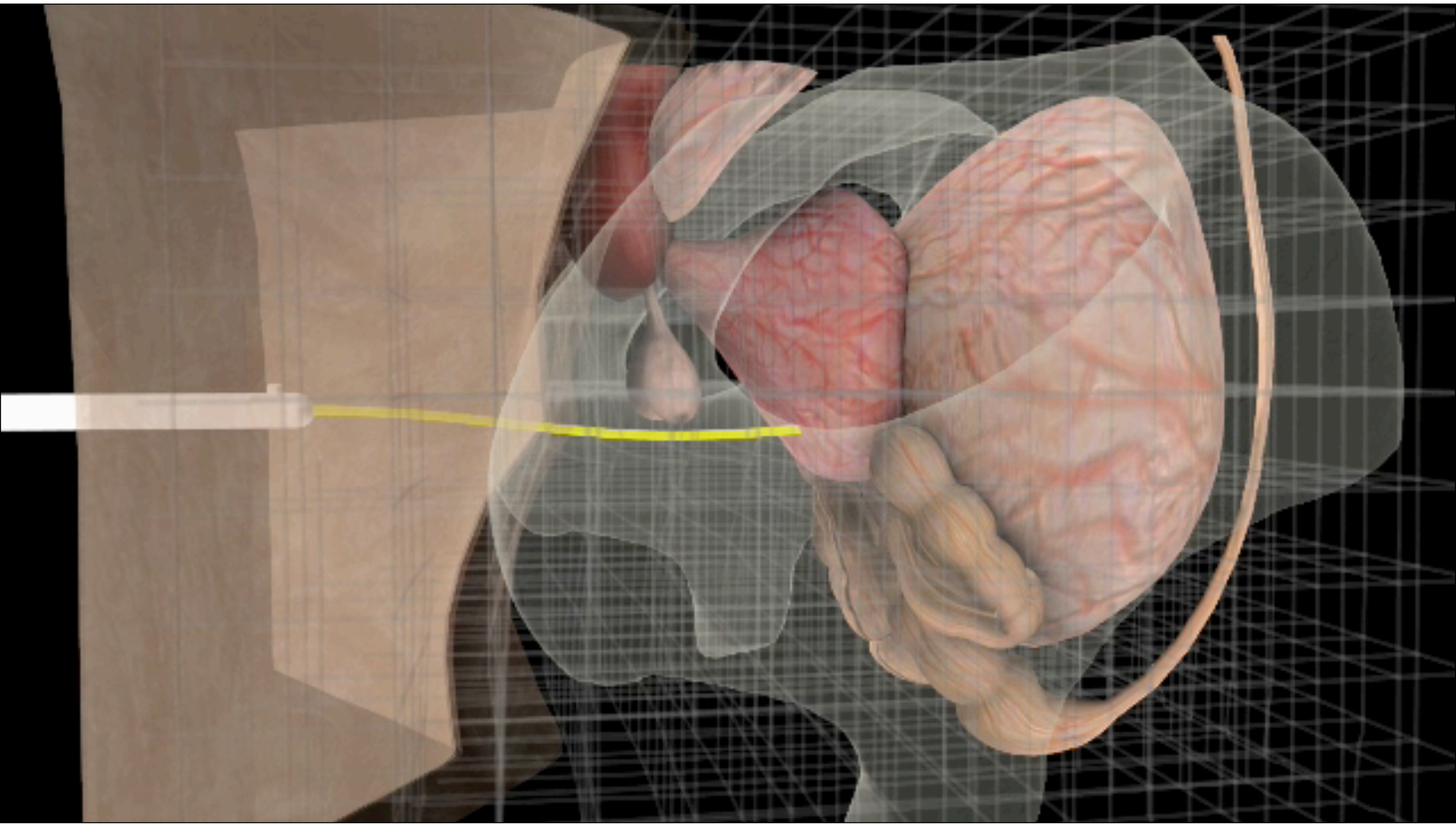
Motion Capture



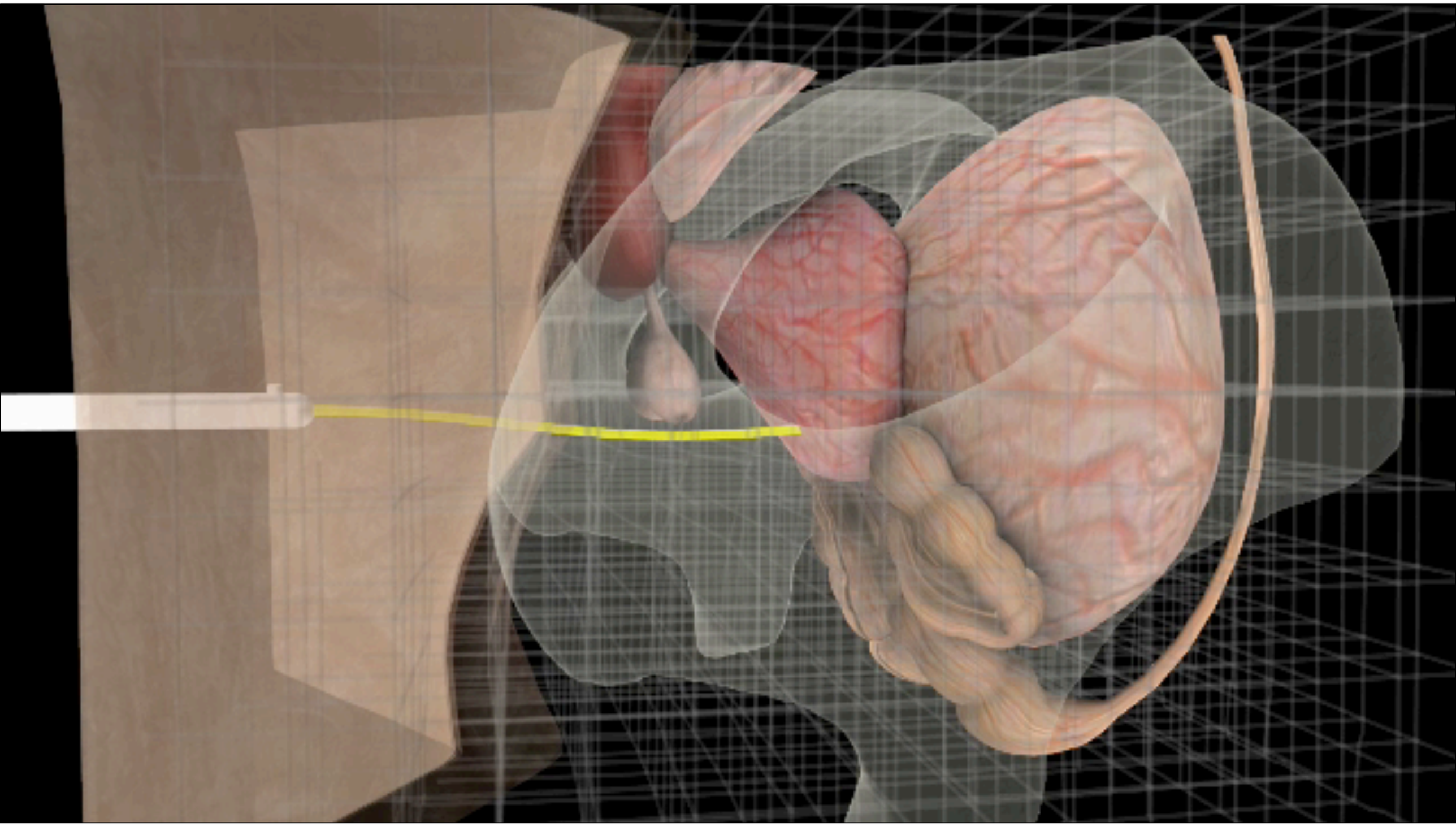
Physical Simulation: Cloth

Physical Simulation: Cloth

Physical Simulation: Surgery



Physical Simulation: Surgery



Animation

"Bring things to life"

- Communication tool
- Aesthetic issues often dominate technical issues

An extension of modeling

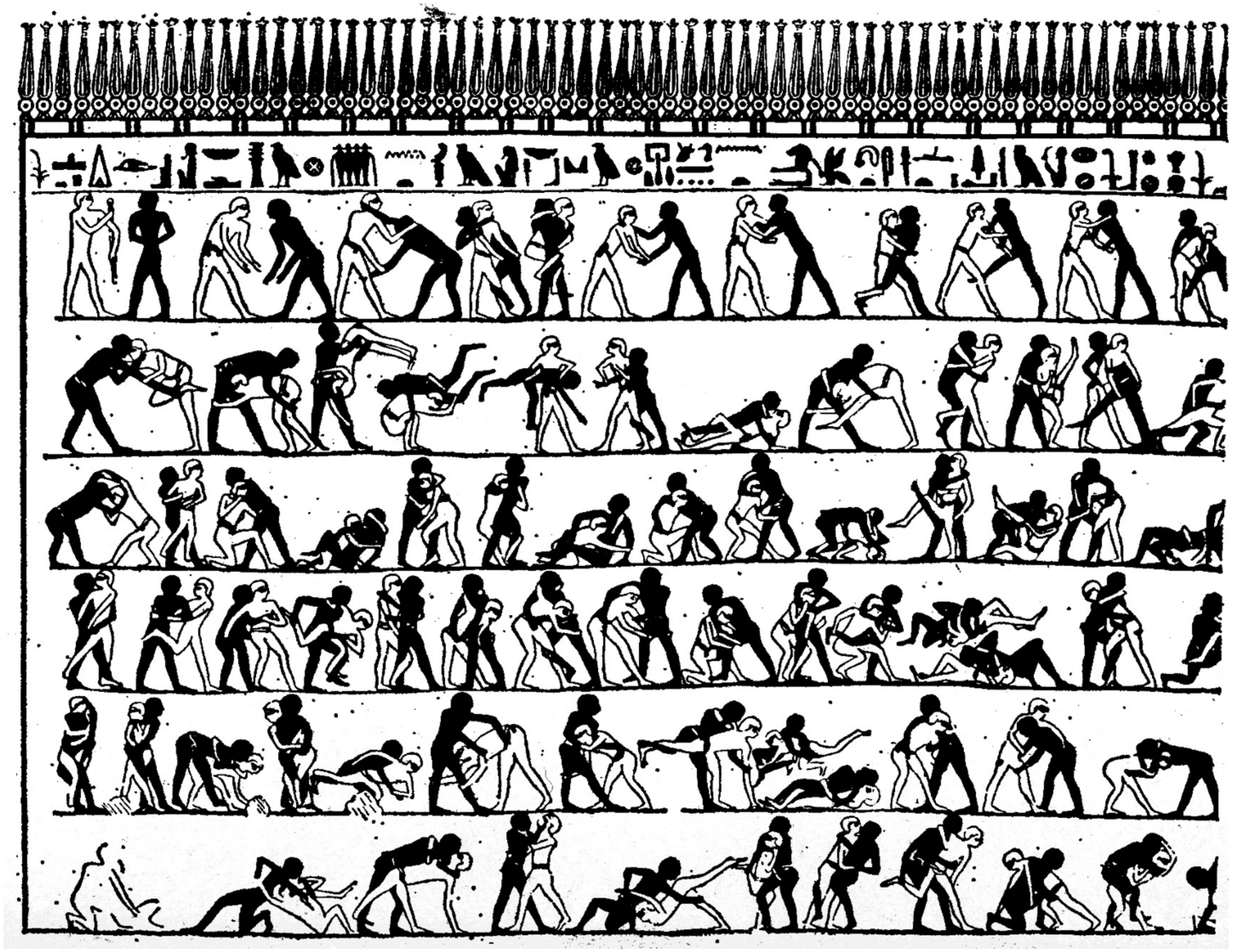
- Represent scene models as a function of space

Output: sequence of images that when viewed sequentially provide a sense of motion

- Film: 24 frames per second
- Video: 30 fps
- Virtual reality: 90+ fps low latency

Historical Points in Animation

History of Animation



(wrestling movements, tomb of Khnumhotep, Egypt 2400 BCE)

History of Animation

(Phenakistoscope, 1831)

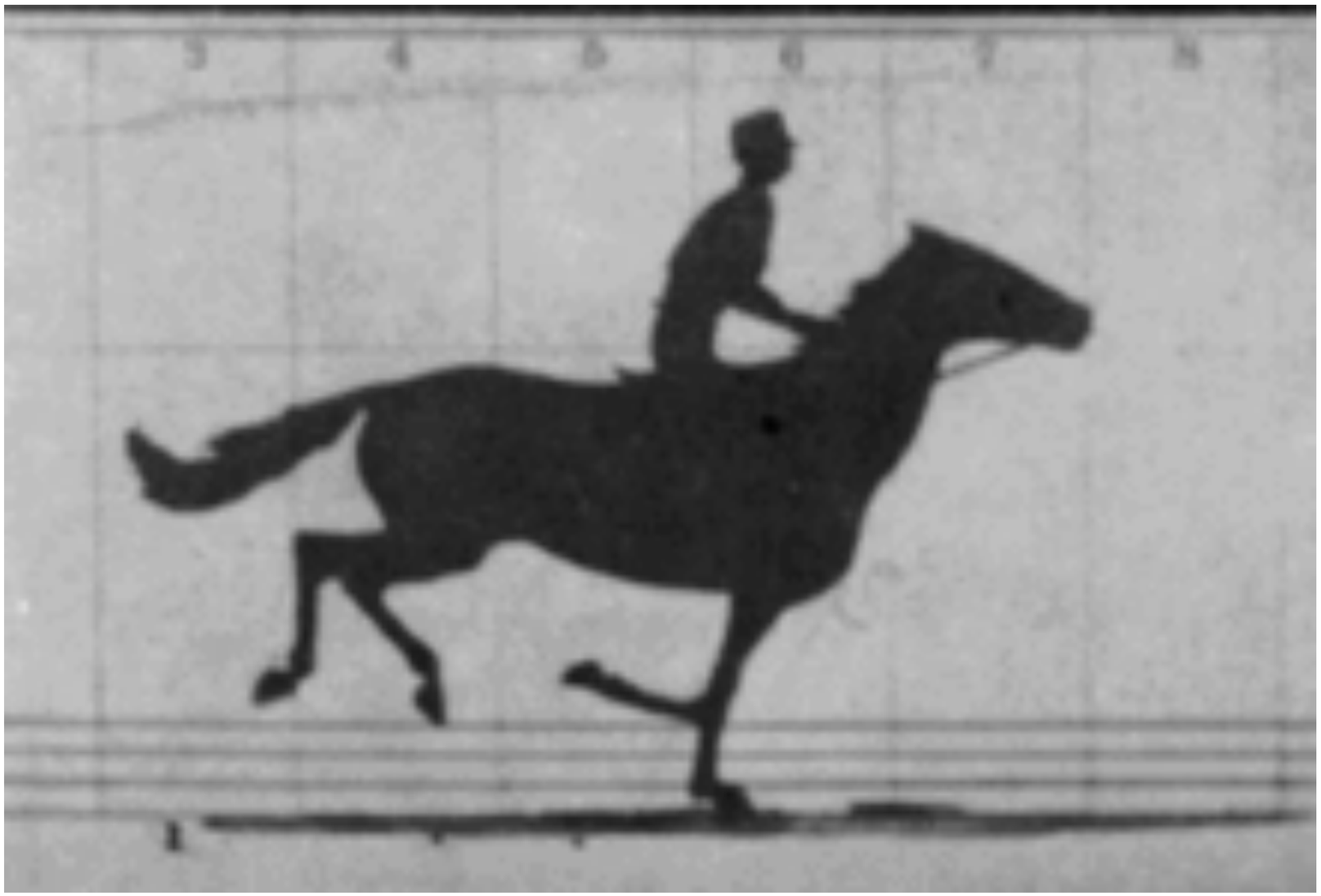
History of Animation

(Phenakistoscope, 1831)

First Film

Originally used as scientific tool rather than for entertainment

Critical technology that accelerated development of animation

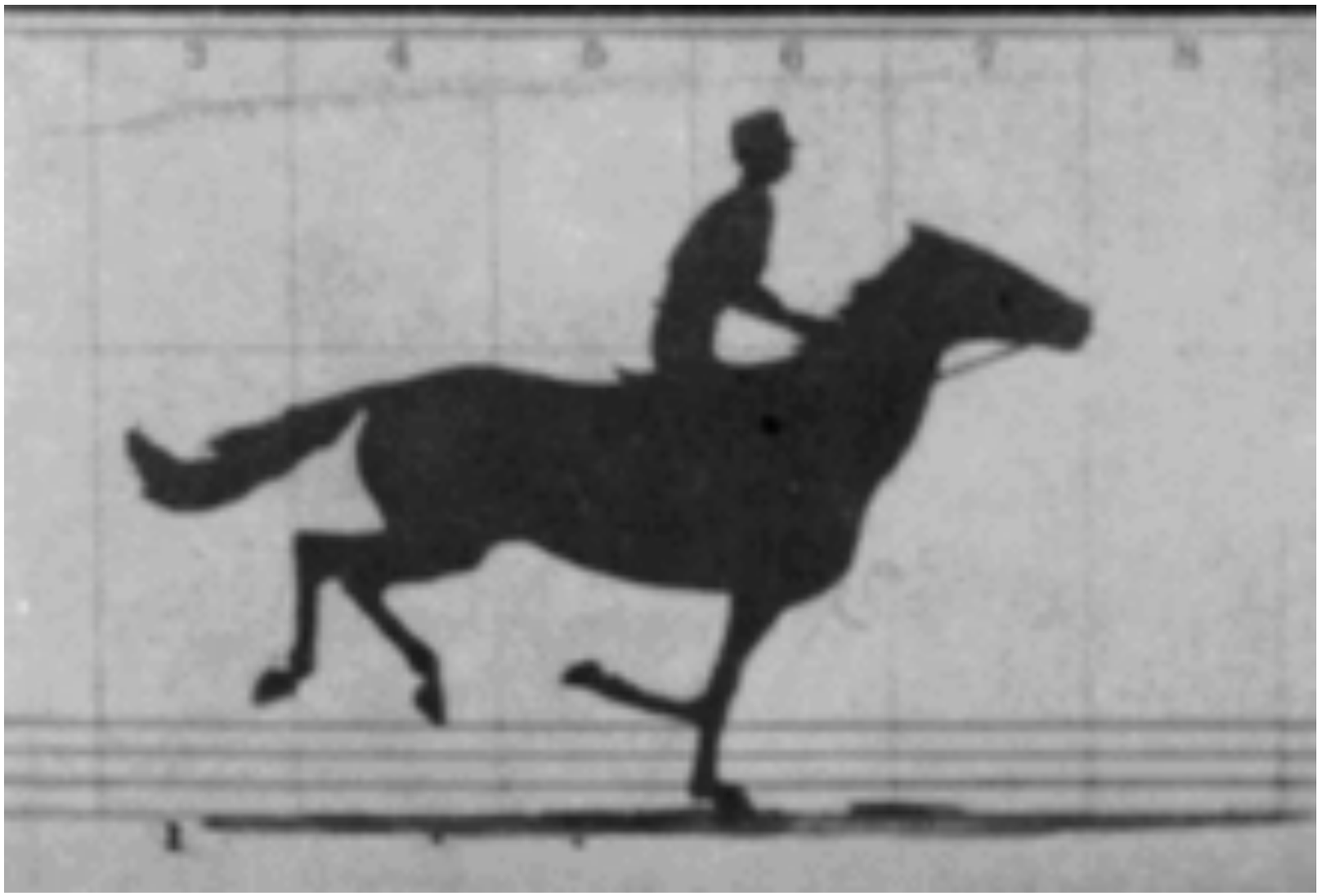


Edward Muybridge, *"Sallie Gardner"* (1878)

First Film

Originally used as scientific tool rather than for entertainment

Critical technology that accelerated development of animation

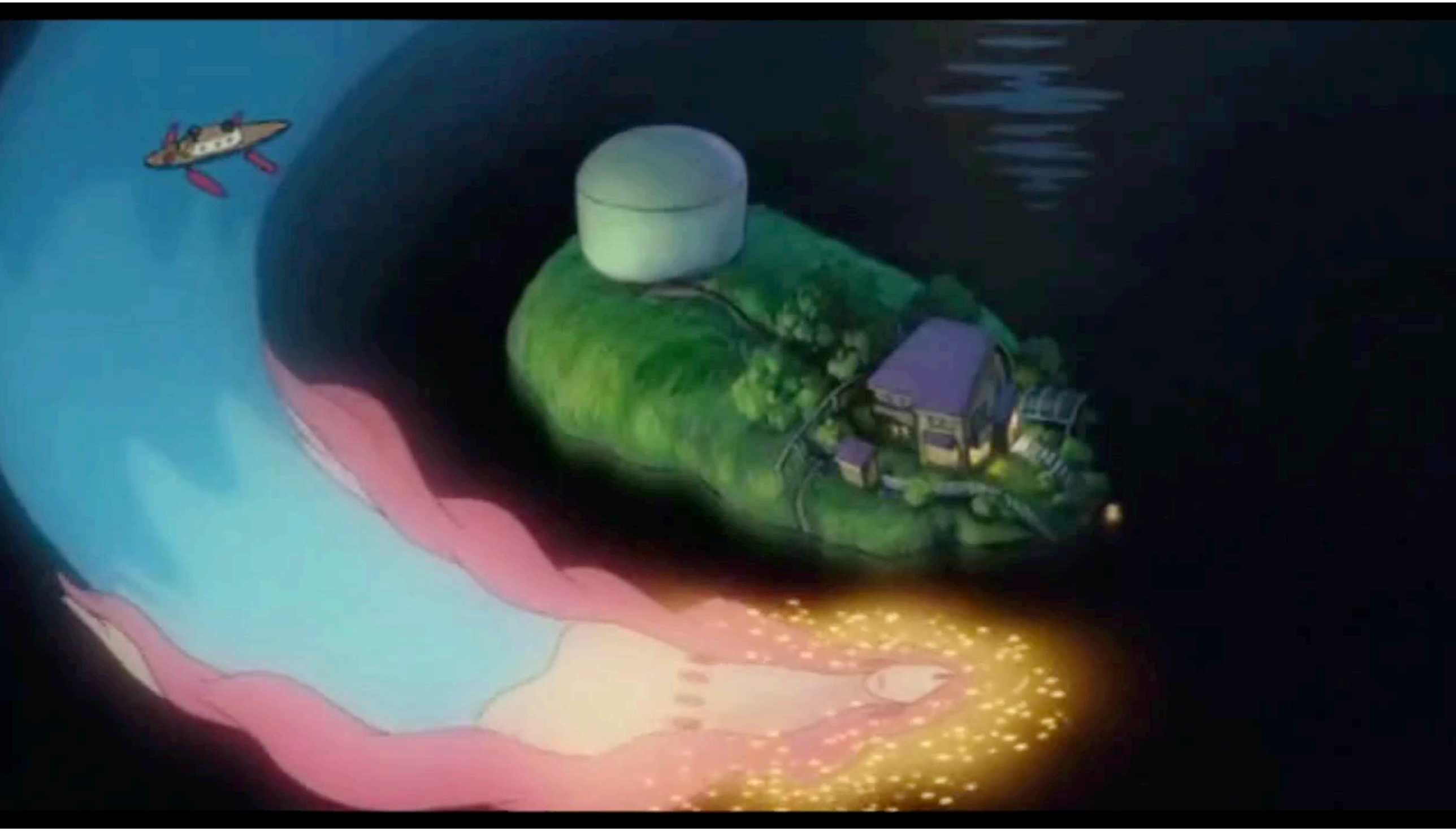


Edward Muybridge, *"Sallie Gardner"* (1878)

First Hand-Drawn Feature-Length Animation

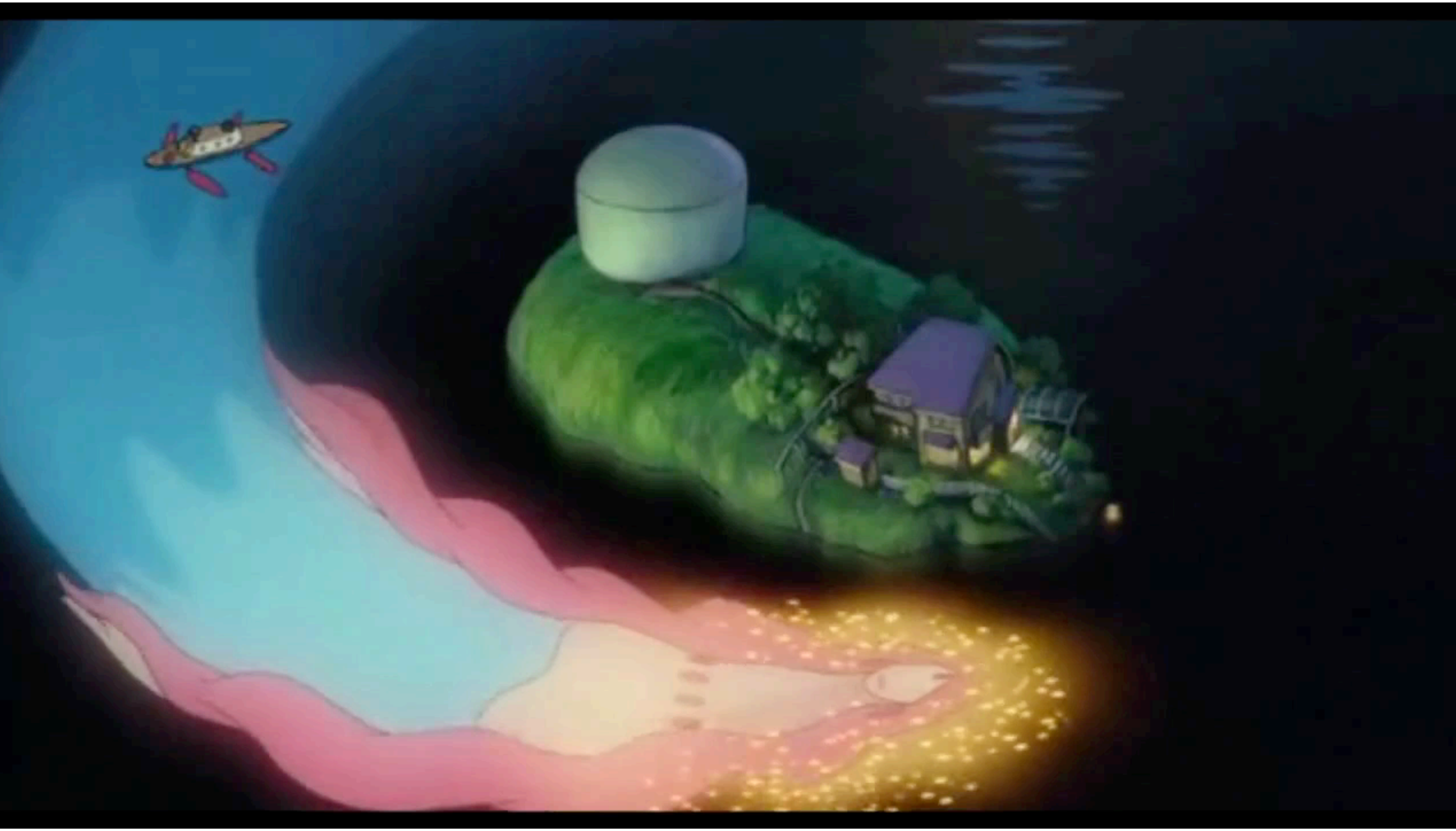
Disney, “Snow White and the Seven Dwarfs” (1937)

Hand-Drawn Animation - Present Day



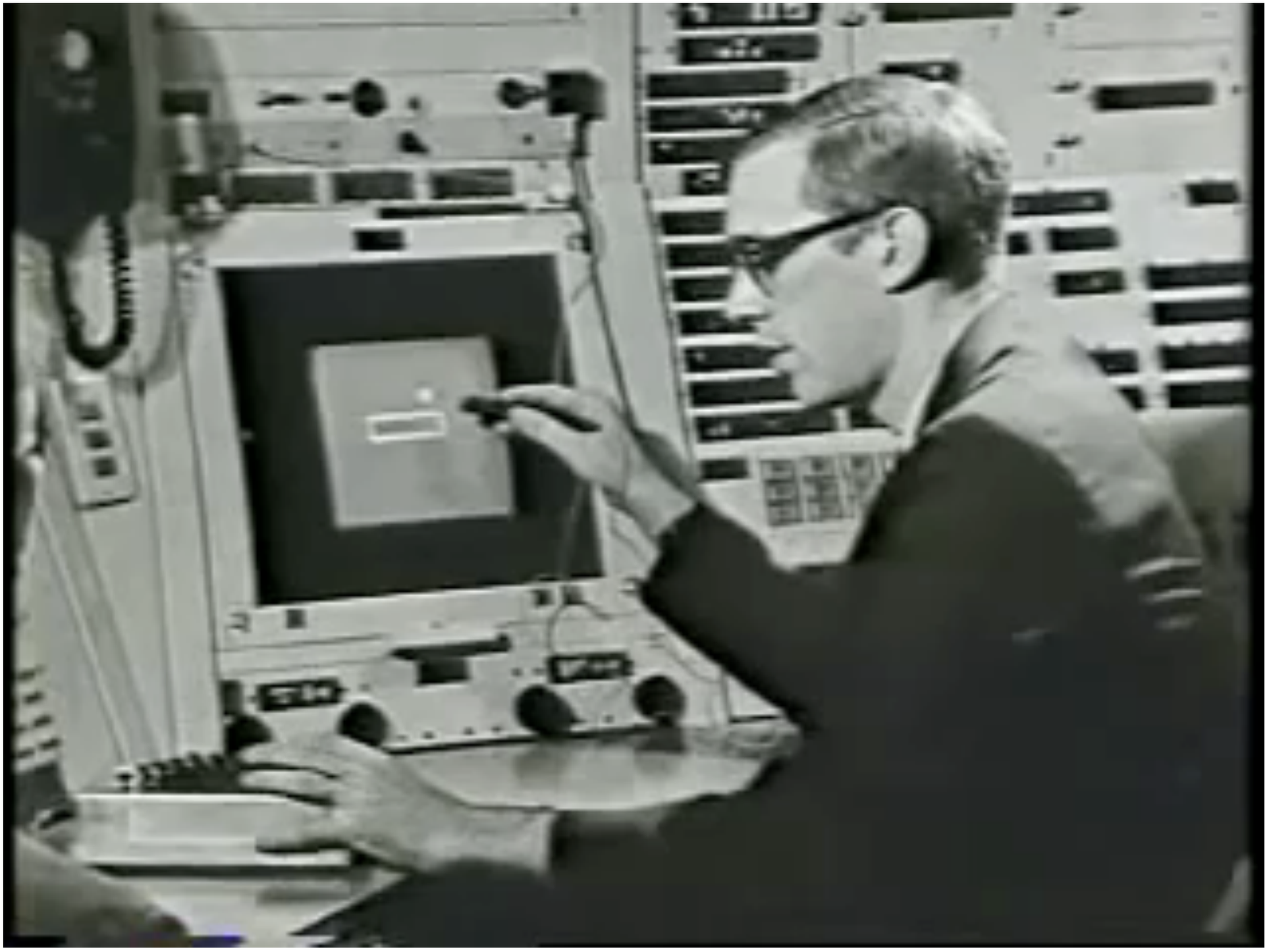
Studio Ghibli, “Ponyo” (2008)

Hand-Drawn Animation - Present Day



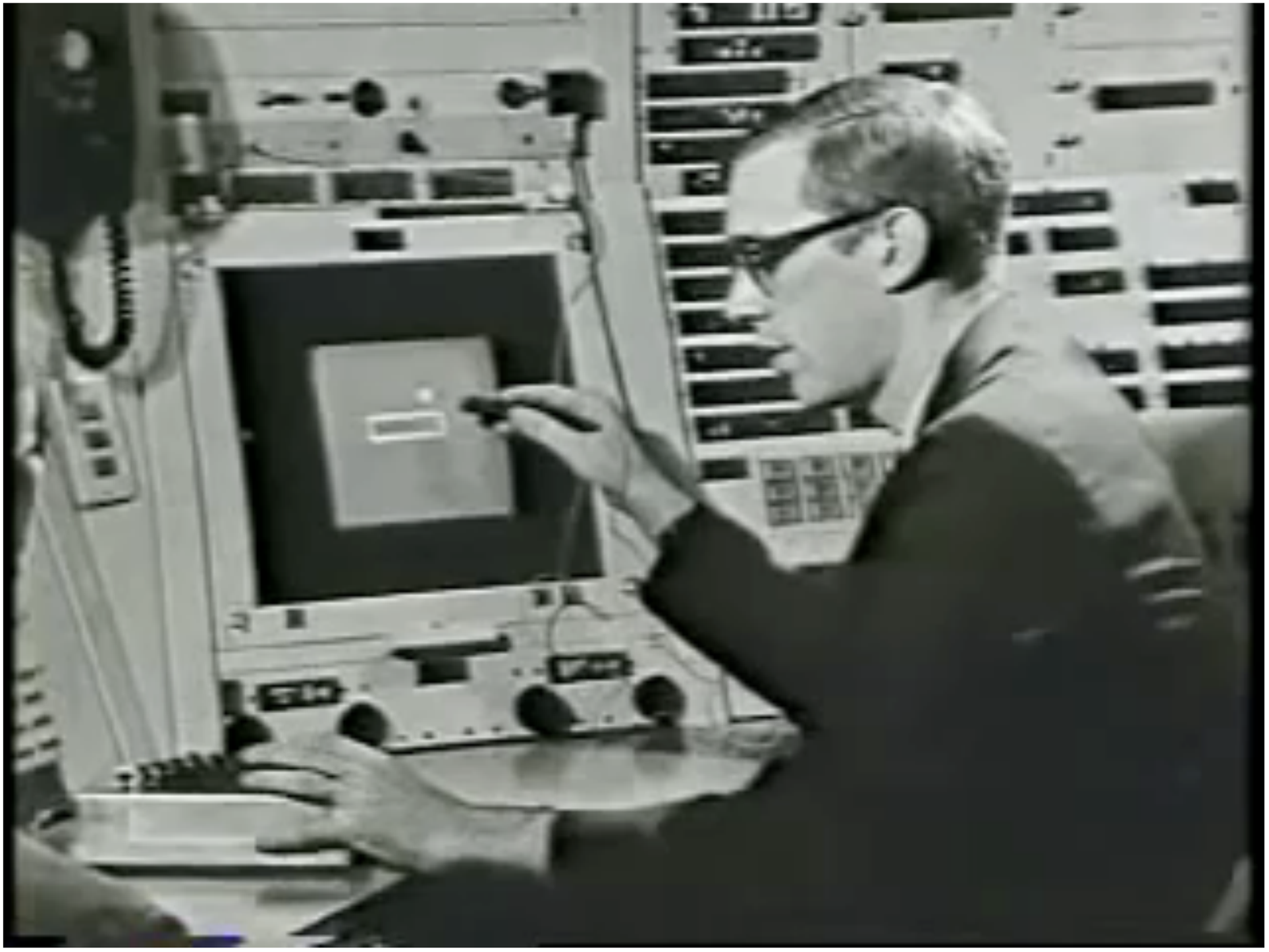
Studio Ghibli, “Ponyo” (2008)

First Digital-Computer-Generated Animation



Ivan Sutherland, “Sketchpad” (1963) – Light pen, vector display

First Digital-Computer-Generated Animation



Ivan Sutherland, “Sketchpad” (1963) – Light pen, vector display

Early Computer Animation

Nikolay Konstantinov, “Kitty” (1968)

Early Computer Animation

Nikolay Konstantinov, “Kitty” (1968)

Early Computer Animation

Ed Catmull & Frederick Parke, “Computer Animated Faces” (1972)

Early Computer Animation

Ed Catmull & Frederick Parke, “Computer Animated Faces” (1972)

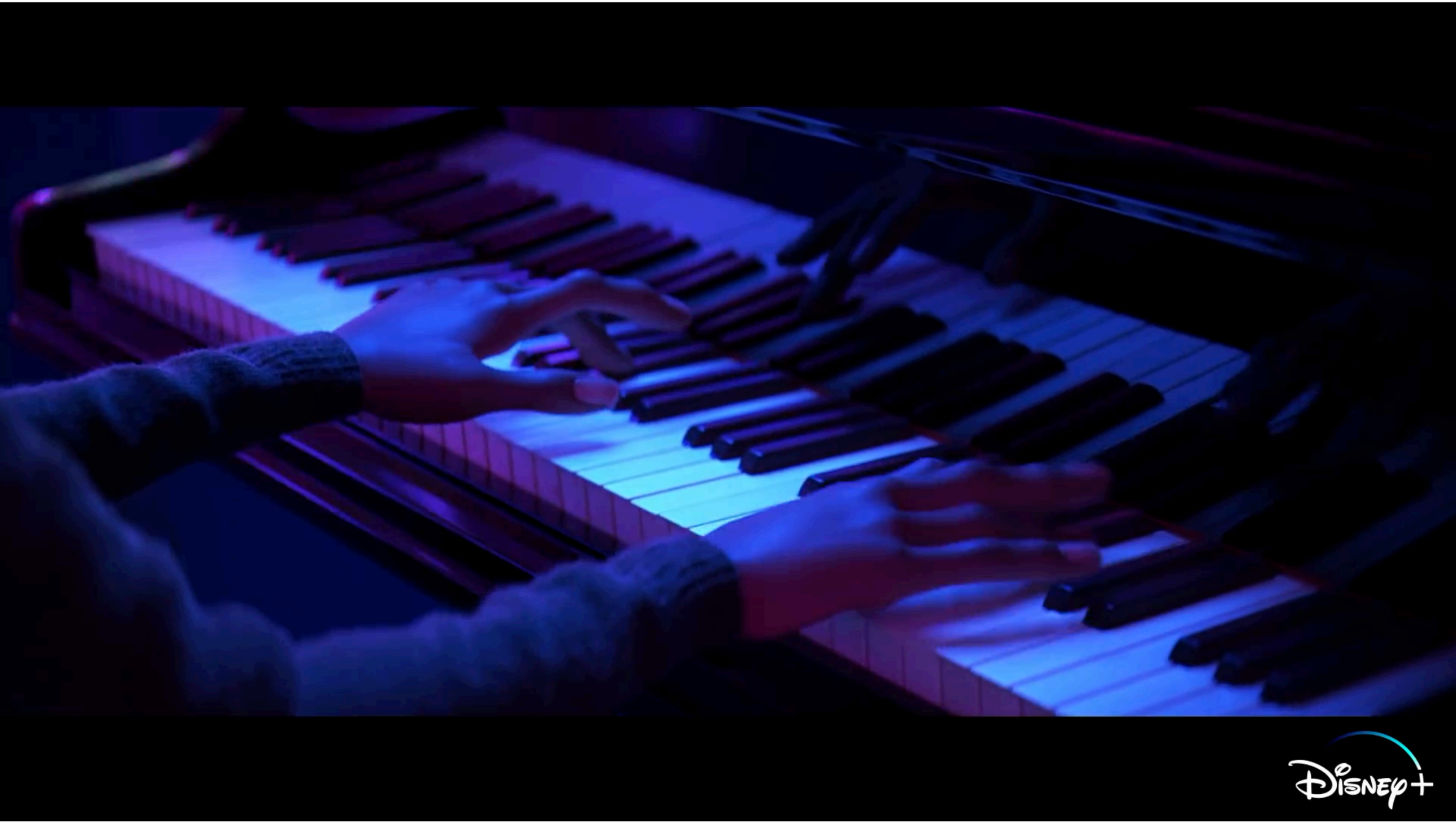
First CG Feature Film

Pixar, “Toy Story” (1995)

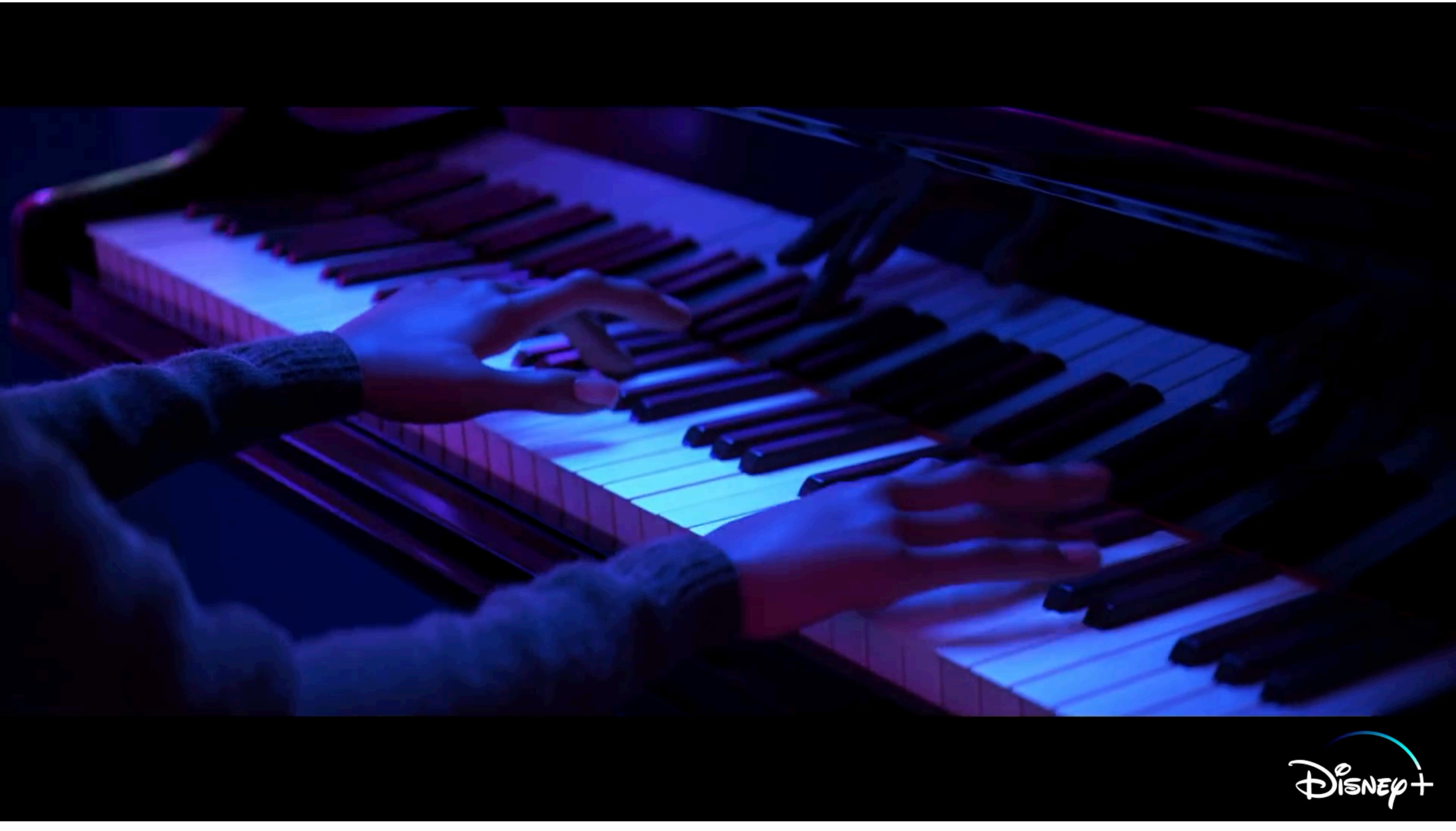
First CG Feature Film

Pixar, “Toy Story” (1995)

Computer Animation - Present Day



Computer Animation - Present Day



Animation Principles

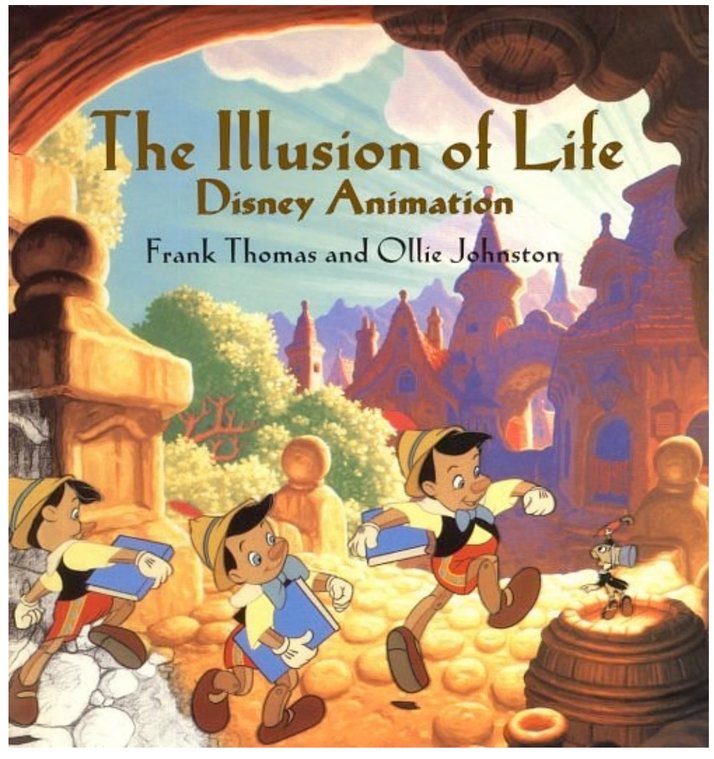
Depiction of Motion

(Duchamp, 1912)

Animation Principles

From

- "Principles of Traditional Animation Applied to 3D Computer Animation" - John Lasseter, ACM Computer Graphics, 21(4), 1987



In turn from

- "The Illusion of Life"
  Frank Thomas and Ollie Johnson

Same for 2D and 3D

http://www.siggraph.org/education/materials/HyperGraph/animation/character_animation/principles/prin_trad_anim.htm

12 Animation Principles

1. Squash and stretch
2. Anticipation
3. Staging
4. Straight ahead and pose-to-pose
5. Follow through
6. Ease-in and ease-out
7. Arcs
8. Secondary action
9. Timing
10. Exaggeration
11. Solid drawings
12. Appeal

Aside from #4, these all either replicate physical phenomena or replication human signaling or combination of both.

12 Animation Principles

Cento Lodgiani, https://vimeo.com/93206523

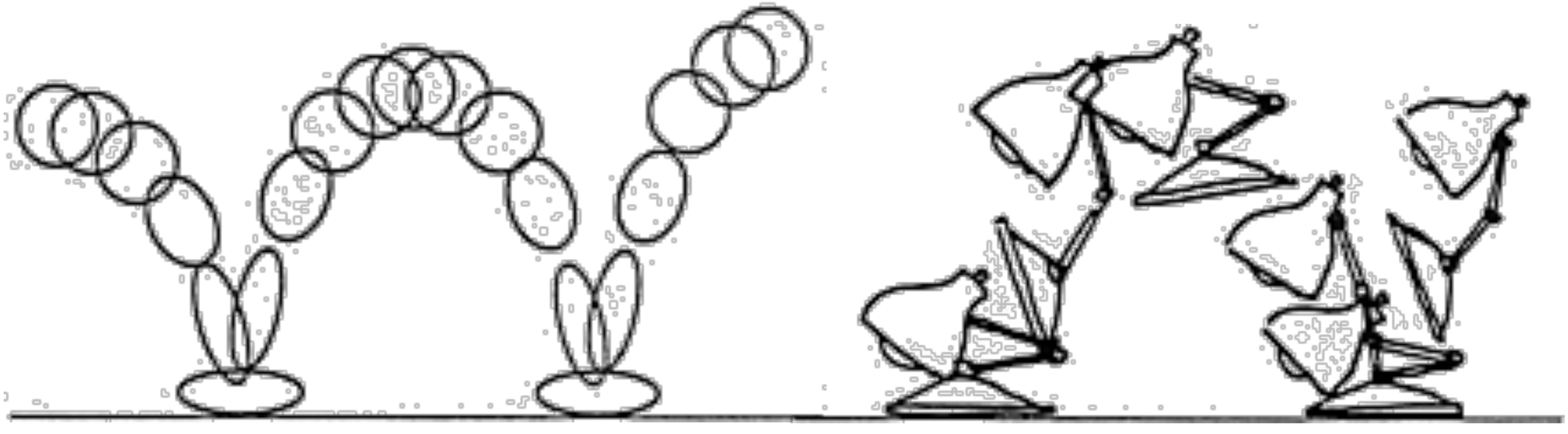
12 Animation Principles

Cento Lodgiani, https://vimeo.com/93206523

Squash and Stretch

Refers to defining the rigidity and mass of an object by distorting its shape during an action.

Shape of object changes during movement, but not its volume.

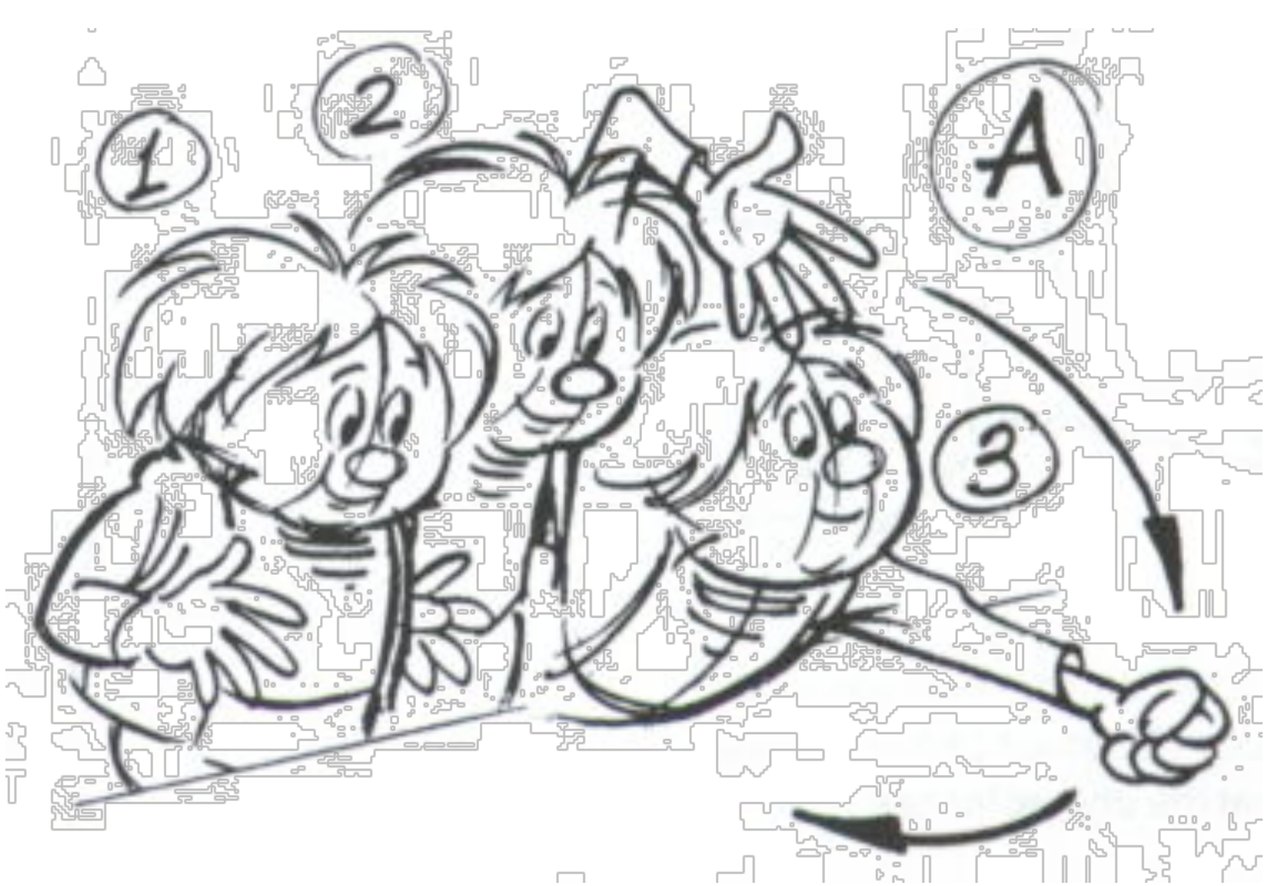


Anticipation

Prepare for each movement

For physical realism

To direct audience's attention



Timing for Animation, Whitaker & Halas

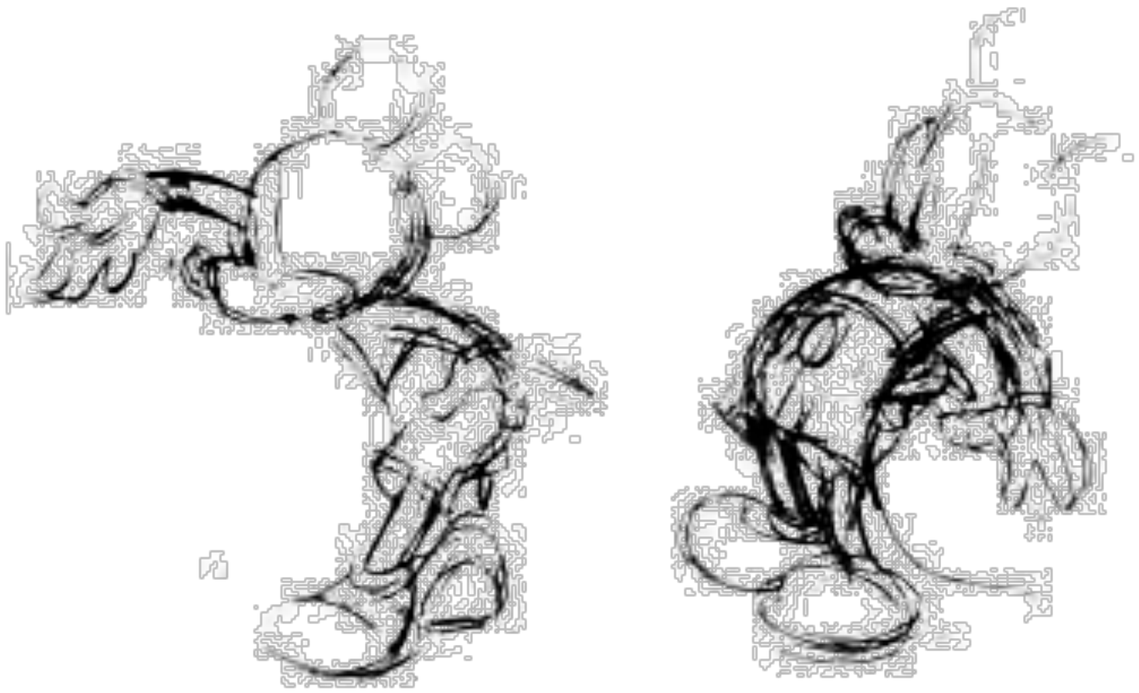
Staging

Picture is 2D

Make situation clear

Audience looking in right place

Action clear in silhouette



Disney Animation: The Illusion of Life

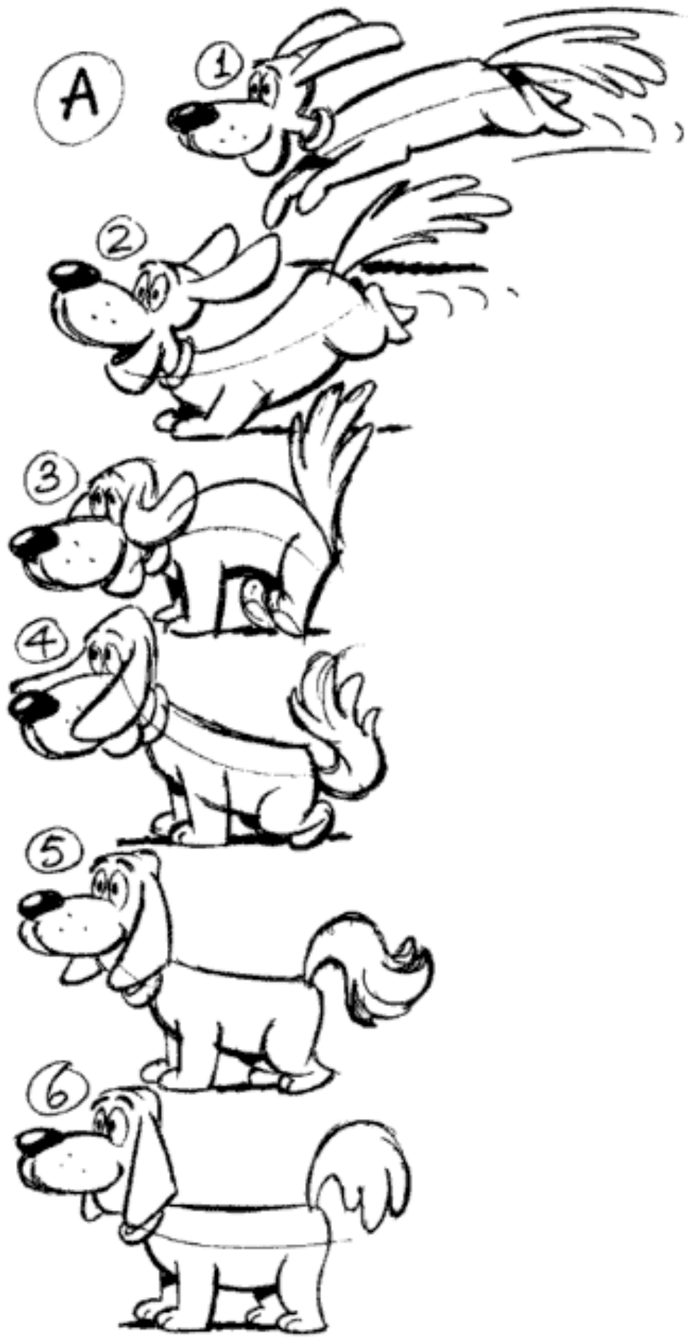
Follow Through

Overlapping motion

Motion doesn’t stop suddenly

Pieces continue at different rates

One motion starts while previous is finishing, keeps animation smooth

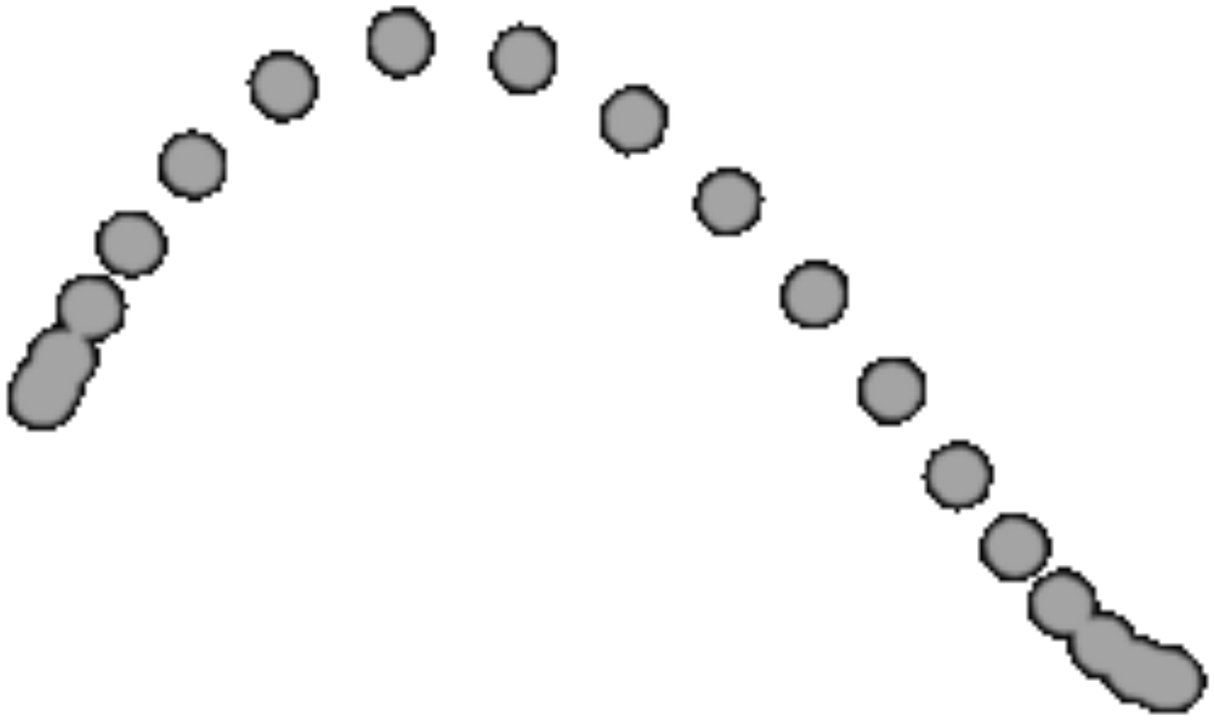


Timing for Animation, Whitaker & Halas

Ease-In and Ease-Out

Movement doesn’t start & stop abruptly.

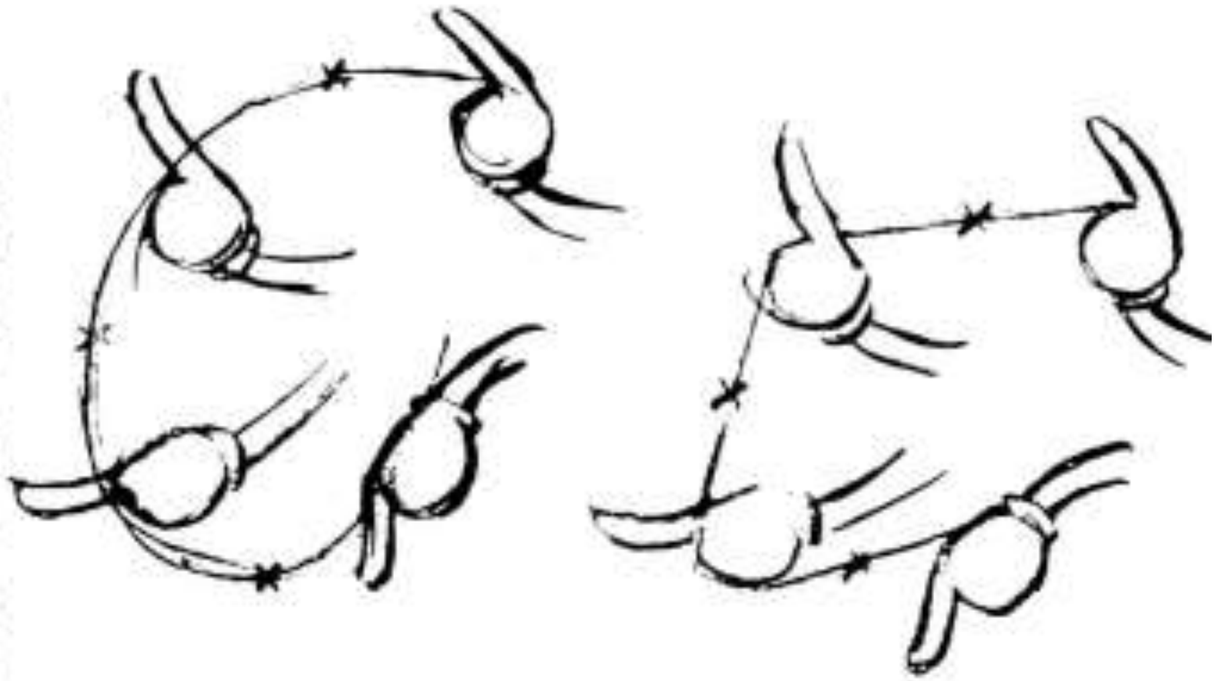
Also contributes to weight and emotion



Arcs

Move in curves, not in straight lines

This is how living creatures move



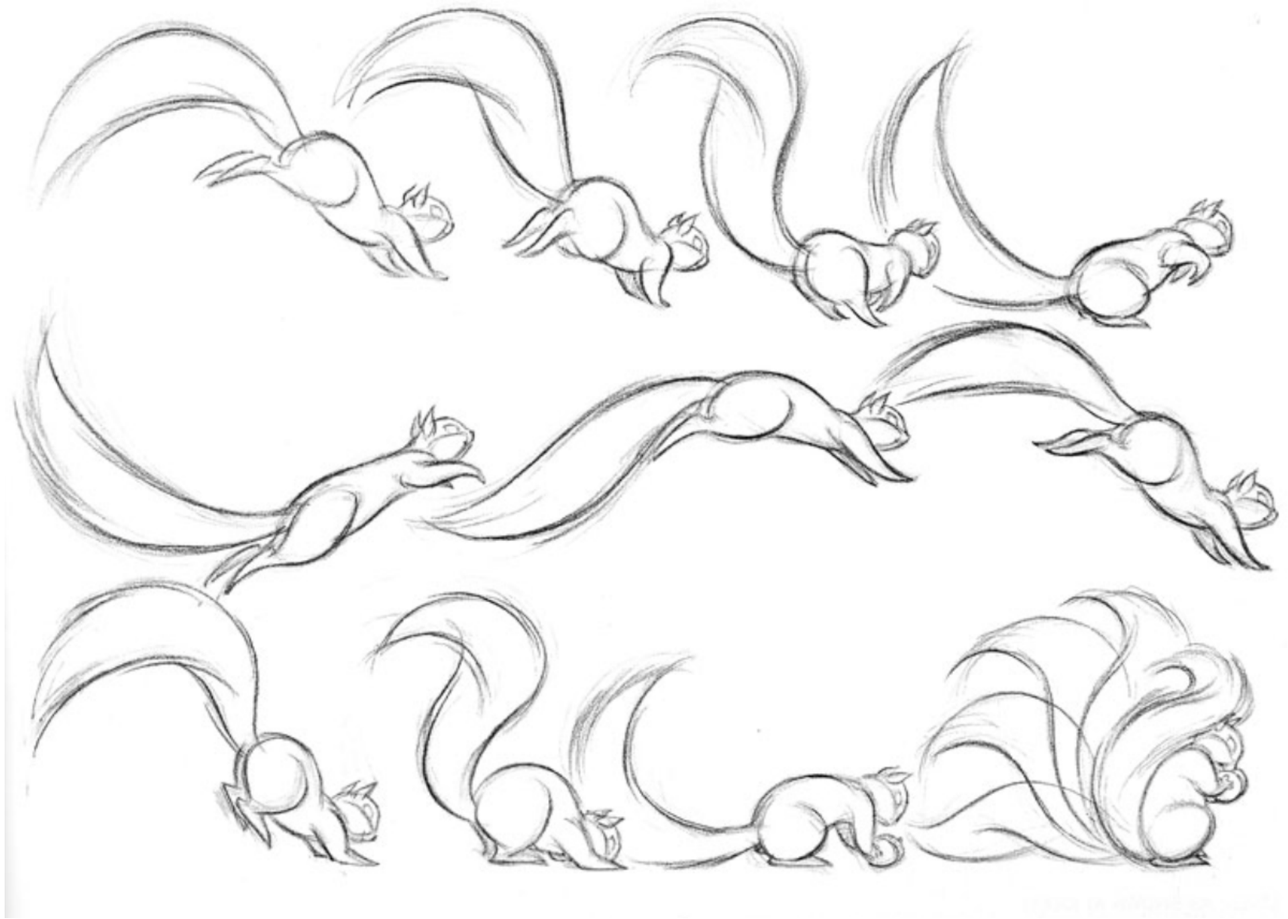
Disney Animation: The Illusion of Life

Secondary Action

Motion that results from some other action

Needed for interest and realism

Shouldn’t distract from primary motion

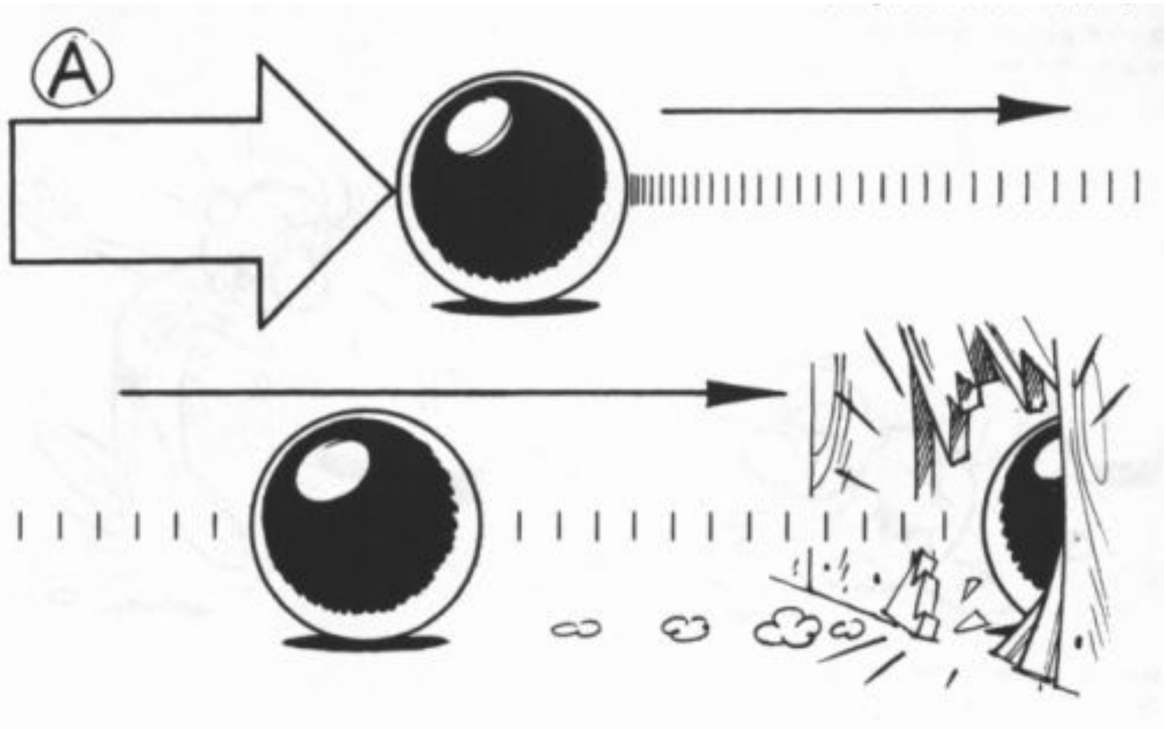


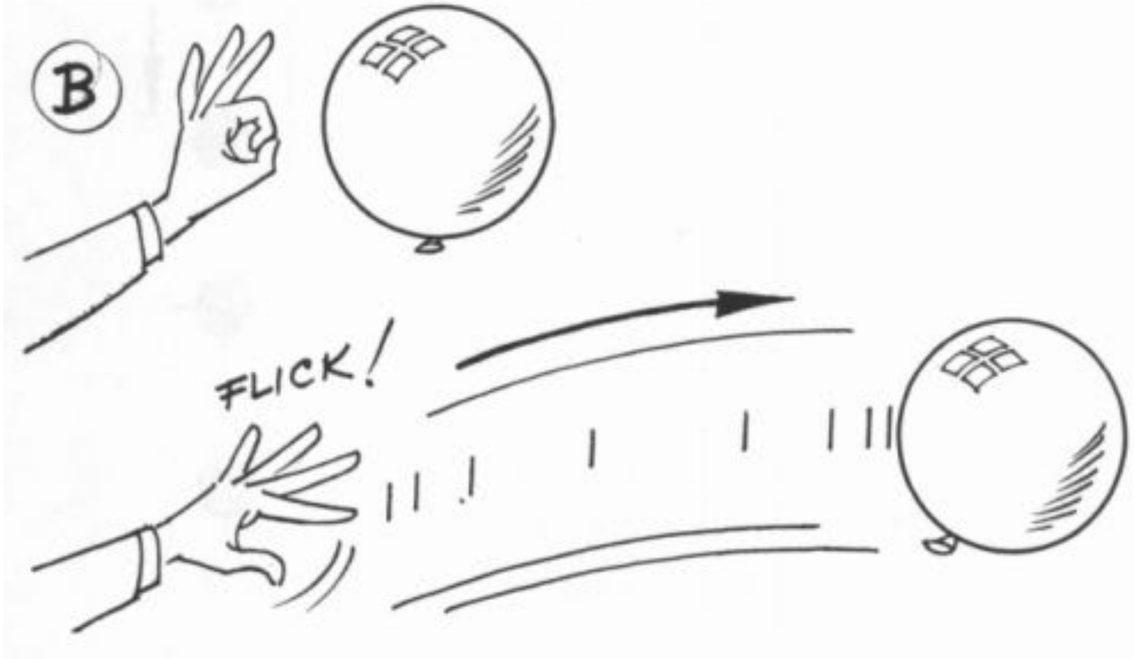
Cartoon Animation, Preston Blair

Timing

Rate of acceleration conveys weight

Speed and acceleration of character’s movements convey emotion





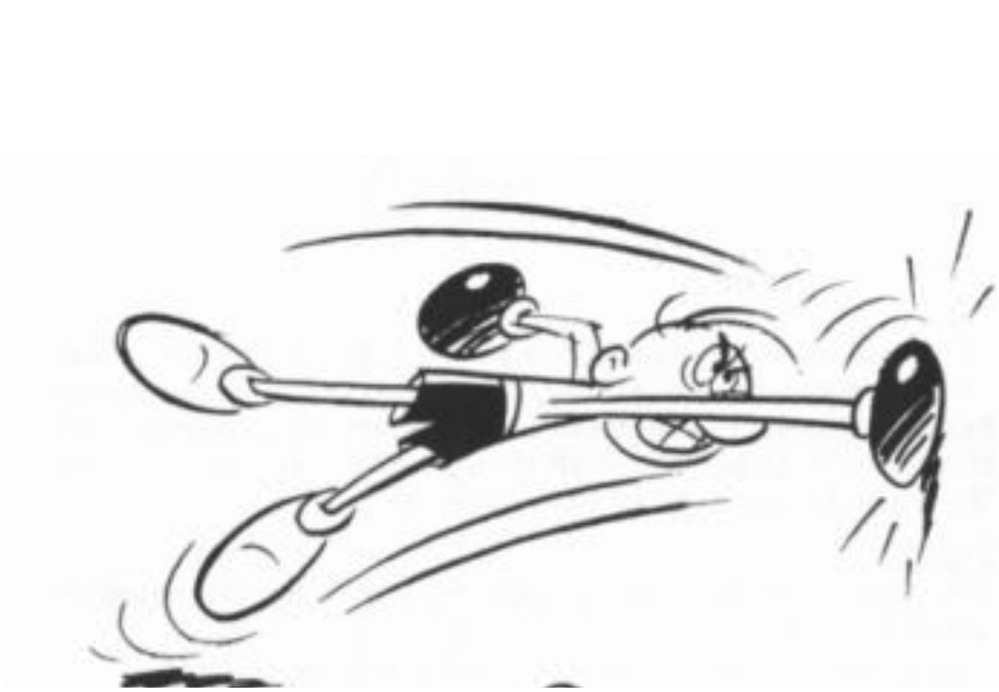
Timing for Animation, Whitaker & Halas

Exaggeration

Helps make actions clear

Helps emphasize story points and emotion

Must balance with non-exaggerated parts



Timing for Animation, Whitaker & Halas

Appeal

Attractive to the eye, strong design

Avoid symmetries

Disney Animation: The Illusion of Life

Appeal: Personality

Action of character is result of its thoughts

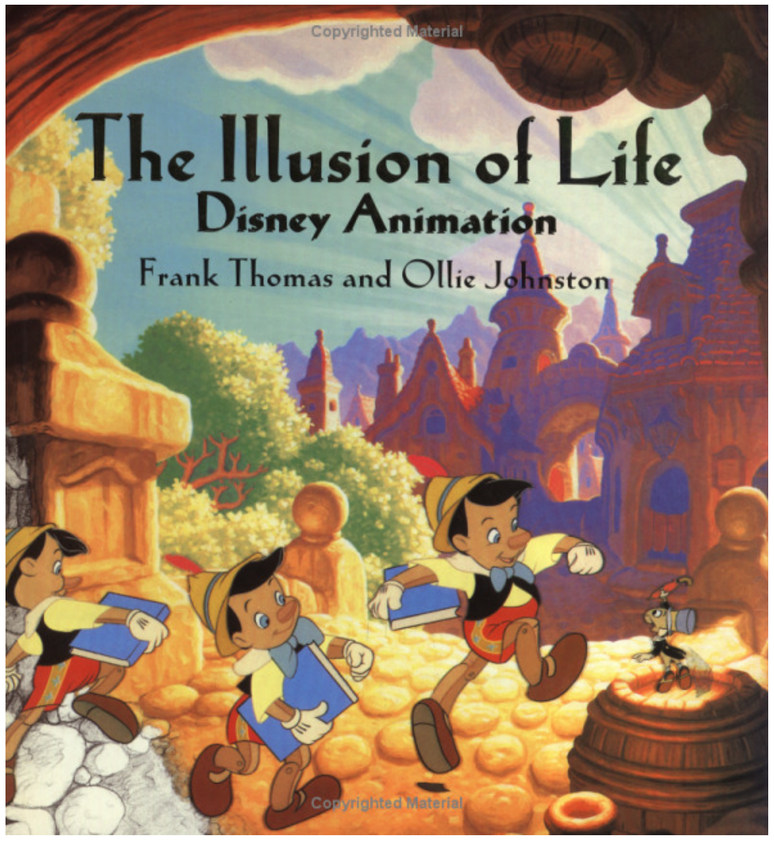
Know purpose & mood before animating each action

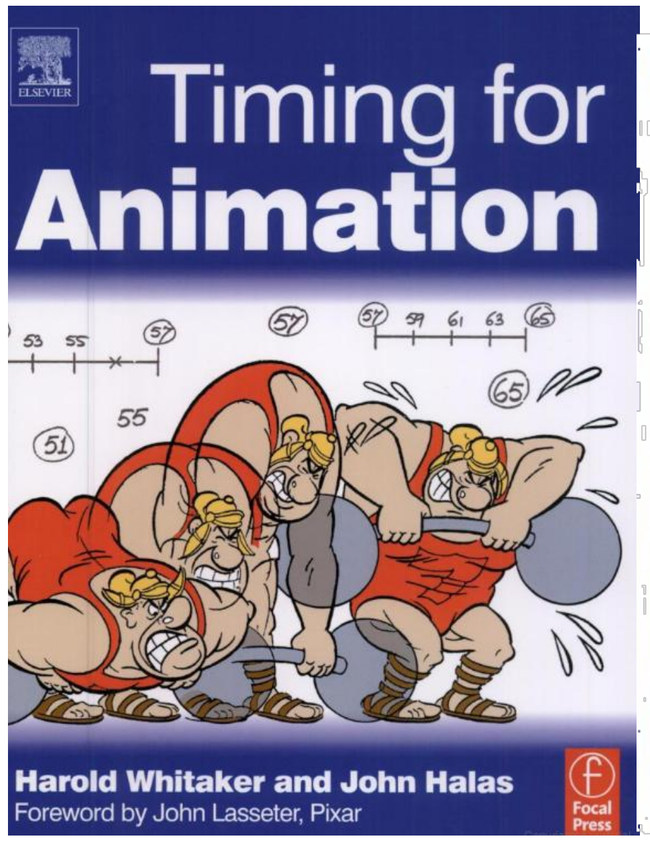
No two characters move the same way

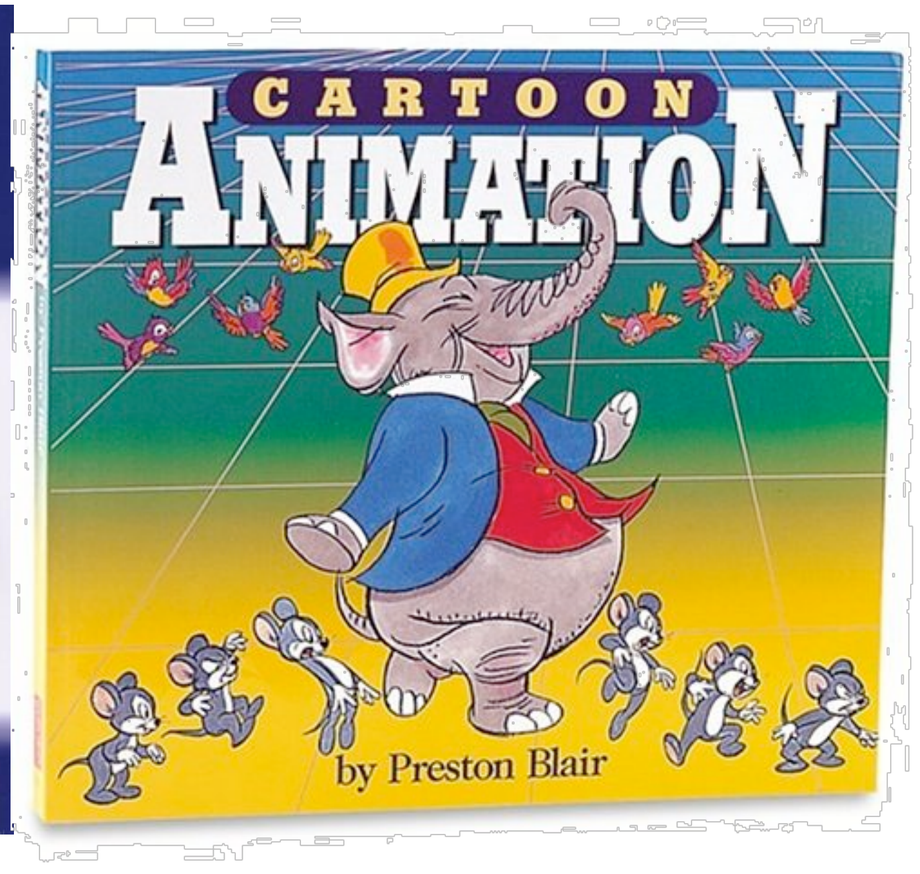
Appeal: Personality

Appeal: Personality

Further Reading







Motion Blur

Fast moving things look blurry

- Human eye
- Finite exposure time in cameras

Another sampling problem!

- Single image is a sample of incoming light a on moment in time
- Should be integral over the time frame interval.
- Without blur: strobing and aliasing
- Ray tracing: tag rays with time inside interval
    - Simultaneously anti-aliasing in space and time!

Blur over part of frame interval

- Often measured in degrees (0..360) due to original design of film cameras.

Motion Blur

No Motion Blur
CS184, UC Berkeley, Prof. O'Brien

With Motion Blur
CS184, UC Berkeley, Prof. O'Brien

Motion Blur

No Motion Blur
CS184, UC Berkeley, Prof. O'Brien

With Motion Blur
CS184, UC Berkeley, Prof. O'Brien

Motion Blur

No Motion Blur
CS184, UC Berkeley, Prof. O'Brien

With Motion Blur
CS184, UC Berkeley, Prof. O'Brien

Motion Blur

Brave 2012

Motion Blur

Brave 2012

Keyframe Animation

Animator (e.g. lead animator) creates keyframes

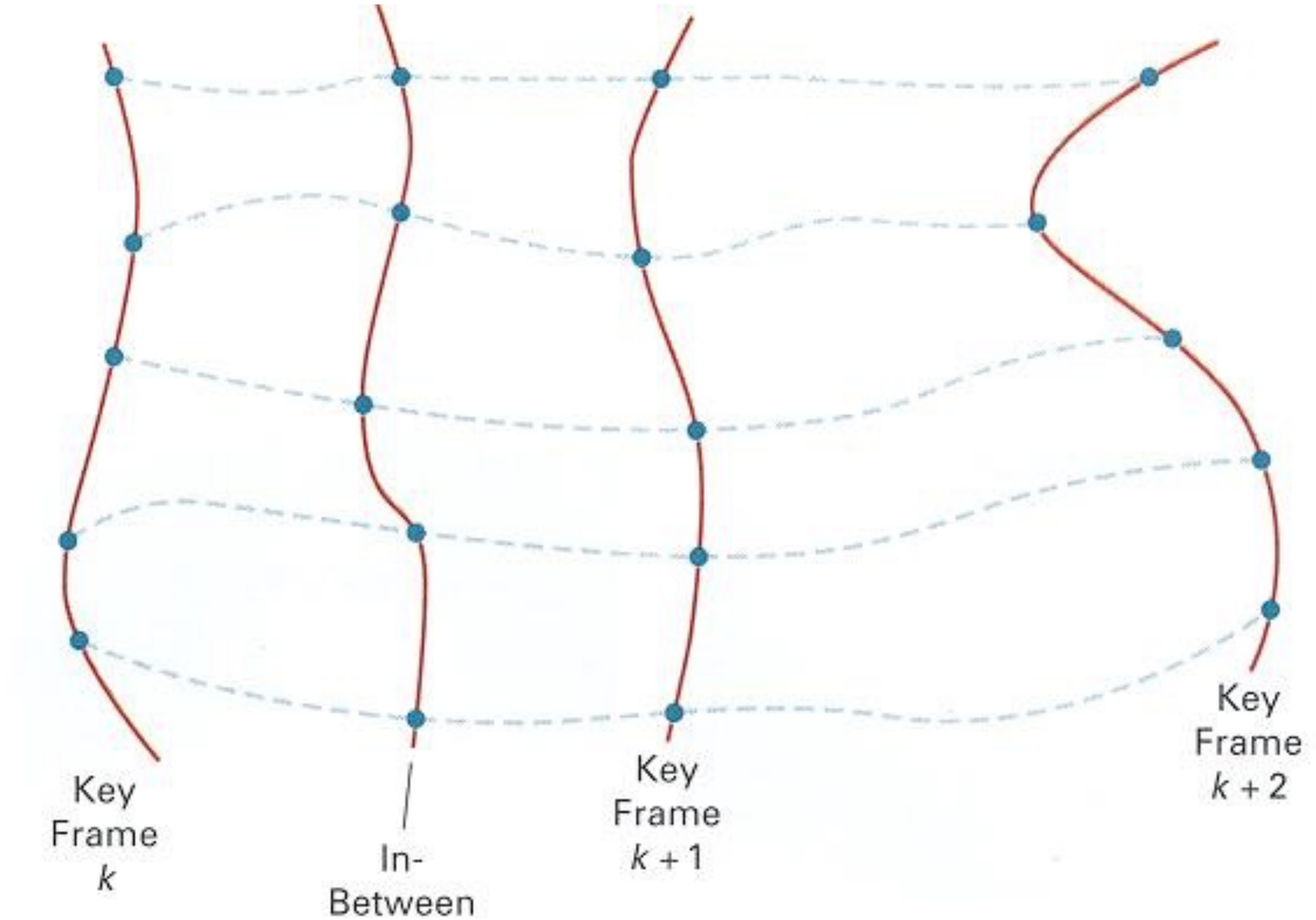
Assistant (person or computer) creates in-between frames (“tweening”)

Keyframes

“Tweens”

Keyframe Interpolation

Think of each frame as a vector of parameter values

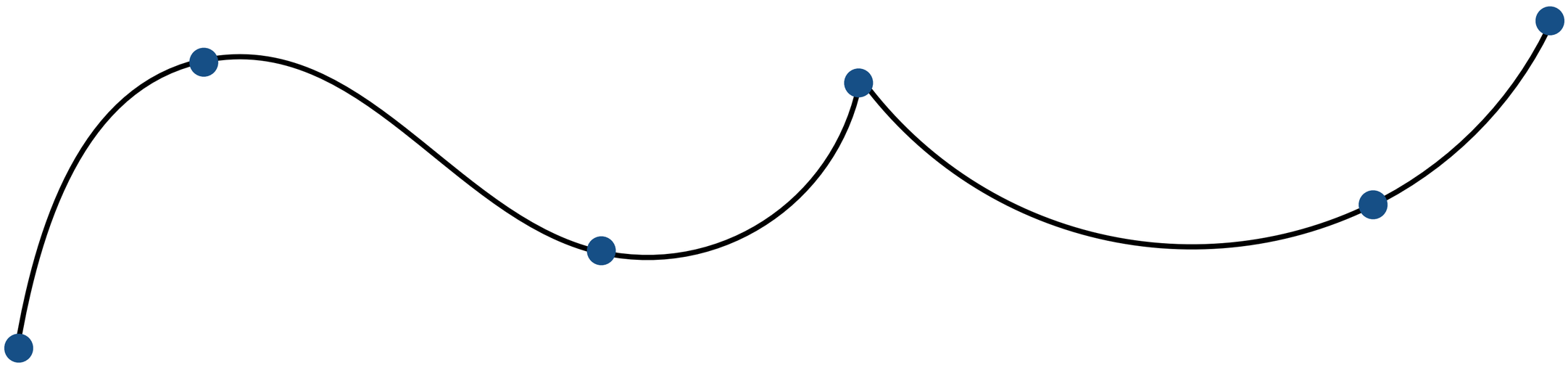


Hearn, Baker and Carithers, Figure 16.11

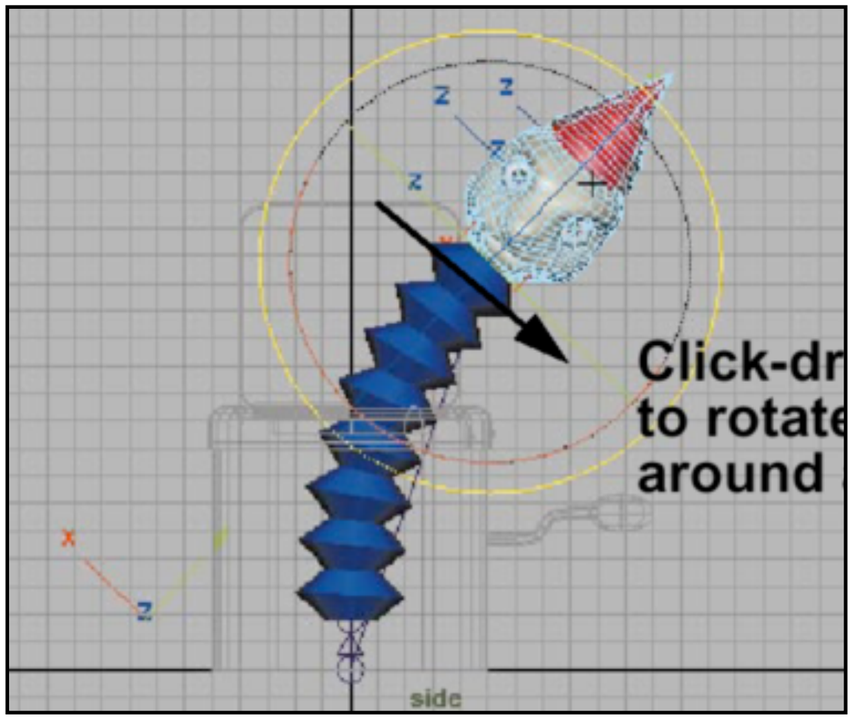
Keyframe Interpolation of Each Parameter

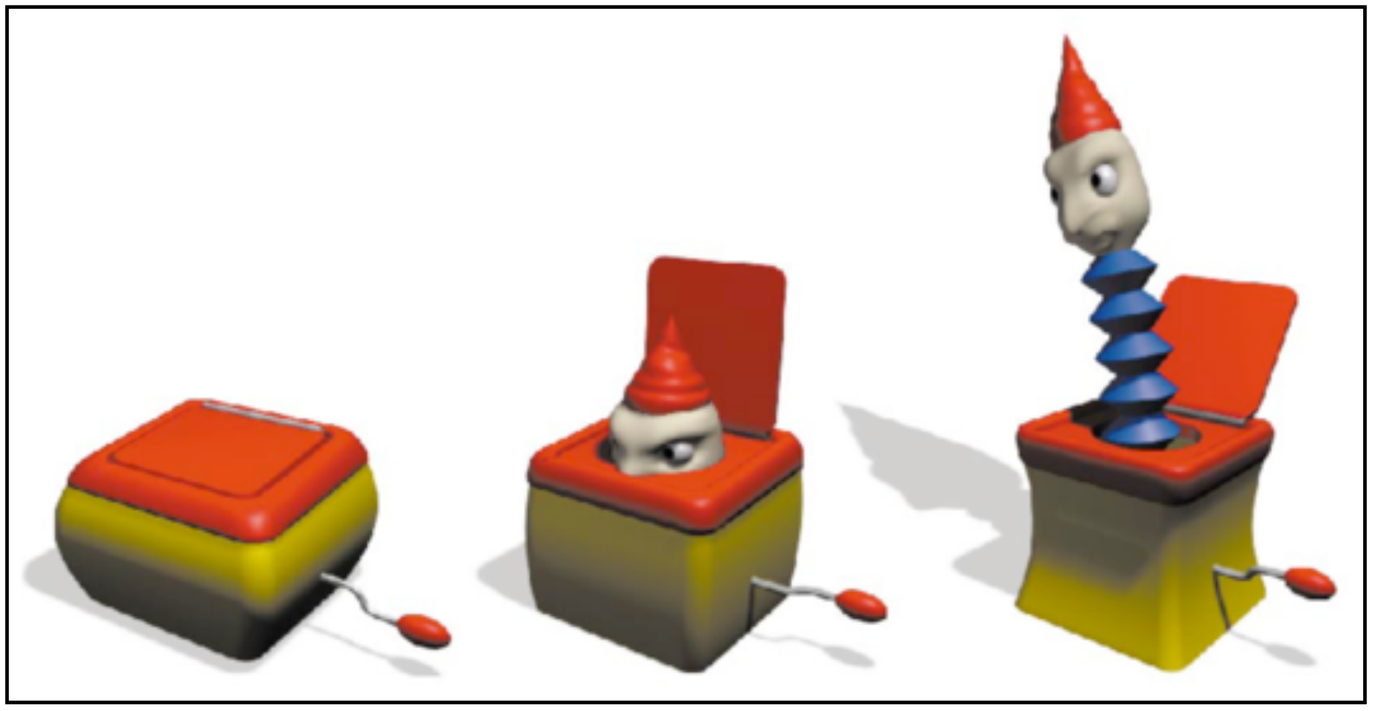
Linear interpolation usually not good enough

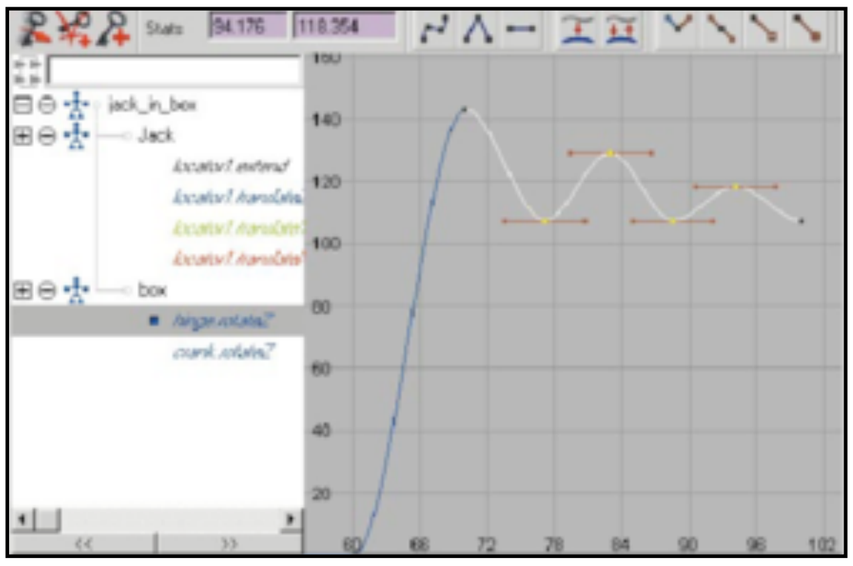
Recall splines for smooth / controllable interpolation



Keyframe Interpolation



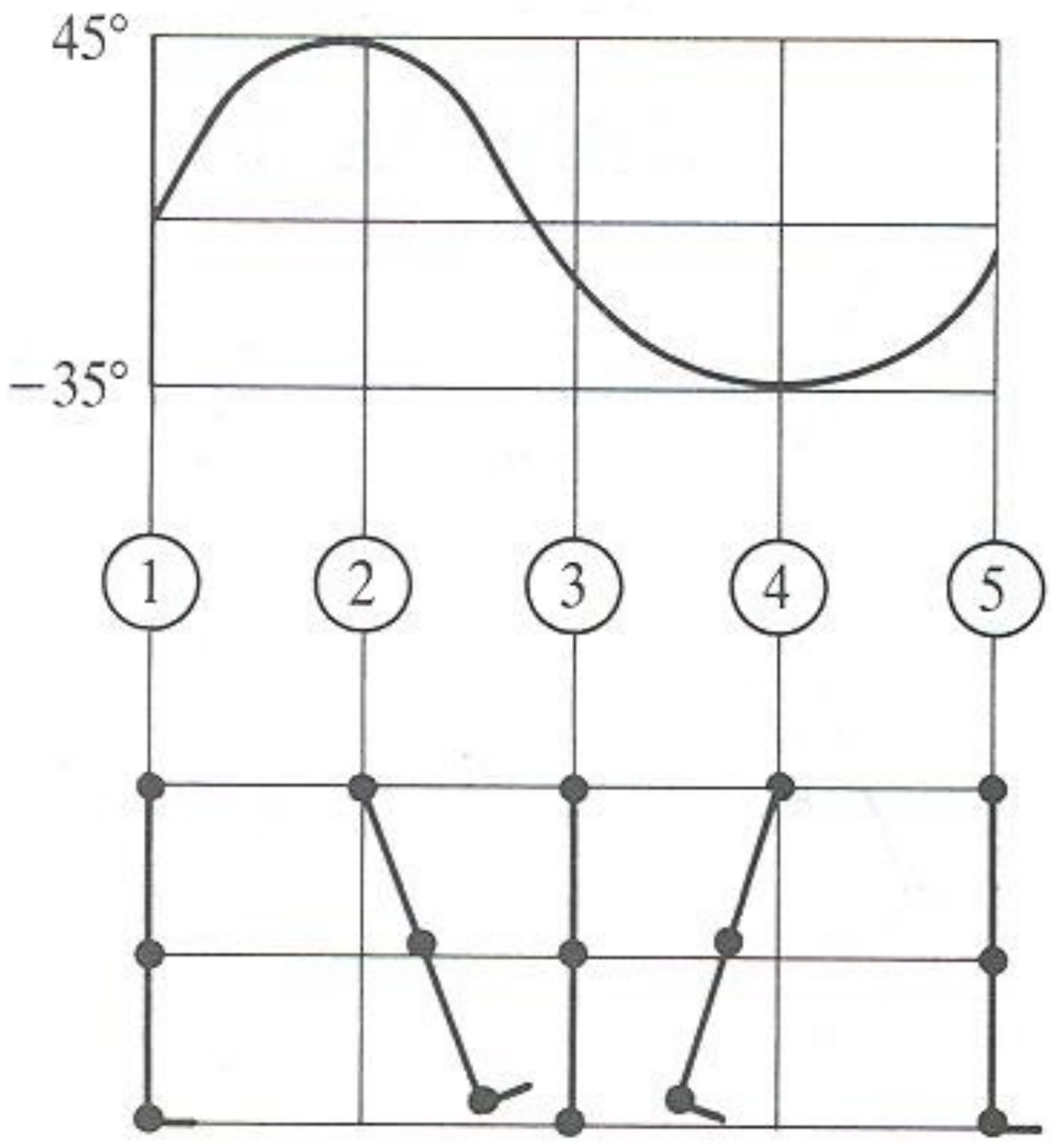




Requires skill and patience!

Example: Walk Cycle

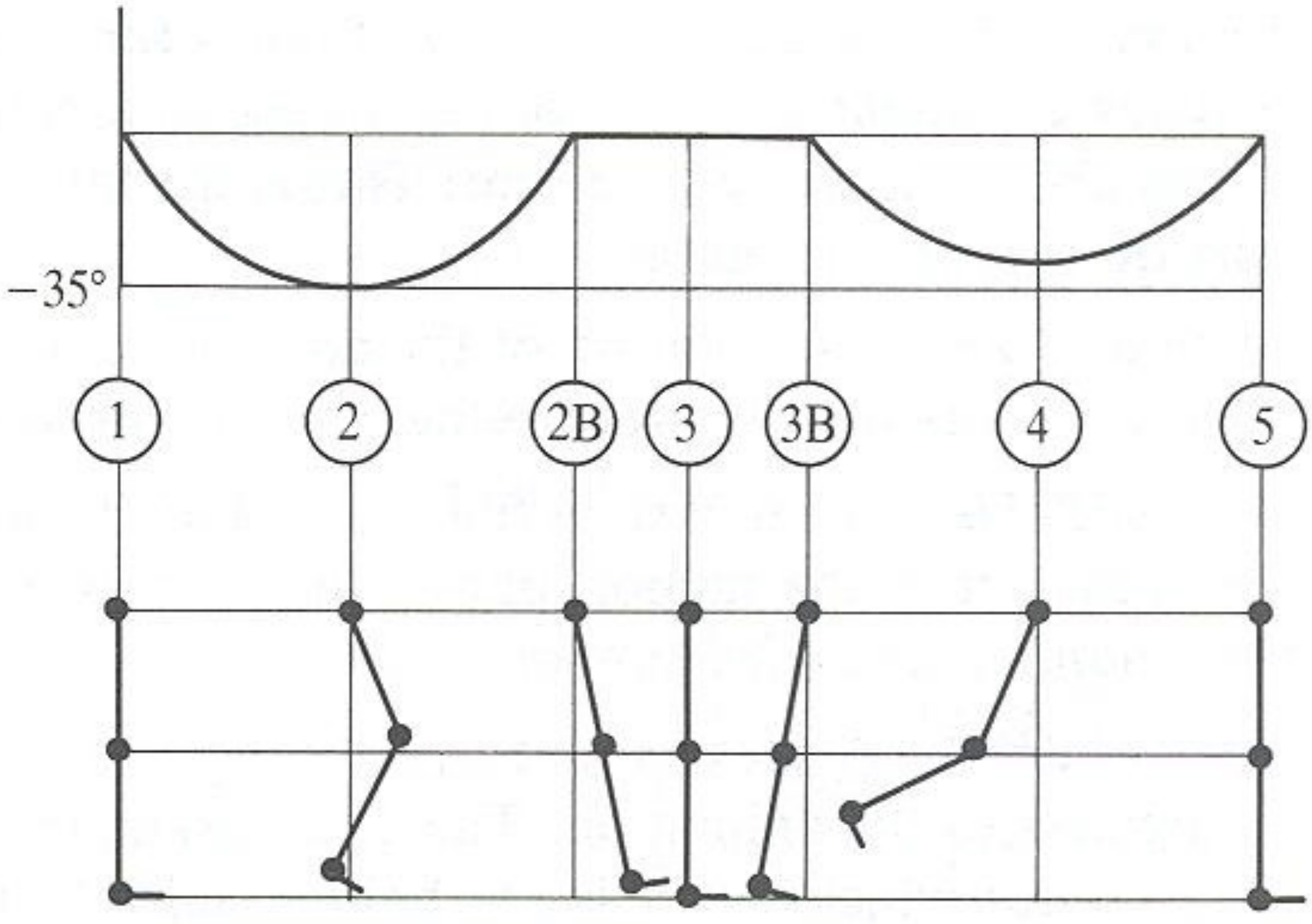
Hip joint angle



Watt & Watt

Example: Walk Cycle

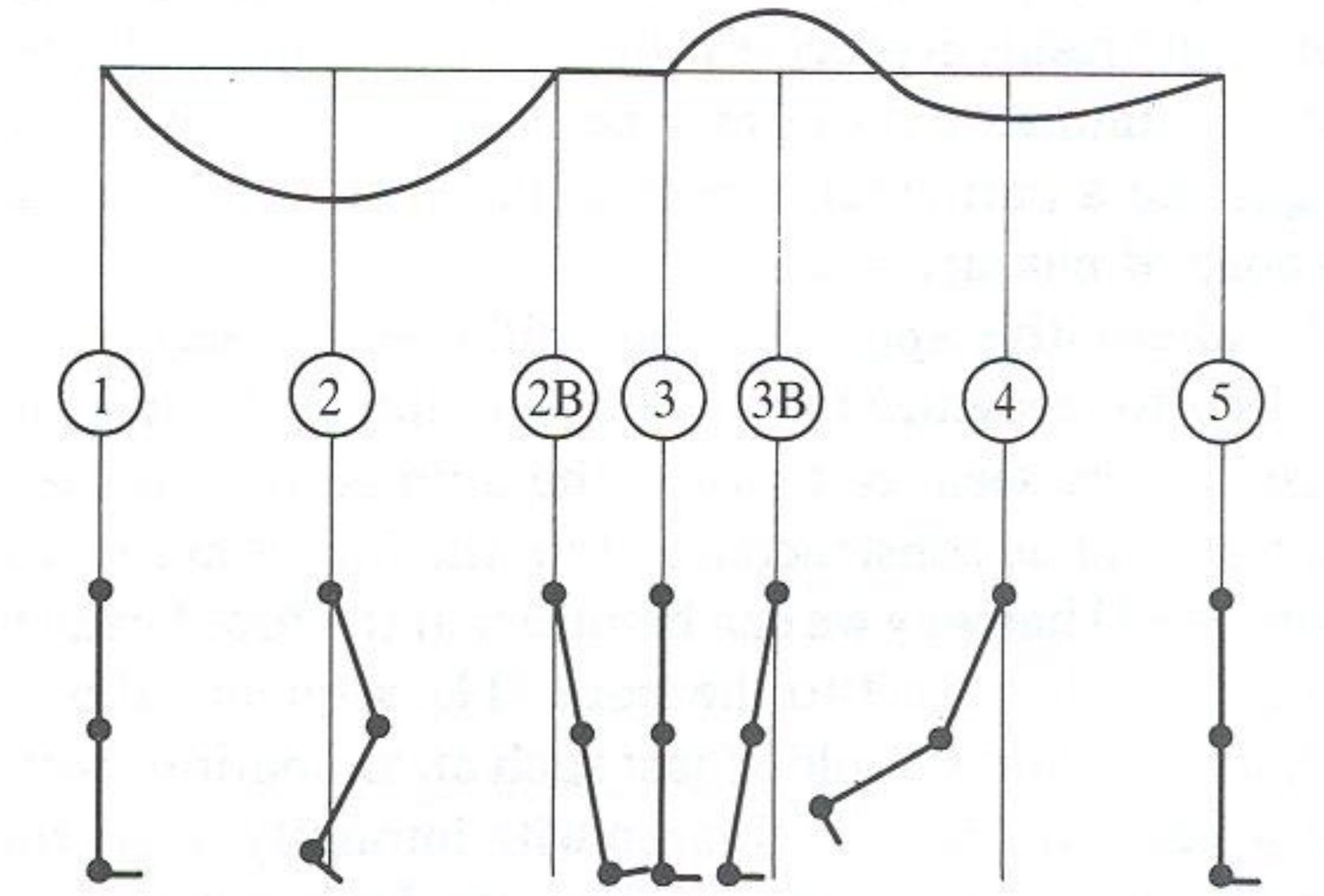
Knee joint angle



Watt & Watt

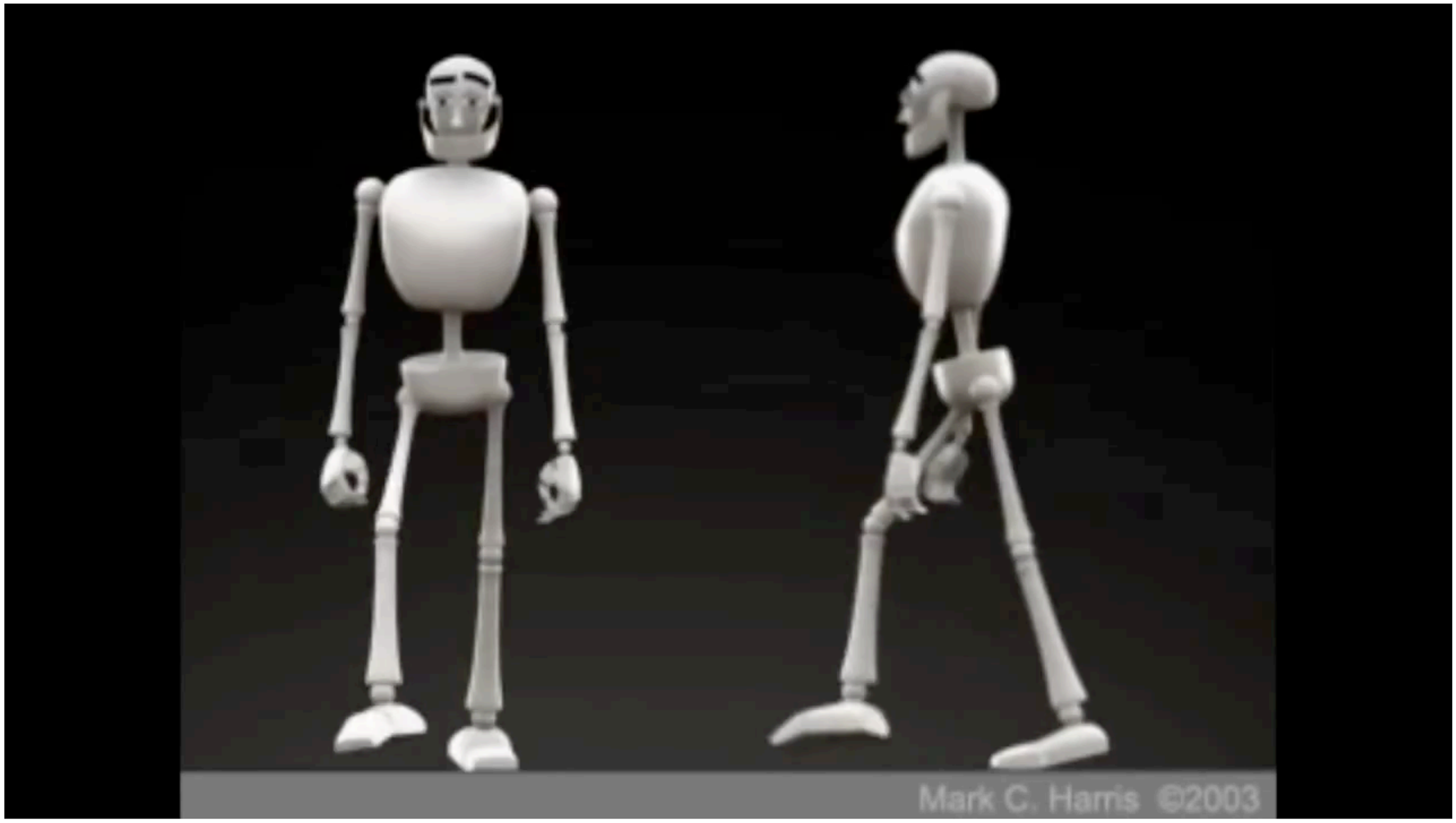
Example: Walk Cycle

Ankle joint angle

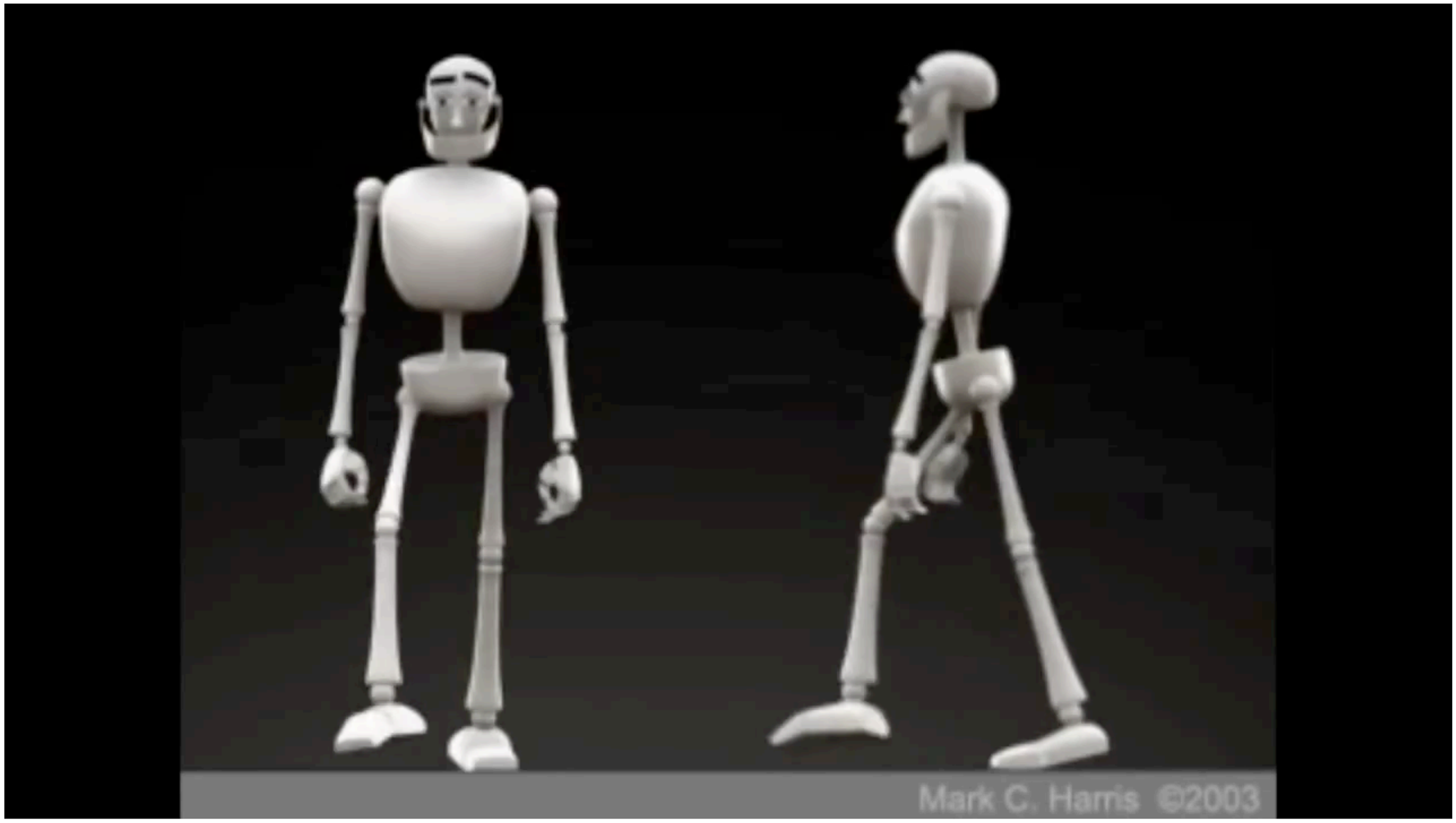


Watt & Watt

Example Walk Cycle

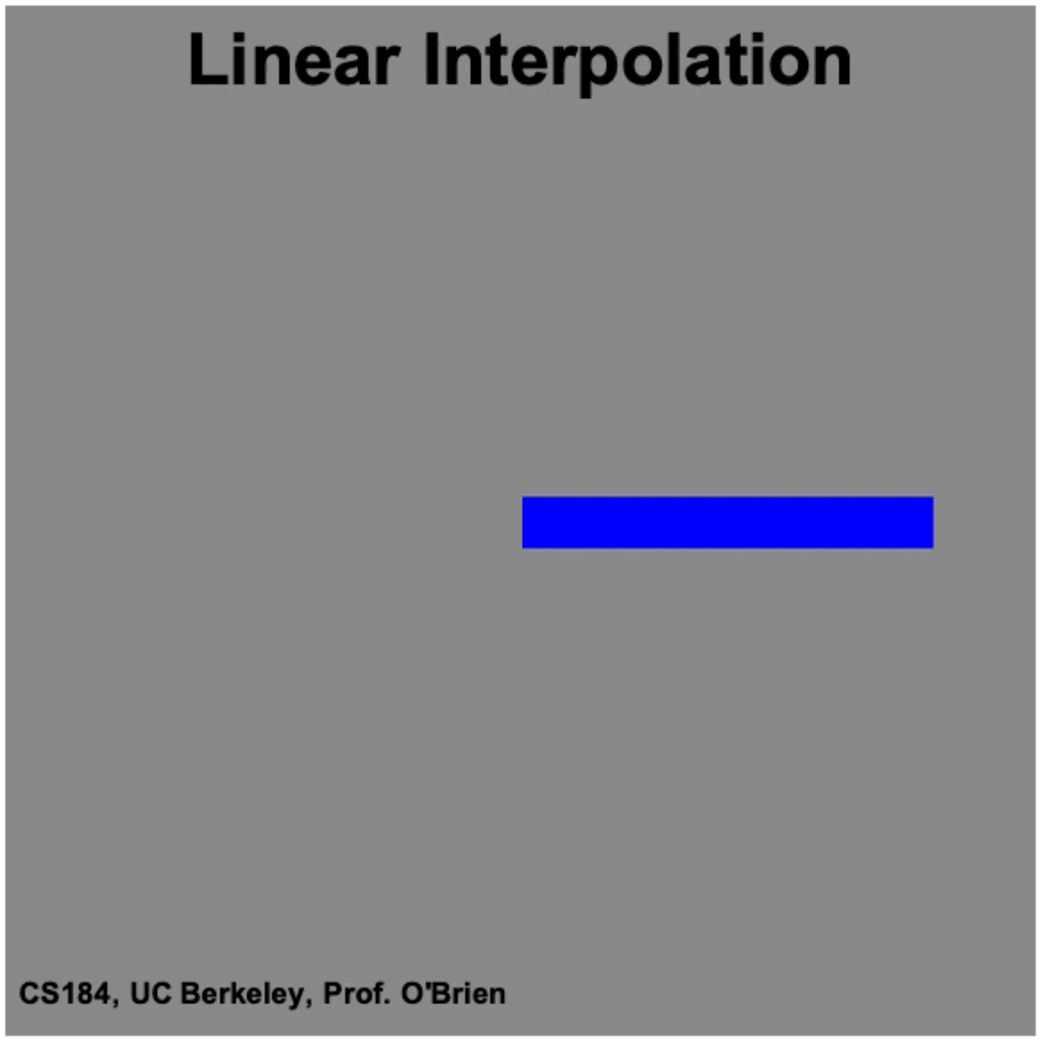


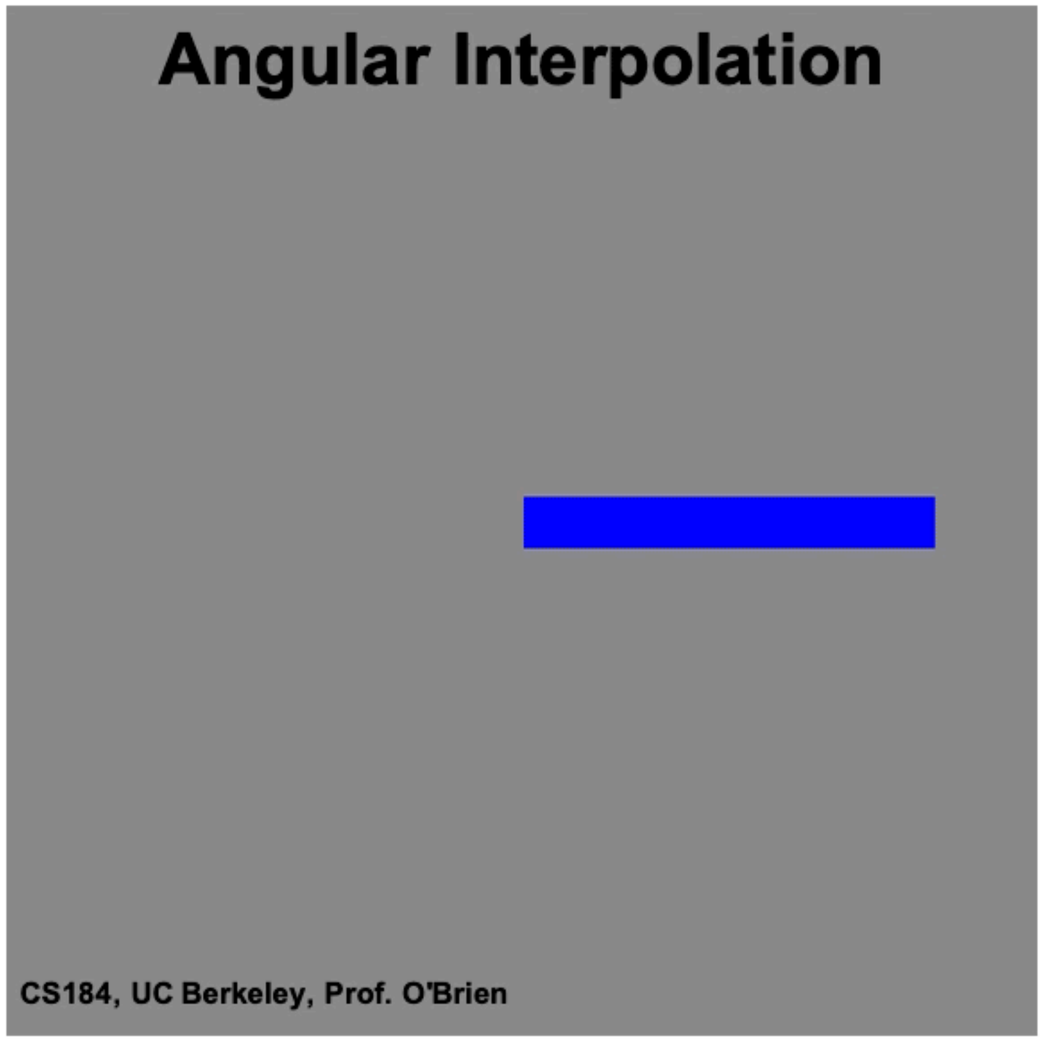
Example Walk Cycle



Interpolation

Interpolation should be semantically correct

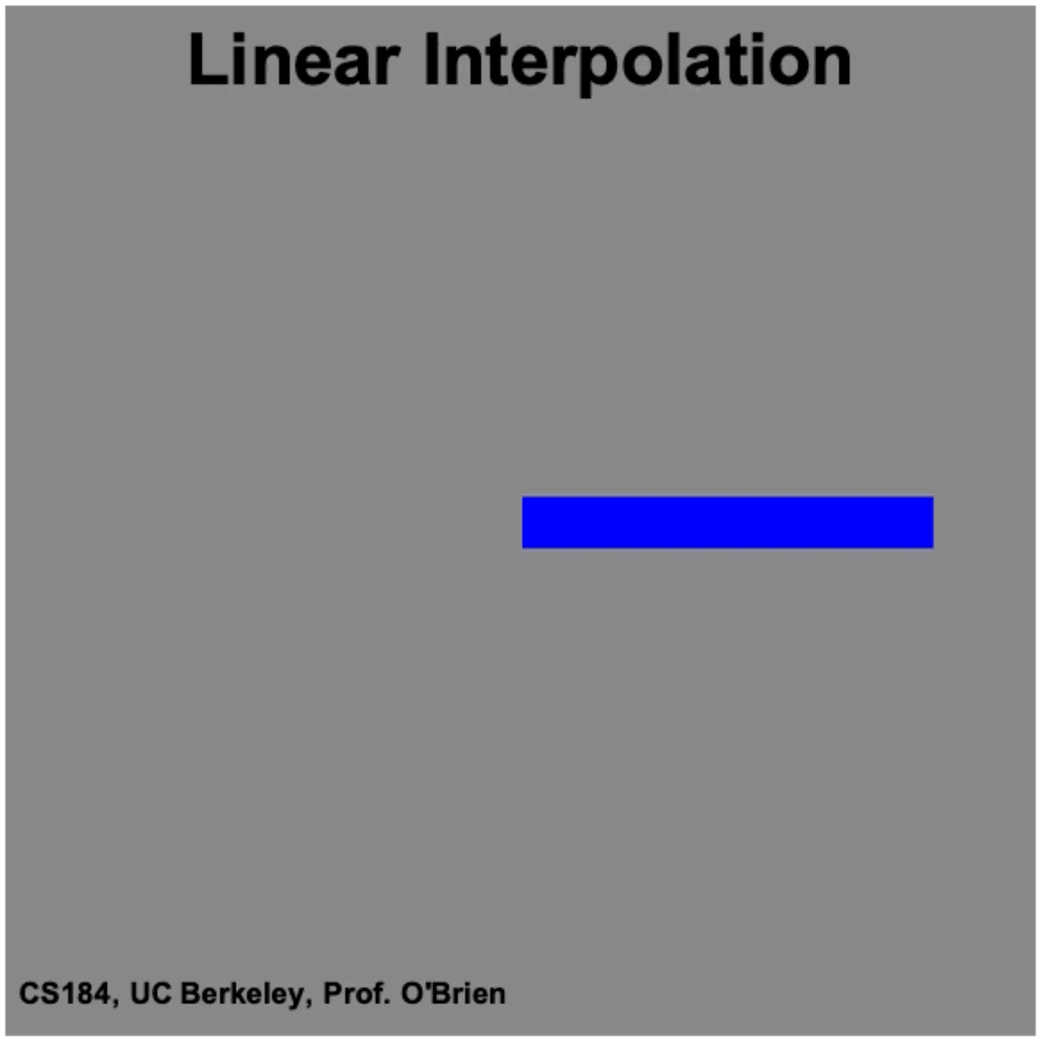


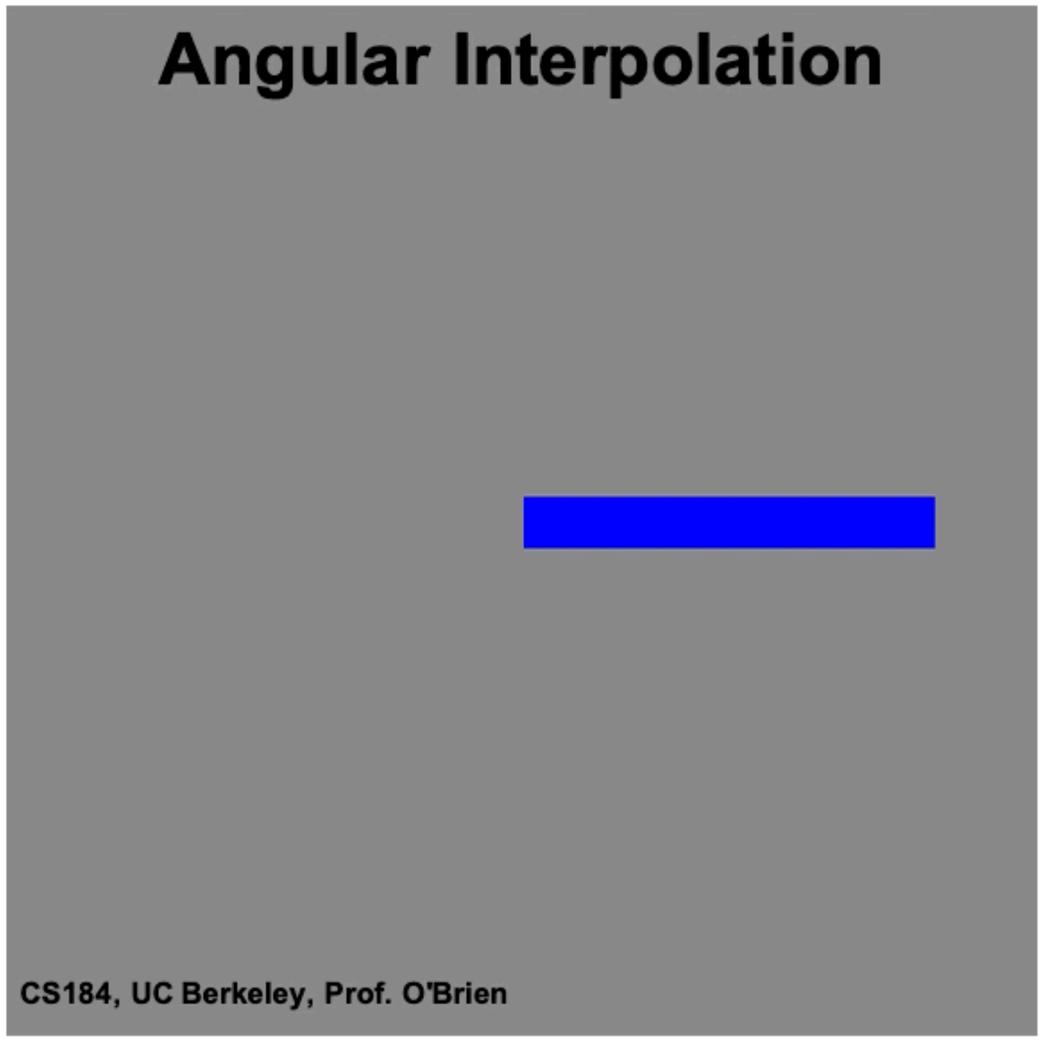


Key frames are every 45 degrees

Interpolation

Interpolation should be semantically correct

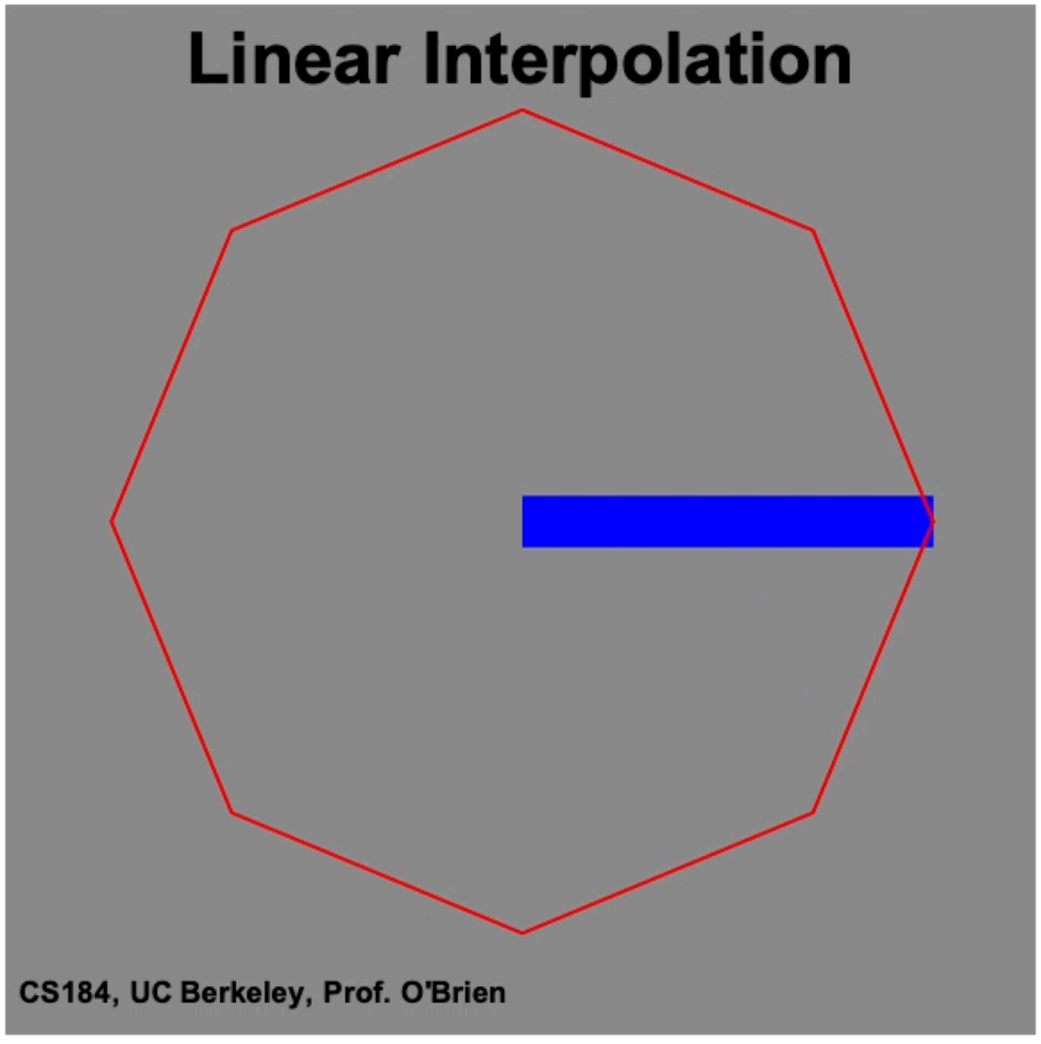


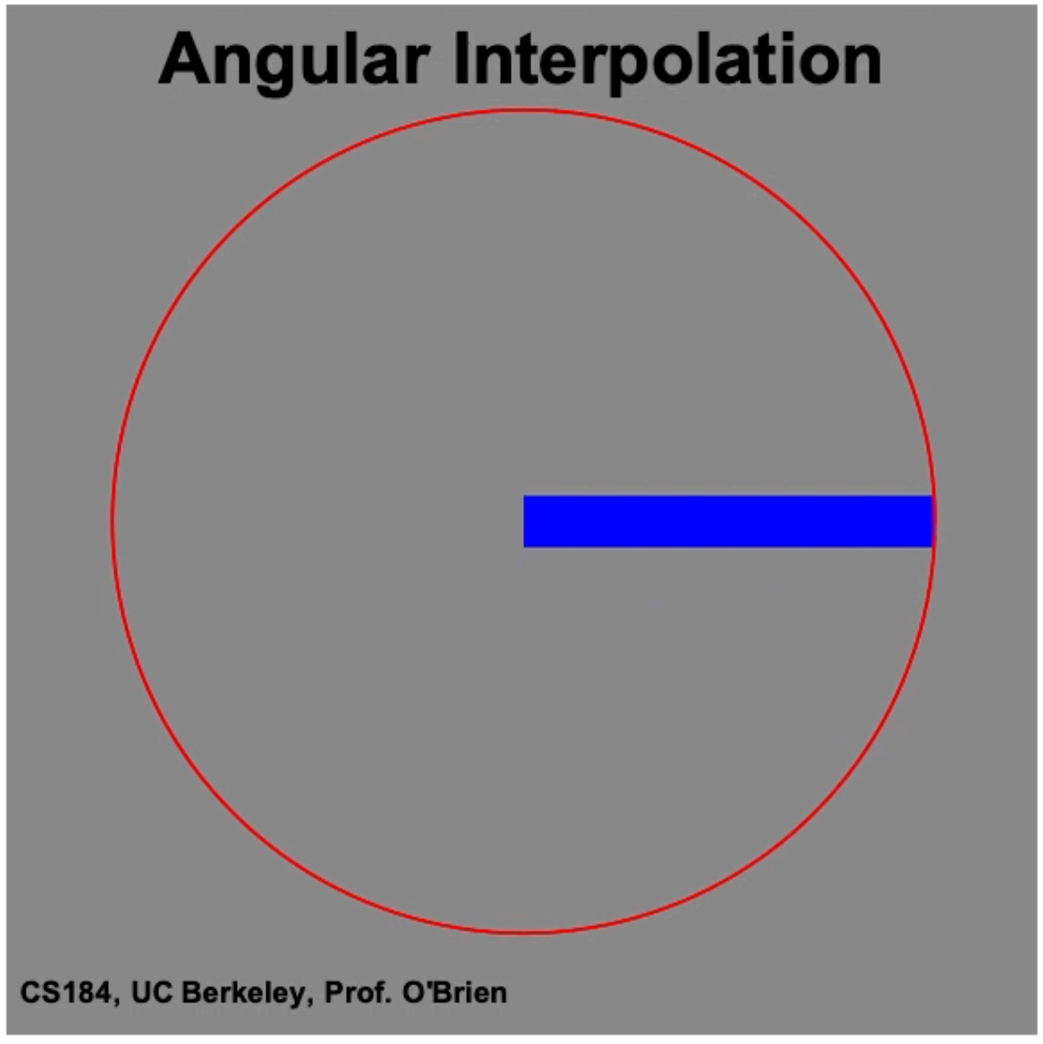


Key frames are every 45 degrees

Interpolation

Interpolation should be semantically correct

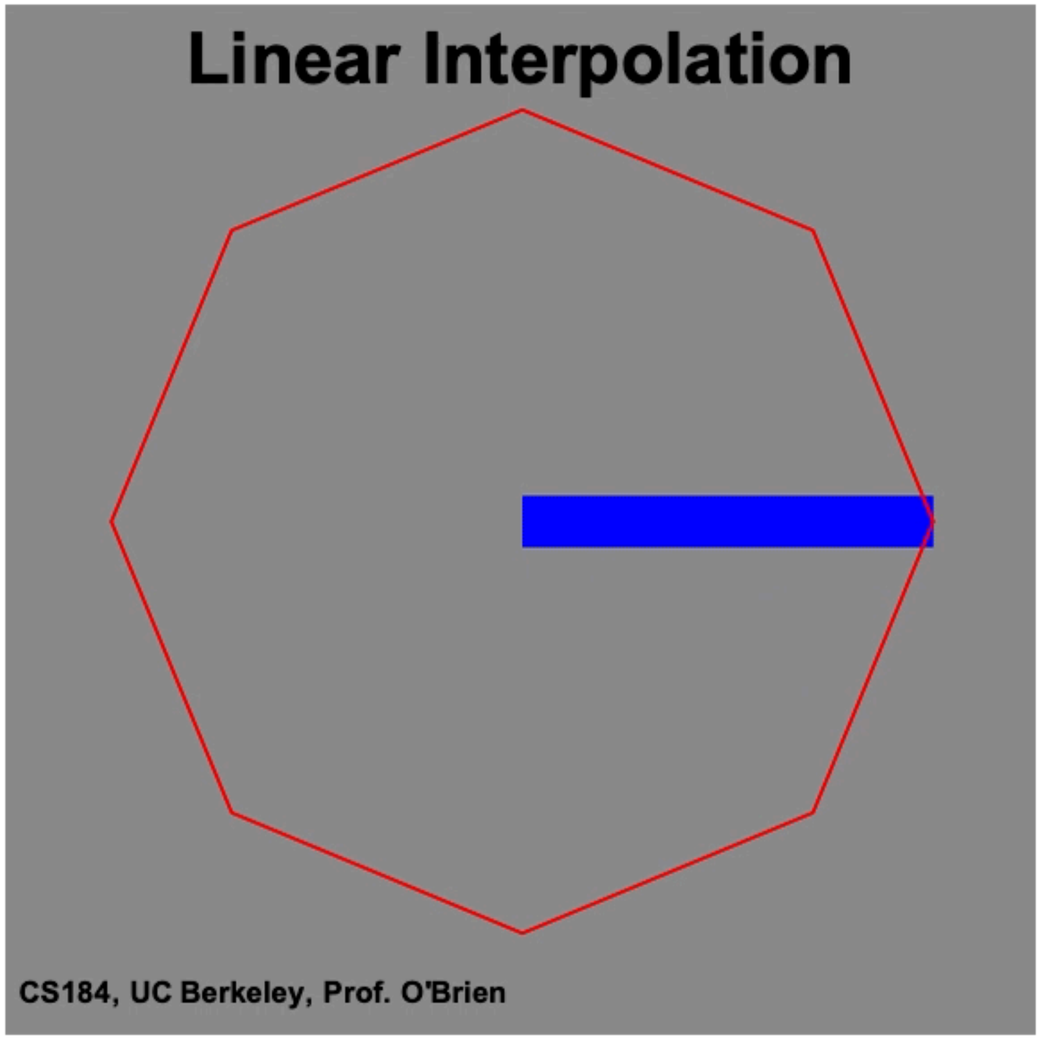


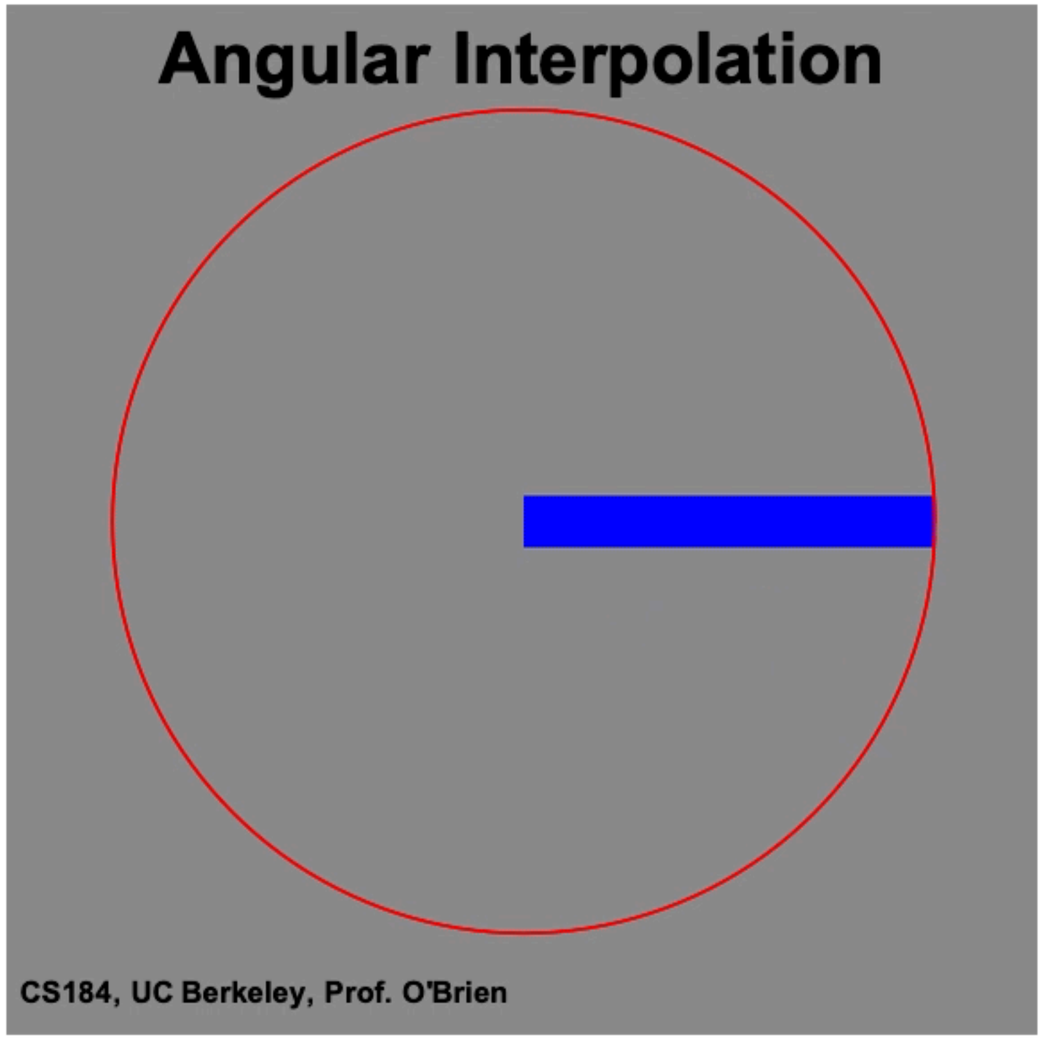


Key frames are every 45 degrees

Interpolation

Interpolation should be semantically correct



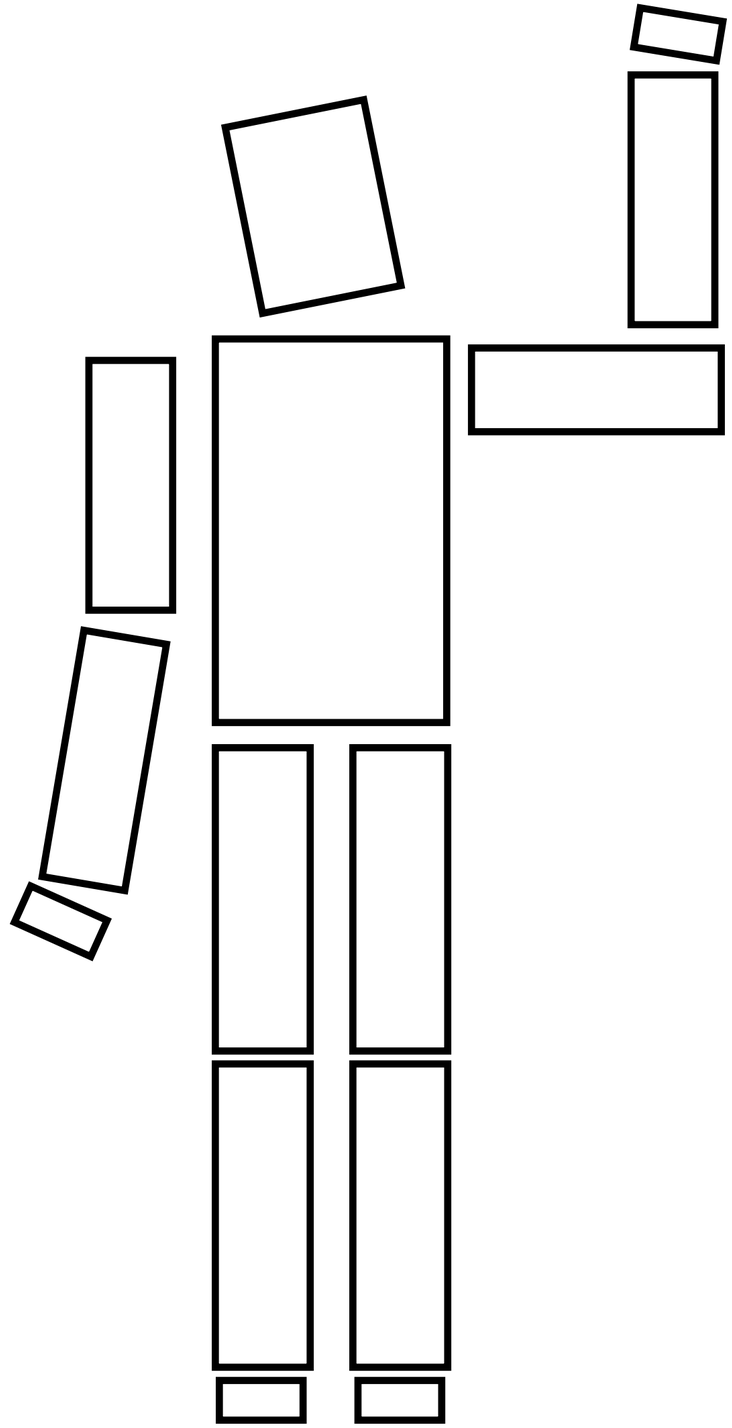


Key frames are every 45 degrees

Forward Kinematics

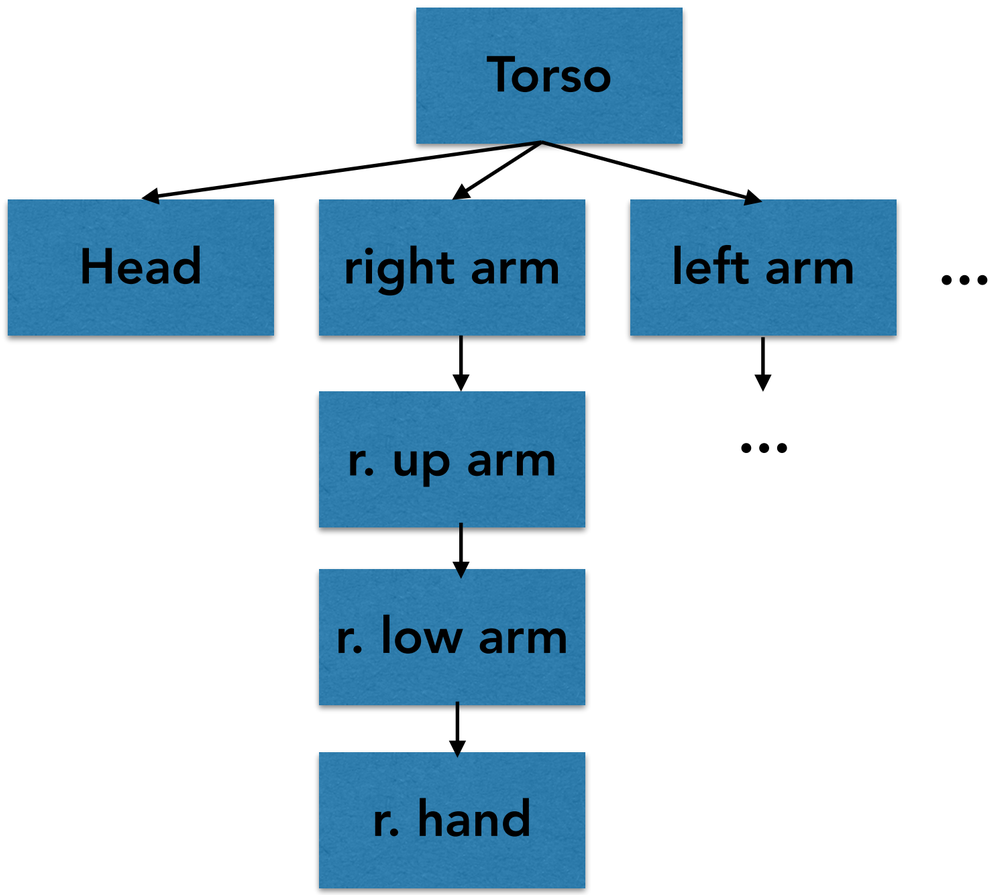
Skeleton - Hierarchical Representation

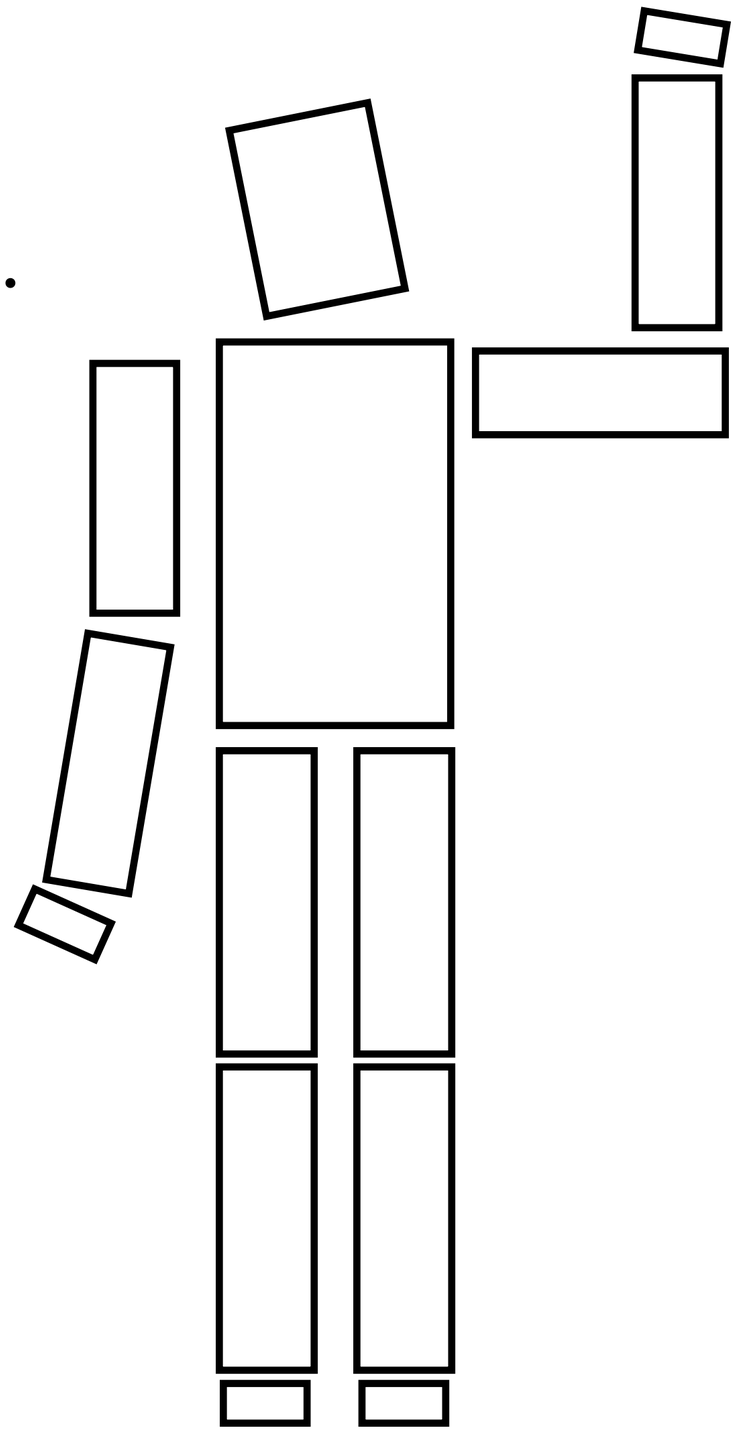
torso
- head
- right arm
  - upper arm
    - lower arm
      - hand
- left arm
  - upper arm
    - lower arm
      - hand
- right leg
  - upper leg
    - lower leg
      - foot
- left leg
  - upper leg
    - lower leg
      - foot



Skeleton - Hierarchical Representation

torso
  head
  right arm
    upper arm
      lower arm
        hand
  left arm
    upper arm
      lower arm
        hand
right leg
  upper leg
    lower leg
      foot
left leg
  upper leg
    lower leg
      foot





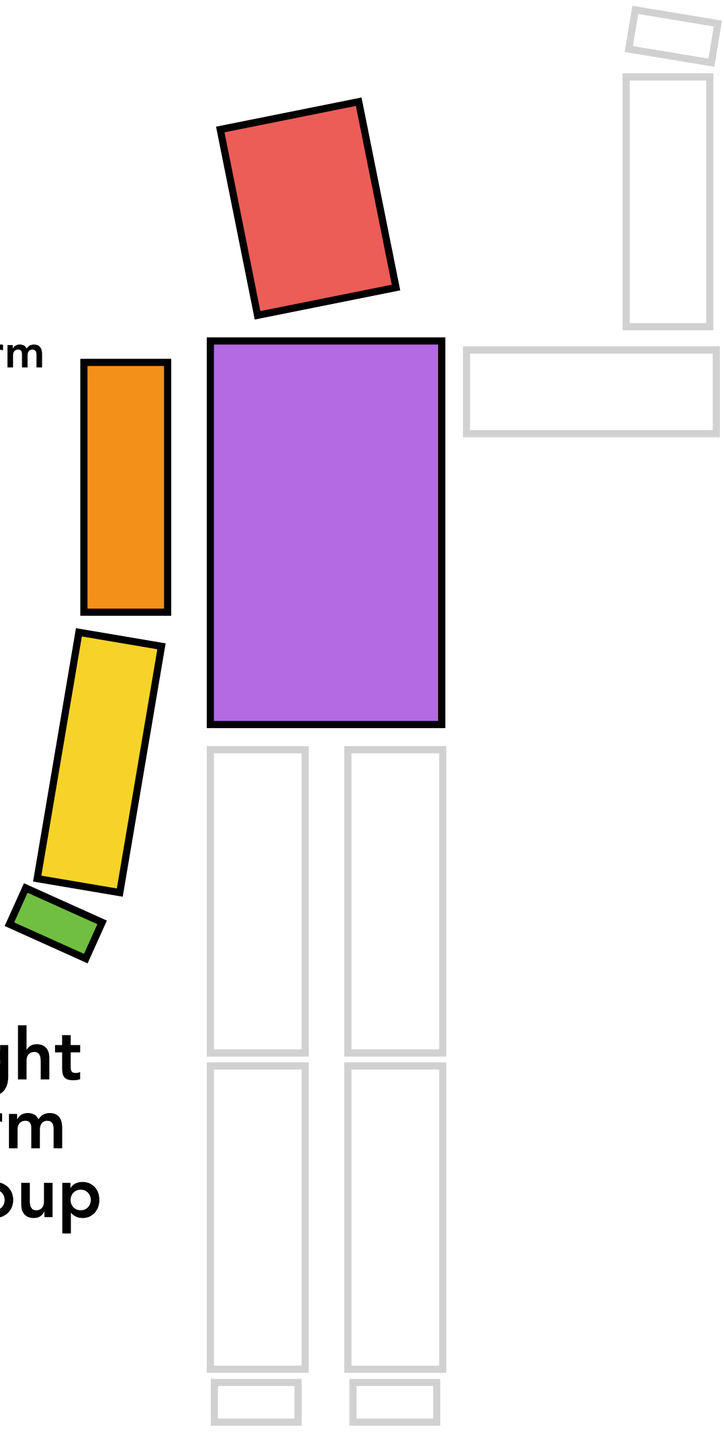
Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
            pushmatrix();
                translate(0, -3);
                rotate(elbowRotation);
                drawLowerArm();
                pushmatrix();
                    translate(0, -3);
                    rotate(wristRotation);
                    drawHand();
                popmatrix();
            popmatrix();
        popmatrix();
    ....
```

right hand

right lower arm group

right arm group



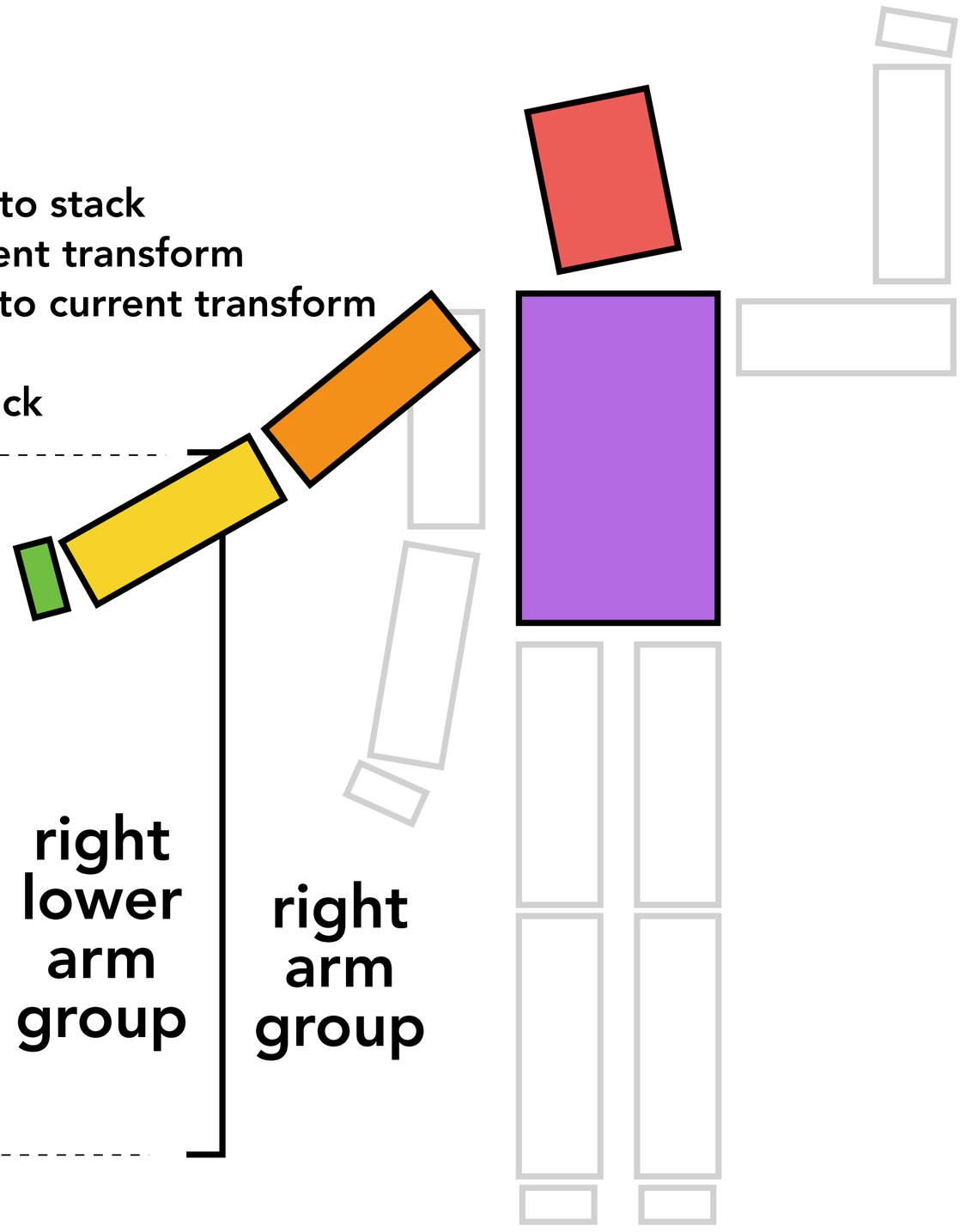
Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
        → rotate(rightShoulderRotation);
            drawUpperArm();
        pushmatrix();
            translate(0, -3);
            rotate(elbowRotation);
            drawLowerArm();
            pushmatrix();
                translate(0, -3);
                rotate(wristRotation);
                drawHand();
            popmatrix();
        popmatrix();
    popmatrix();
    ....
```

right hand

right lower arm group

right arm group



Motion Capture

Motion Capture

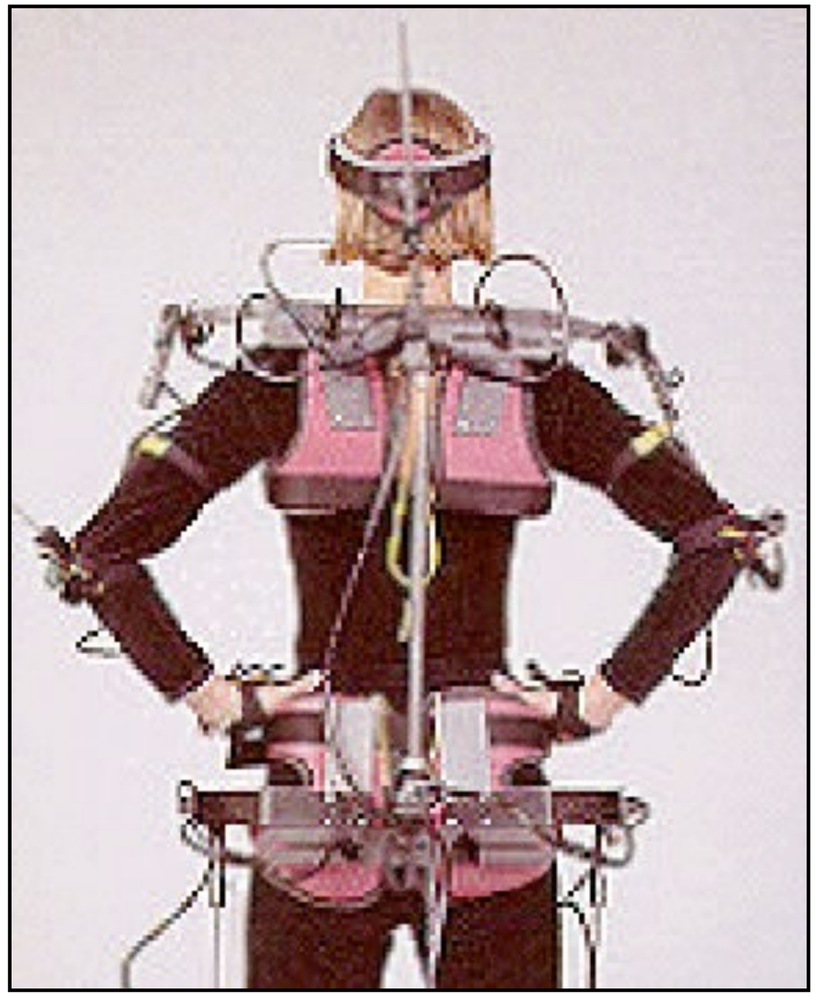
Data-driven approach to creating animation sequences

- Record real-world performances
- Extract pose as a function of time from raw data

Motion Capture Equipment

Optical
(More on following slides)

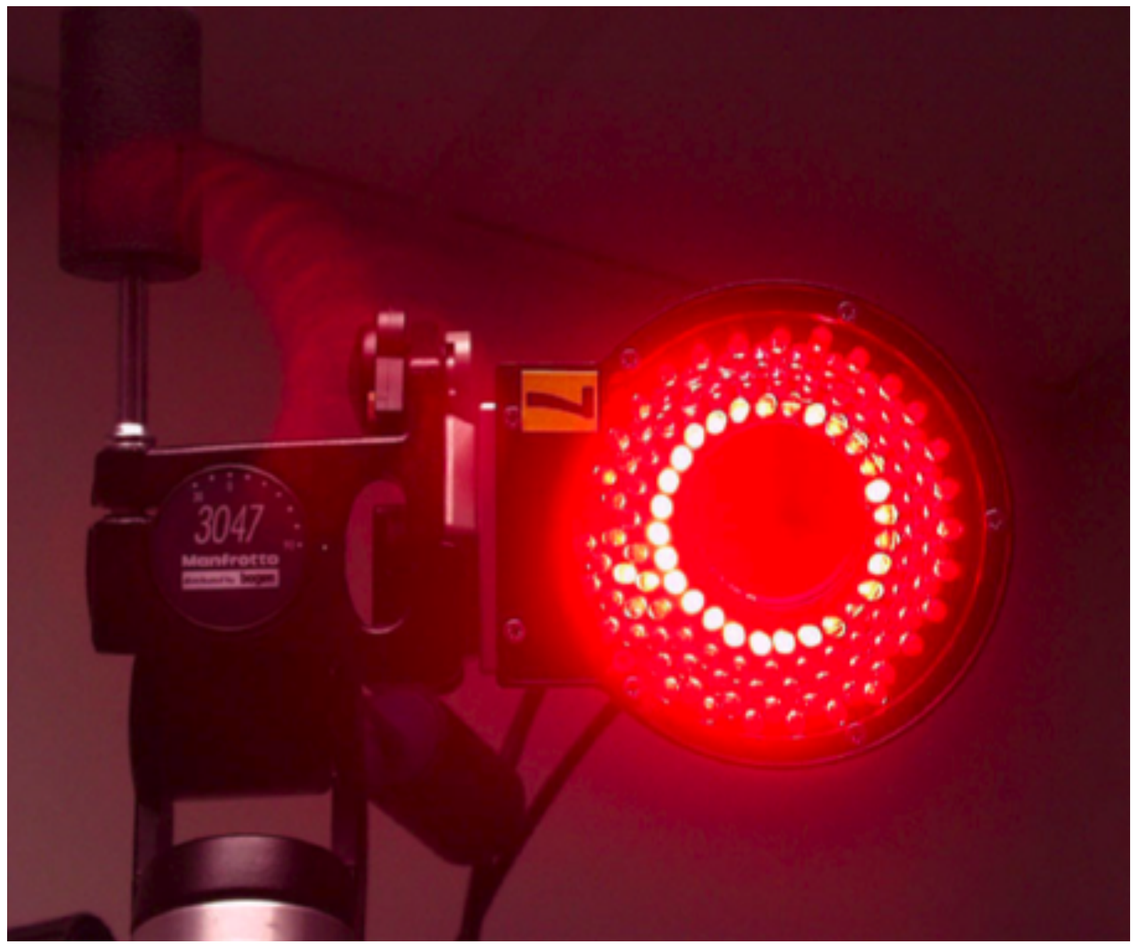
Magnetic
Sense magnetic fields to infer position / orientation.
Tethered.



Mechanical
Measure joint angles directly.
Restricts motion.

Optical Motion Capture

Retroflective markers attached to subject

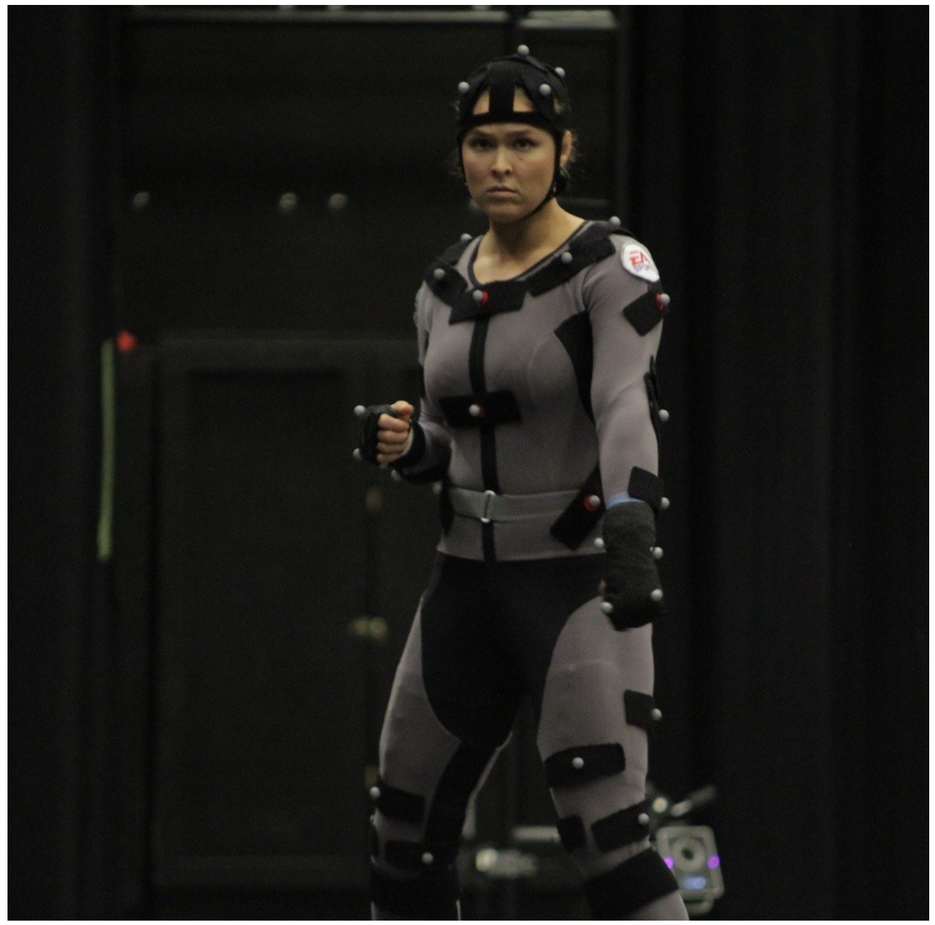


IR illumination and cameras

- Markers on subject
- Positions by triangulation from multiple cameras
- 8+ cameras, 240 Hz, occlusions are difficult

Slide credit: Steve Marschner

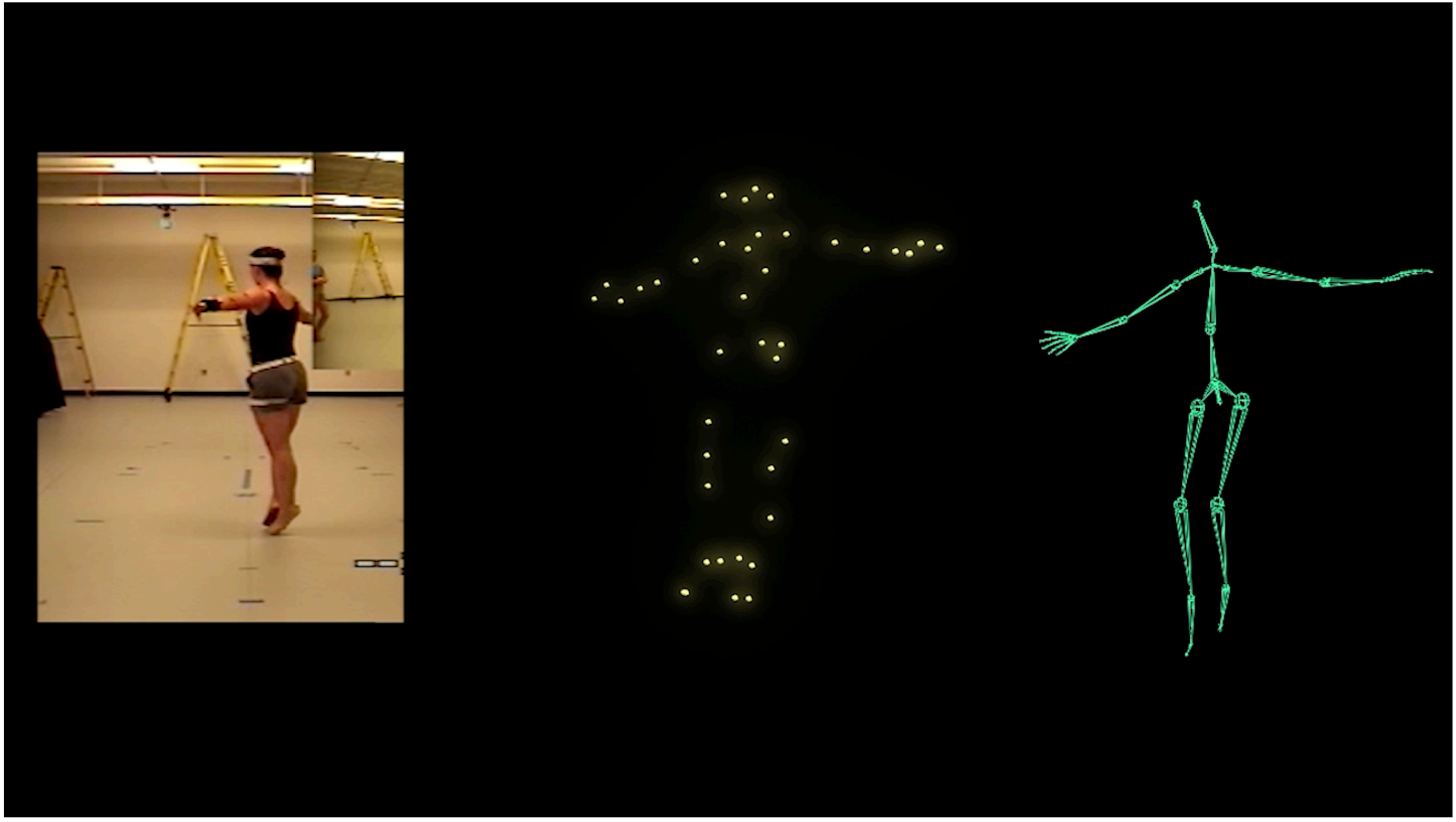
Optical Motion Capture



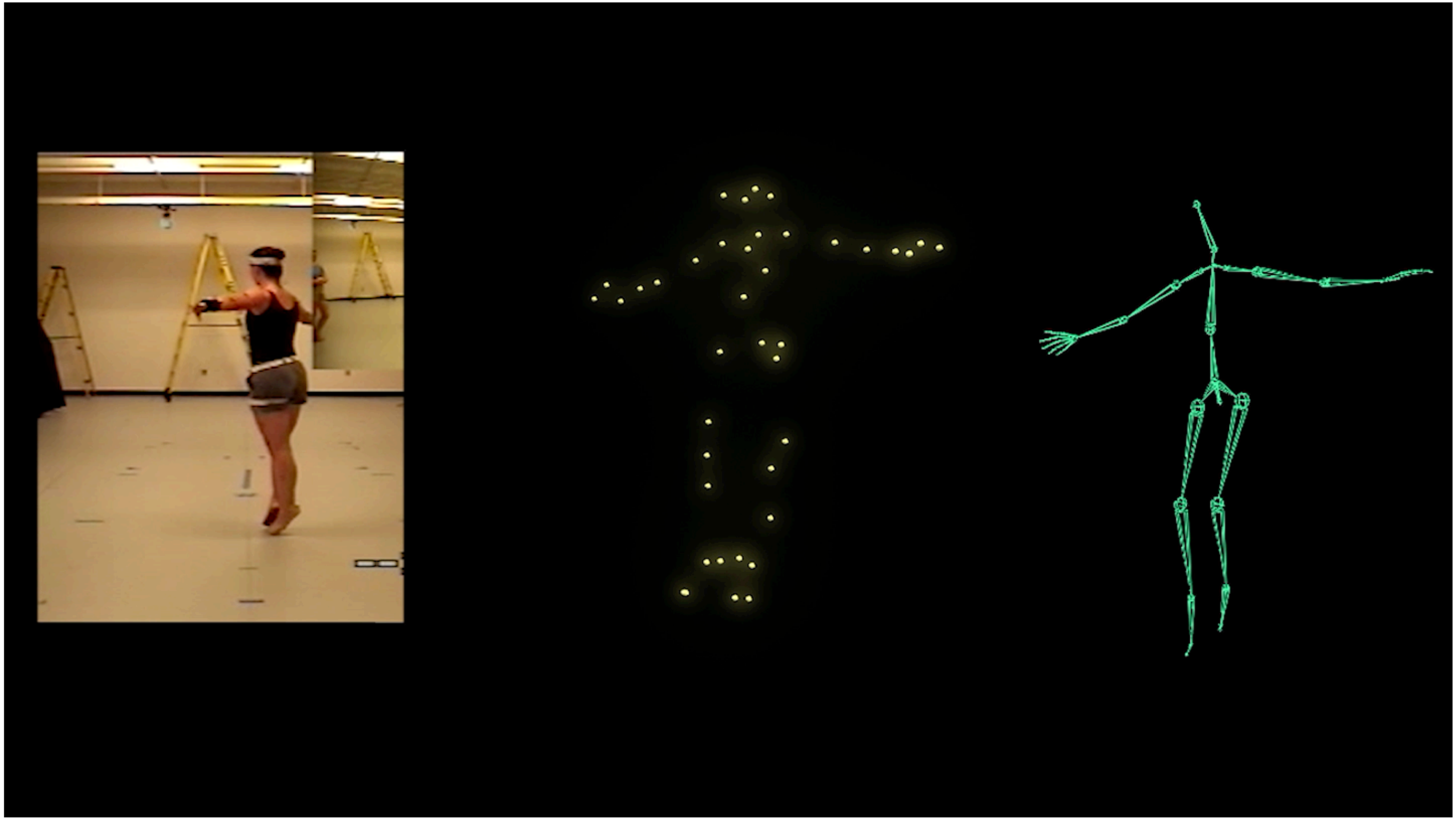
Source: http://fightland.vice.com/blog/ronda-rousey-20-the-queen-of-all-media

Ronda Roussey in Electronic Arts' motion capture studio

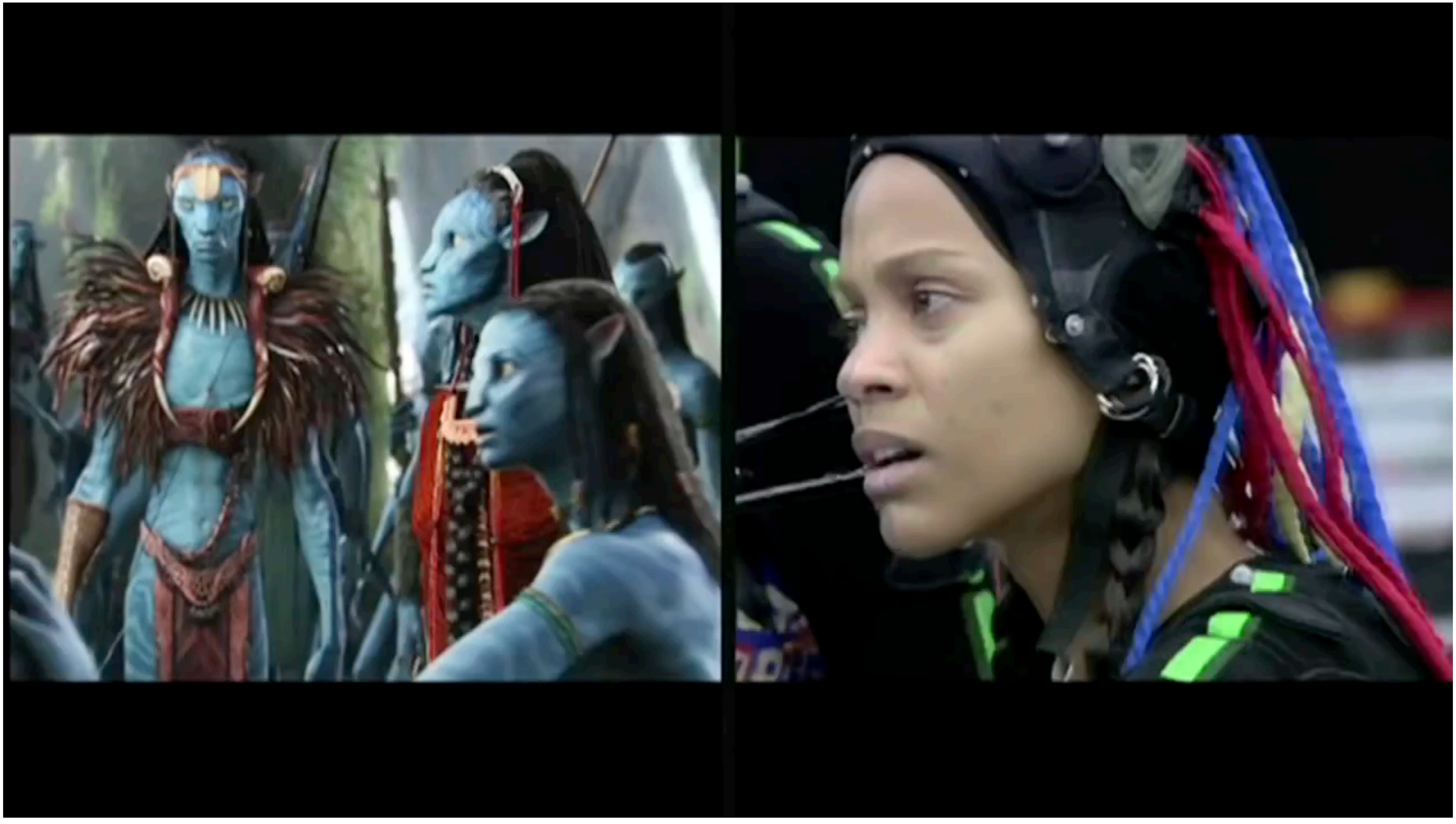
Motion Capture



Motion Capture

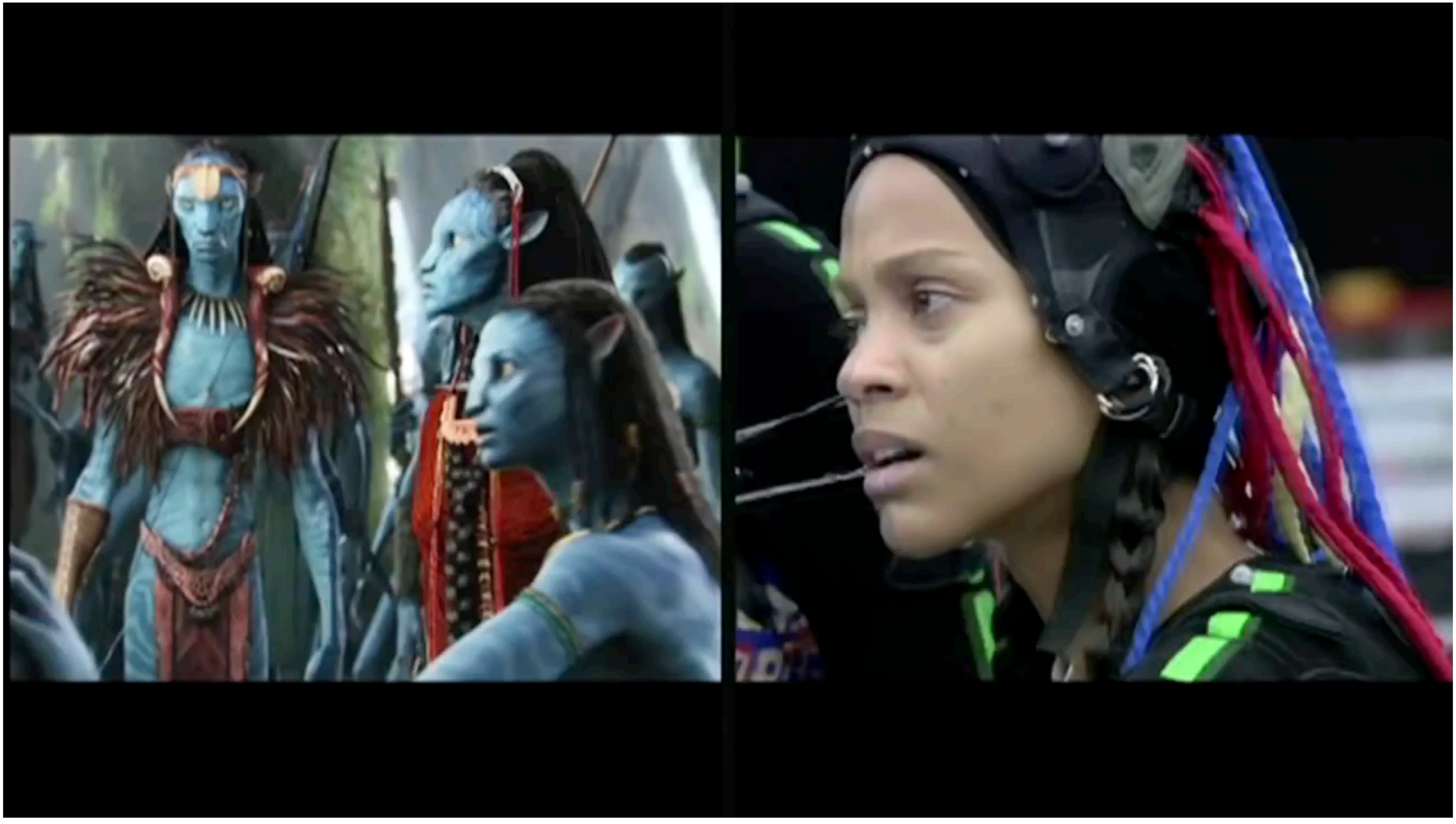


Facial Motion Capture



Discovery, “Avatar: Motion Capture Mirrors Emotions”, https://youtu.be/1wK1Ixr-UmM

Facial Motion Capture



Discovery, “Avatar: Motion Capture Mirrors Emotions”, https://youtu.be/1wK1Ixr-UmM

Challenges of Facial Animation

And animation in general.

Uncanny valley

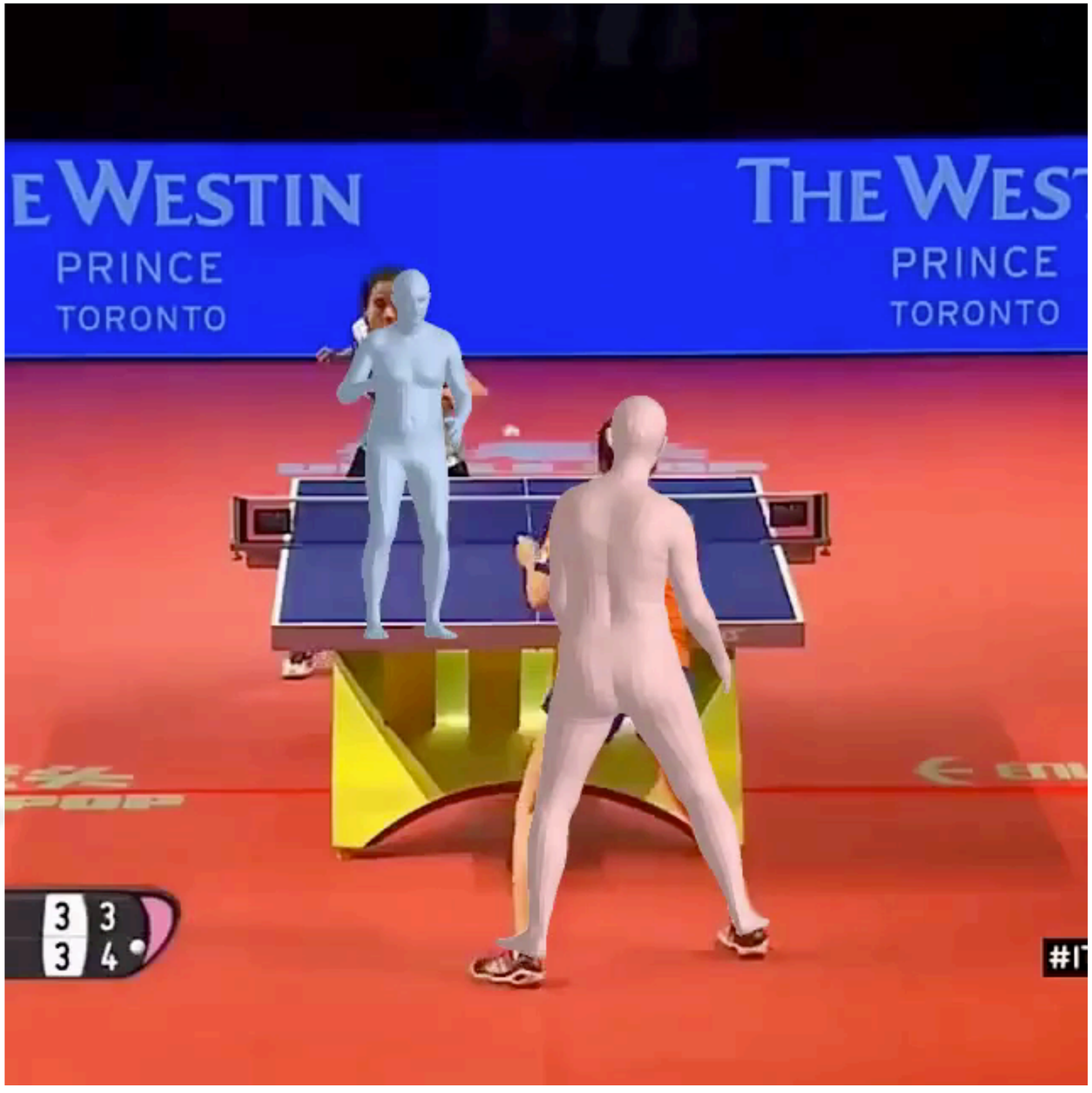
- In robotics and graphics
- As artificial character appearance approaches human realism, our emotional response goes negative, until it achieves a sufficiently convincing level of realism in expression

Cartoon. Brave, Pixar

Semi-realistic. Polar Express, Warner Bros.

Markerless Motion Capture

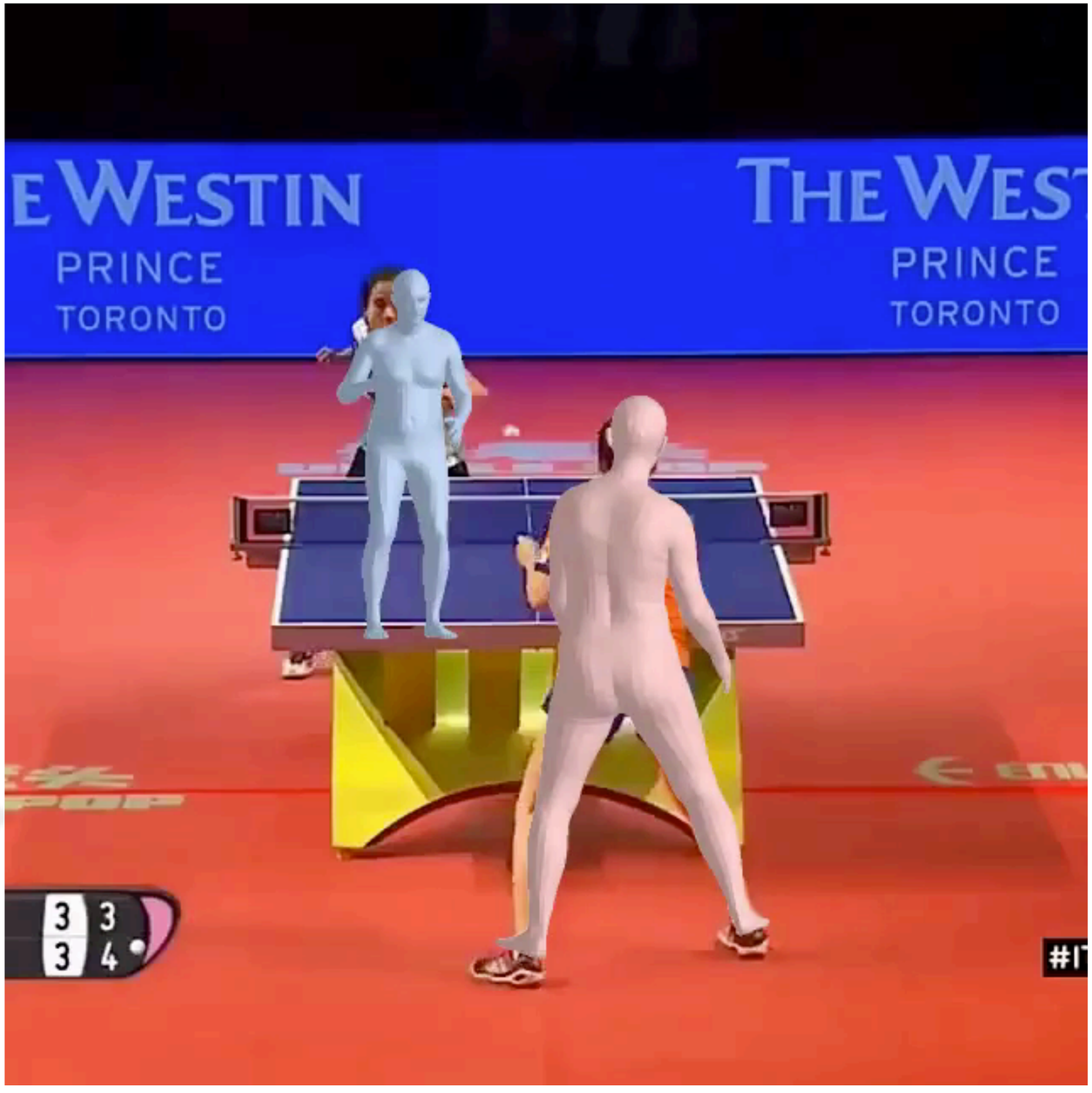
Kanazawa et al. 2018



Kanazawa et al. 2019

Markerless Motion Capture

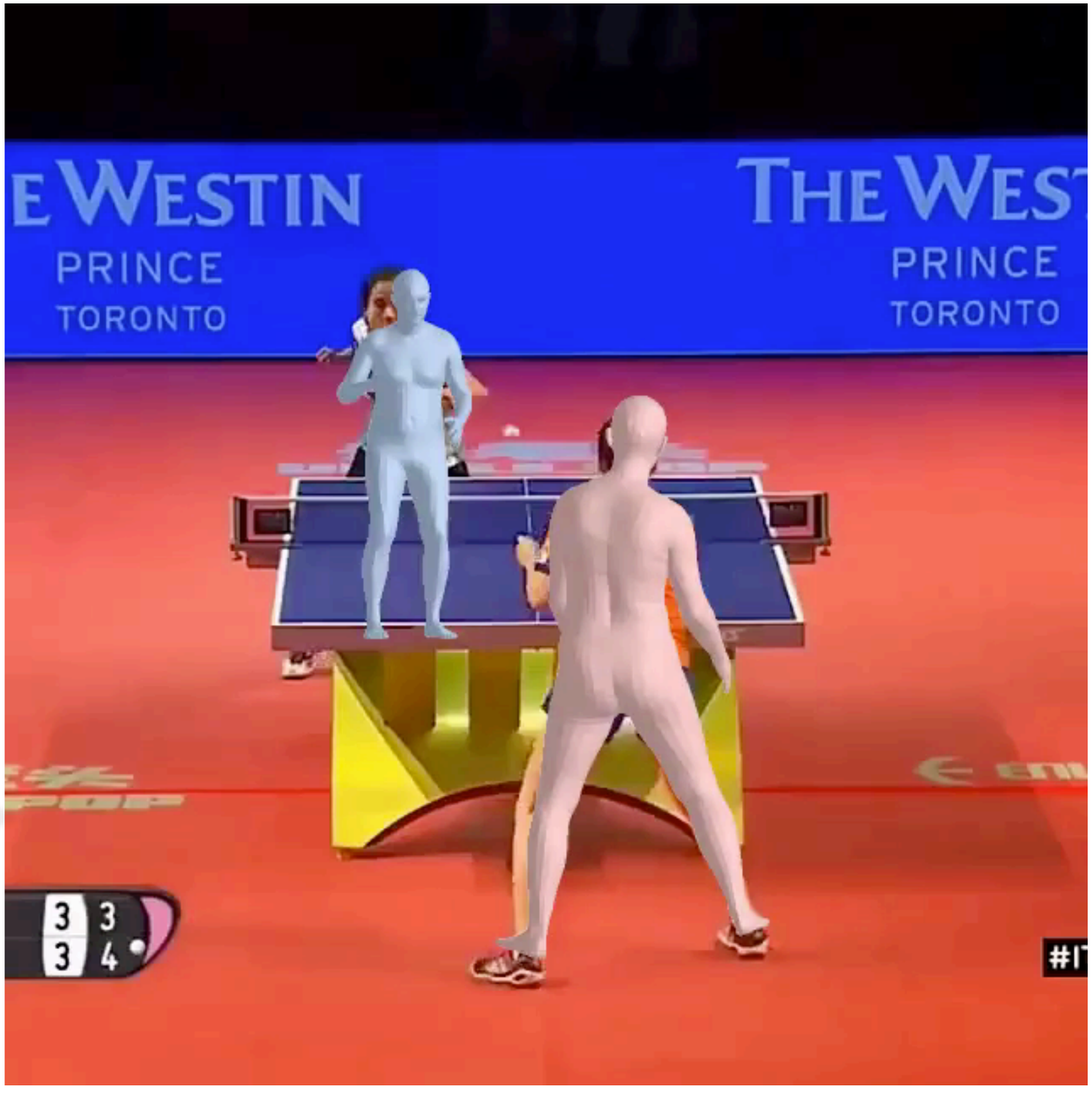
Kanazawa et al. 2018



Kanazawa et al. 2019

Markerless Motion Capture

Kanazawa et al. 2018



Kanazawa et al. 2019

Motion Capture Pros and Cons

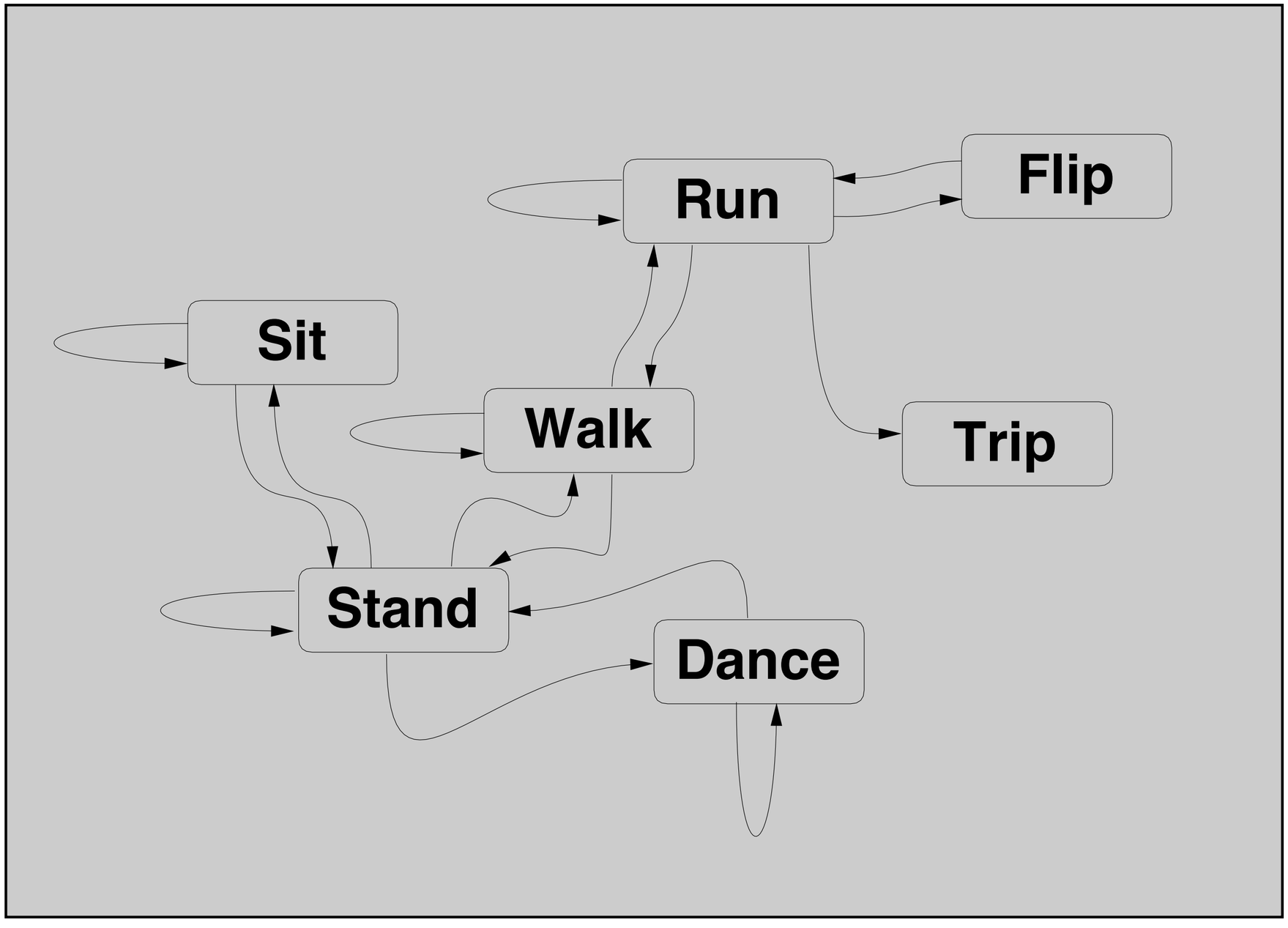
Strengths

- Can capture large amounts of real data quickly
- Realism can be high

Weaknesses

- Complex and costly set-ups
- Captured animation may not meed artistic needs, requiring alterations

Transition Graphs

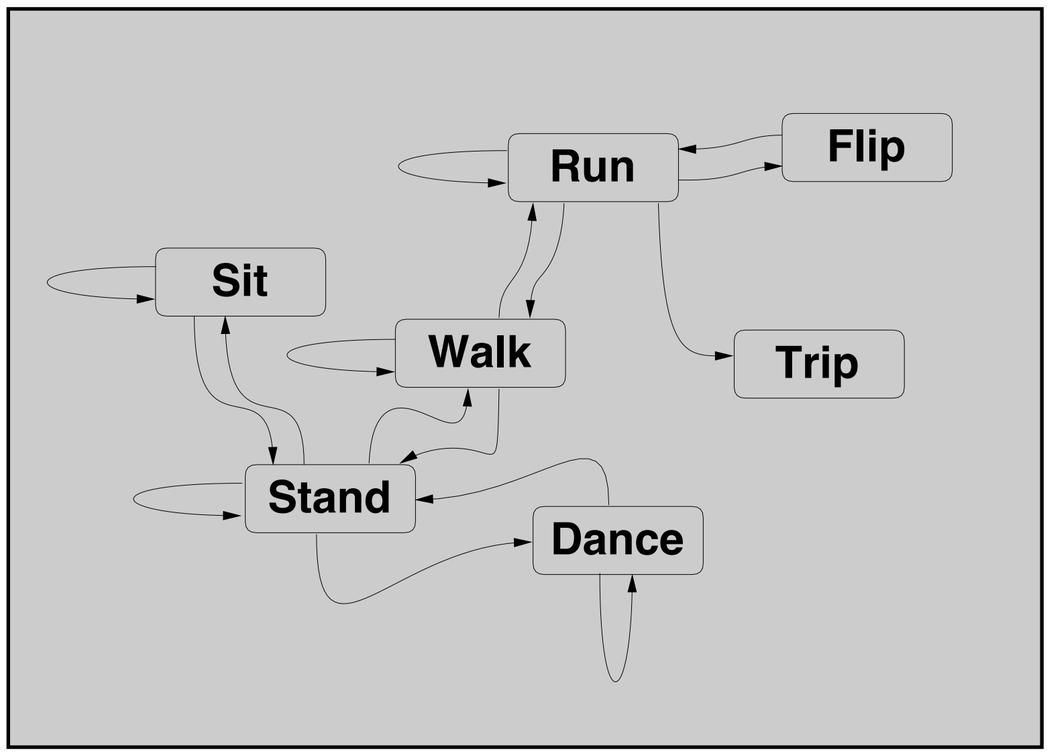


Motion Graphs

Hand build motion graphs often used in games

- Significant amount of work required
- Limited transitions by design

Motion graphs can also be built automatically



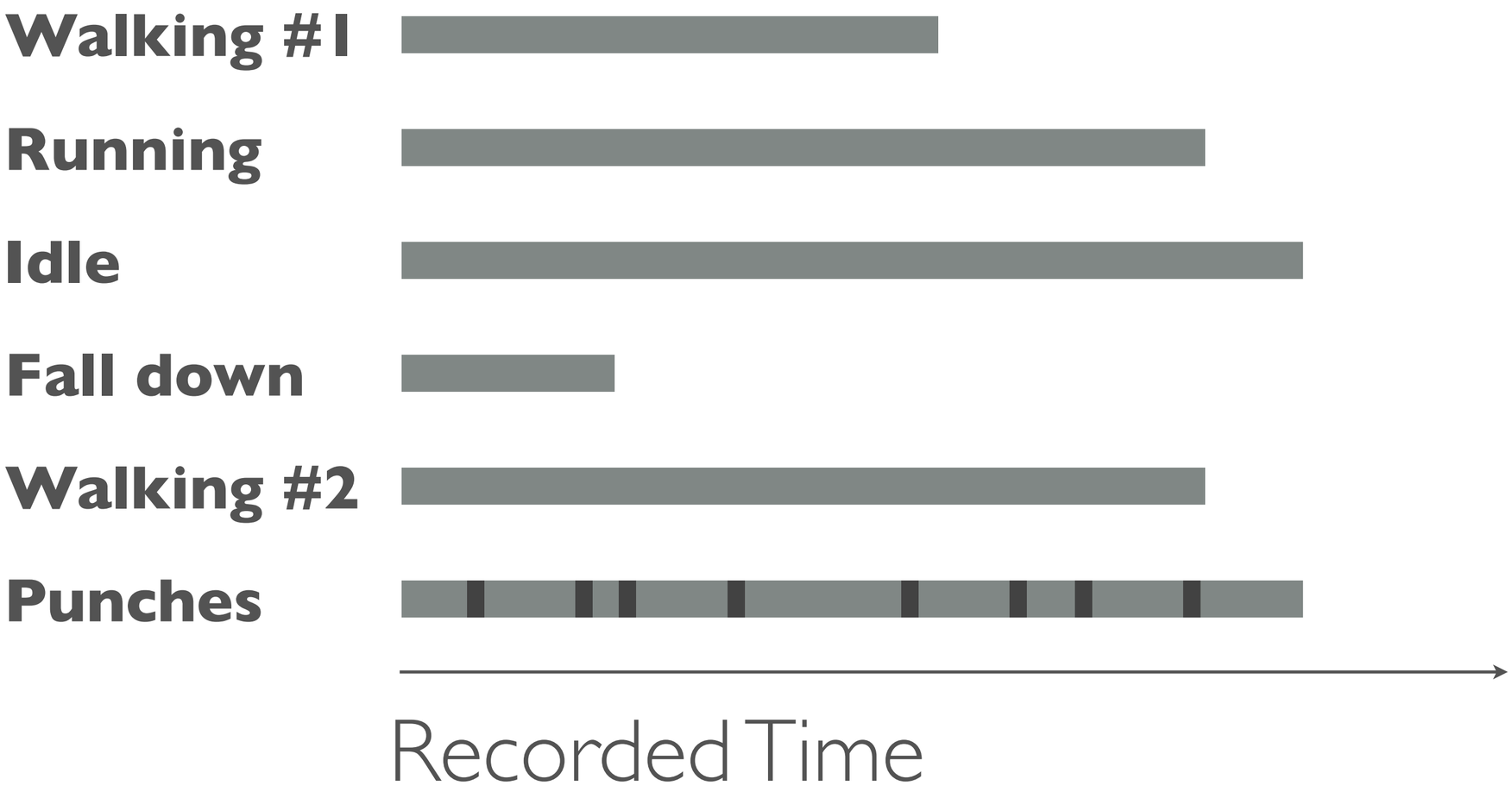
Motion Graphs

Similarity metric

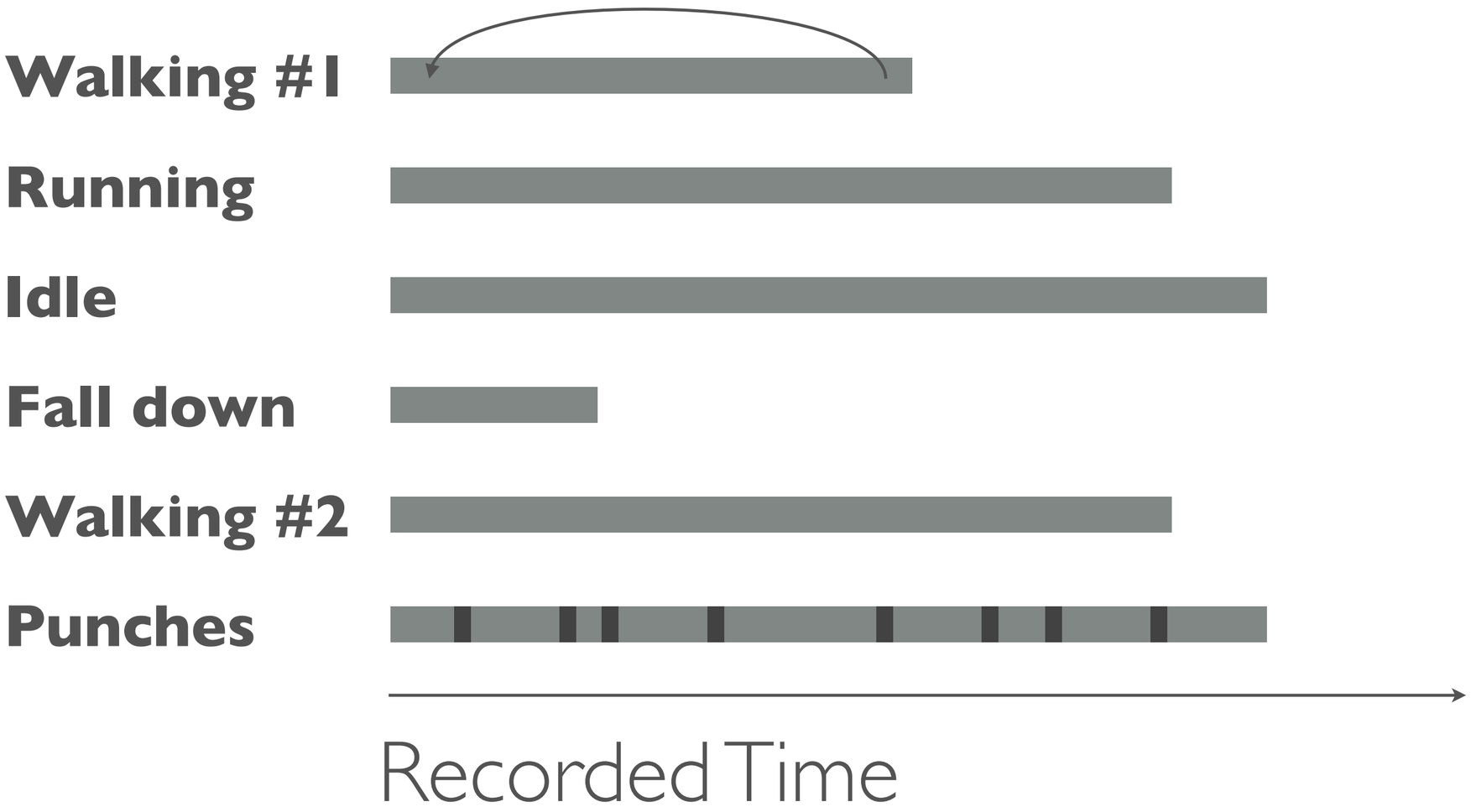
- Measurement of how similar two frames of motion are
  - Based on joint angles or point positions
  - Must include some measure of velocity
  - Ideally independent of capture setup and skeleton

Capture a “large” database of motions

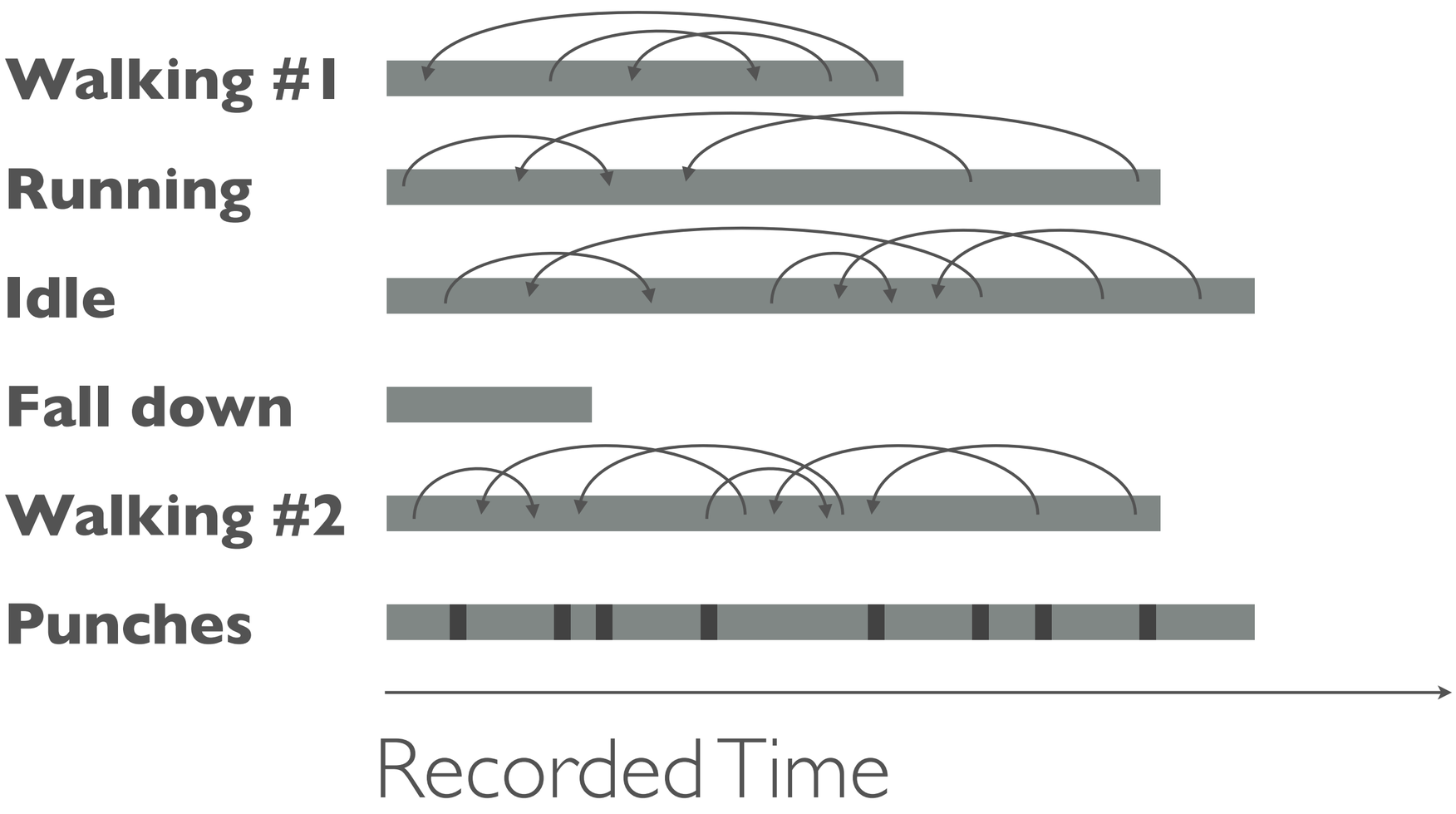
Typical Motion Graph



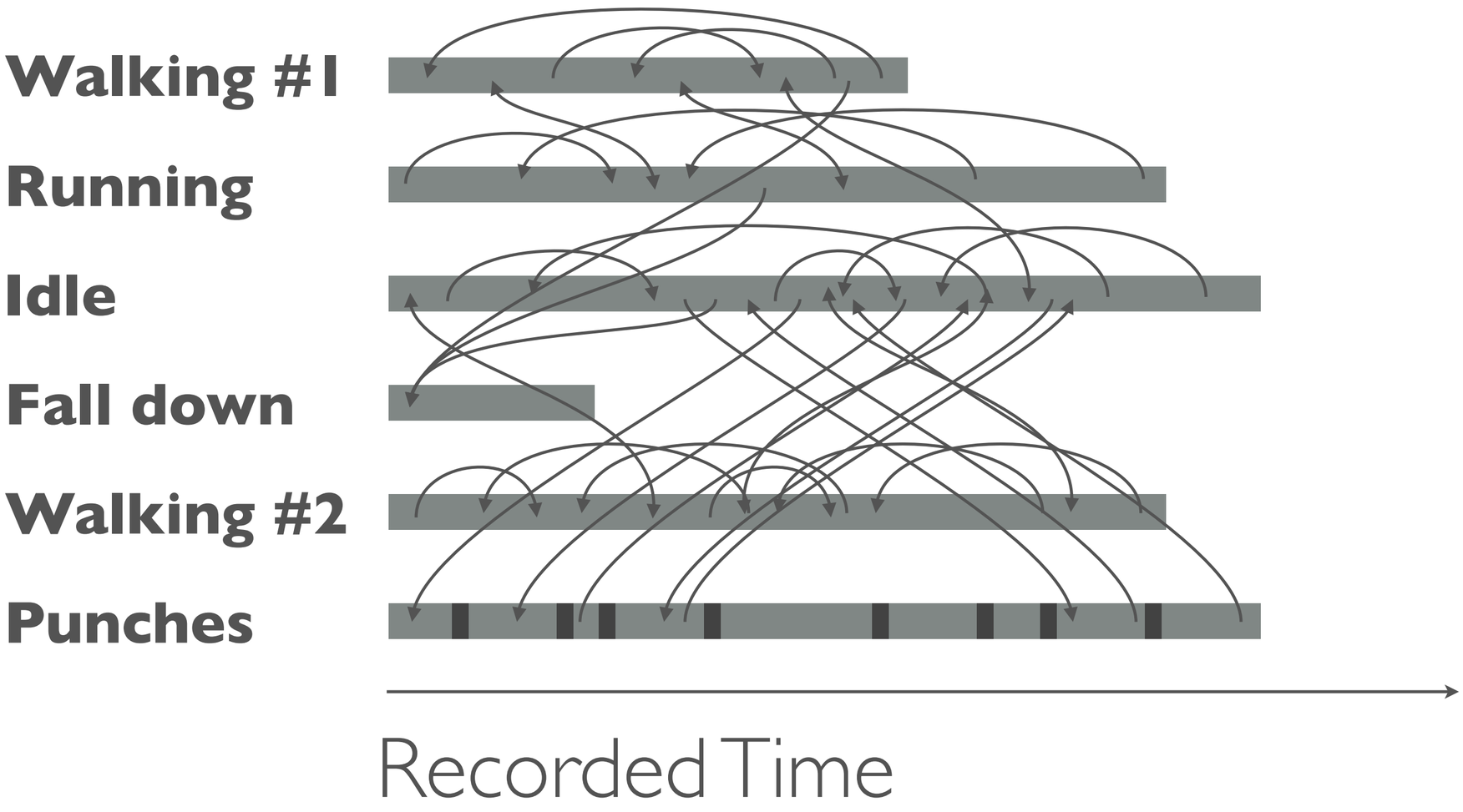
Typical Motion Graph



Typical Motion Graph



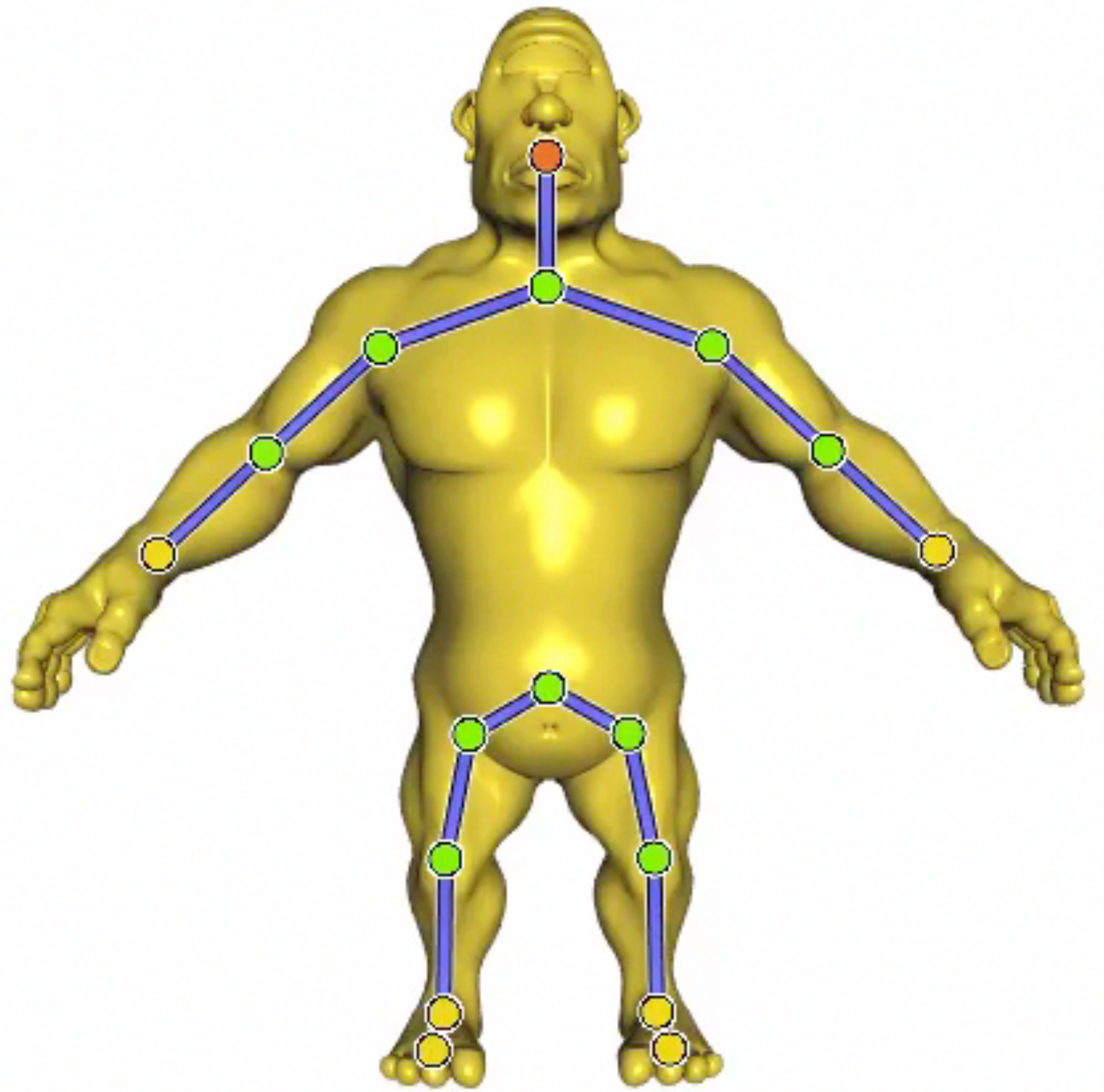
Typical Motion Graph



Skinning

Skinning

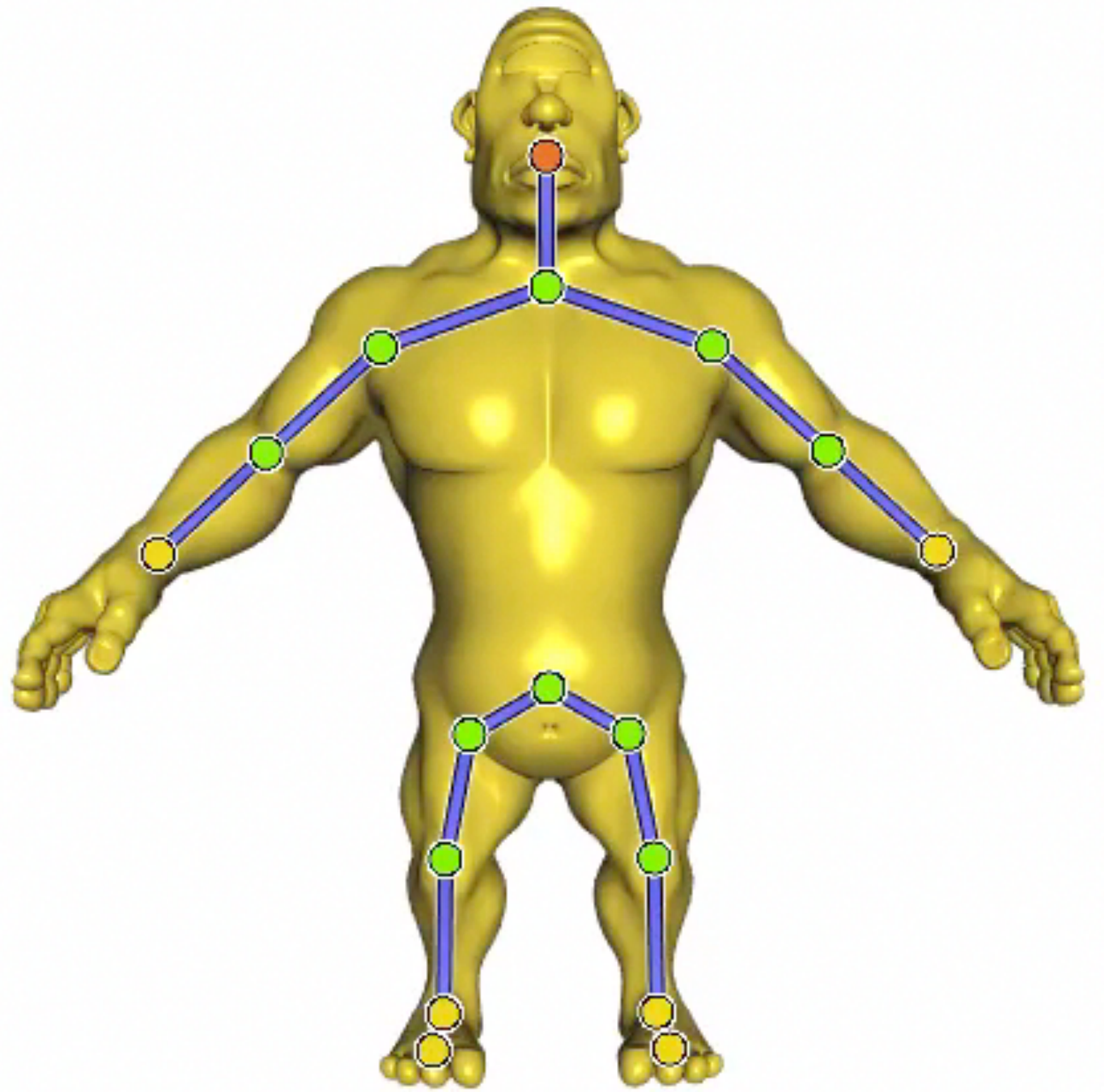
Goal: move the surface along with assigned bones or "handles"



Courtesy of Alec Jacobson

Skinning

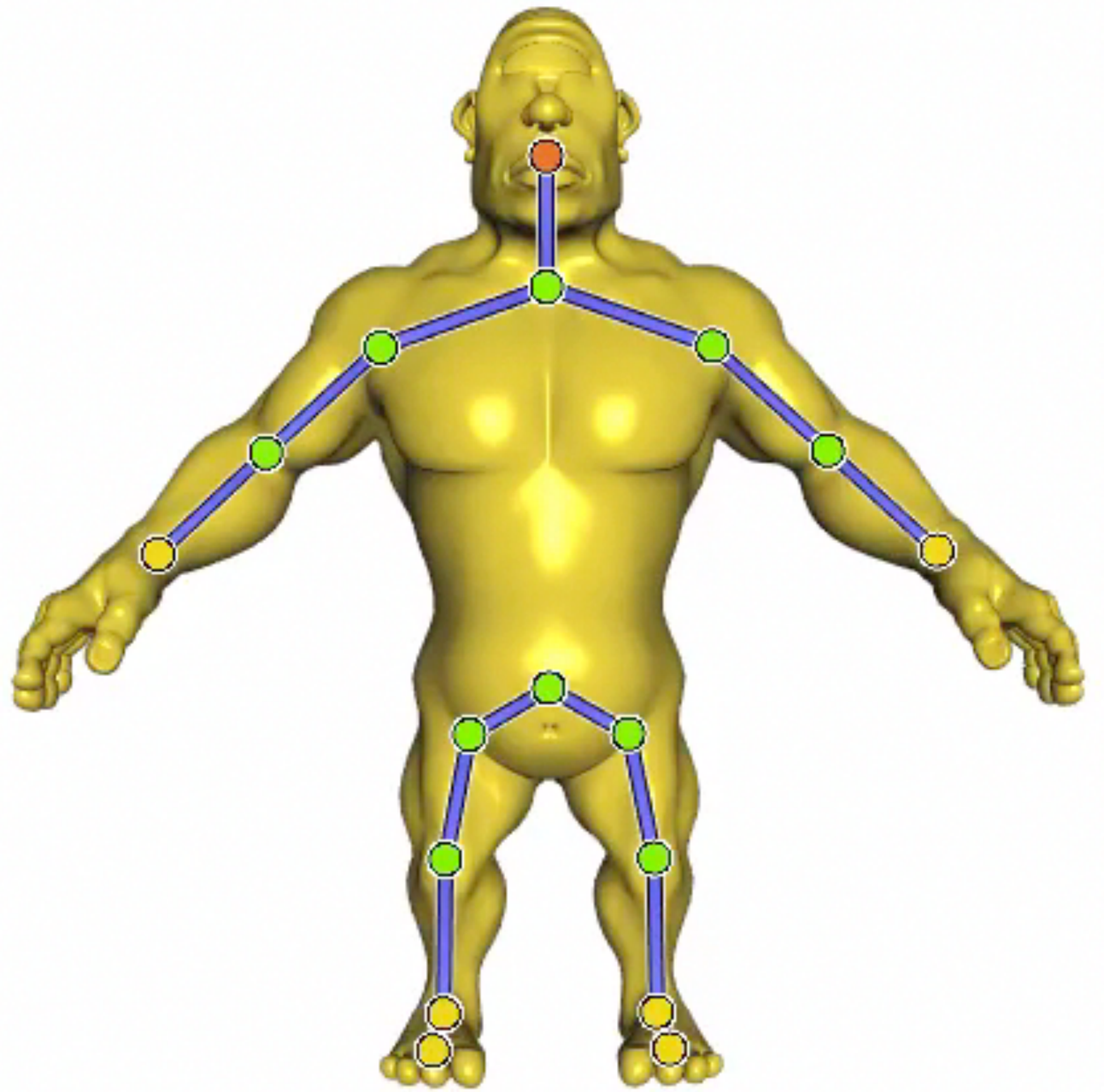
Goal: move the surface along with assigned bones or "handles"



Courtesy of Alec Jacobson

Skinning

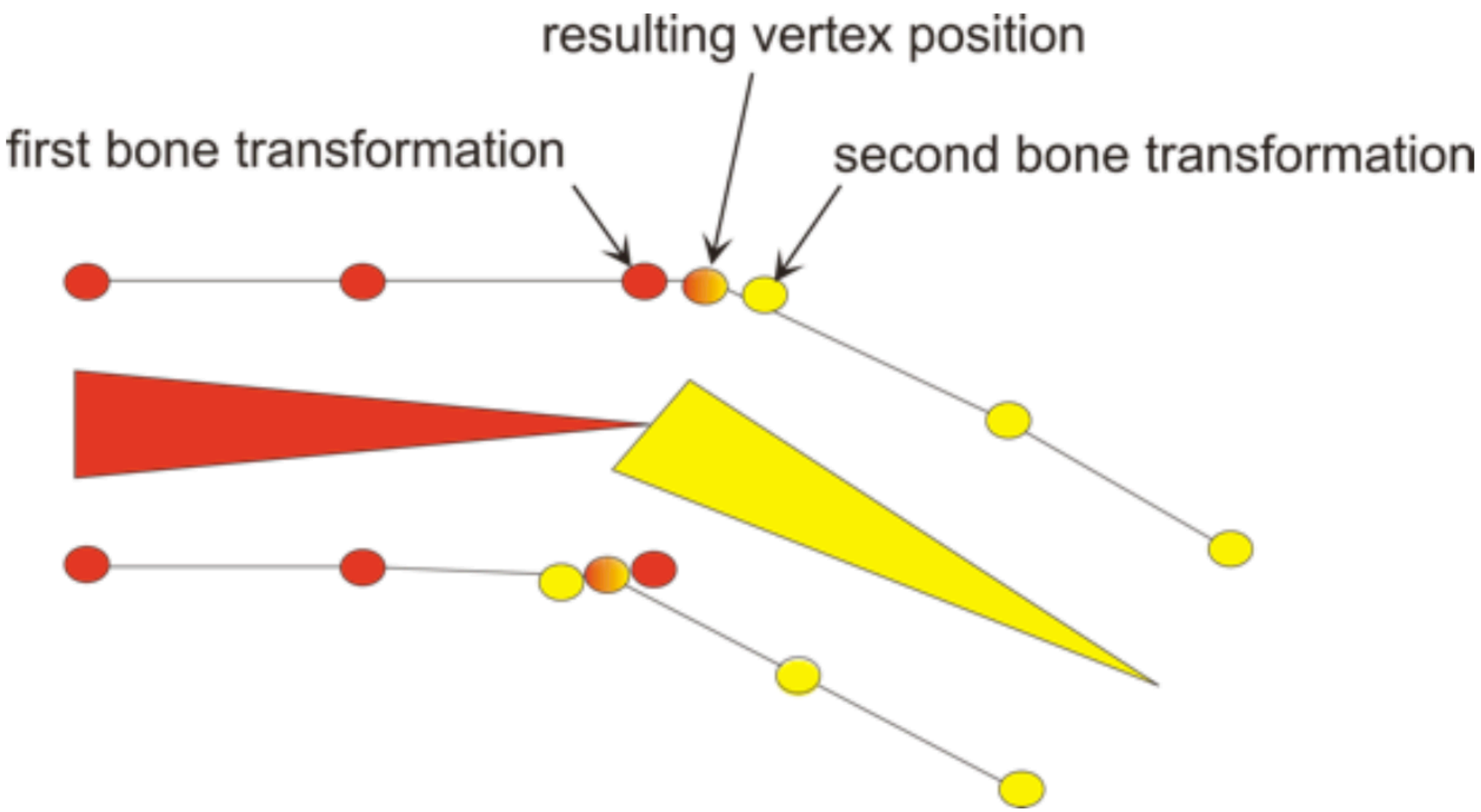
Goal: move the surface along with assigned bones or "handles"



Courtesy of Alec Jacobson

Basic Idea

1. Transform each vertex with each bone rigidly
2. Blend the results using weights, or assignments



Common Approach: Linear Blend Skinning (LBS)

Blend contribution linearly.

Super simple to implement. Great for real time.

$$\mathbf{v}' = \sum_{j \in H} w_j(\mathbf{v}) \mathbf{T}_j \begin{pmatrix} \mathbf{v} \\ 1 \end{pmatrix}$$

Common Approach: Linear Blend Skinning (LBS)

Blend contribution linearly.

Super simple to implement. Great for real time.

$$\mathbf{v}' = \sum_{j \in H} w_j(\mathbf{v}) \mathbf{T}_j \begin{pmatrix} \mathbf{v} \\ 1 \end{pmatrix}$$

New vertex

Original vertex

Common Approach: Linear Blend Skinning (LBS)

Blend contribution linearly.

Super simple to implement. Great for real time.

Bone j transformation

$$\mathbf{v}' = \sum_{j \in H} w_j(\mathbf{v}) \mathbf{T}_j \begin{pmatrix} \mathbf{v} \\ 1 \end{pmatrix}$$

New vertex

Original vertex

Common Approach: Linear Blend Skinning (LBS)

Blend contribution linearly.

Super simple to implement. Great for real time.

How much influence this bone has on v (often sparse)

Bone j transformation

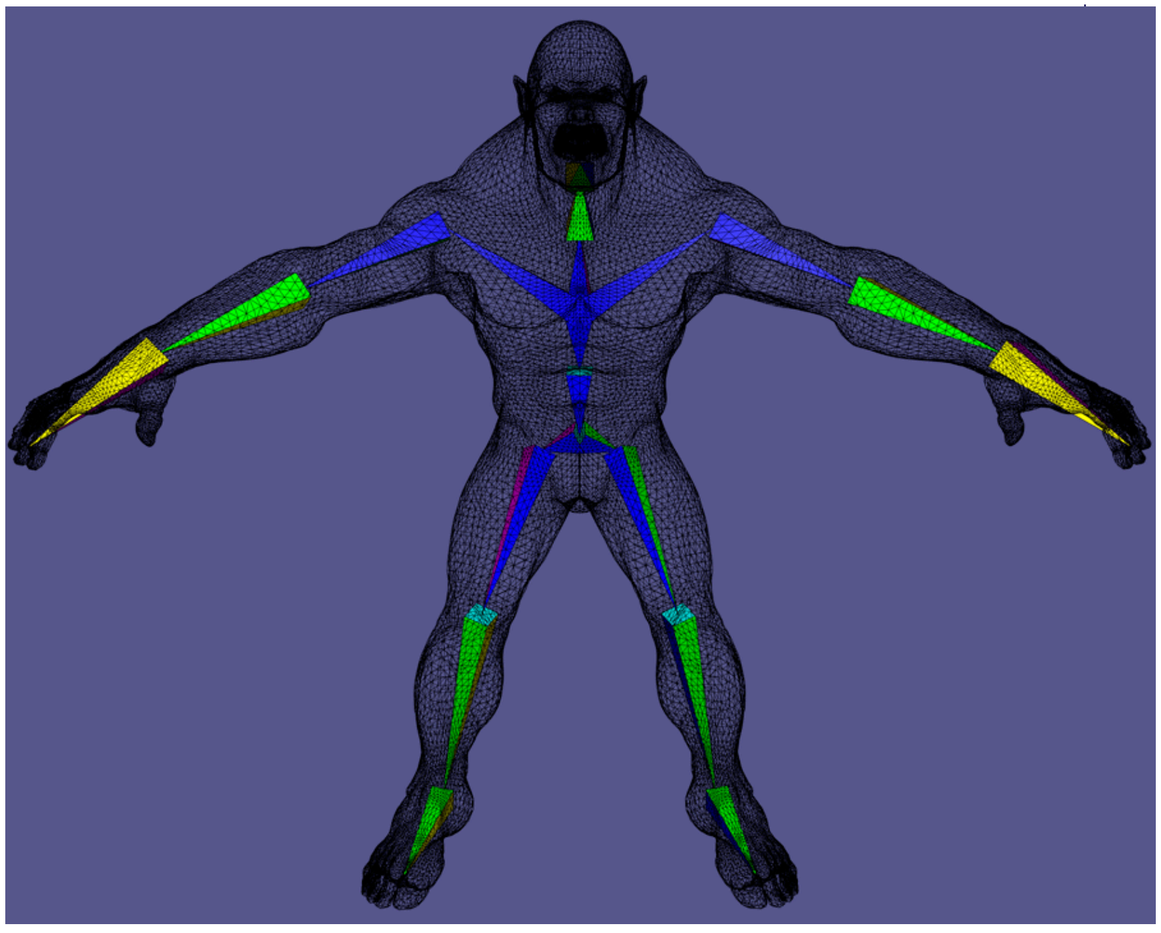
$$\mathbf{v}' = \sum_{j \in H} w_j(\mathbf{v}) \mathbf{T}_j \begin{pmatrix} \mathbf{v} \\ 1 \end{pmatrix}$$

New vertex

Original vertex

Illustration of Rig & Skinning Weights

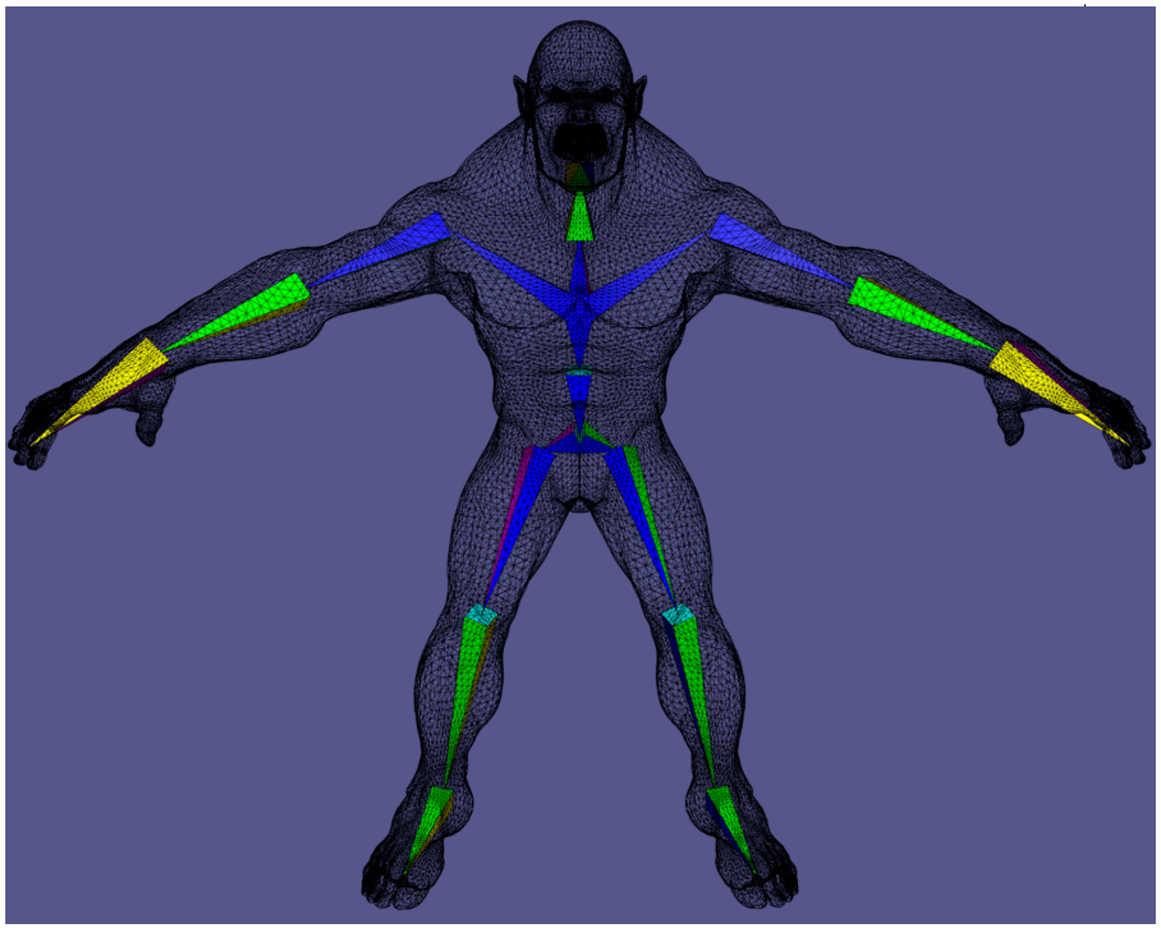
Bone transformations



Skinning Weights

Illustration of Rig & Skinning Weights

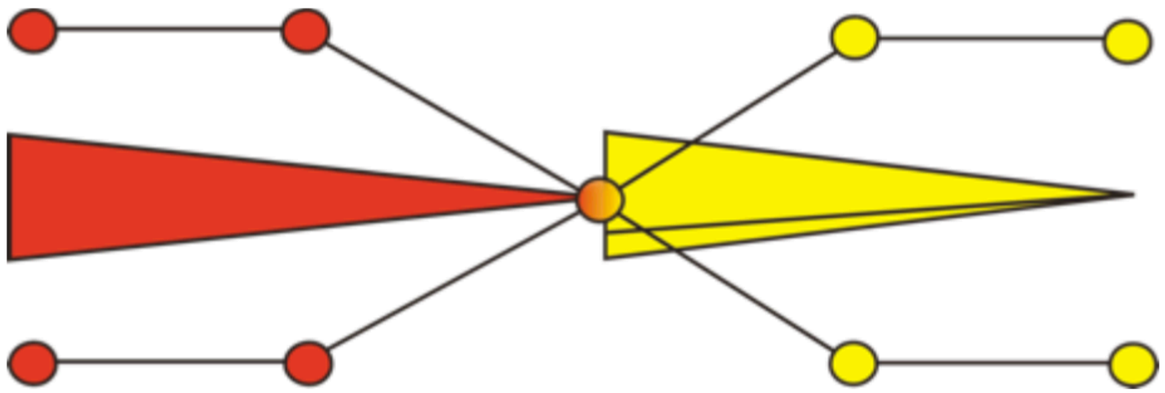
Bone transformations



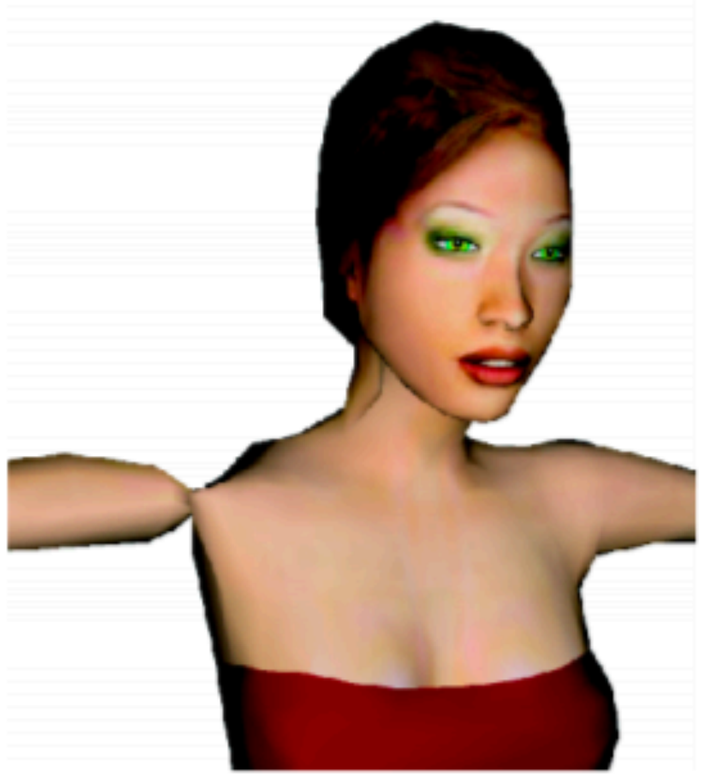
Skinning Weights

Problems with LBS

When joint rotates 180 degrees



"Candy Wrapper" artifact

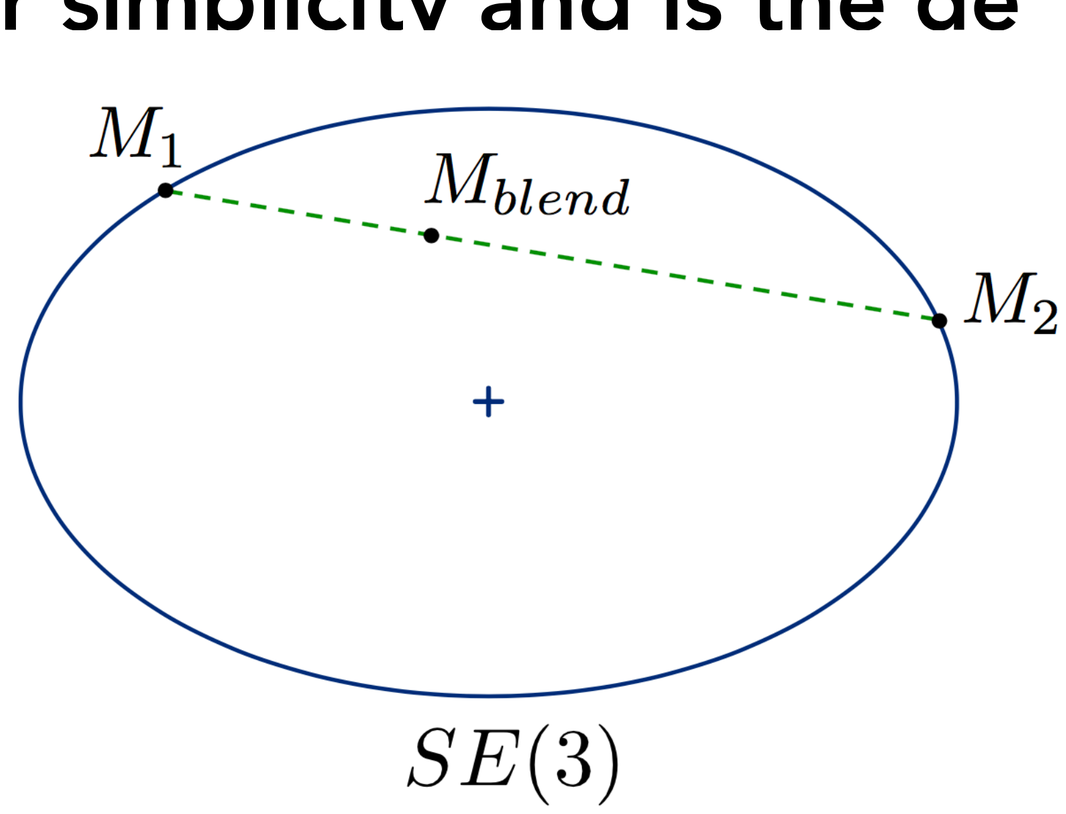


Why?

Can’t linearly combine rigid transformations!

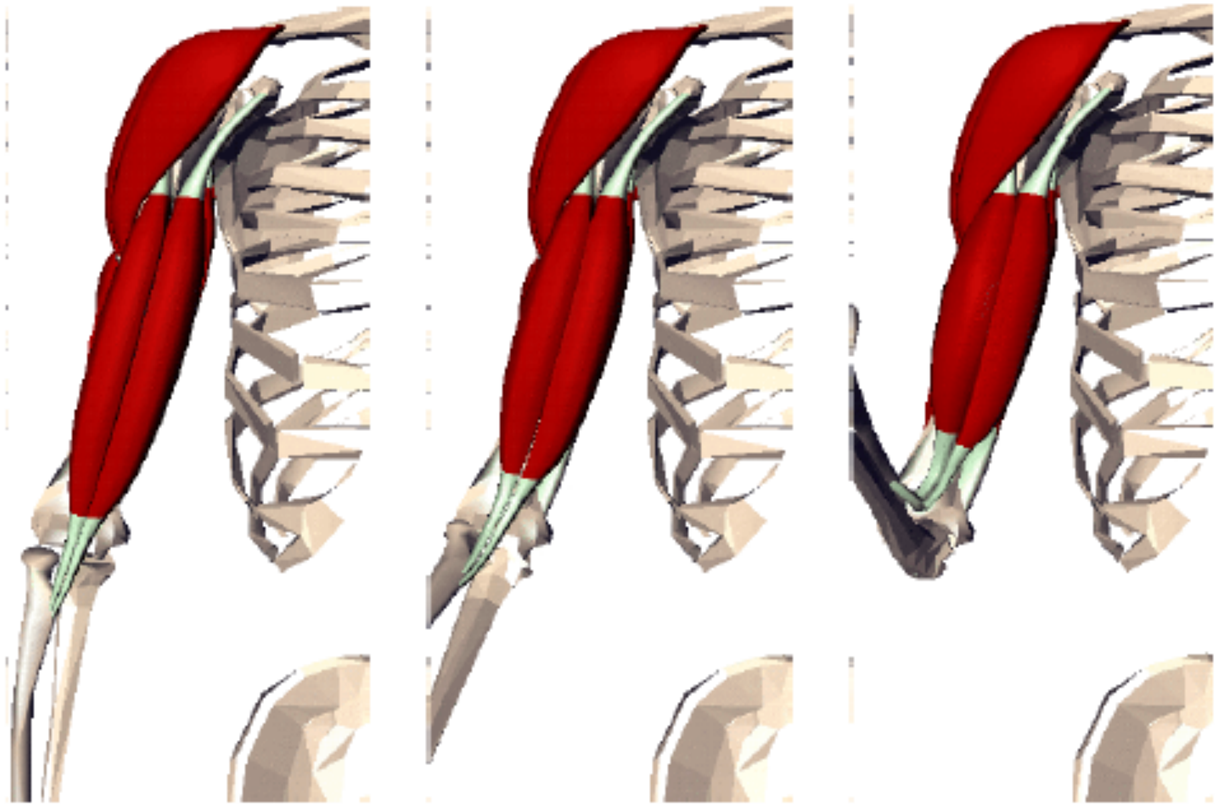
Better methods exist like Dual Quaternion Skinning [Kavan et al. TOG 2008]

Still LBS is most popular for simplicity and is the de facto industry standard

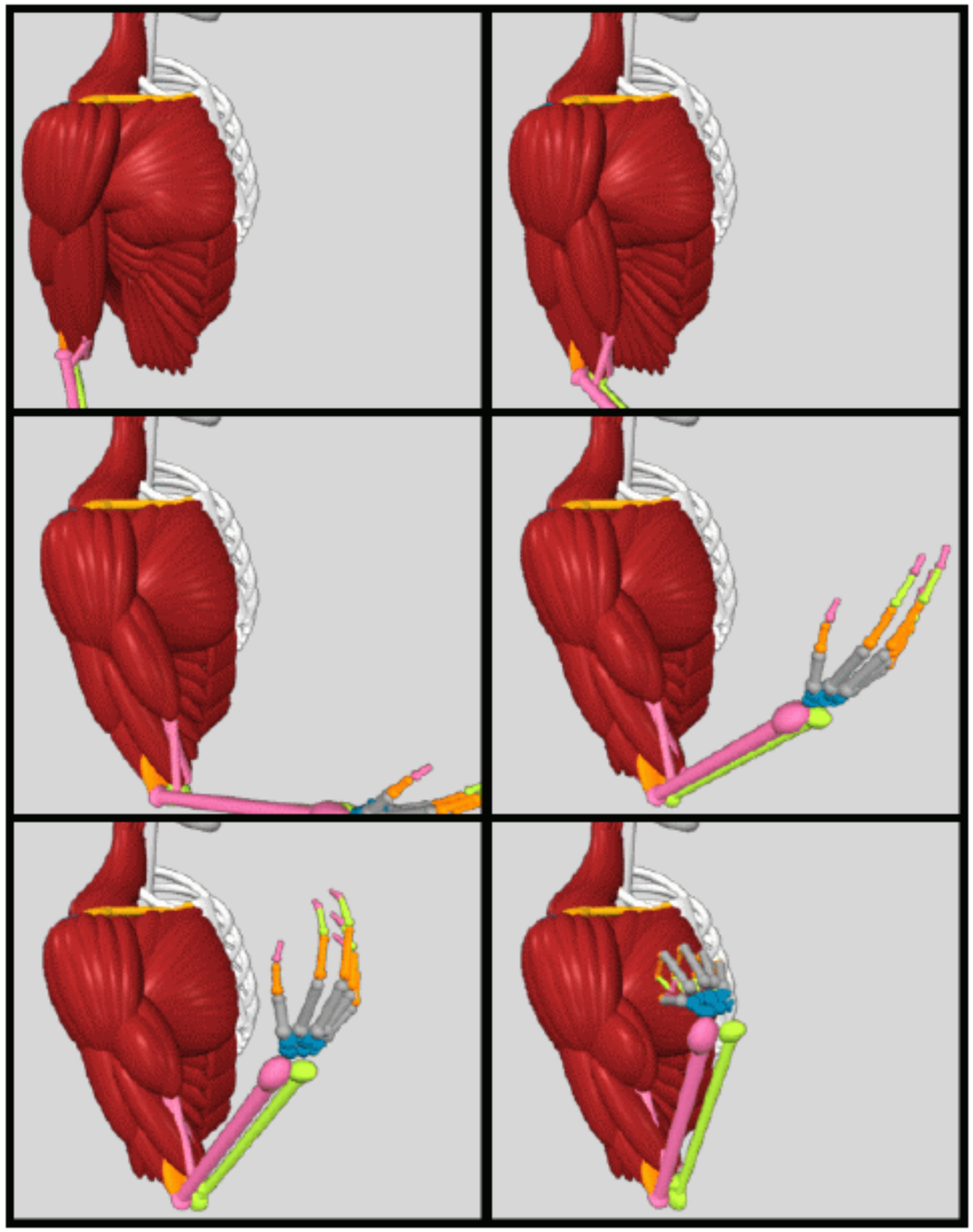


Anatomical Models

Complex musculature built up from lots of simple primitives.

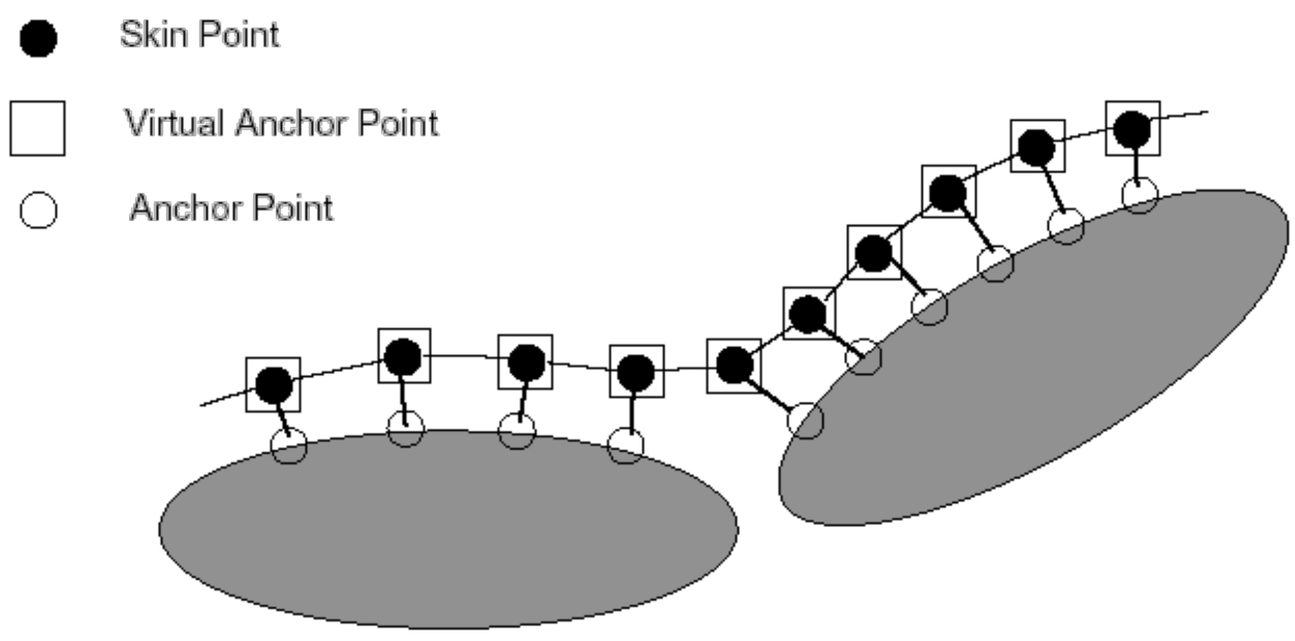


Anatomy-Based Modeling of the Human Musculature. Scheepers et al.



Anatomical Models

Skin can be attached to the muscles with springs/ dampers and physically simulated with collisions against bone & muscle

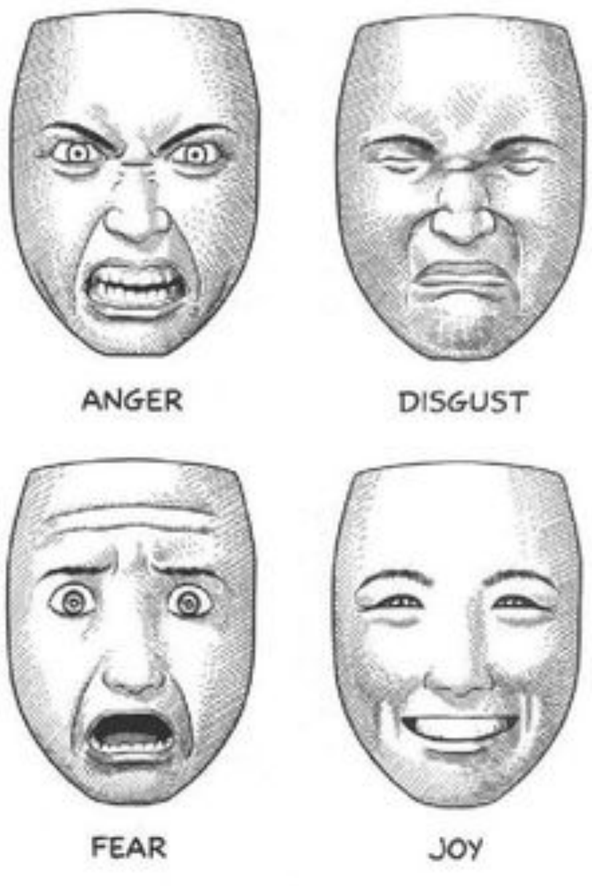


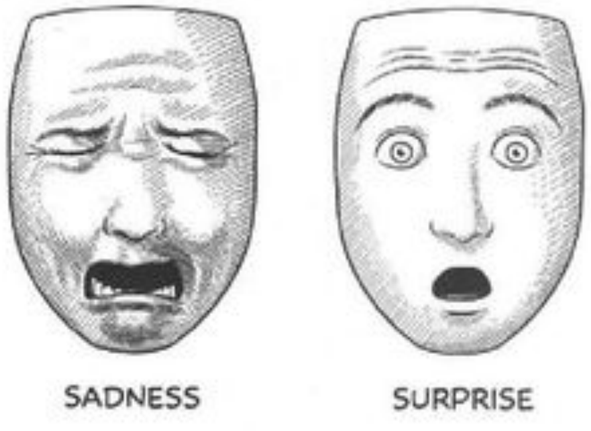
Anatomically based modeling,
Wilhelms & Van Gelder

Blend Shapes

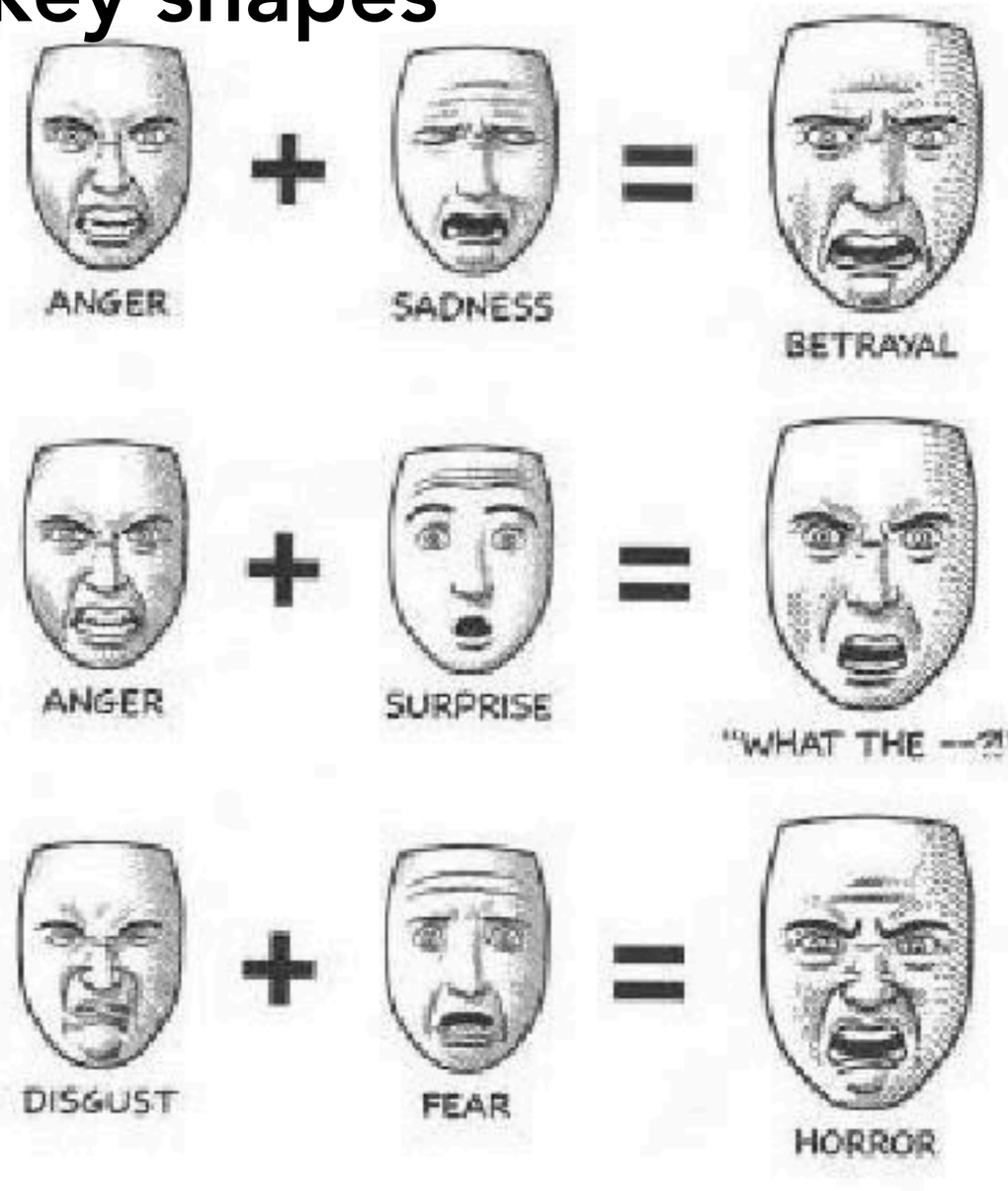
Blend Shapes

Not all deformation is from bones.
Interpolate surfaces between key shapes



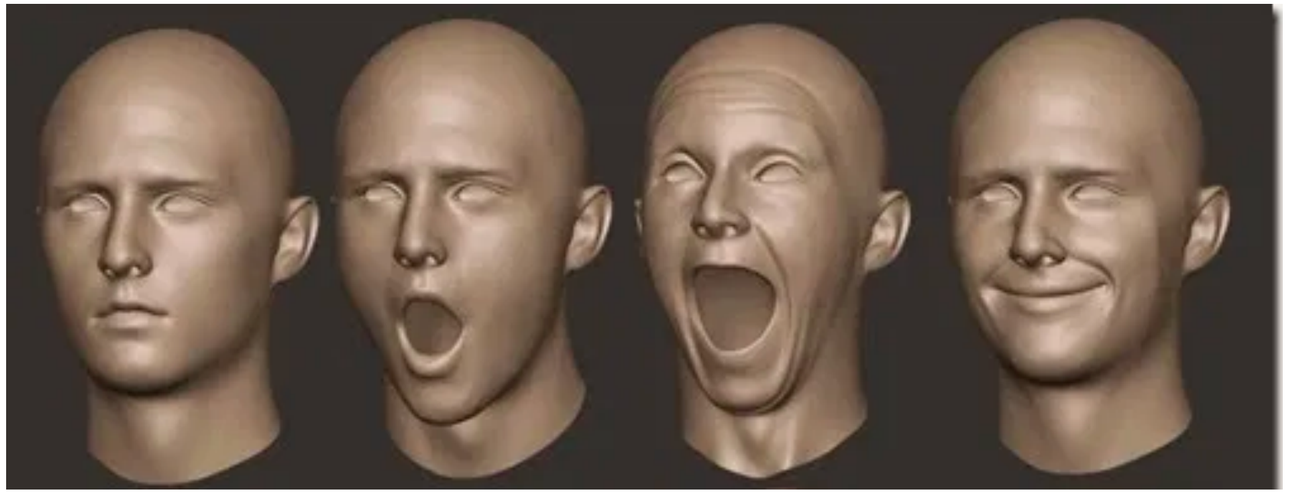


Making Comics: Storytelling Secrets Of Comics, Manga, and Graphics Novels by Scott McCloud



Blend Shapes

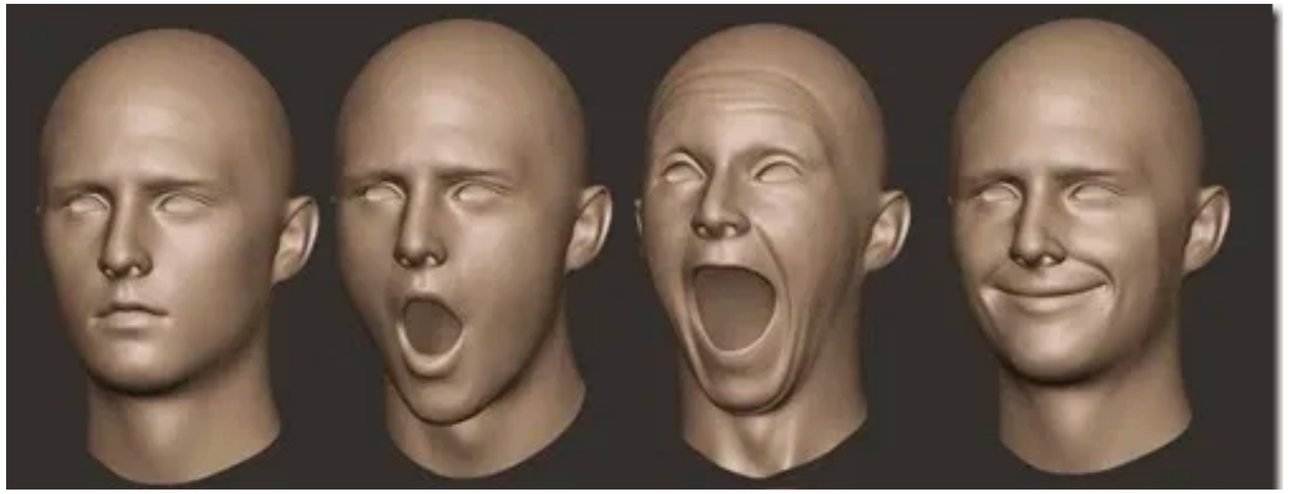
$$B = \mathrm{vec}\left(\begin{bmatrix} \Delta x_1 & \Delta y_1 & \Delta z_1 \\ & \vdots & \\ & \vdots & \\ \Delta x_N & \Delta y_N & \Delta z_N \end{bmatrix}\right)$$



Blend Shapes

- A set of vertex offsets to neutral shape

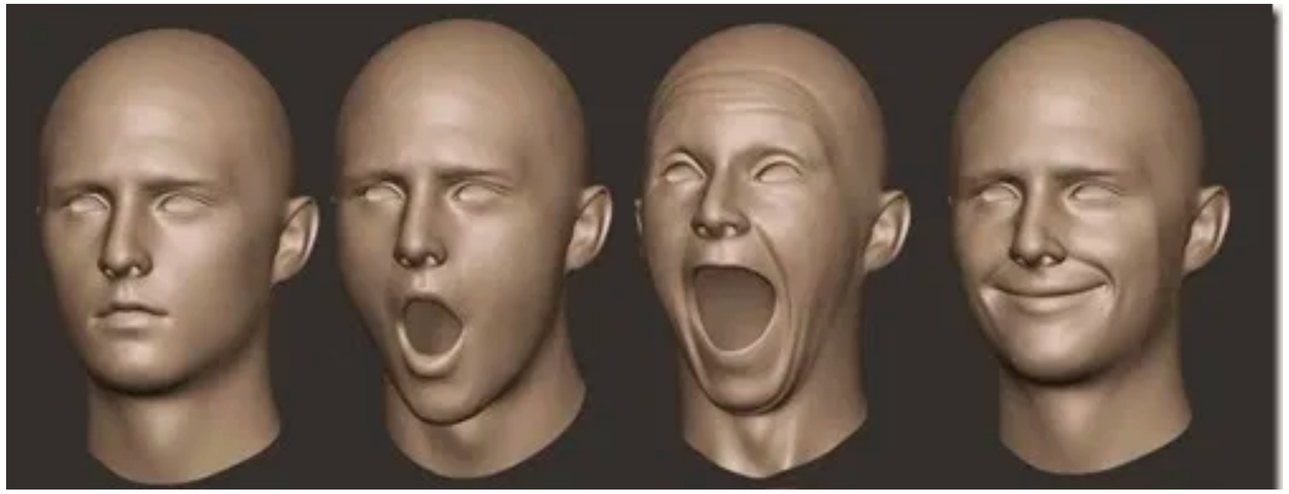
$$B = \mathrm{vec}\left(\begin{bmatrix} \Delta x_1 & \Delta y_1 & \Delta z_1 \\ & \vdots & \\ & \vdots & \\ \Delta x_N & \Delta y_N & \Delta z_N \end{bmatrix} \right)$$



Blend Shapes

- A set of vertex offsets to neutral shape
- Linearly interpolate these key blend shapes for control

$$B = \mathrm{vec}\left(\begin{bmatrix} \Delta x_1 & \Delta y_1 & \Delta z_1 \\ & \vdots & \\ \Delta x_N & \Delta y_N & \Delta z_N \end{bmatrix} \right)$$

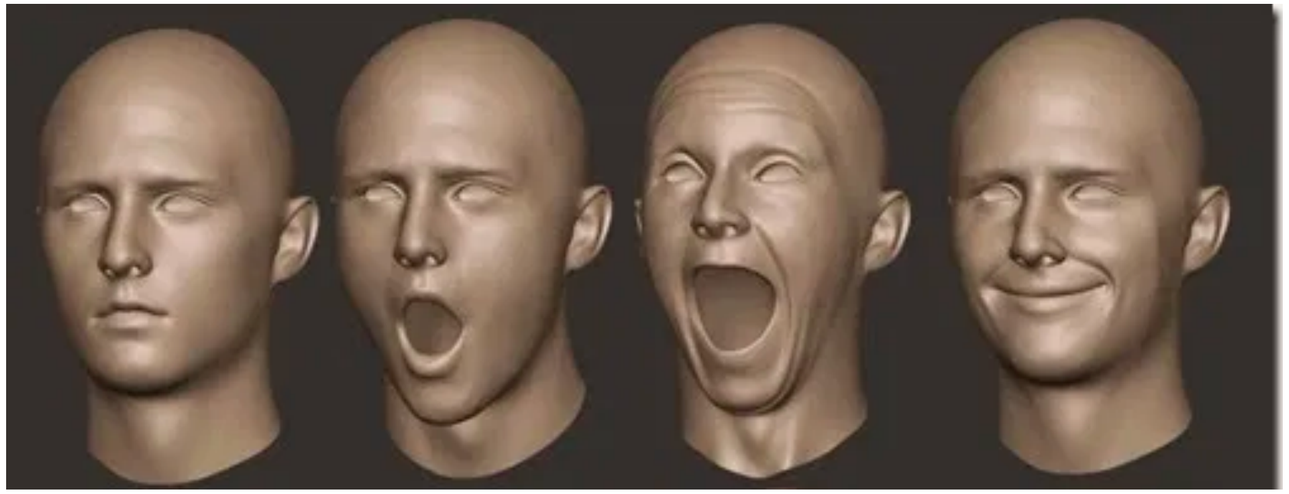


$$V = \sum_i \beta_i B_i$$

Blend Shapes

- A set of vertex offsets to neutral shape
- Linearly interpolate these key blend shapes for control
- Often used for expressions

$$B = \mathrm{vec}\left(\begin{bmatrix} \Delta x_1 & \Delta y_1 & \Delta z_1 \\ & \vdots & \\ \Delta x_N & \Delta y_N & \Delta z_N \end{bmatrix}\right)$$

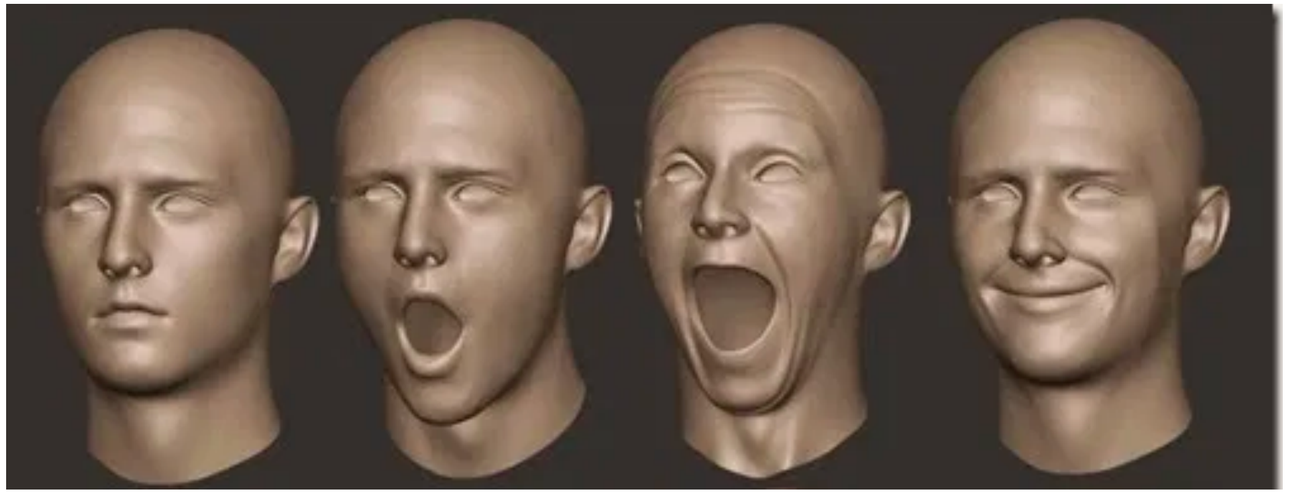


$$V = \sum_i \beta_i B_i$$

Blend Shapes

- A set of vertex offsets to neutral shape
- Linearly interpolate these key blend shapes for control
- Often used for expressions
- Works for deformations that are linear, *i.e.* the average of two shapes is a valid shape

$$B = \mathrm{vec}\left(\begin{bmatrix} \Delta x_1 & \Delta y_1 & \Delta z_1 \\ & \vdots & \\ \Delta x_N & \Delta y_N & \Delta z_N \end{bmatrix}\right)$$



$$V = \sum_i \beta_i B_i$$

Blend Shapes

by Félix Ferrand

Blend Shapes

by Félix Ferrand

Rigging

Rigging

Rigging

Augment character with controls to easily change its pose, create facial expressions, bulge muscles, etc.

Rigging

Augment character with controls to
easily change its pose, create facial expressions, bulge muscles, etc.

Rigging is like the strings on a marionette.

Rigging

Augment character with controls to
easily change its pose, create facial expressions, bulge muscles, etc.

Rigging is like the strings on a marionette.

Capture space of meaningful deformations.

Rigging

Augment character with controls to
easily change its pose, create facial expressions, bulge muscles, etc.

Rigging is like the strings on a marionette.

Capture space of meaningful deformations.

Varies from character to character.

Rigging

Augment character with controls to
easily change its pose, create facial expressions, bulge muscles, etc.

Rigging is like the strings on a marionette.

Capture space of meaningful deformations.

Varies from character to character.

Rigging

Augment character with controls to
easily change its pose, create facial expressions, bulge muscles, etc.

Rigging is like the strings on a marionette.

Capture space of meaningful deformations.

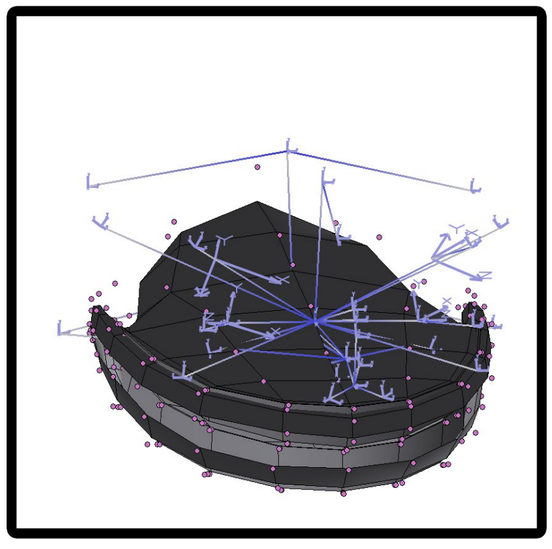
Varies from character to character.

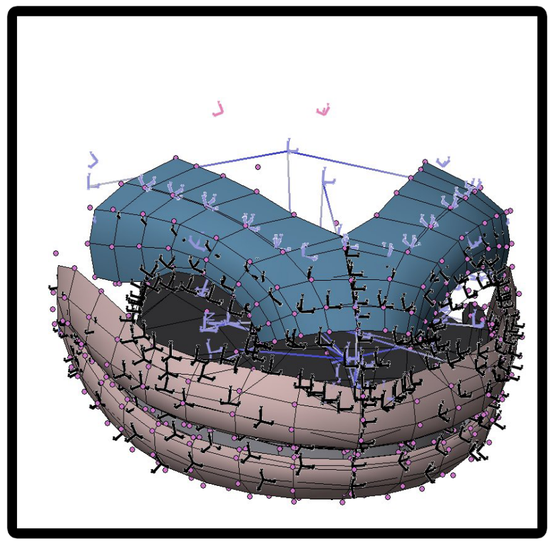
Skeleton is ONE type of rigging

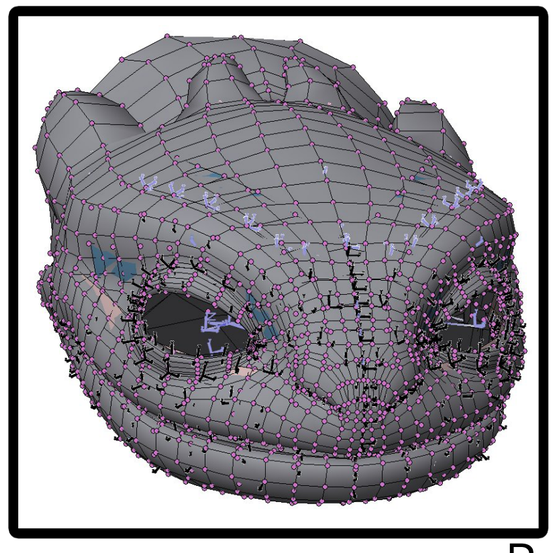
Example of A Diverse Set of Sophisticated Rigs

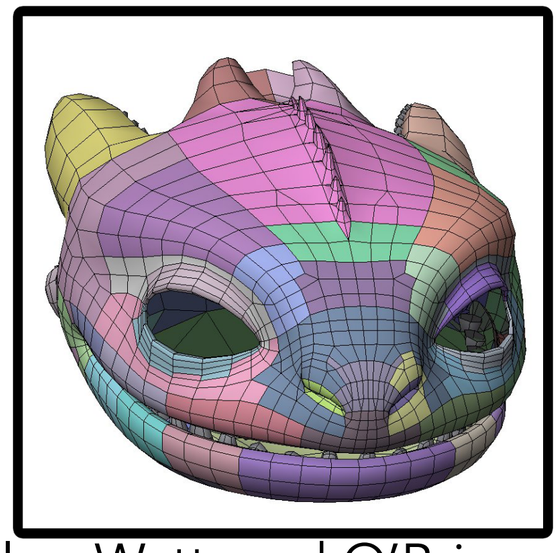
Example of A Diverse Set of Sophisticated Rigs

Rig-Based Facial Animation



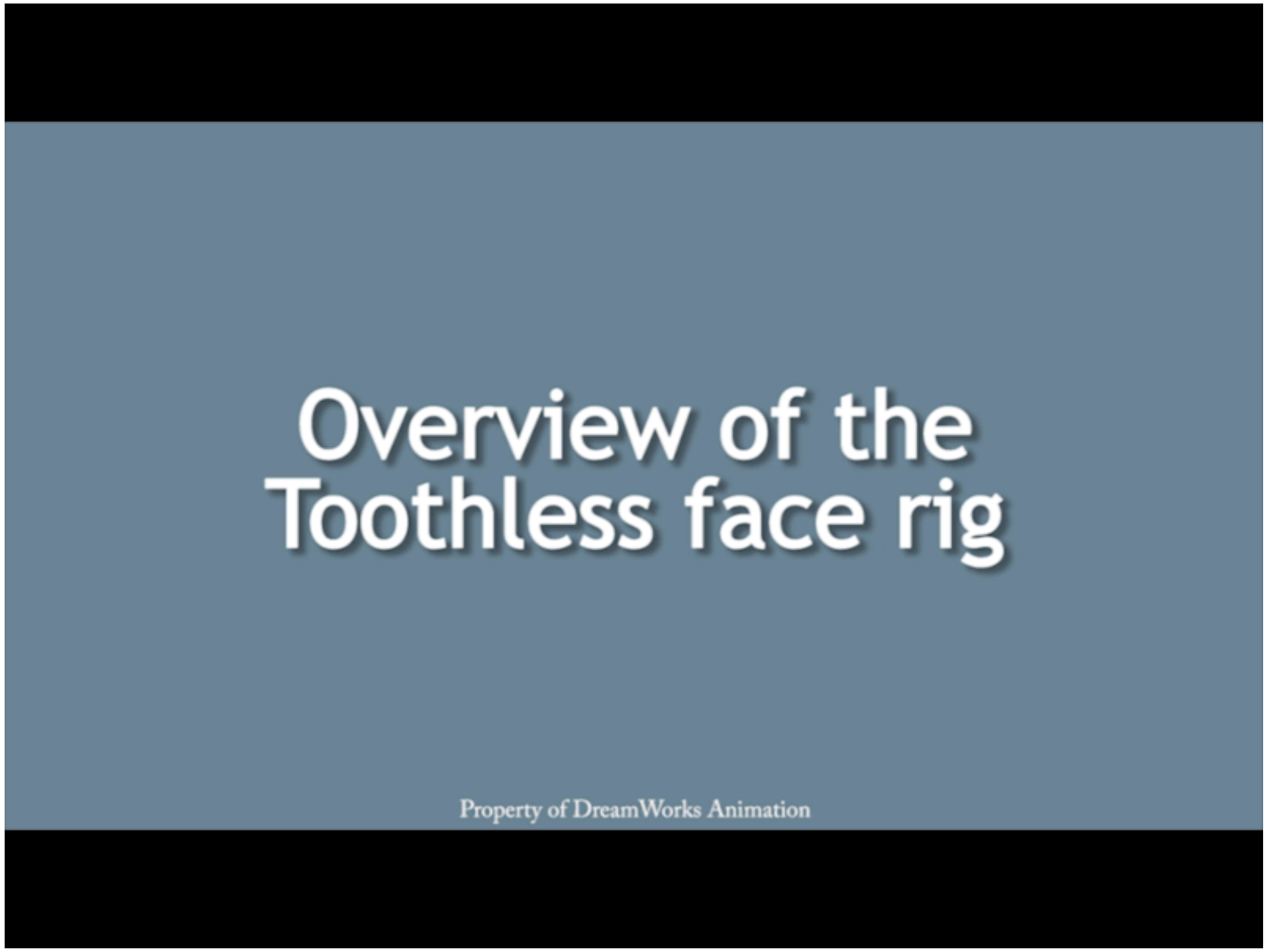






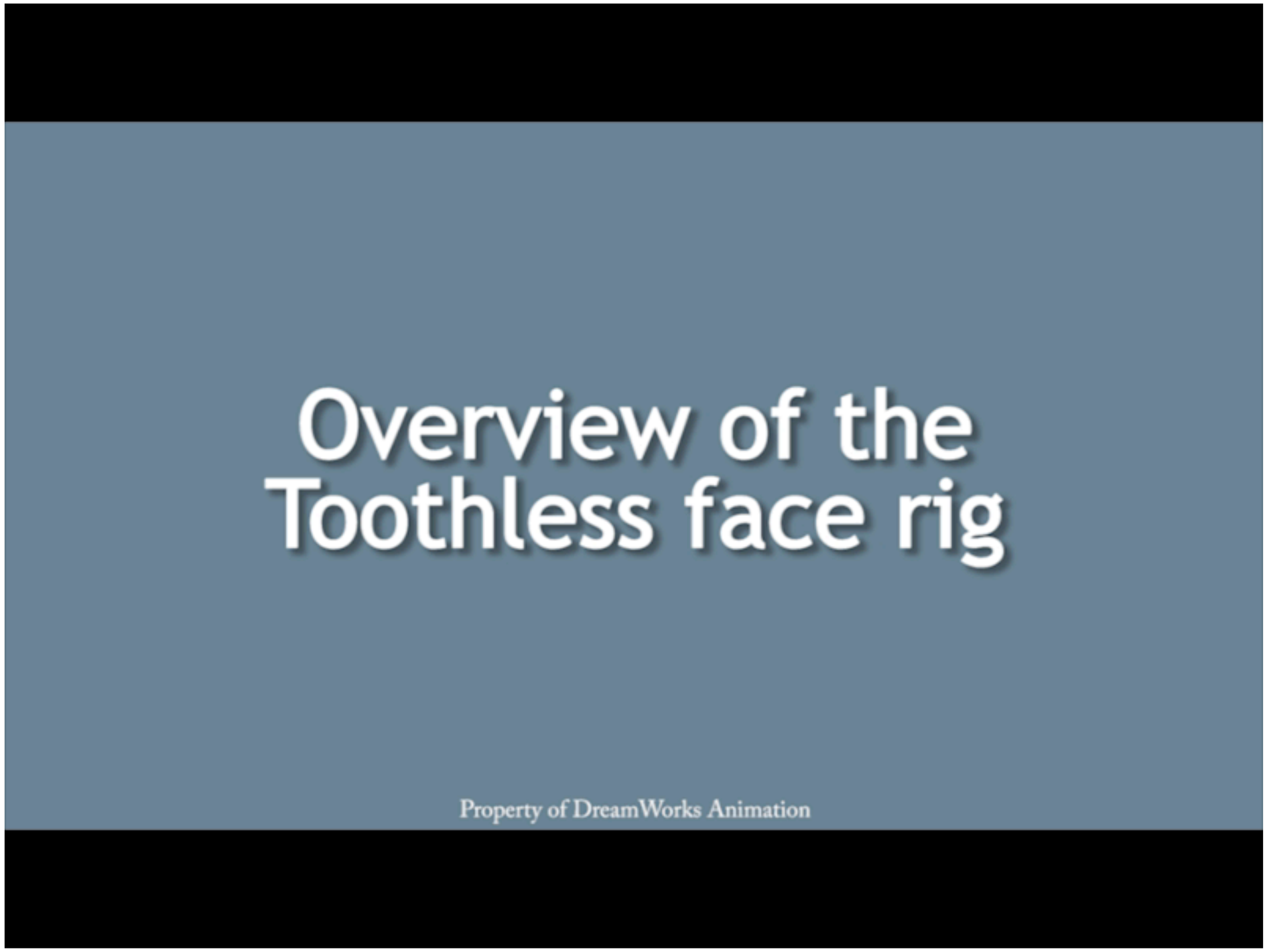
Bailey, Watt, and O'Brien

Rib-Based Facial Animation



Bailey, Watt, and O’Brien

Rib-Based Facial Animation



Bailey, Watt, and O’Brien

Data Driven Models

Human Bodies Scans

Civilian American and European Surface Anthropometry Resource Project (CAESAR)

$$\{V_1, V_2, V_3, \ldots V_N\} \in \mathscr{R}^{3\times \#V}$$

All meshes have same topology and same vertex semantics.

e.g. the same vertex number is the the tip of nose for every mesh

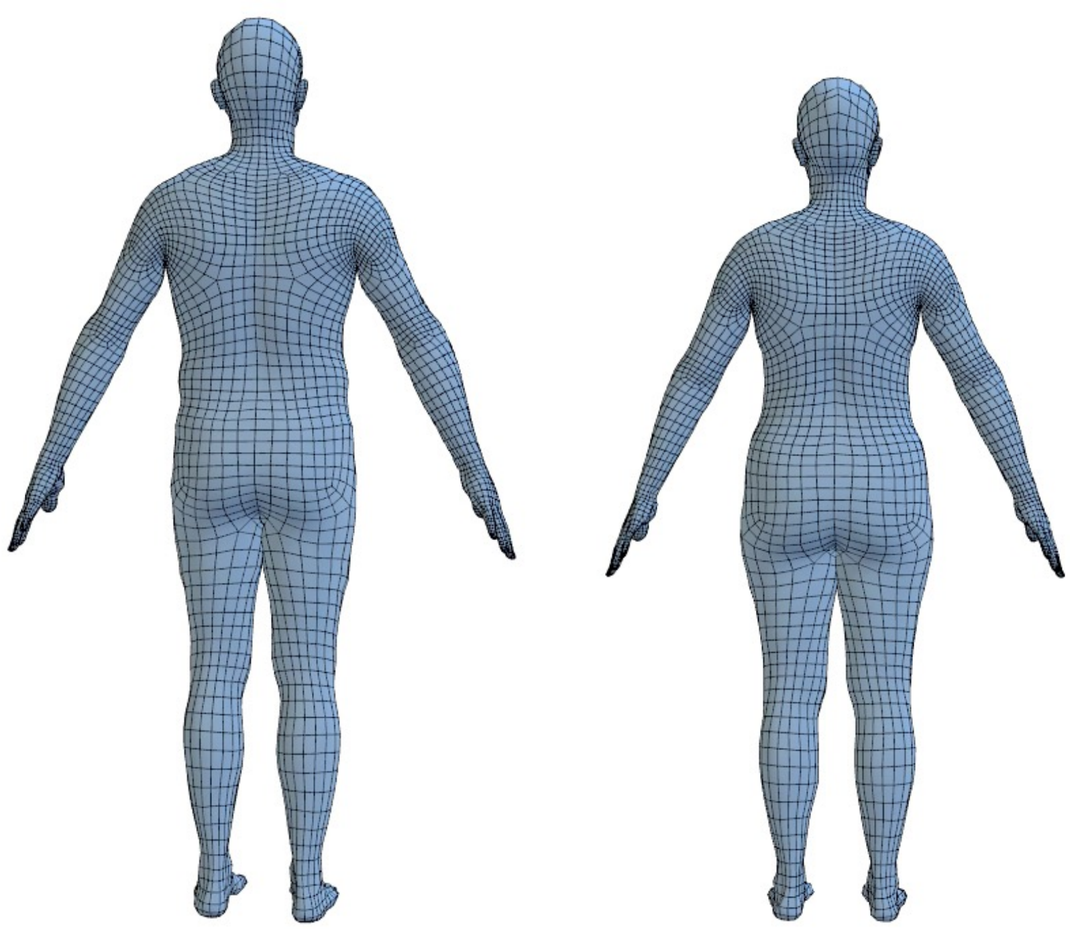
Statistical Analysis of Body Shapes

We have a data of aligned people in the exact same pose.

$$\{V_1, V_2, V_3, \ldots V_N\} \in \mathscr{R}^{3\times \#V}$$

Now we can ask questions like: what is the average?

Average shape of ~2000 men and women in US and Europe



Figures Courtesy of Michael Black

Principal Component Analysis

For learning "Shape" Blend Shapes, apply PCA!

$$\left\{V_1, V_2, V_3, \ldots V_N\right\} \in \mathscr{R}^{3\times \#V}$$

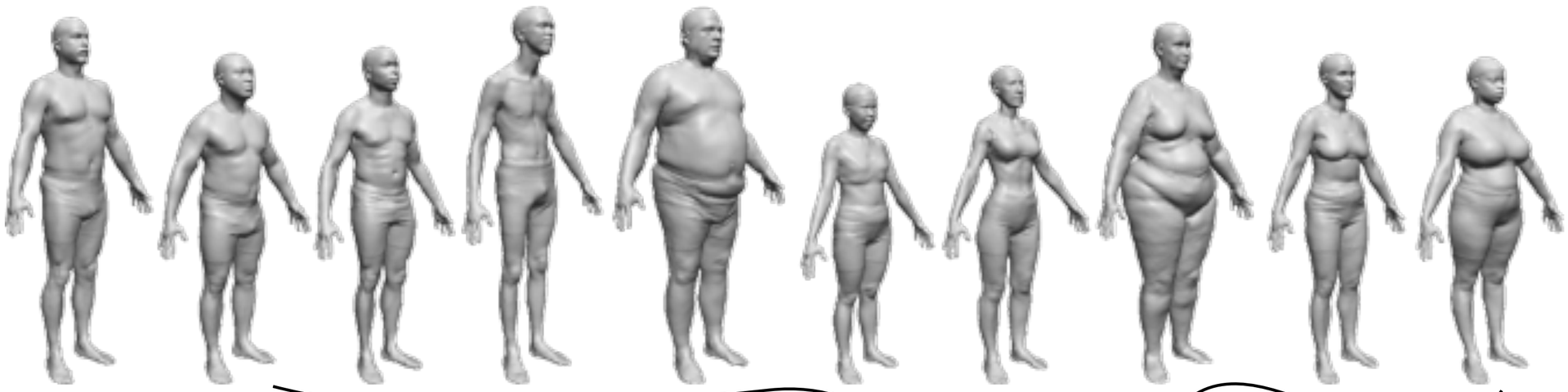
$$\cong \sum_i \beta_i B_i + \bar{V}$$

Eigenvalues

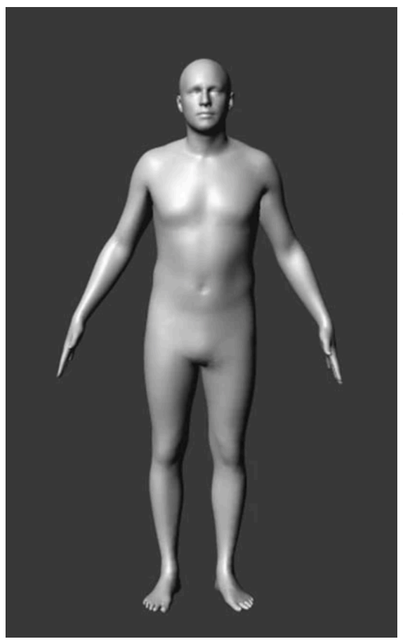
Eigenvectors / Blend Shapes

Principal Component Analysis

Simplified for people who haven't seen PCA before



$$\{V_1, V_2, V_3, \ldots V_N\} \in \mathscr{R}^{3\times \#V}$$



$$\bar{V} = \frac{\sum V_i}{N}$$

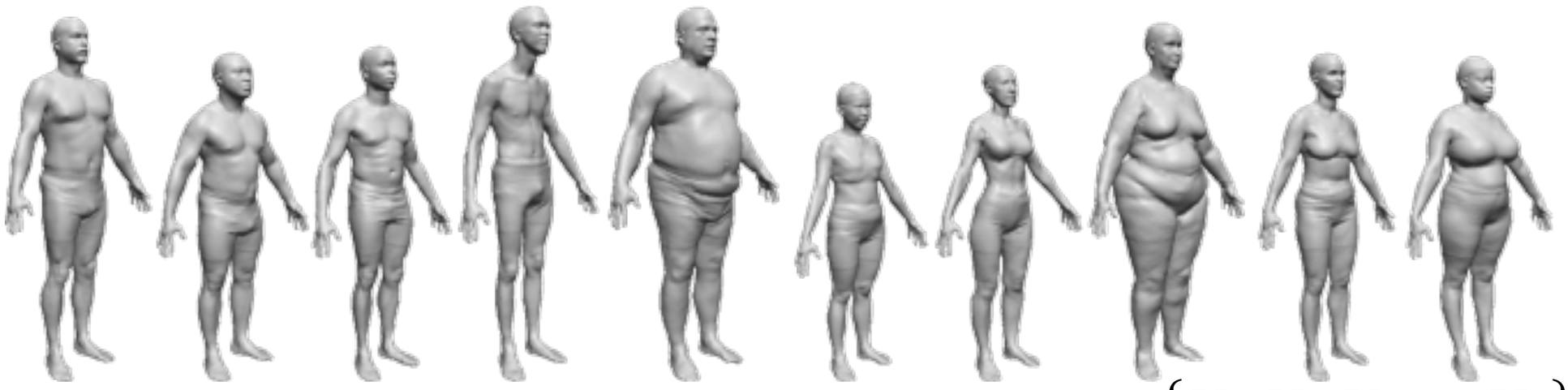
$$R_i = V_i - \bar{V}$$

Residual

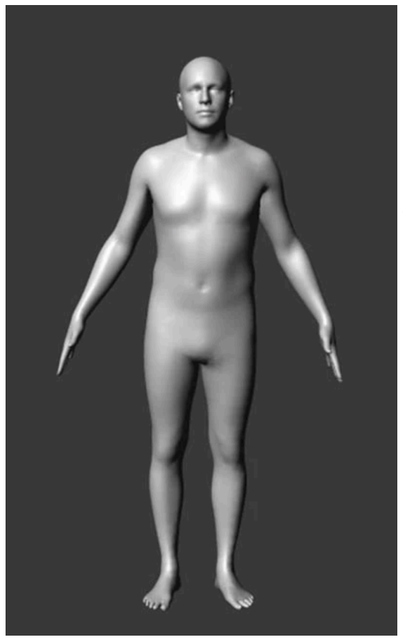
- $\bar{V}$ minimizes $\sum |R_i|^2$
- What do the R_i look like?

Principal Component Analysis

Simplified for people who haven't seen PCA before



$$\left\{V_1, V_2, V_3, \ldots V_N\right\} \in \mathscr{R}^{3\times \#V}$$



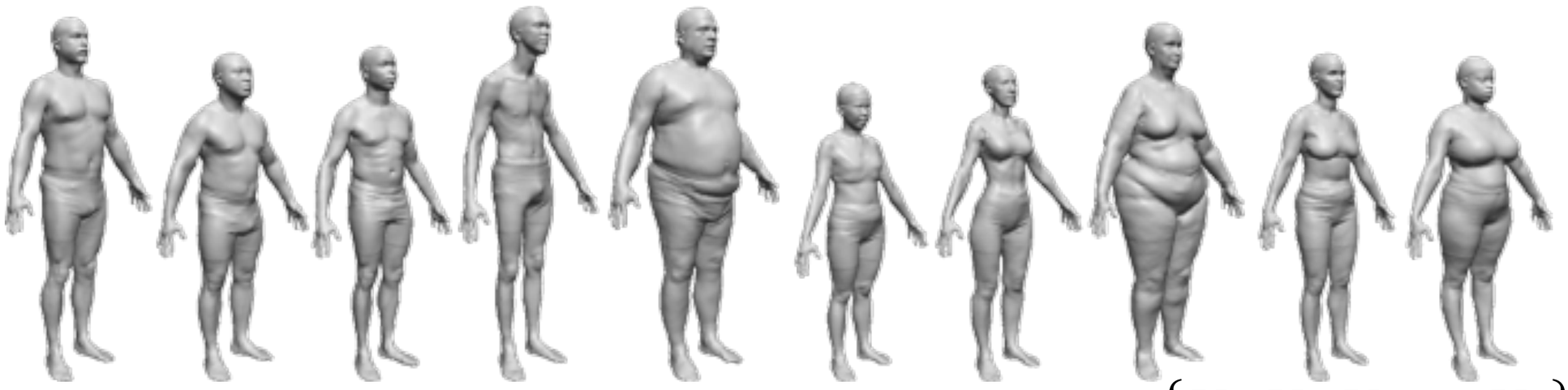
$\bar{V}$

$$R_i = V_i - \bar{V}$$

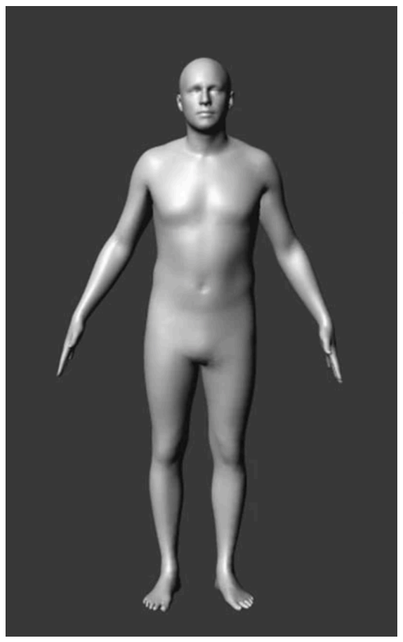
$$\hat{B}_1 = \underset{\hat{B}_1}{\operatorname{argmin}} \left(\min_{\{\alpha_i\}} \sum \left| R_i - \hat{B}_1 \alpha_i \right|^2 \right) \quad , \quad |\hat{B}_1| = 1$$

Principal Component Analysis

Simplified for people who haven't seen PCA before



$$\{V_1, V_2, V_3, \ldots V_N\} \in \mathscr{R}^{3\times \#V}$$



$$\bar{V}$$

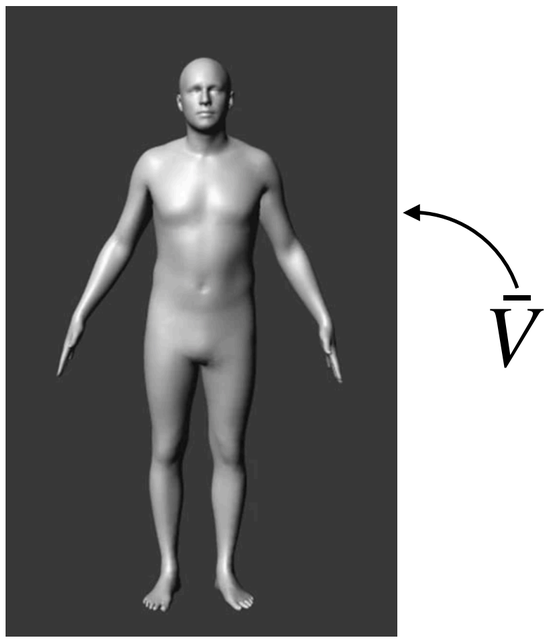
$$R_i = V_i - \bar{V}$$

$$\hat{B}_1 = \underset{\hat{B}_1}{\operatorname{argmin}} \left(\min_{\{\alpha_i\}} \sum \left| R_i - \hat{B}_1 \alpha_i \right|^2 \right) \quad , \quad |\hat{B}_1| = 1$$

$$\alpha_i = R_i \cdot \hat{B}_1$$

Principal Component Analysis

Simplified for people who haven't seen PCA before



$\bar{V}$

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$

$$R_i = V_i - \bar{V}$$

$$\hat{B}_1 = \underset{\hat{B}_1}{\operatorname{argmin}} \left(\min_{\{\alpha_i\}} \sum \left| R_i - \hat{B}_1 \alpha_i \right|^2 \right) \quad , \quad |\hat{B}_1| = 1$$

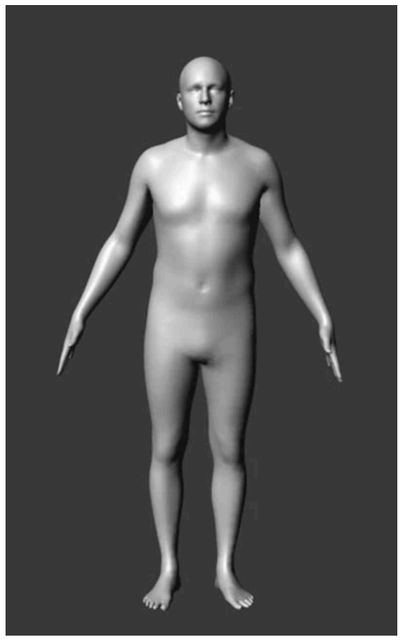
$$= \underset{\hat{B}_1}{\operatorname{argmax}} \left(\sum \left| R_i \cdot \hat{B}_1 \right|^2 \right) \quad , \quad |\hat{B}_1| = 1$$

Principal Component Analysis

Simplified for people who haven't seen PCA before

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$

$$R_i = V_i - \bar{V}$$



$$\bar{V}$$

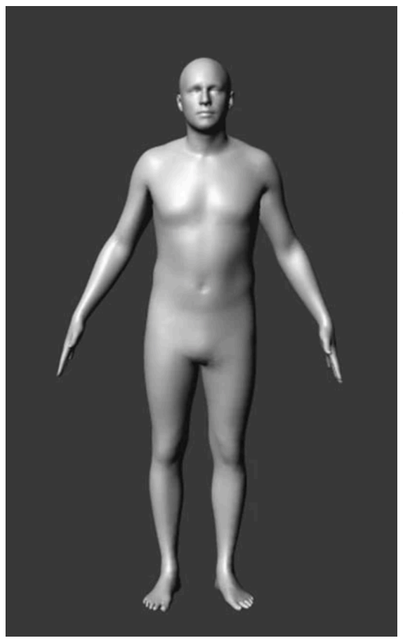
$$\hat{B}_1 = \underset{\hat{B}_1}{\operatorname{argmin}} \left(\min_{\{\alpha_i\}} \sum \left| R_i - \hat{B}_1 \alpha_i \right|^2 \right) \quad , \quad |\hat{B}_1| = 1$$

$$= \underset{\hat{B}_1}{\operatorname{argmax}} \left(\sum \left| R_i \cdot \hat{B}_1 \right|^2 \right) \quad , \quad |\hat{B}_1| = 1$$

How does this turn into this?

Principal Component Analysis

Simplified for people who haven't seen PCA before



$\bar{V}$

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$

$$R_i = V_i - \bar{V}$$

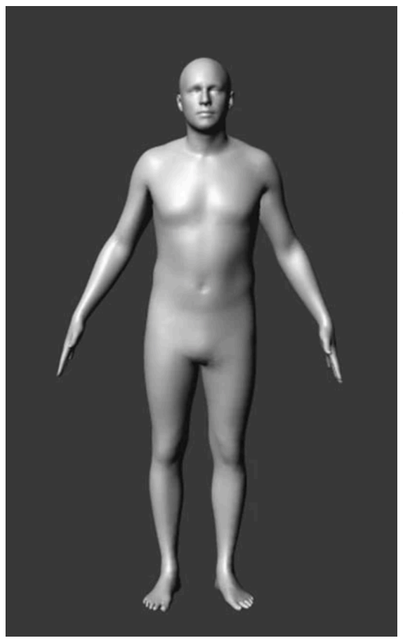
$$\hat{B}_1 = \underset{\hat{B}_1}{\operatorname{argmin}} \left(\min_{\{\alpha_i\}} \sum \left| R_i - \hat{B}_1 \alpha_i \right|^2 \right) \quad , \quad |\hat{B}_1| = 1$$

$$= \underset{\hat{B}_1}{\operatorname{argmax}} \left(\sum \left| R_i \cdot \hat{B}_1 \right|^2 \right) \quad , \quad |\hat{B}_1| = 1$$

$$= \underset{\hat{B}_1}{\operatorname{argmax}} \left(\sum \left| \hat{B}_1^{\mathsf{T}} \cdot R_i^{\mathsf{T}} \cdot R_i \cdot \hat{B}_1 \right| \right) \quad , \quad |\hat{B}_1| = 1$$

Principal Component Analysis

Simplified for people who haven't seen PCA before



$\bar{V}$

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$

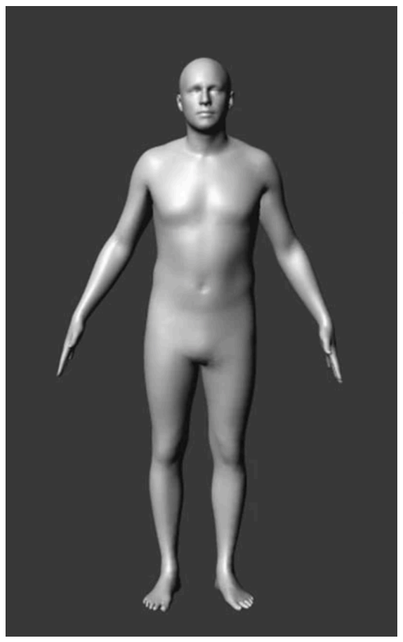
$$R_i = V_i - \bar{V}$$

$$\hat{B}_1 = \underset{\hat{B}_1}{\text{argmax}} \left(\sum \left| \hat{B}_1^{\mathsf{T}} \cdot R_i^{\mathsf{T}} \cdot R_i \cdot \hat{B}_1 \right| \right) \quad , \quad |\hat{B}_1| = 1$$

$$= \underset{\hat{B}_1}{\text{argmax}} \left(\hat{B}_1^{\mathsf{T}} \mathbf{R}^{\mathsf{T}} \mathbf{R} \hat{B}_1 \right) \quad , \quad |\hat{B}_1| = 1$$

Principal Component Analysis

Simplified for people who haven't seen PCA before



$\bar{V}$

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$

$$R_i = V_i - \bar{V}$$

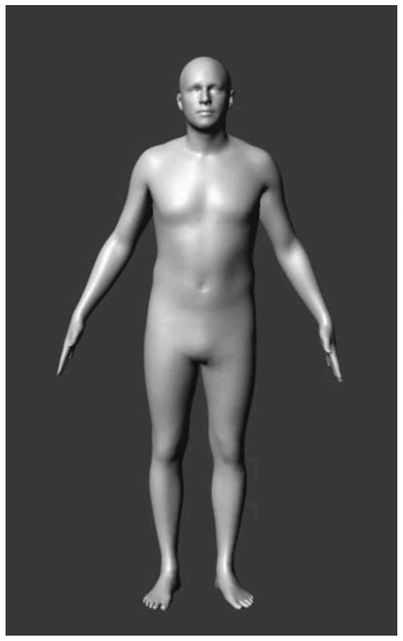
$$\hat{B}_1 = \underset{\hat{B}_1}{\text{argmax}} \left(\sum \left| \hat{B}_1^{\mathsf{T}} \cdot R_i^{\mathsf{T}} \cdot R_i \cdot \hat{B}_1 \right| \right) \quad , \quad |\hat{B}_1| = 1$$

$$= \underset{\hat{B}_1}{\text{argmax}} \left(\hat{B}_1^{\mathsf{T}} \mathbf{R}^{\mathsf{T}} \mathbf{R} \hat{B}_1 \right) \quad , \quad |\hat{B}_1| = 1$$

Thus $\hat{B}_1$ must be the eigenvector of $\mathbf{M} = \mathbf{R}^{\mathsf{T}}\mathbf{R}$ with largest eigenvalue!

Principal Component Analysis

Simplified for people who haven't seen PCA before



$\bar{V}$

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$

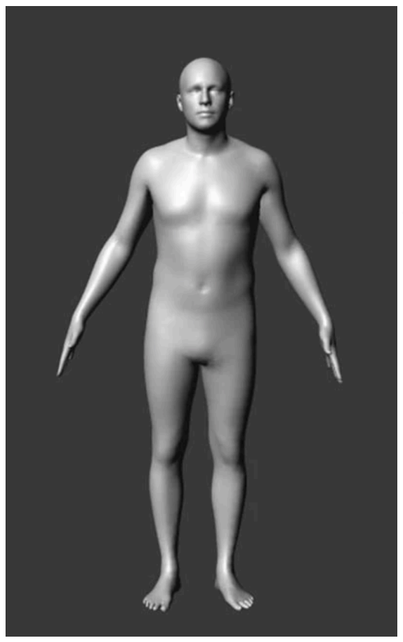
$$R_i = V_i - \bar{V}$$

$$R_i' = R_i - \hat{B}_1\alpha_i \quad = R_i - \hat{B}_1(R_i \cdot \hat{B}_1)$$

Principal Component Analysis

Simplified for people who haven't seen PCA before

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$



$\bar{V}$

$$R_i = V_i - \bar{V}$$

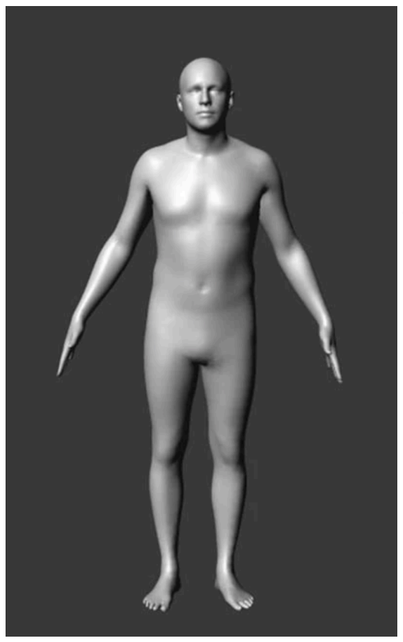
$$R_i' = R_i - \hat{B}_1\alpha_i \quad = R_i - \hat{B}_1(R_i \cdot \hat{B}_1)$$

Thus $\hat{B}_2$ must be the eigenvector of $\mathbf{R'}^{\mathsf{T}}\mathbf{R'}$ with largest eigenvalue!

Principal Component Analysis

Simplified for people who haven't seen PCA before

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$



$\bar{V}$

$$\bar{V} = \frac{1}{n}\sum_{i=1}^{n} V_i$$

$$R_i = V_i - \bar{V}$$

$$R_i' = R_i - \hat{B}_1\alpha_i \quad = R_i - \hat{B}_1(R_i \cdot \hat{B}_1)$$

Thus $\hat{B}_2$ must be the eigenvector of $\mathbf{R'}^{\mathsf{T}}\mathbf{R'}$ with largest eigenvalue!

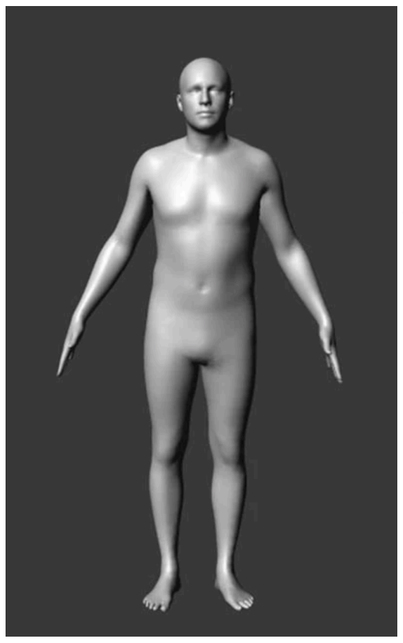
Equivalently:

$\hat{B}_2$ must be the eigenvector of $\mathbf{R}^{\mathsf{T}}\mathbf{R}$ with <u>second</u> largest eigenvalue!

Principal Component Analysis

Simplified for people who haven't seen PCA before

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$



$\bar{V}$

$$\bar{V} = \frac{1}{n}\sum_{i=1}^{n} V_i$$

$$R_i = V_i - \bar{V}$$

$$R_i' = R_i - \hat{B}_1\alpha_i \quad = R_i - \hat{B}_1(R_i \cdot \hat{B}_1)$$

Thus $\hat{B}_2$ must be the eigenvector of $\mathbf{R'}^{\mathsf{T}}\mathbf{R'}$ with largest eigenvalue!

Equivalently:

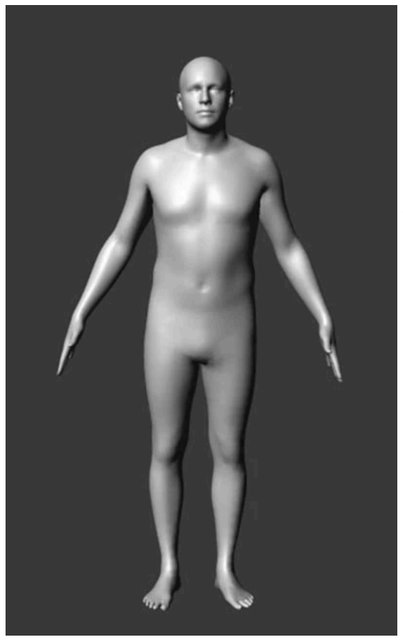
$\hat{B}_2$ must be the eigenvector of $\mathbf{R}^{\mathsf{T}}\mathbf{R}$ with <u>second</u> largest eigenvalue!

We can keep going…

Principal Component Analysis

Simplified for people who haven't seen PCA before

$$\{V_1, V_2, \cdots, V_n\}, V \in \mathbf{R}^{3\times|V|}$$

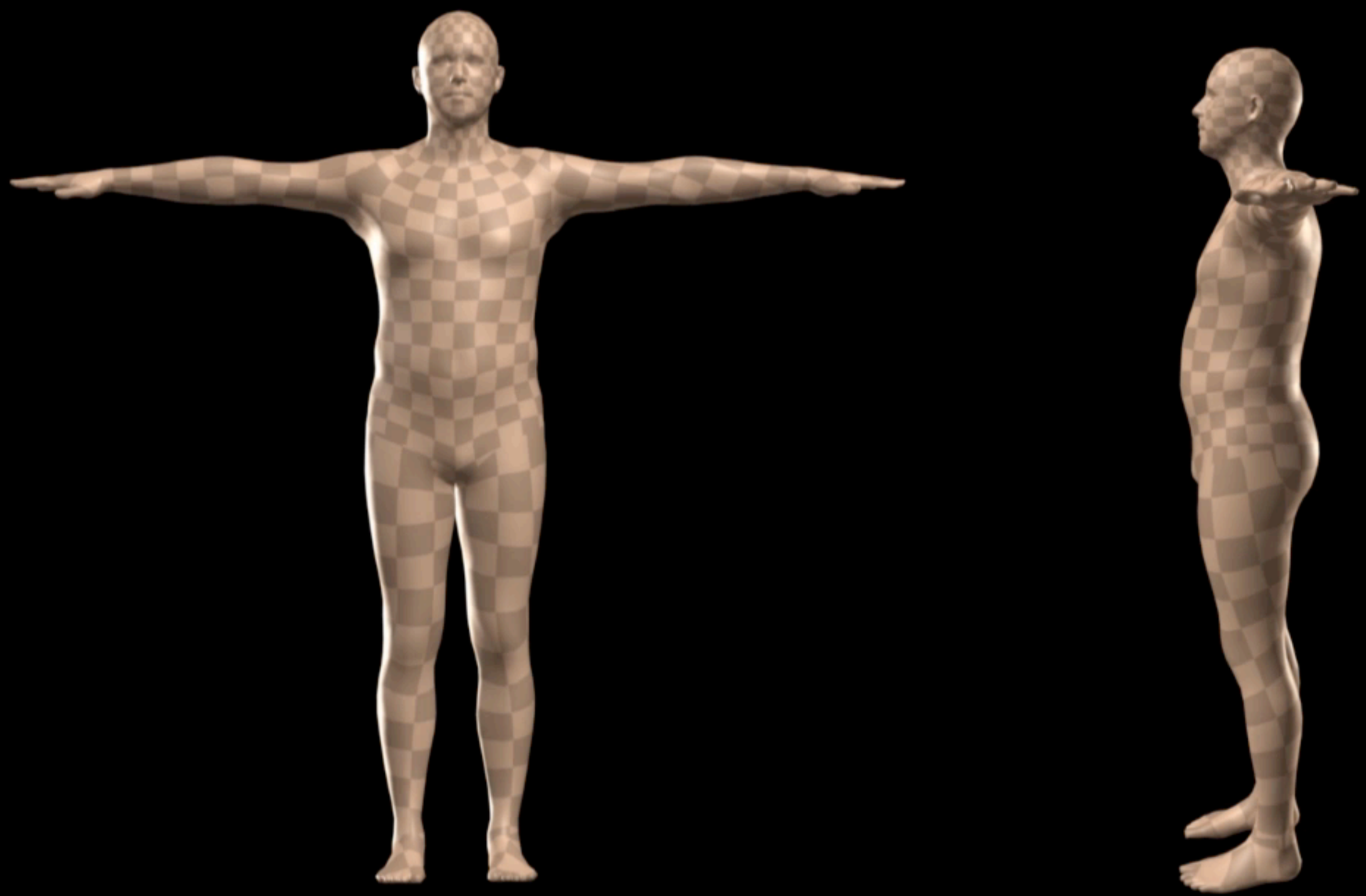


$\bar{V}$

- **The eigenvectors of $\mathbf{R}^\mathsf{T}\mathbf{R}$, $\left\{\hat{B}_i\right\}$, form a complete basis**
  - **Any $V = \bar{V} + \sum \hat{B}_i \beta_i$ for some $\{\beta_i\}$**
- **The first several $\hat{B}_i$ capture most of the interesting variation**
  - **Only need to keep K of them where $K \ll 3\#V$ and $K \ll N$**

Principal components: Male

"Shape" Blend Shapes

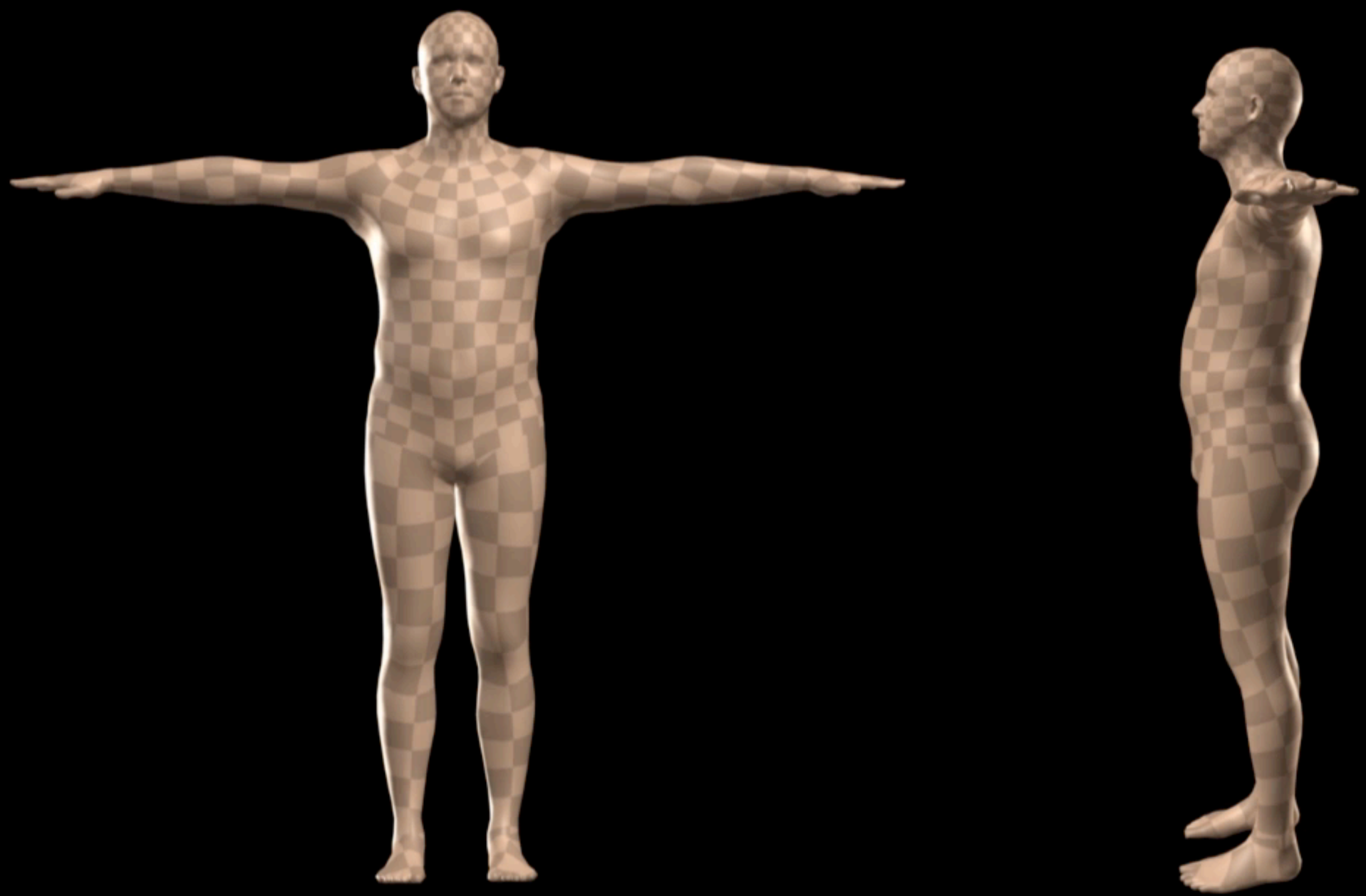


PC 1 varied between +/- 3 std dev

$\hat{B}_i = \mathrm{PC}i$

Principal components: Male

"Shape" Blend Shapes

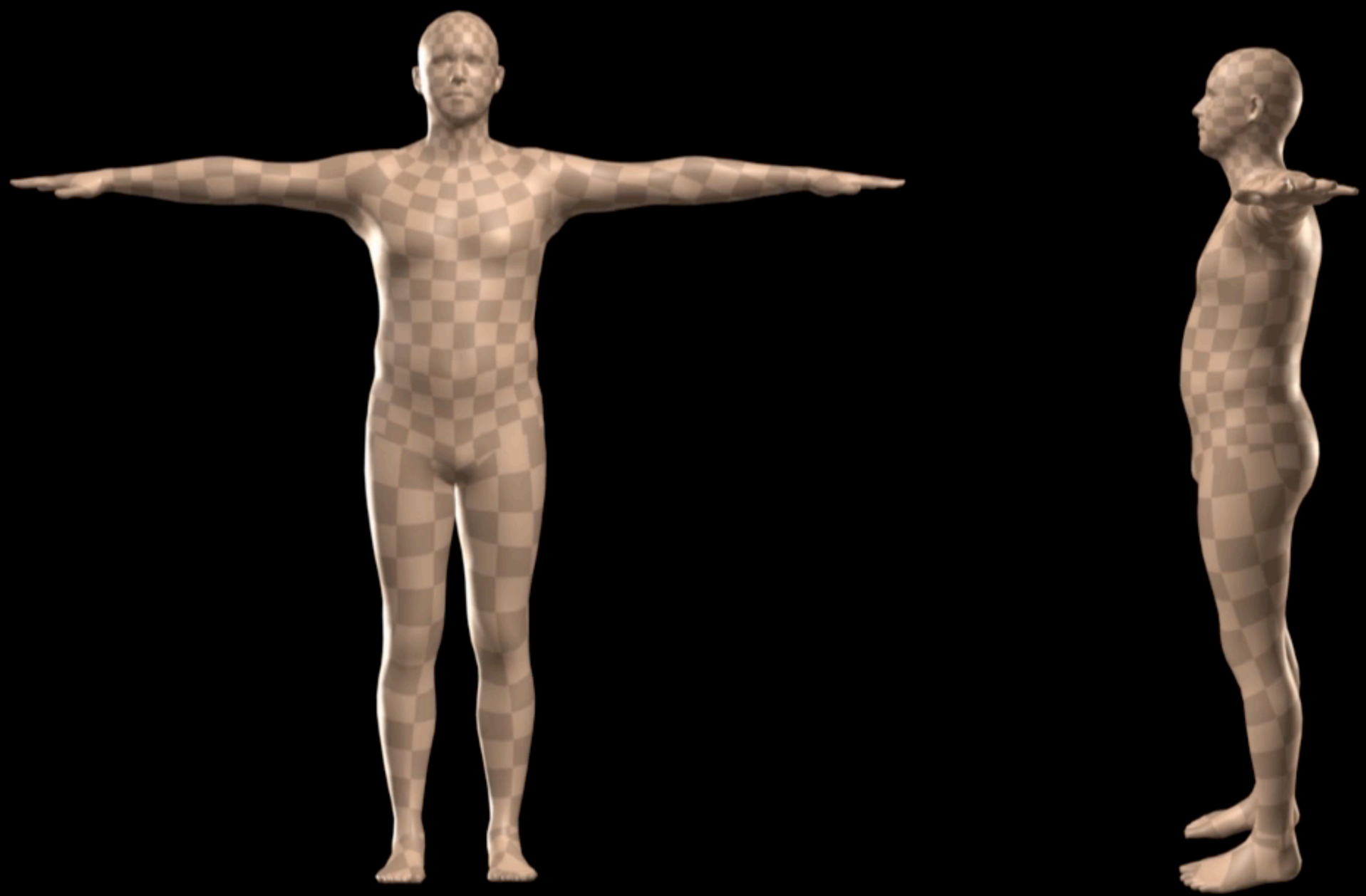


PC 1 varied between +/- 3 std dev

$\hat{B}_i = \mathrm{PC}i$

Principal components: Male

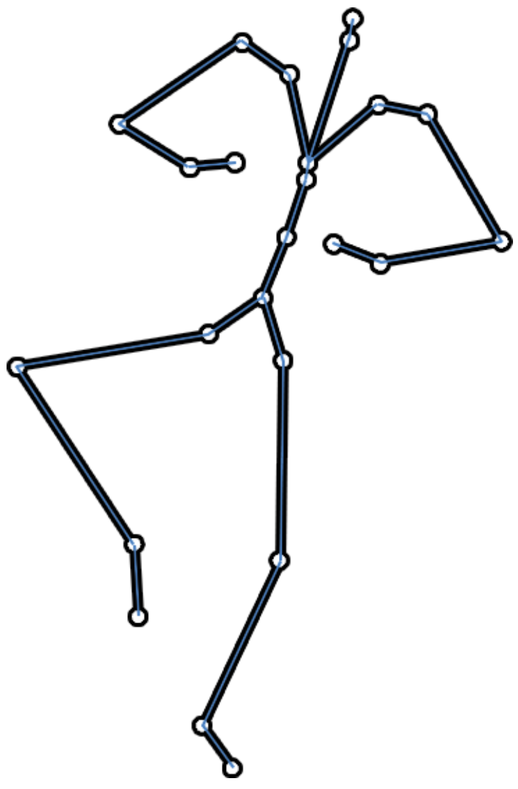
"Shape" Blend Shapes

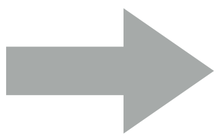


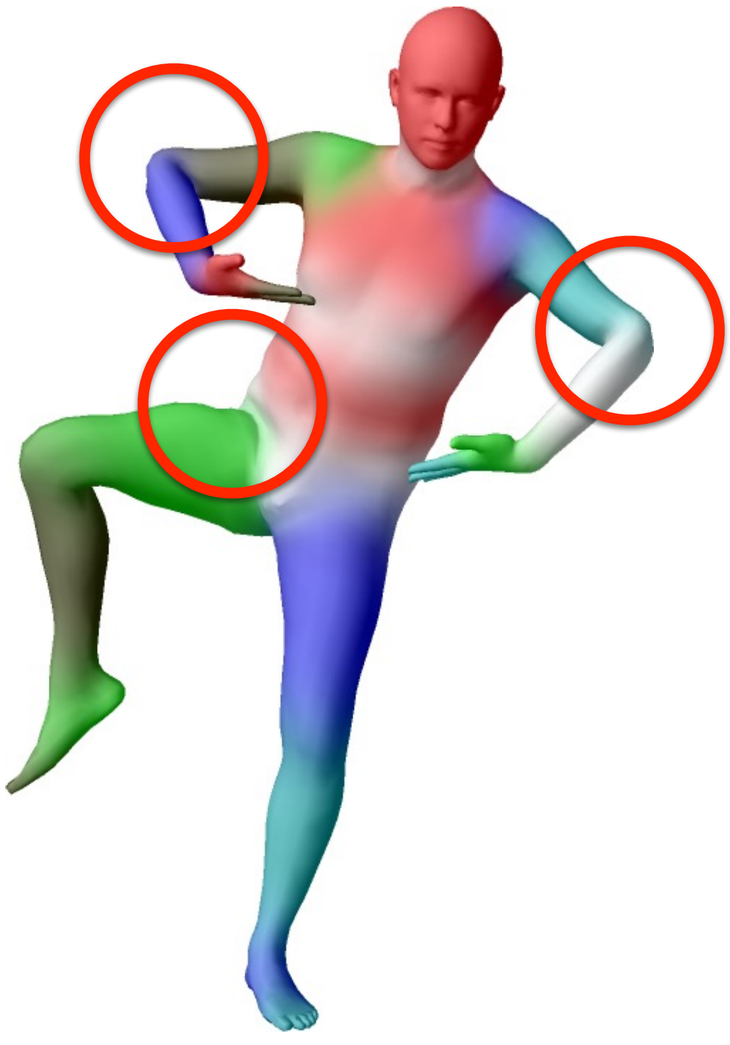
PC 1 varied between +/- 3 std dev

$\hat{B}_i = \mathrm{PC}i$

Pose with kinematics + Linear Blend Skinning



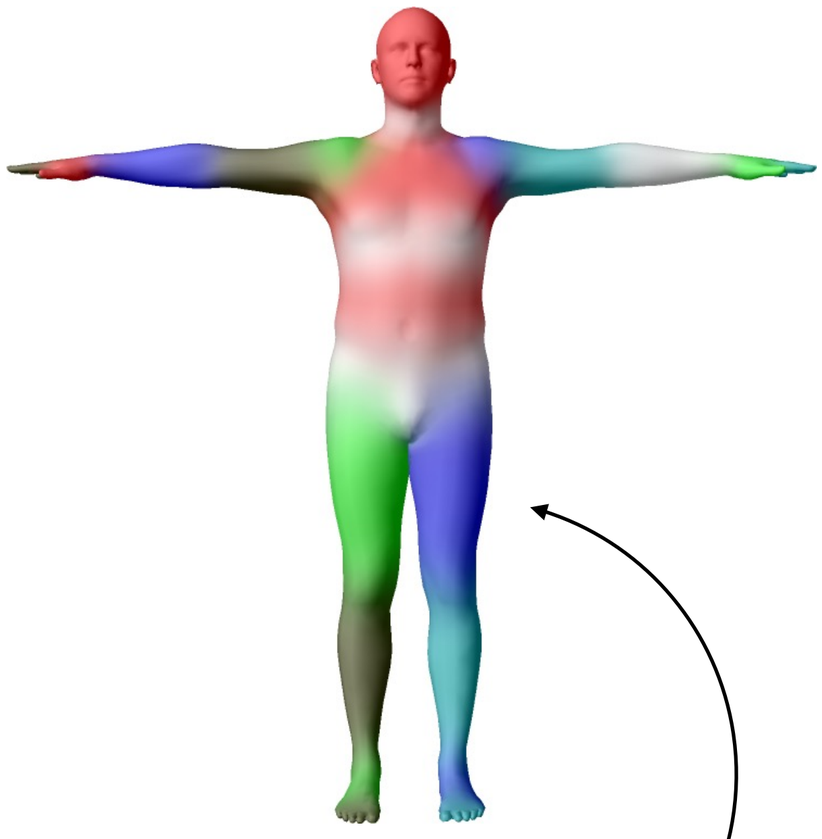


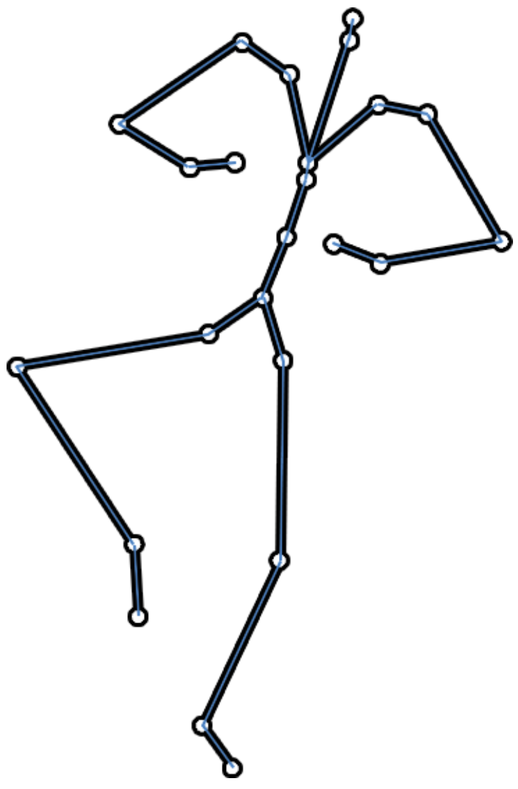


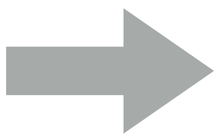
User Input:
pose as relative rotation

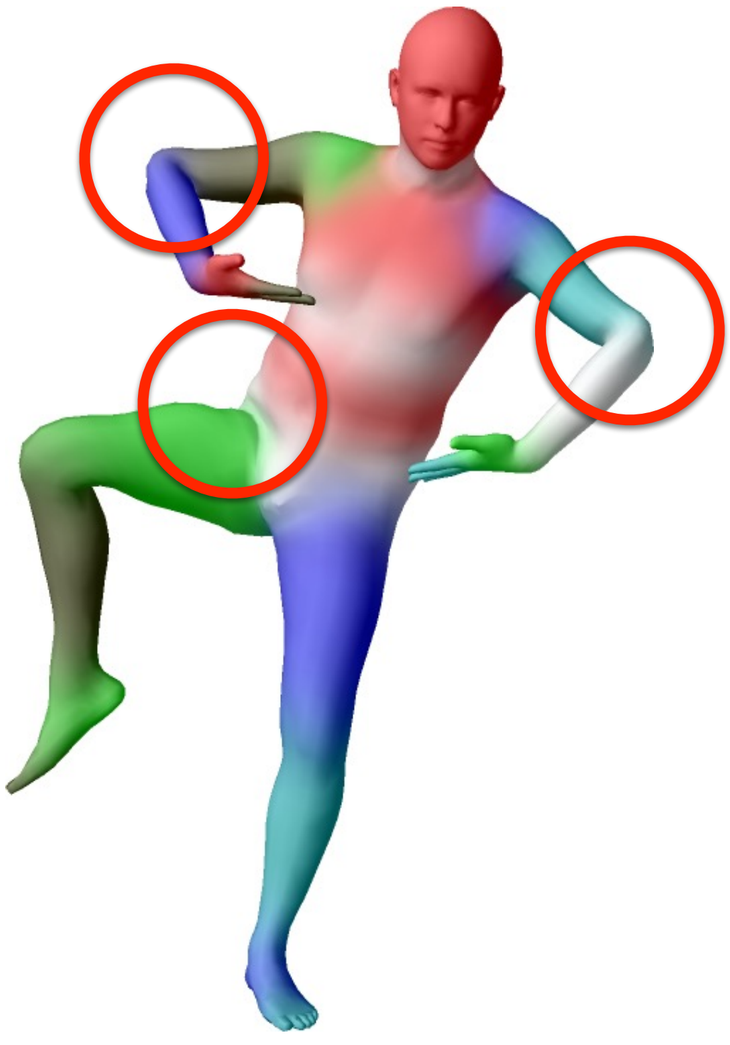
$$\theta \in \mathbb{R}^{3 \times |B|}$$

Pose with kinematics + Linear Blend Skinning







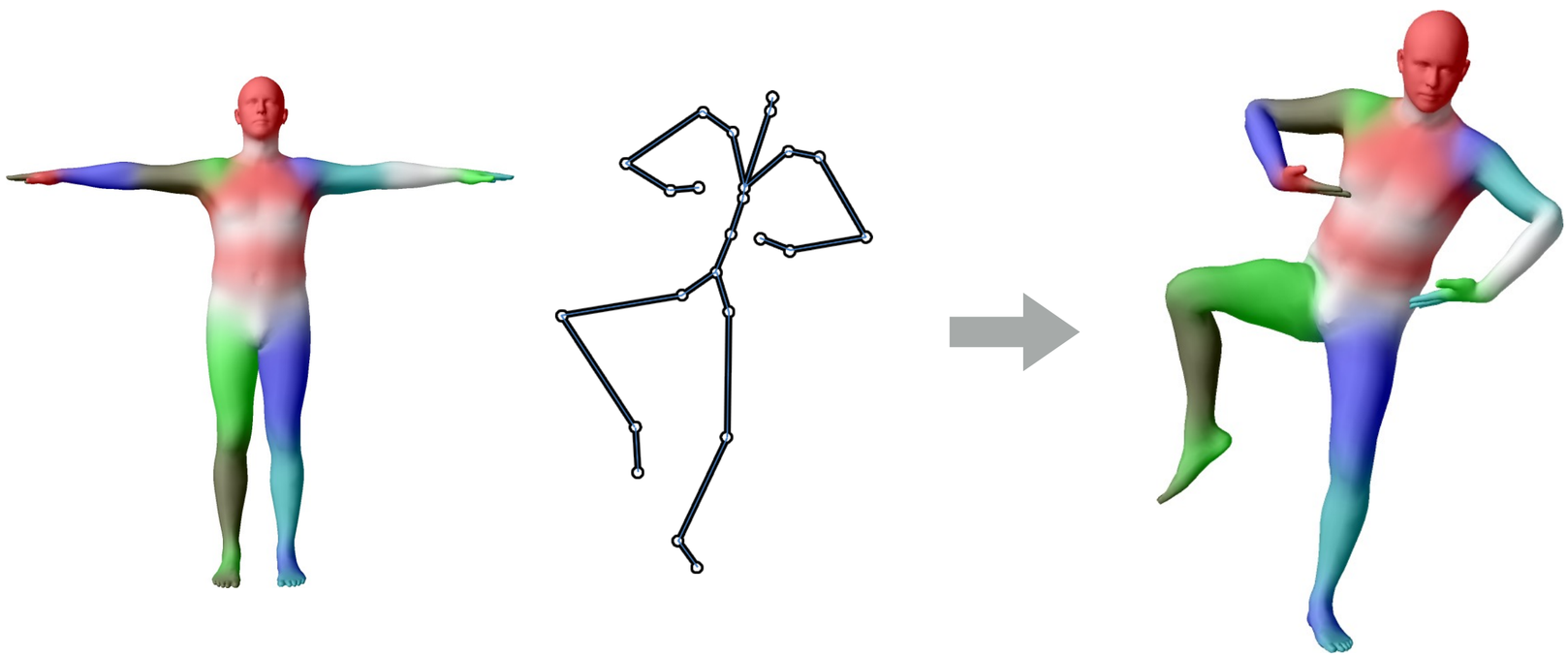


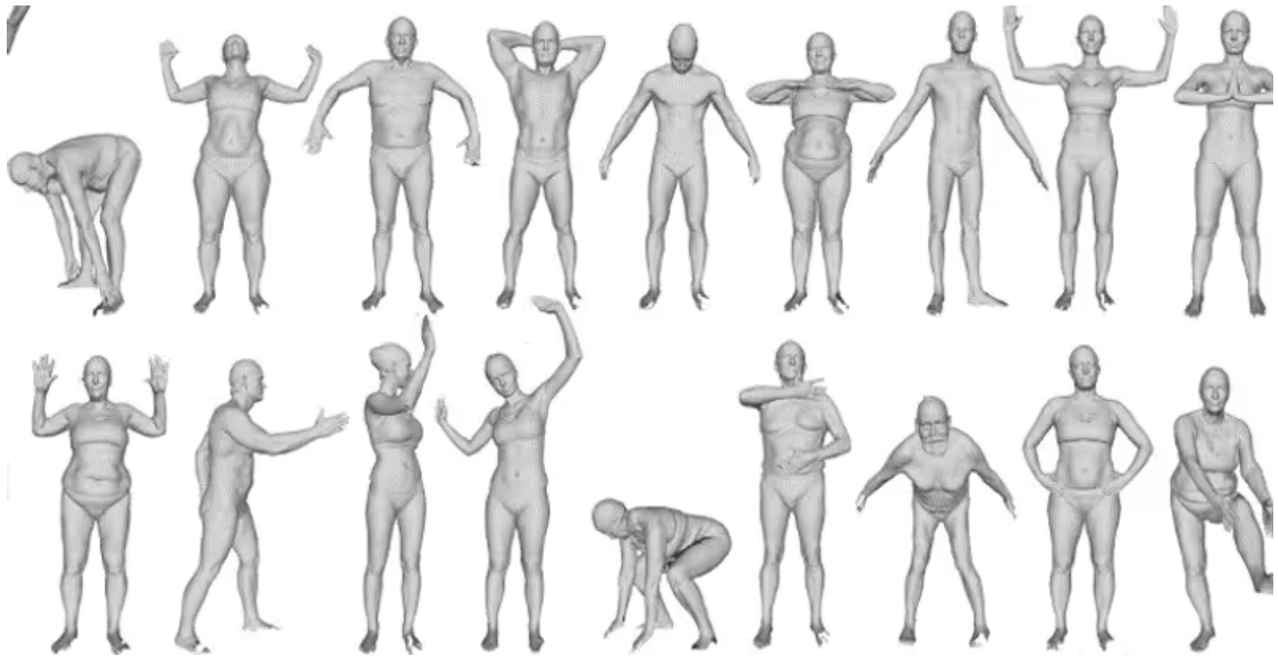
Where did blend weights come from?

User Input:
pose as relative rotation

$$\theta \in \mathbb{R}^{3 \times |B|}$$

Pose with kinematics + Linear Blend Skinning





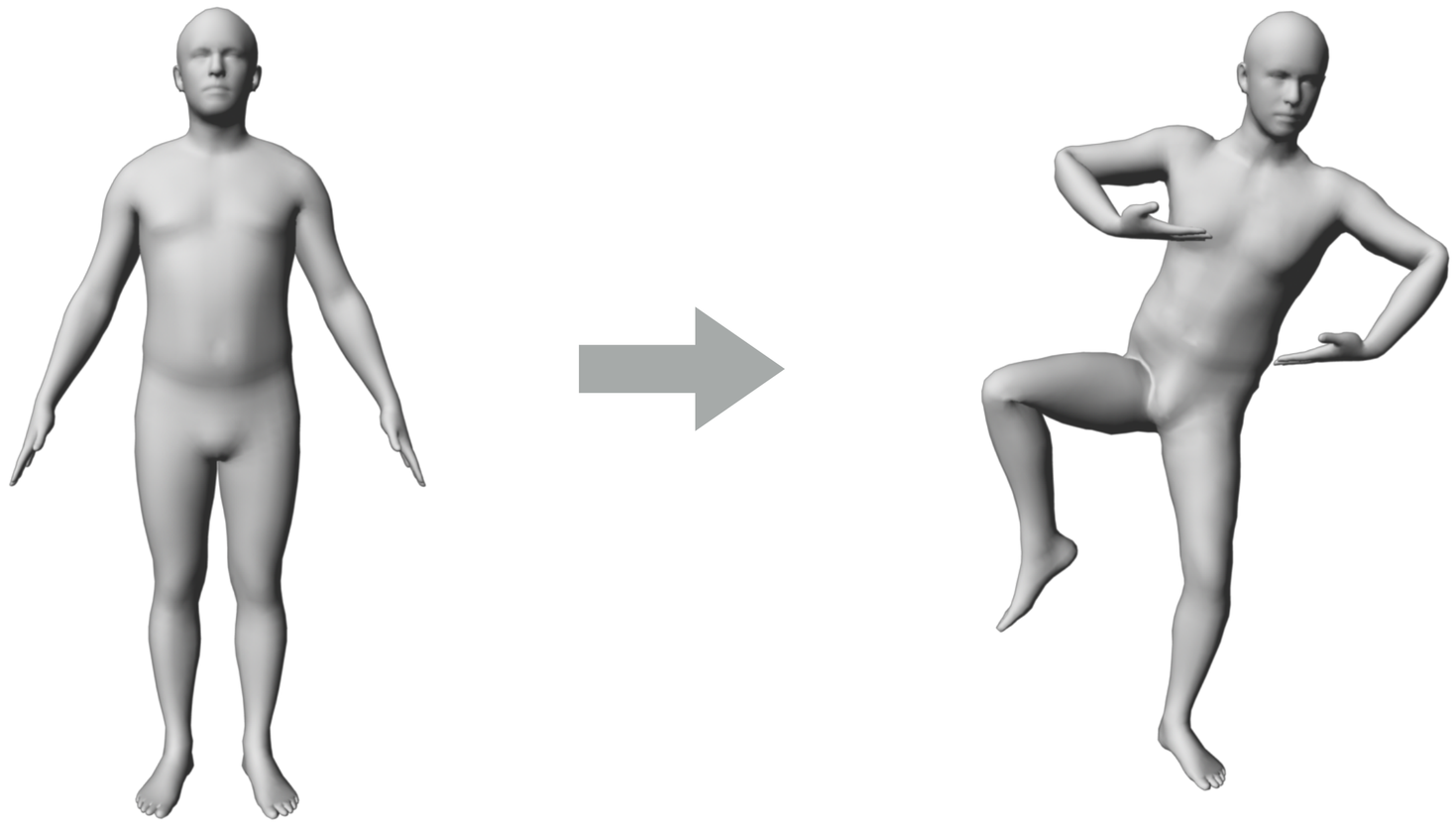
Bodies scanned with different poses

- **Adjust LBS weights to match data**
  - **Match skeleton to scan pose**
  - **Compute skin with LBS**
  - **Error is difference between scan and LBS**

Corrective Blend Shapes

Blend Shape: a set of vertex displacements in rest pose.

Corrective Blend Shape: Offsets as a function of pose, so after LBS it looks good.

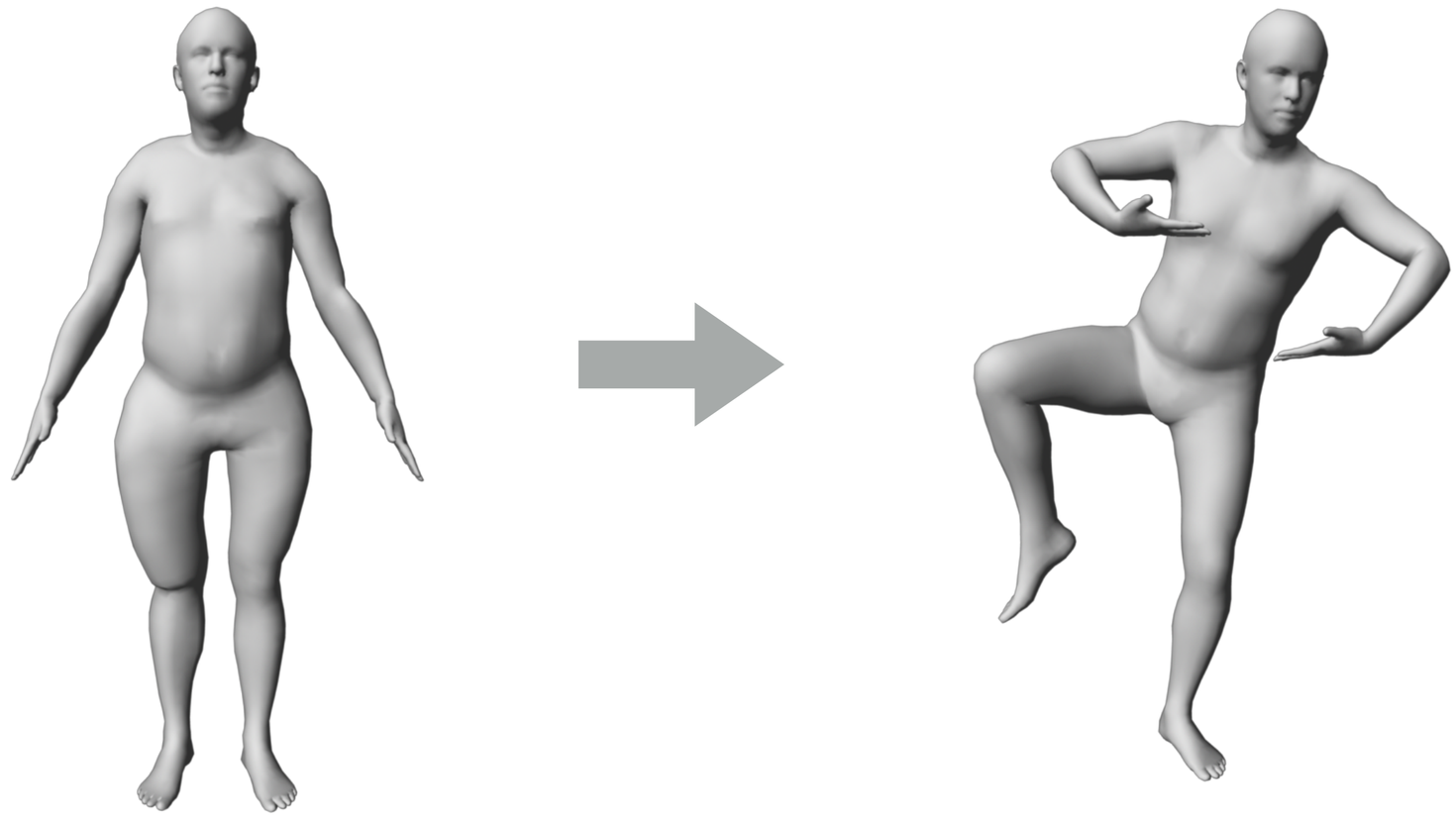


“Pose” blend shapes, bc if fixes issues with posing

Corrective Blend Shapes

Blend Shape: a set of vertex displacements in rest pose.

Corrective Blend Shape: Offsets as a function of pose, so after LBS it looks good.

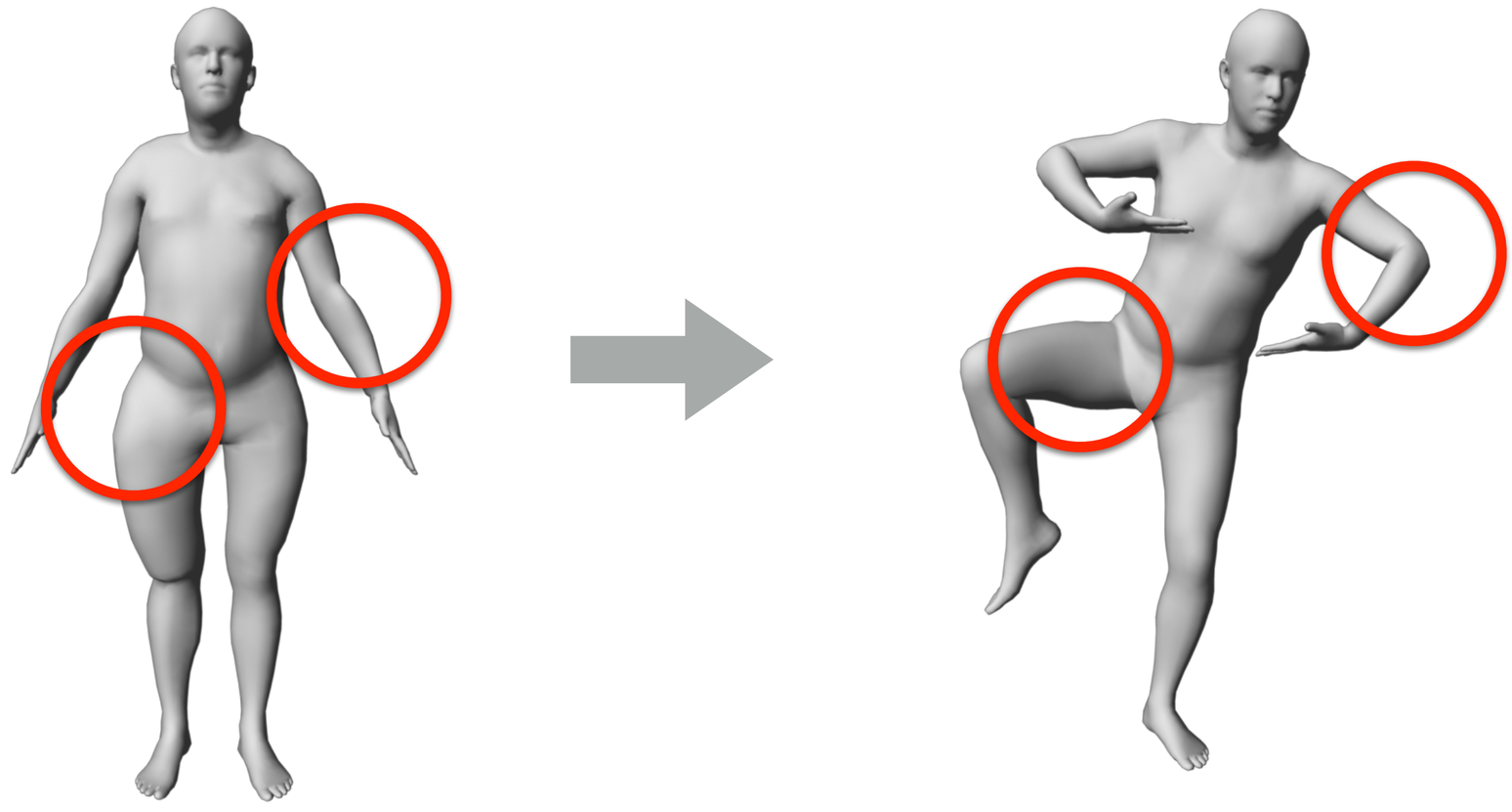


“Pose” blend shapes, bc if fixes issues with posing

Corrective Blend Shapes

Blend Shape: a set of vertex displacements in rest pose.

Corrective Blend Shape: Offsets as a function of pose, so after LBS it looks good.



“Pose” blend shapes, bc if fixes issues with posing

SMPL Summary

SMPL is a blend skinned model with blend shapes.
Model parameters are learned from data.

Template Mesh

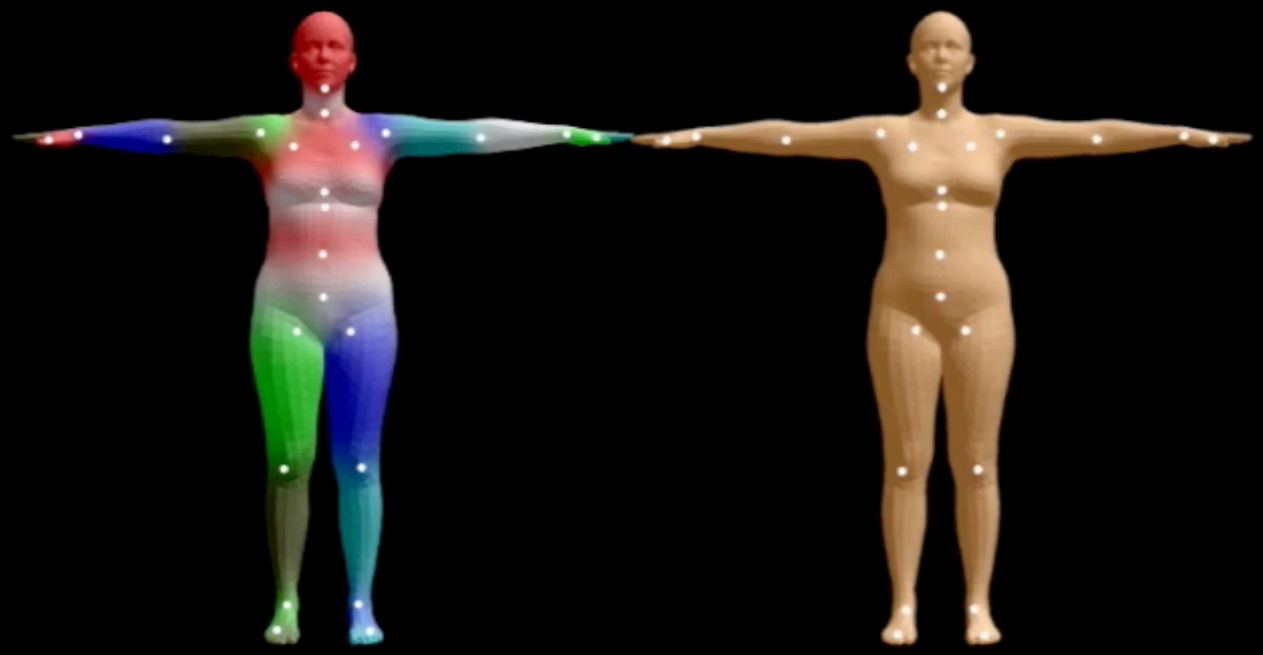
SMPL Summary

SMPL is a blend skinned model with blend shapes.
Model parameters are learned from data.

Template Mesh

SMPL Summary

SMPL is a blend skinned model with blend shapes.
Model parameters are learned from data.

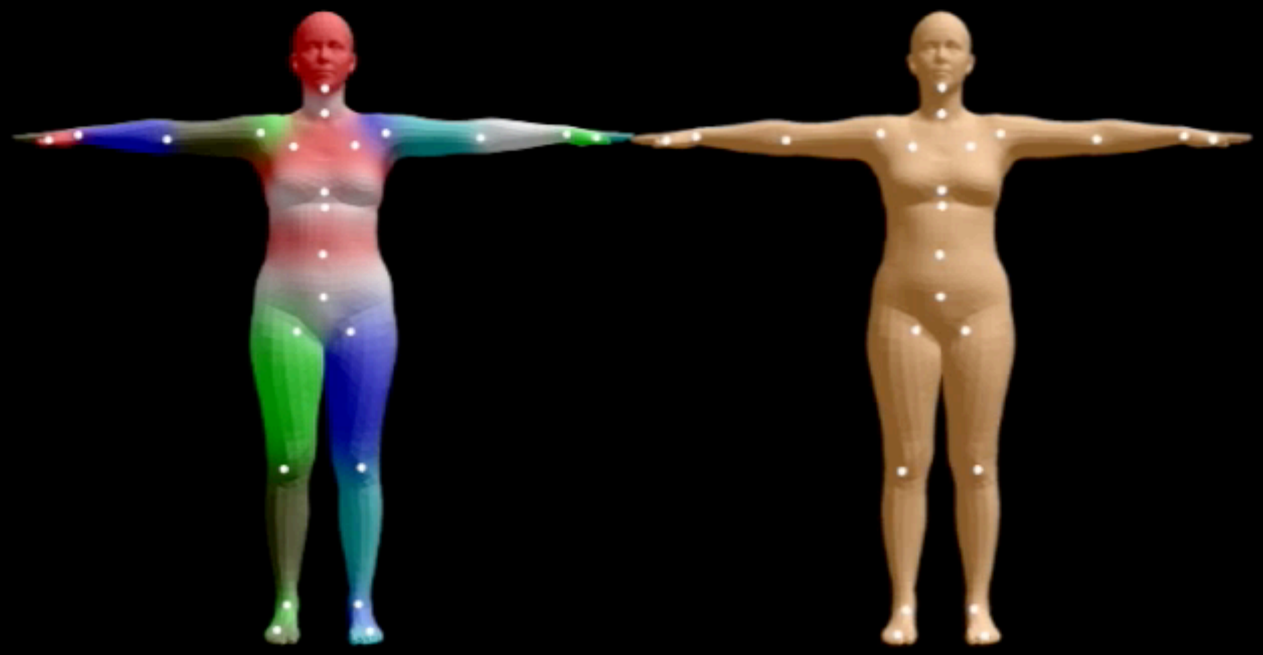


Template Mesh

Shape
Blend Shapes

SMPL Summary

SMPL is a blend skinned model with blend shapes.
Model parameters are learned from data.

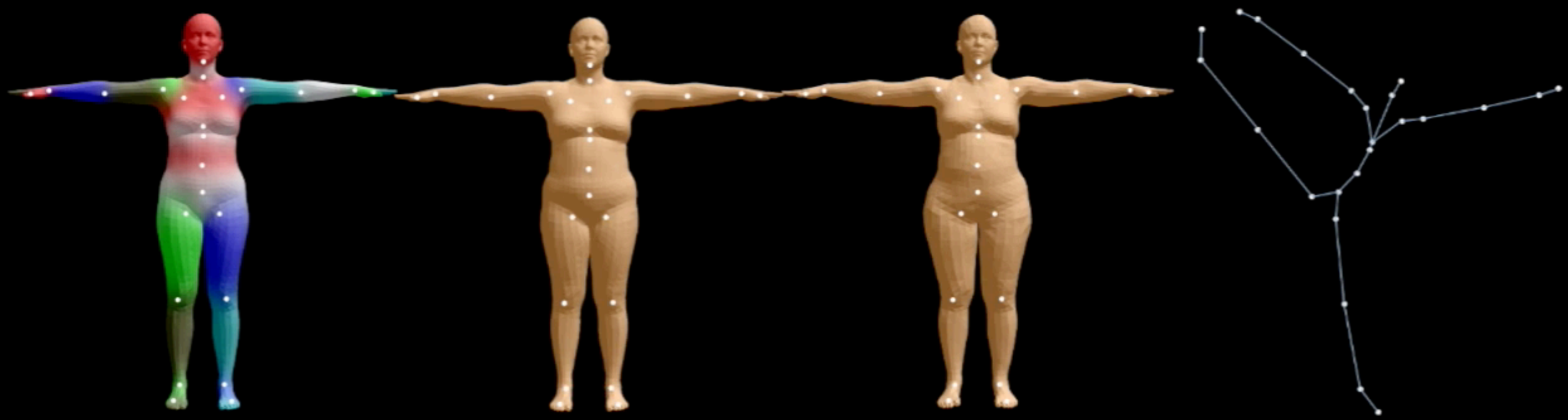


Template Mesh

Shape
Blend Shapes

SMPL Summary

SMPL is a blend skinned model with blend shapes.
Model parameters are learned from data.



Template Mesh

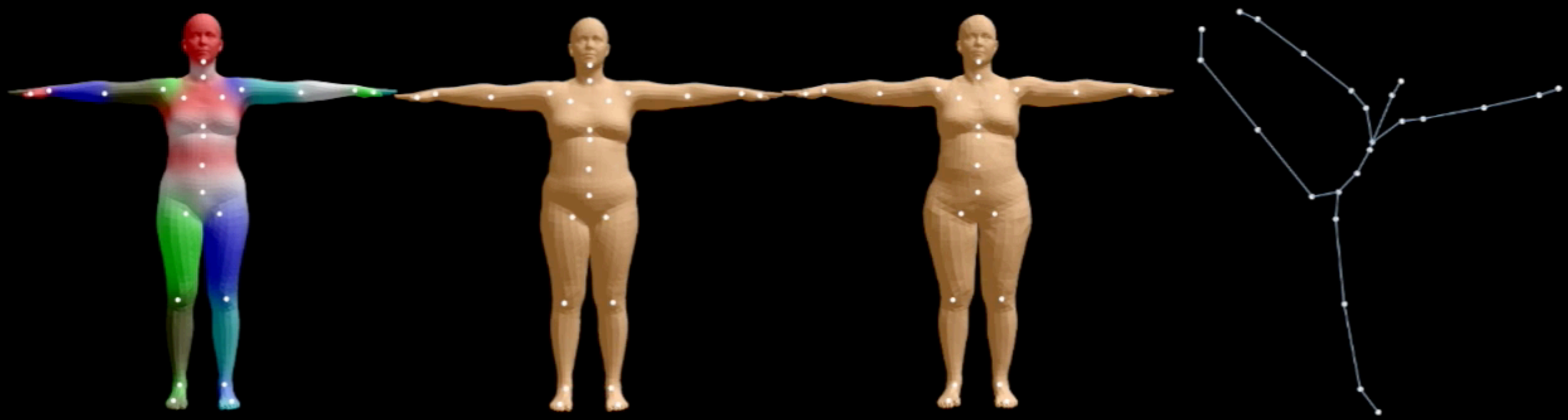
Shape
Blend Shapes

Pose
Blend Shapes

Given Pose

SMPL Summary

SMPL is a blend skinned model with blend shapes.
Model parameters are learned from data.



Template Mesh

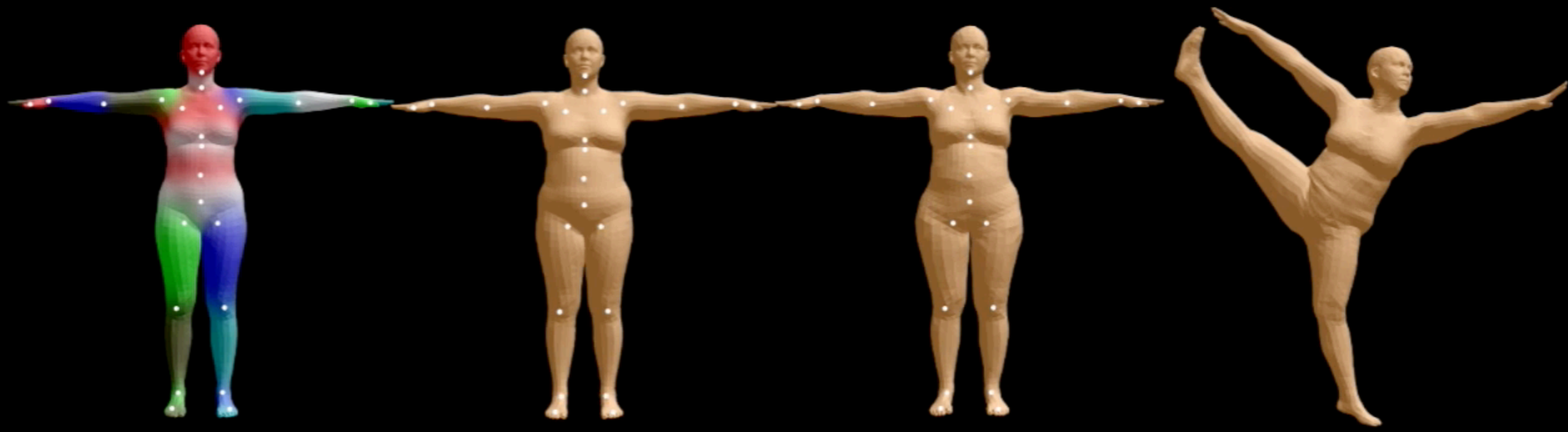
Shape
Blend Shapes

Pose
Blend Shapes

Given Pose

SMPL Summary

SMPL is a blend skinned model with blend shapes.
Model parameters are learned from data.



Template Mesh

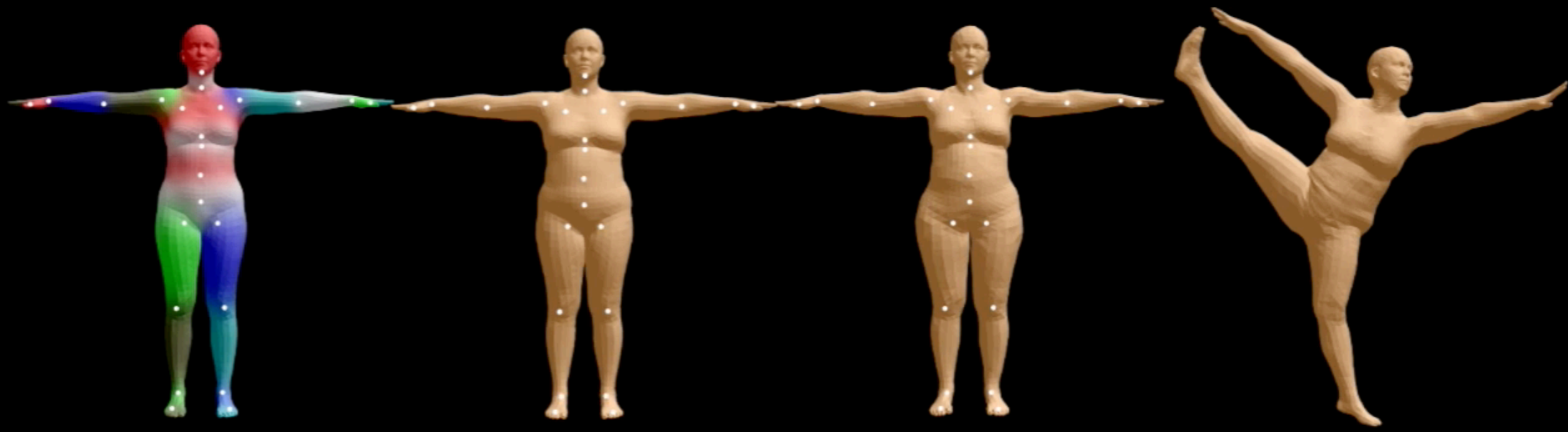
Shape
Blend Shapes

Pose
Blend Shapes

Final Mesh

SMPL Summary

SMPL is a blend skinned model with blend shapes.
Model parameters are learned from data.



Template Mesh

Shape
Blend Shapes

Pose
Blend Shapes

Final Mesh

Inverse Kinematics

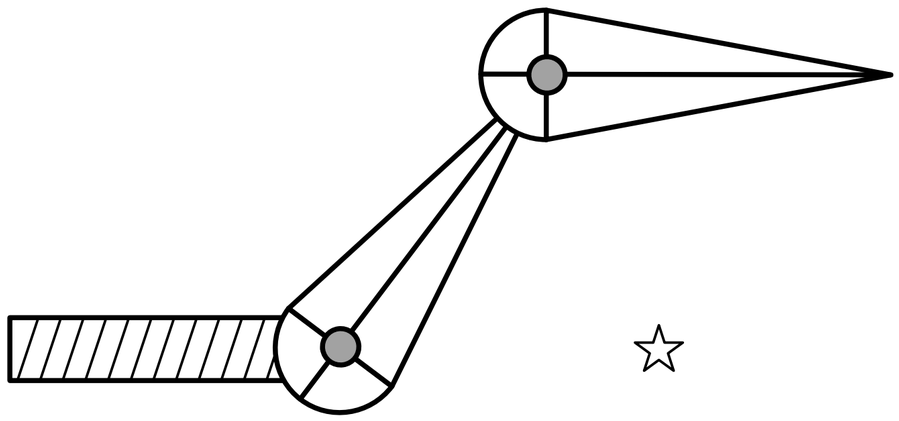
Inverse Kinematics

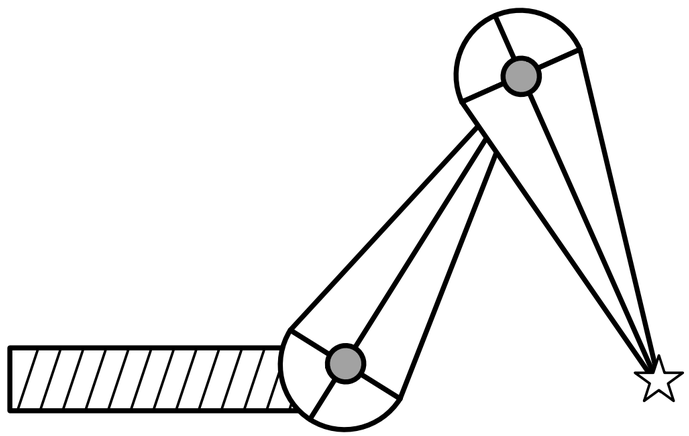
Given

- Root transformation
- Initial configuration
- Desired end point location

Find

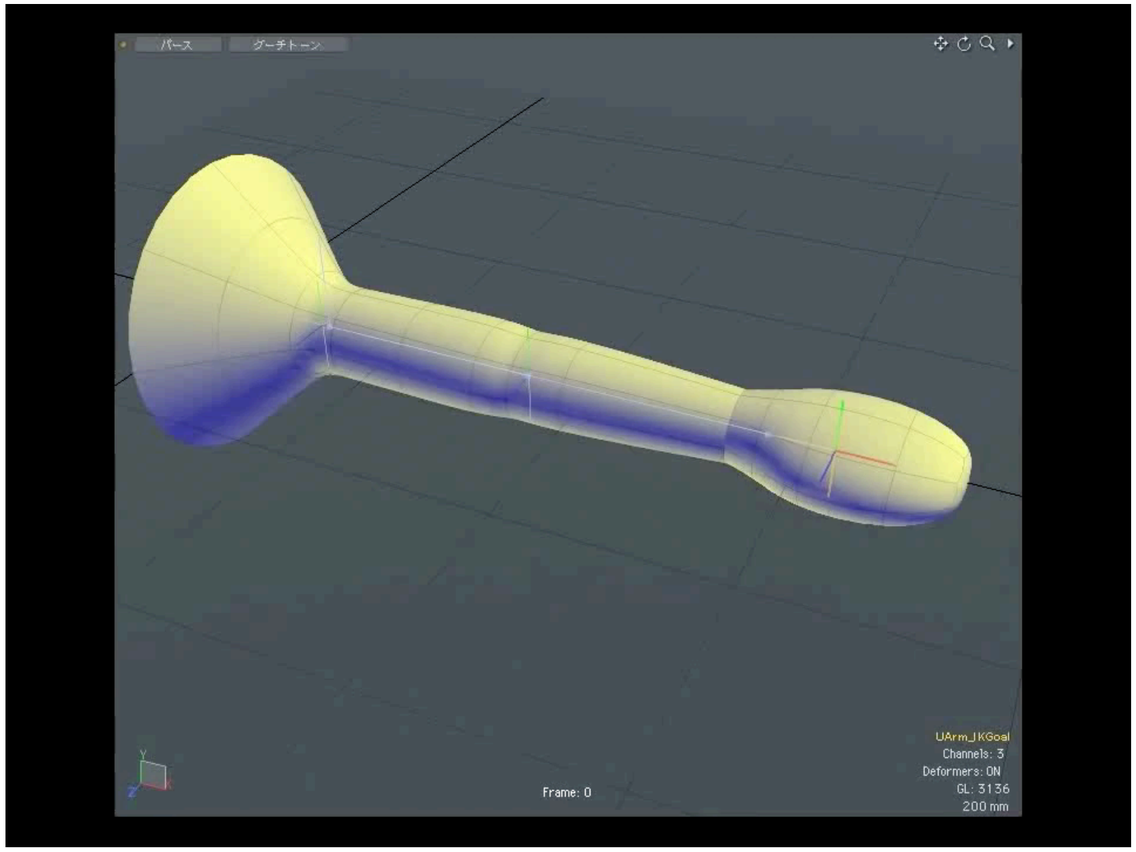
- Interior parameter settings





Inverse Kinematics

Given the end effector position, find the joint angles.

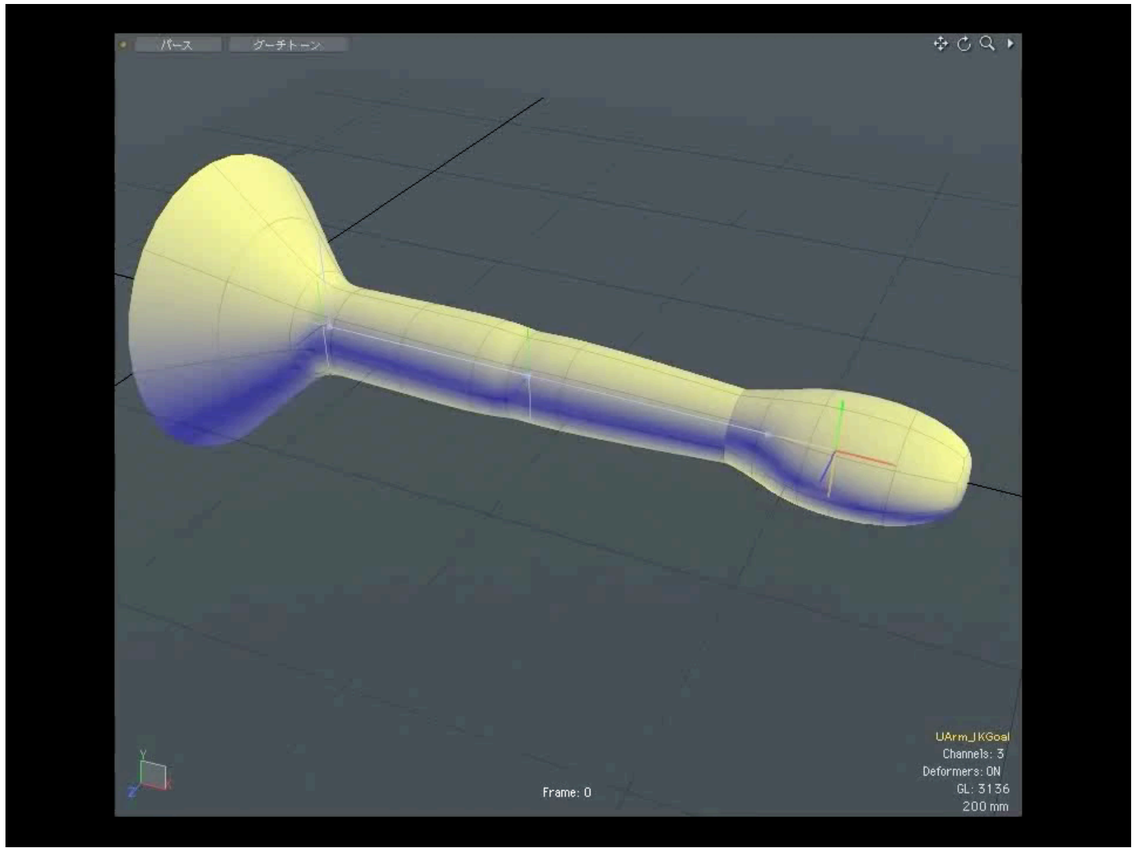


Goals

- Keep end of limb fixed while body moves
- Position end of limb by direct manipulation
- (More general: arbitrary constraints)

Inverse Kinematics

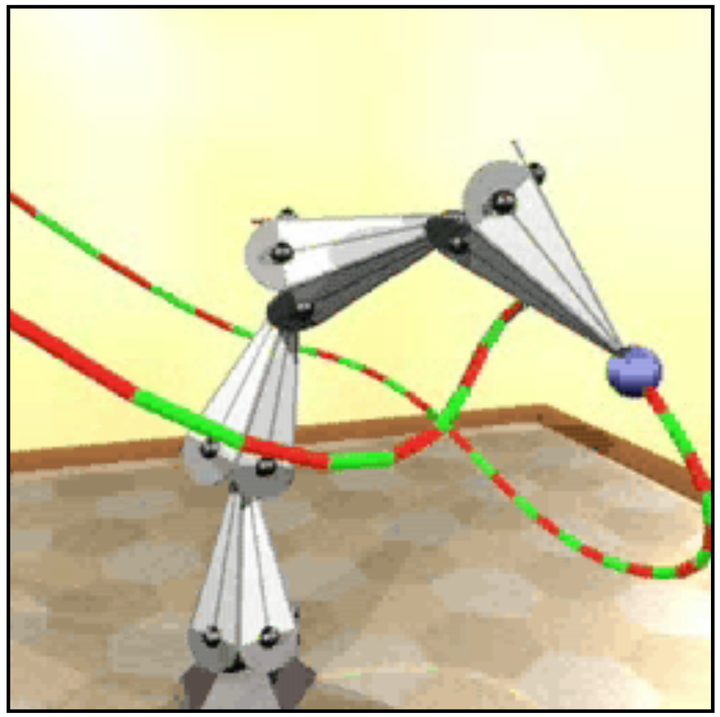
Given the end effector position, find the joint angles.

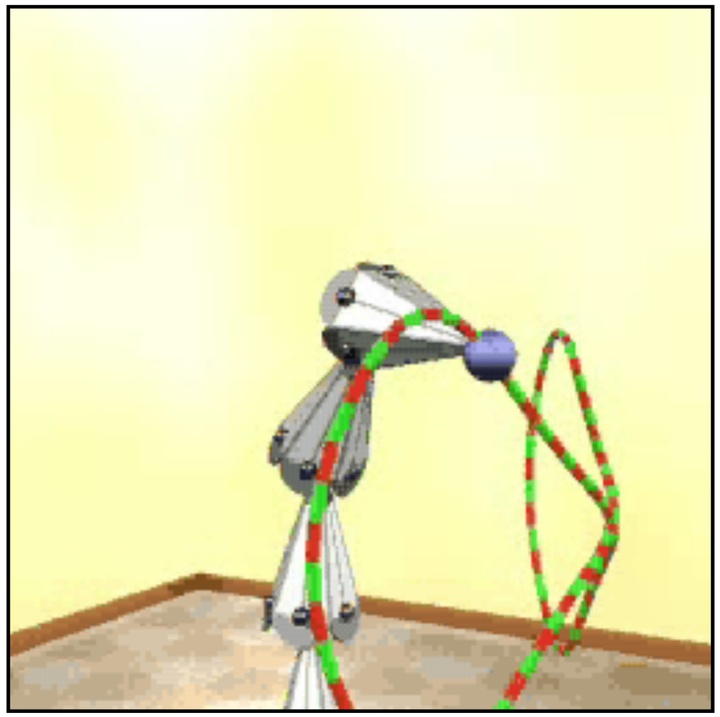


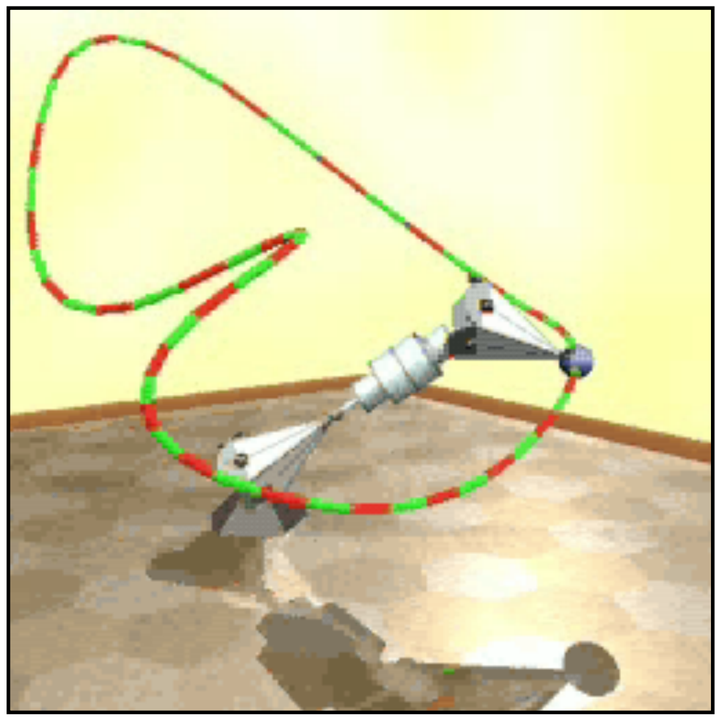
Goals

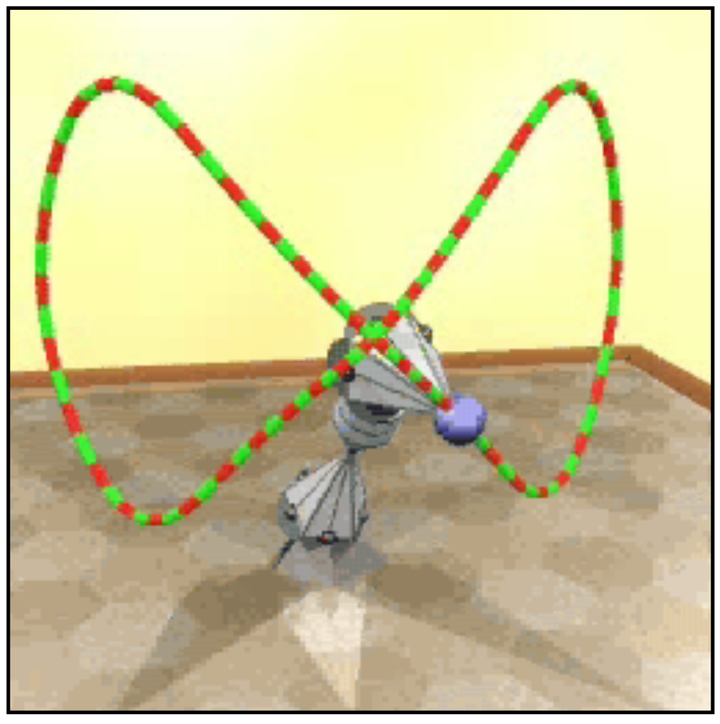
- Keep end of limb fixed while body moves
- Position end of limb by direct manipulation
- (More general: arbitrary constraints)

Inverse Kinematics

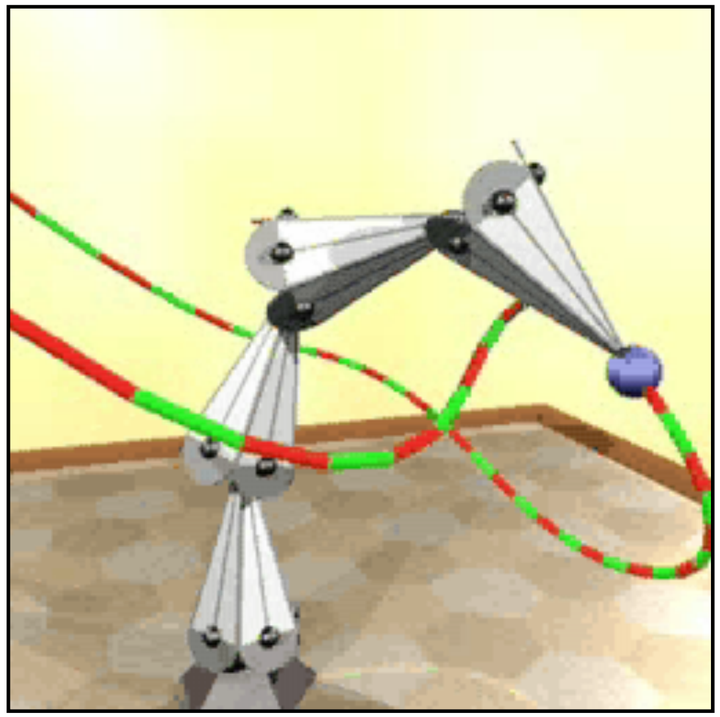


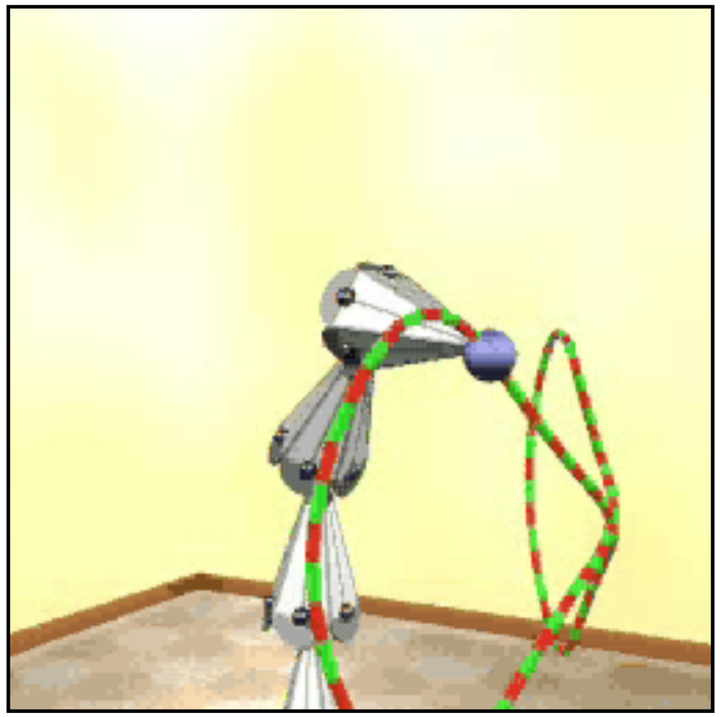


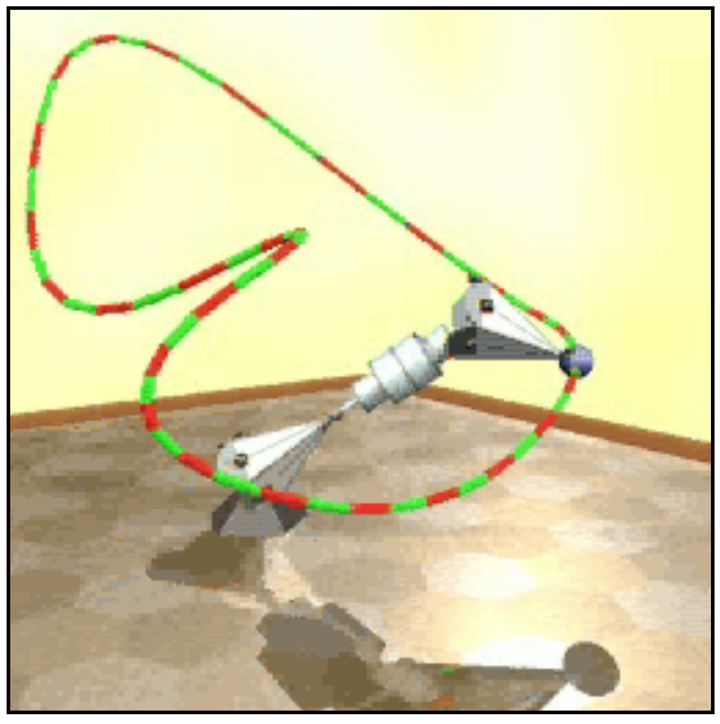


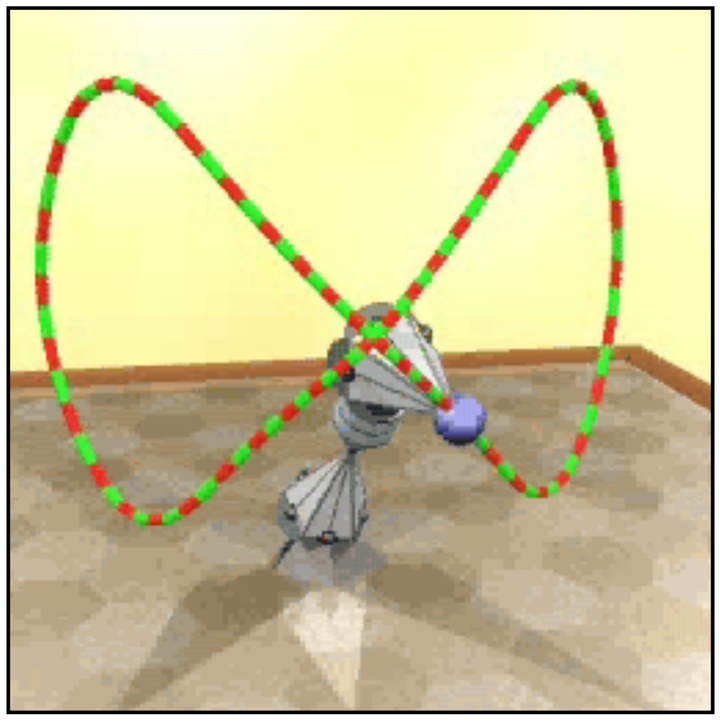


Inverse Kinematics



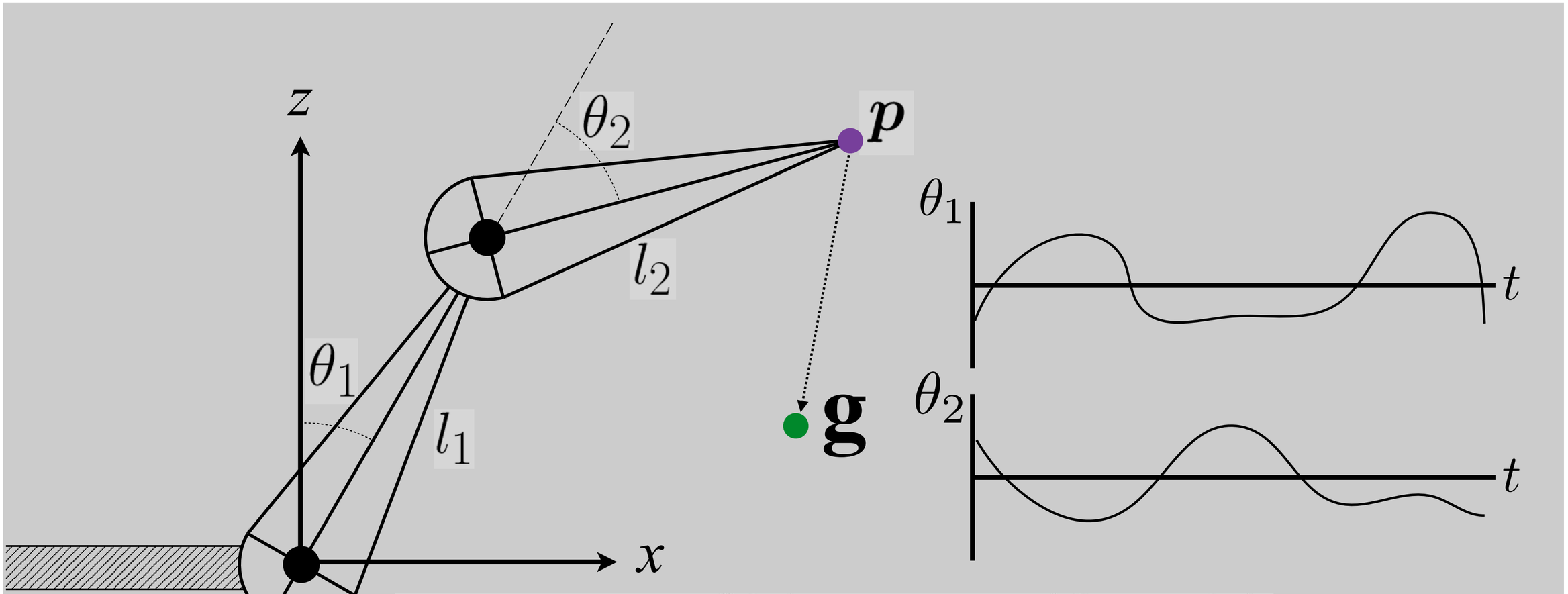






Inverse Kinematics

Simple example forward kinematic function:



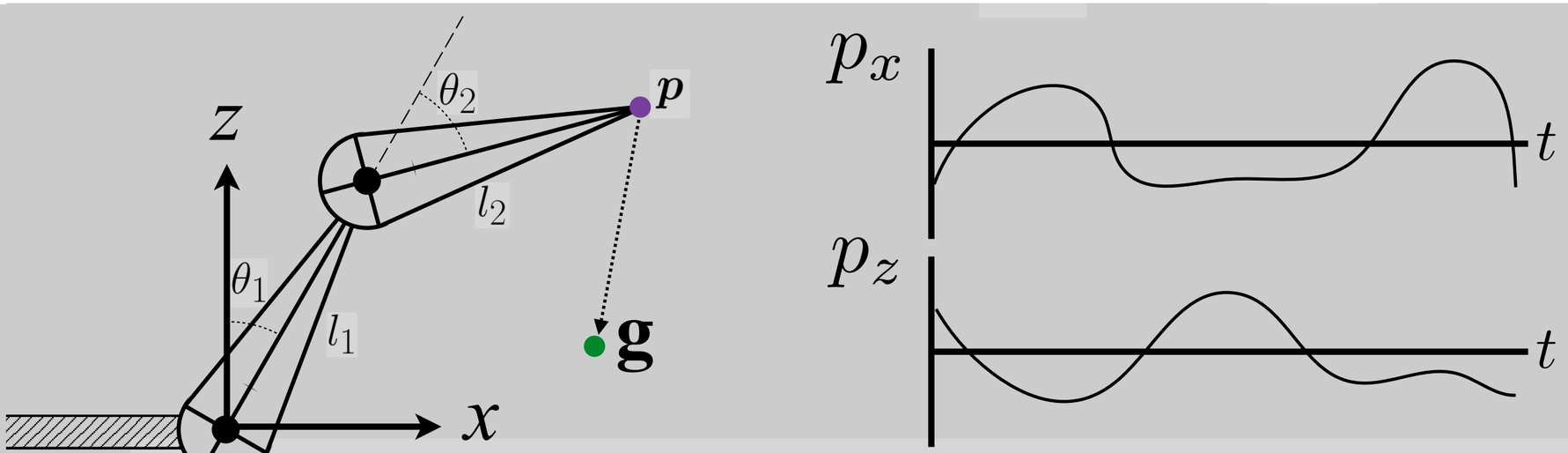
$$p_z = l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2)$$

$$p_x = l_1 \sin(\theta_1) + l_2 \sin(\theta_1 + \theta_2)$$

Warning: Z–up Coordinate System

Inverse Kinematics

Direct inverse kinematics: for two-segment arm, can solve for parameters analytically



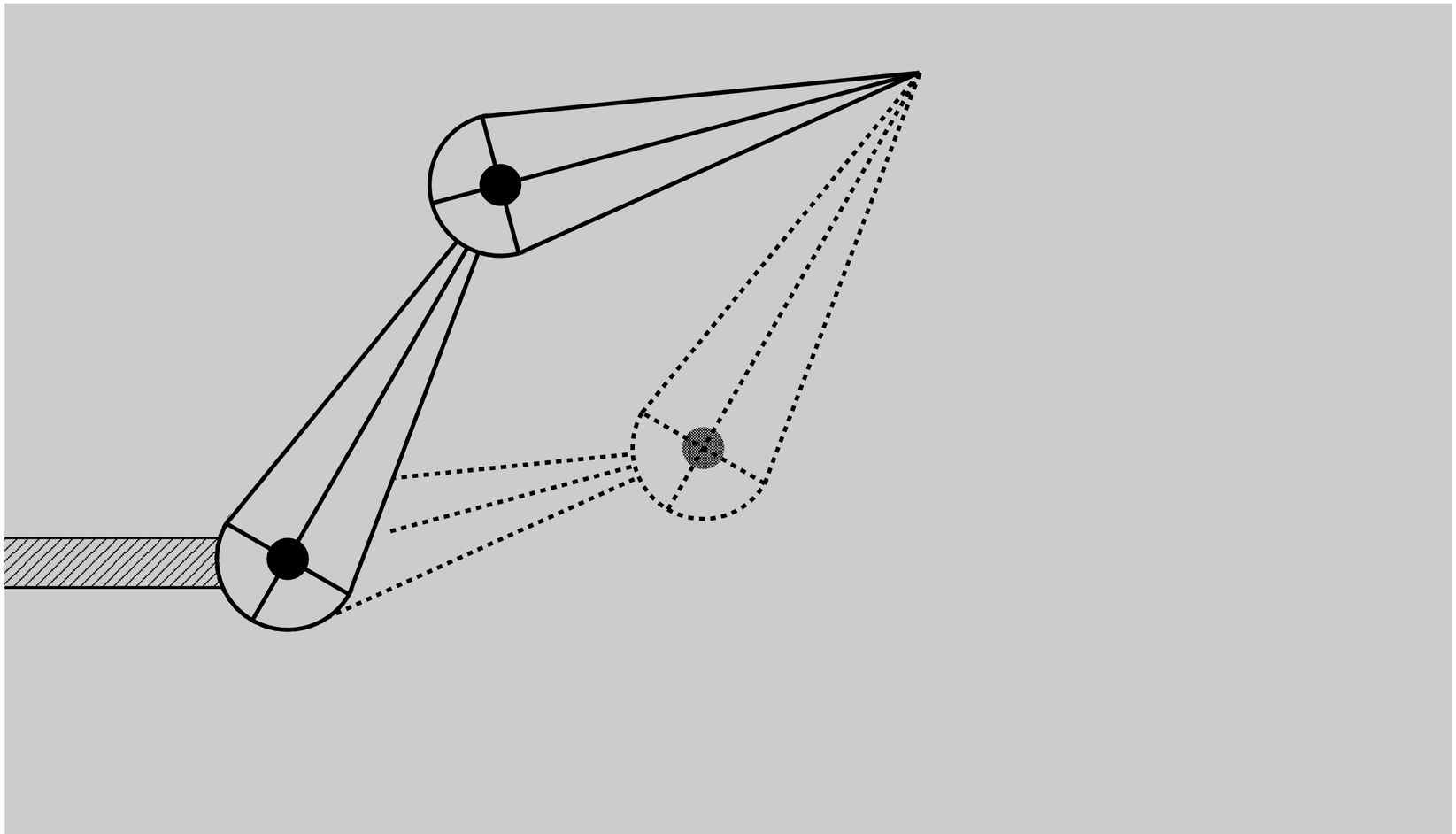
$$\theta_2 = \cos^{-1}\left(\frac{p_z^2 + p_x^2 - l_1^2 - l_2^2}{2 l_1 l_2}\right)$$

$$\theta_1 = \frac{-p_z l_2 \sin(\theta_2) + p_x(l_1 + l_2\cos(\theta_2))}{p_x l_2 \sin(\theta_2) + p_z(l_1 + l_2\cos(\theta_2))}$$

Inverse Kinematics

Why is the problem hard?

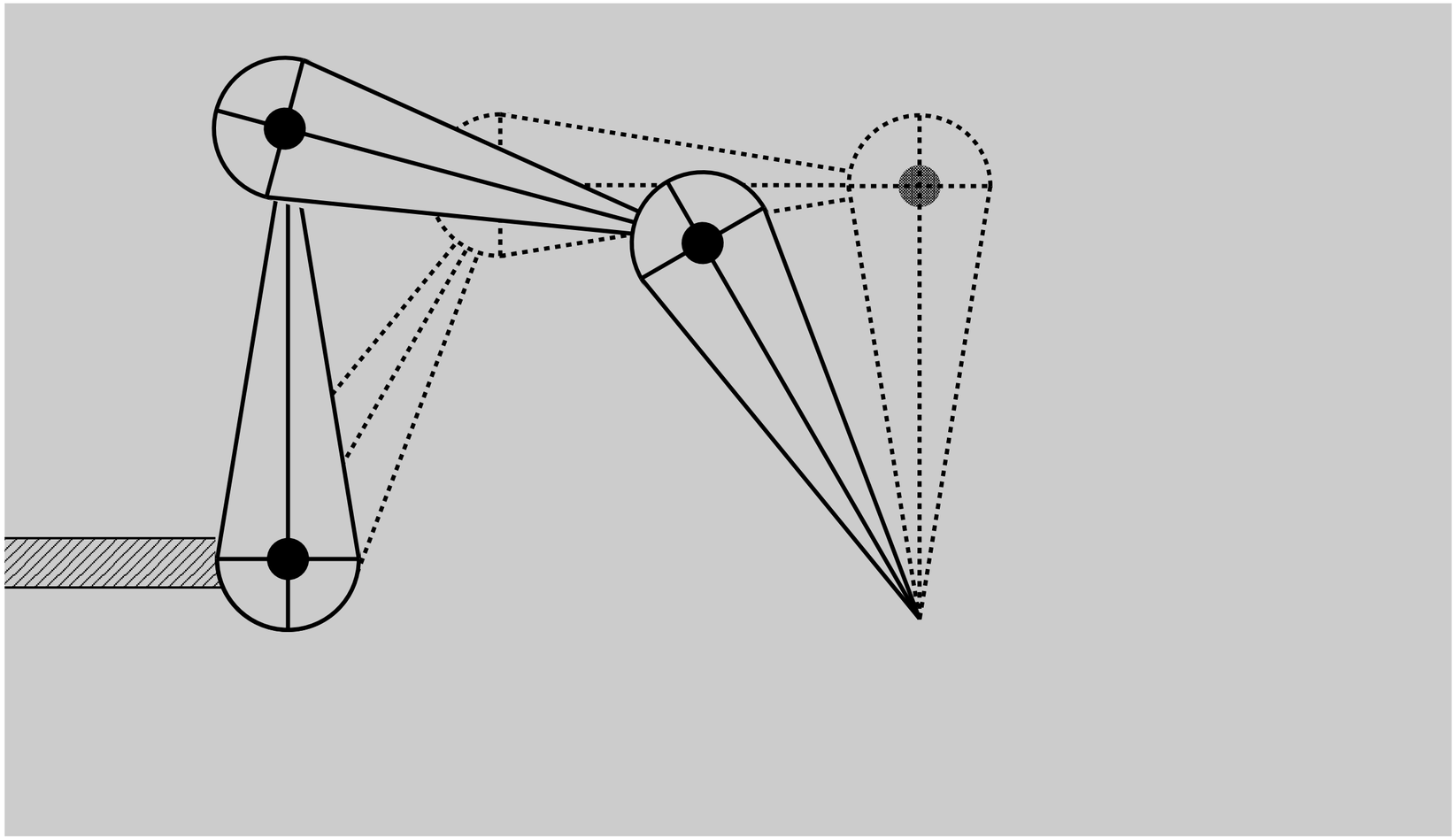
- Multiple solutions separated in configuration space



Inverse Kinematics

Why is the problem hard?

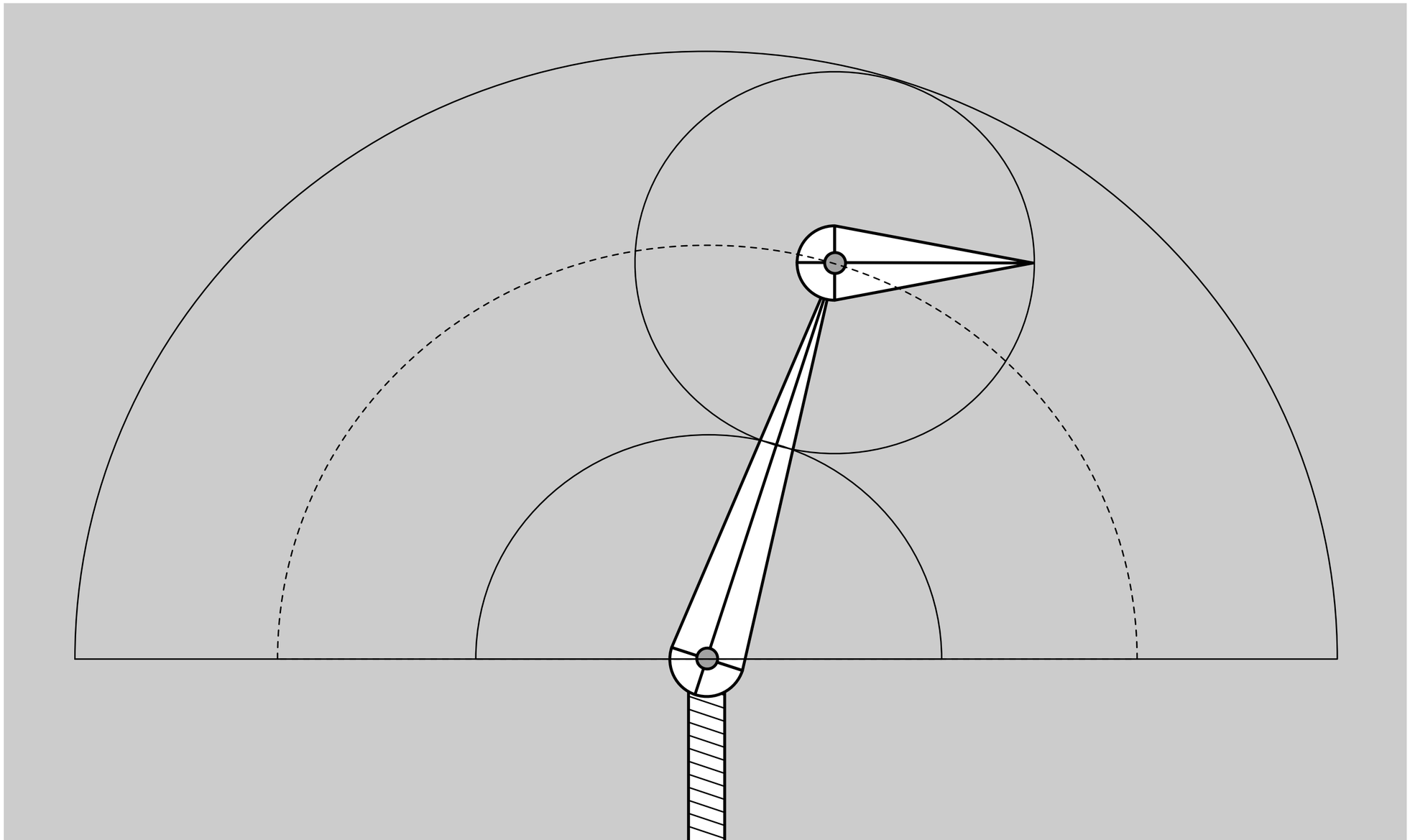
- Multiple solutions connected in configuration space



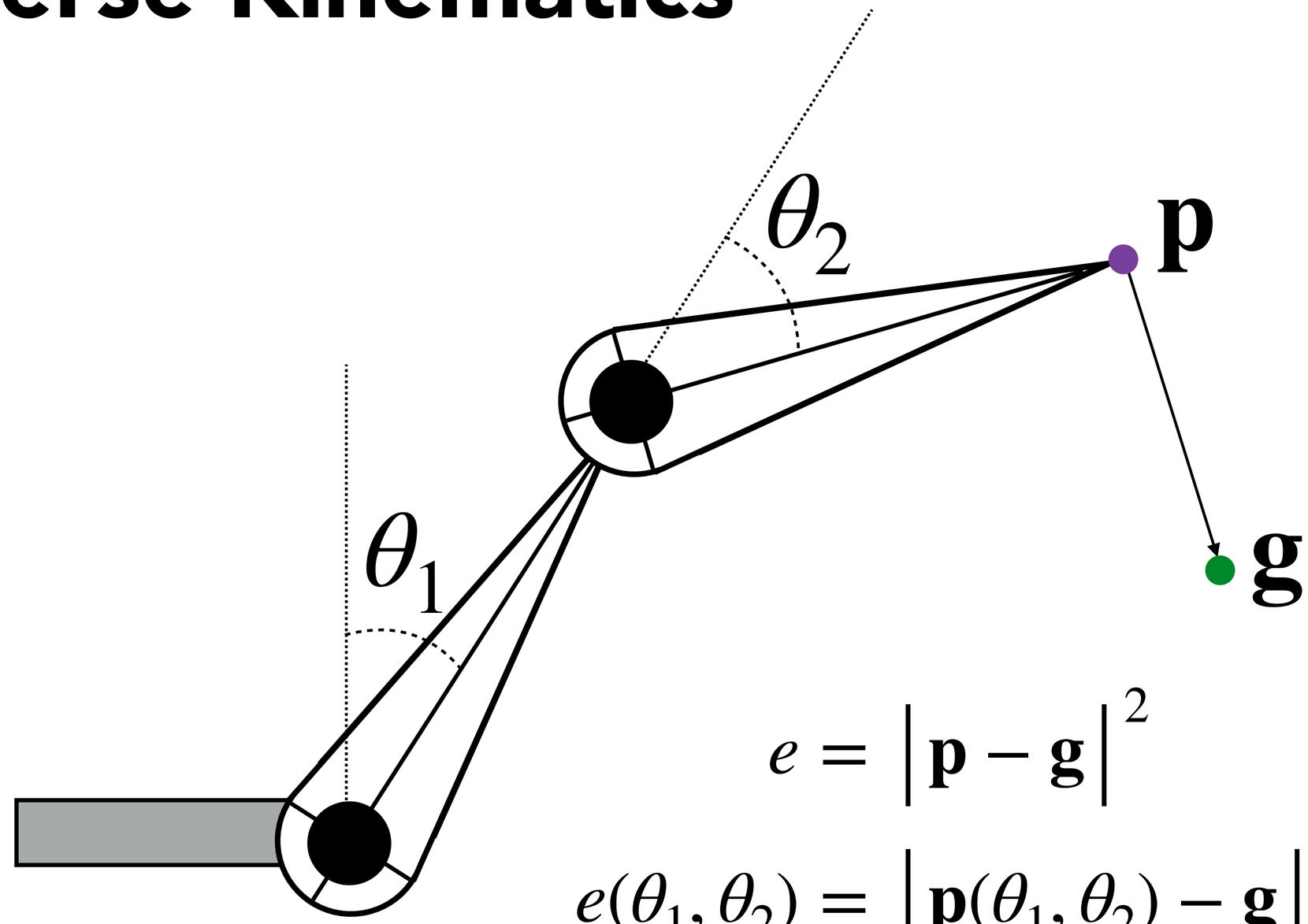
Inverse Kinematics

Why is the problem hard?

- Solutions may not always exist



Inverse Kinematics



$$e = \left| \mathbf{p} - \mathbf{g} \right|^2$$

$$e(\theta_1, \theta_2) = \left| \mathbf{p}(\theta_1, \theta_2) - \mathbf{g} \right|^2$$

Minimize e to get solution.

Inverse Kinematics

Numerical solution to general N-link IK problem

- Choose an initial configuration
- Define an error metric (e.g. square of distance between goal and current position)
- Compute gradient of error as function of configuration
- Apply gradient descent (or Newton's method, or other optimization procedure)

Inverse Kinematics

Cyclic Coordinate Decent (easy, not great)

- For each joint, check if error decreases by going either + or - one degree, take whichever is better
- Keep cycling through the joints (degrees of freedom) until no progress is made.

Stabilized Newton Iterations (harder, but great)

- Use Newton's method with line search.

Gradient Decent (easy with ML tools!)

- Go in gradient direction with line search

Issues:

- Speed
- Quality of result
- Starting point
- Single or varied solutions for same goal
- Local minima?

Inverse Kinematics

Generalize to other constraints

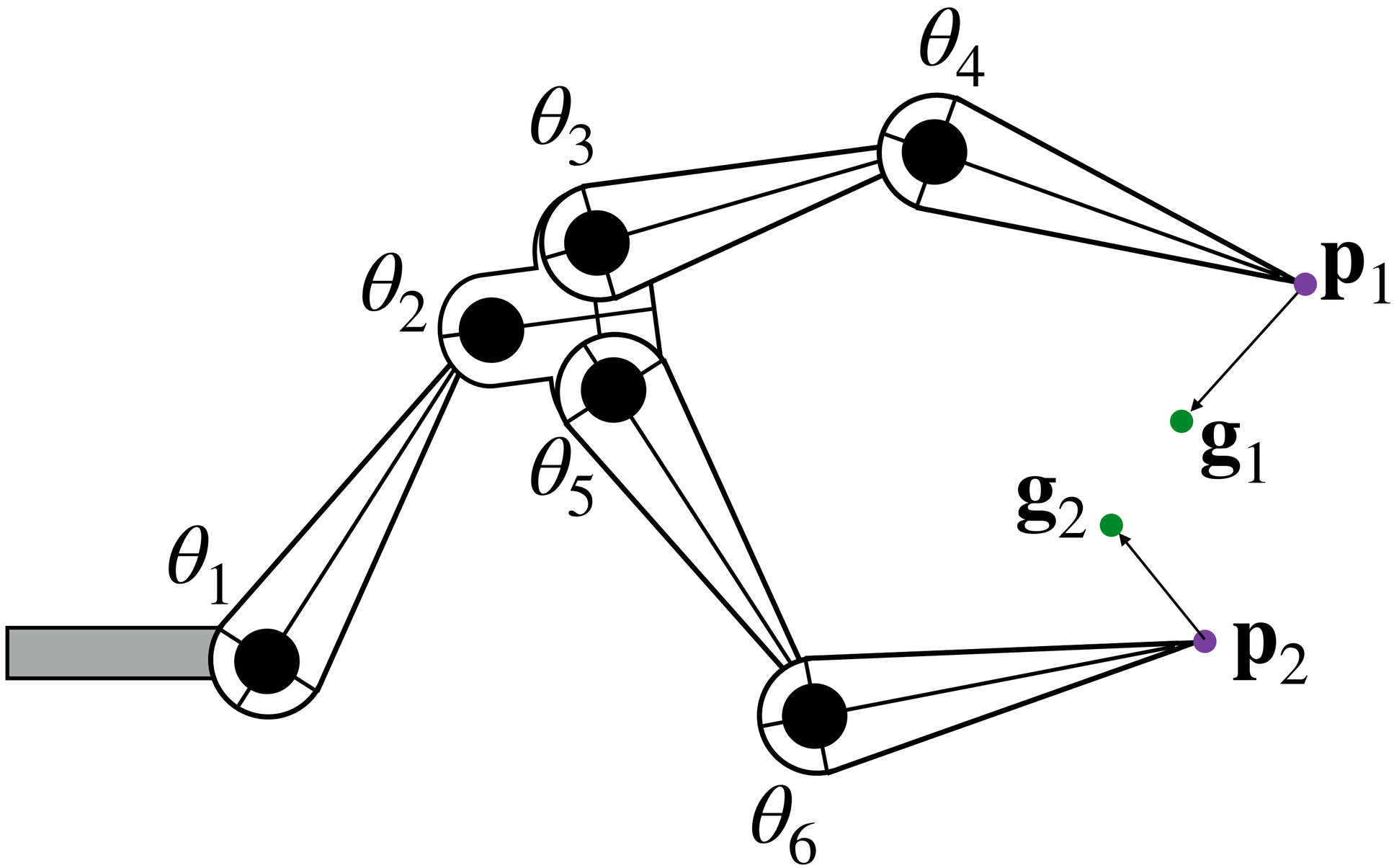
- Balance
- Gaze direction
- Foot plant (no sliding)
- Animation (combine with curves)

Combine multiple Constraints

Same general approach

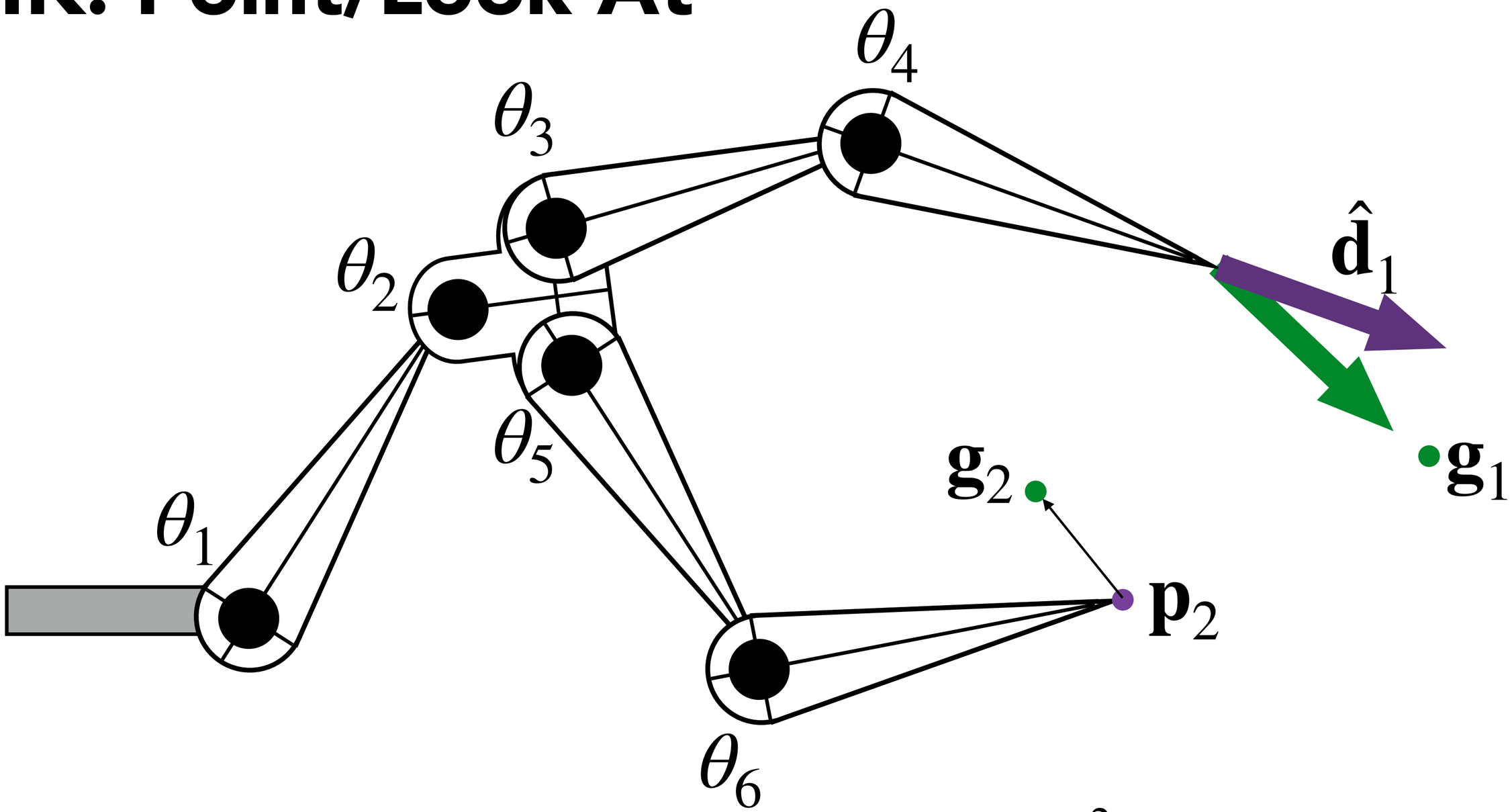
- Define error terms
- Do something to minimize the error terms
  - Cyclic Coordinate Decent, Gradient Decent, Newton, etc.

IK: Multiple Constraints



$$e(\boldsymbol{\theta}) = w_1 \left| \mathbf{p}_1(\boldsymbol{\theta}) - \mathbf{g}_1 \right|^2 + w_2 \left| \mathbf{p}_2(\boldsymbol{\theta}) - \mathbf{g}_2 \right|^2$$

IK: Point/Look At

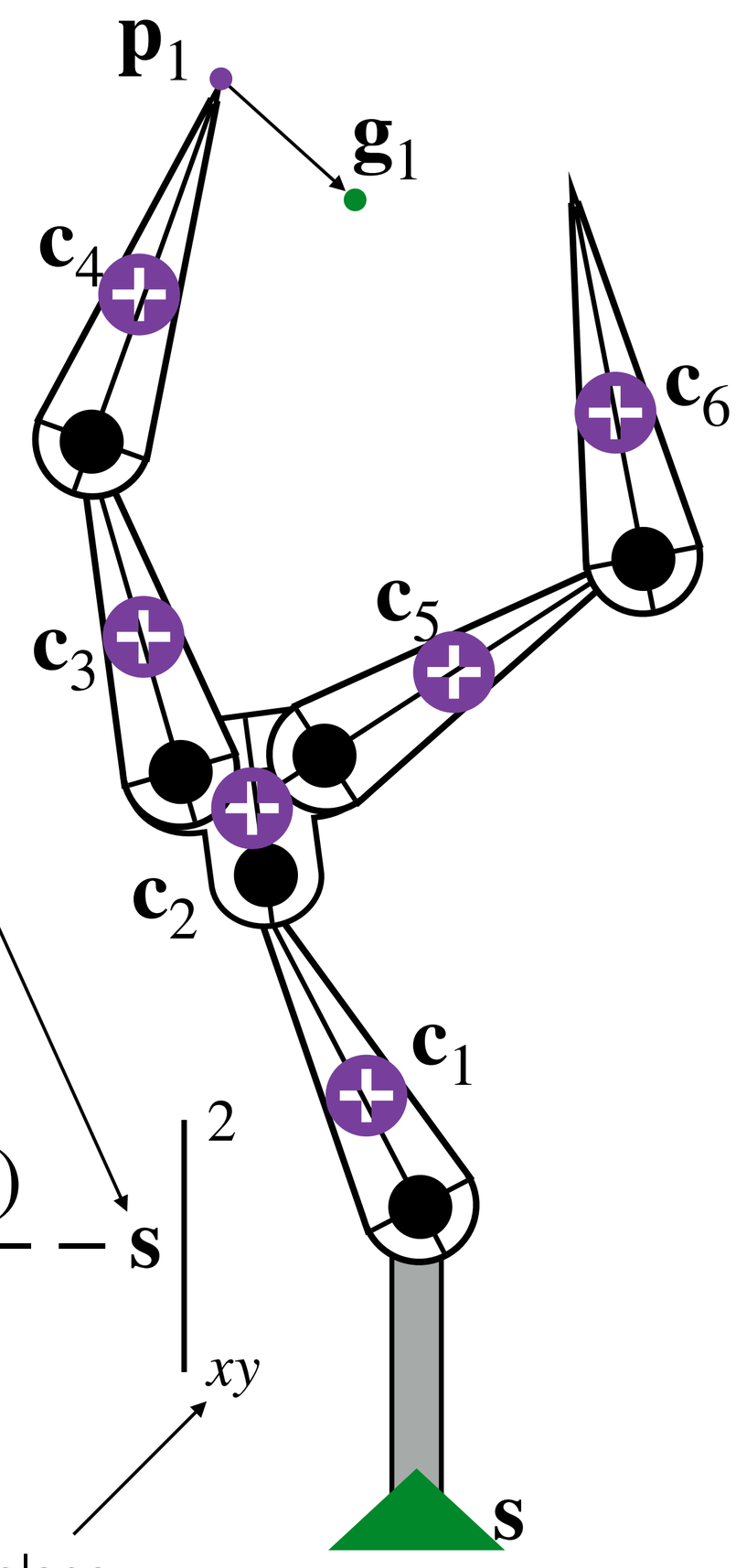


$$e(\boldsymbol{\theta}) = w_1 \left| 1 - \left(\frac{\mathbf{p}_1(\boldsymbol{\theta}) - \mathbf{g}_1}{\left| \mathbf{p}_1(\boldsymbol{\theta}) - \mathbf{g}_1 \right|} \right) \cdot \hat{\mathbf{d}}(\boldsymbol{\theta}) \right|^2 + w_2 \left| \mathbf{p}_2(\boldsymbol{\theta}) - \mathbf{g}_2 \right|^2$$

IK: Balance

Keep COM over support polygon

Support polygon is convex hull of ground contacts



Either center of support polygon or nearest point

body COM

$$e(\boldsymbol{\theta}) = w_1 \left| \mathbf{p}_1(\boldsymbol{\theta}) - \mathbf{g}_1 \right|^2 + w_2 \left| \frac{\sum m_i \mathbf{c}_i(\boldsymbol{\theta})}{\sum m_i} - \mathbf{s} \right|^2_{xy}$$

mass

projected in to XY plane

IK: Keyframing

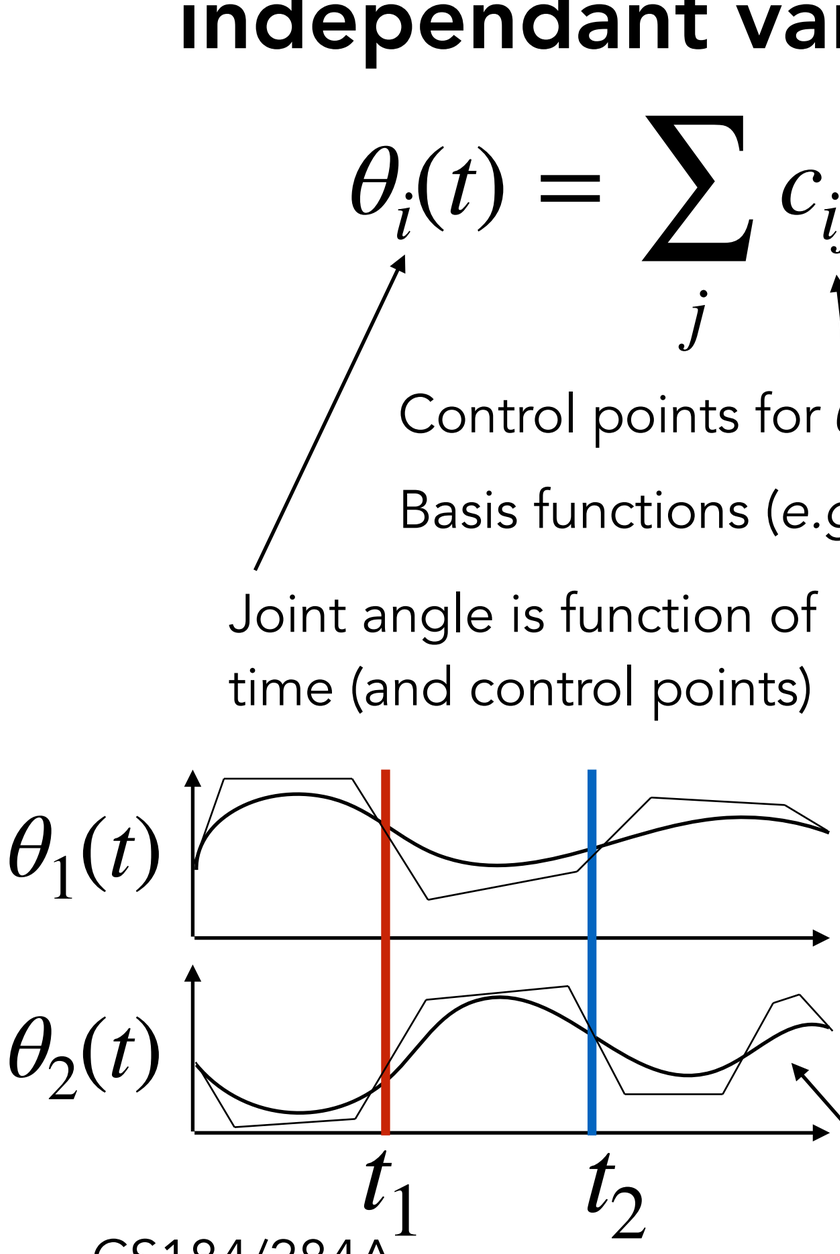
- **With key-framed animation, the θ are not independant variables:**

$$\theta_i(t) = \sum_j c_{ij} B_j(t)$$

Control points for θ_i

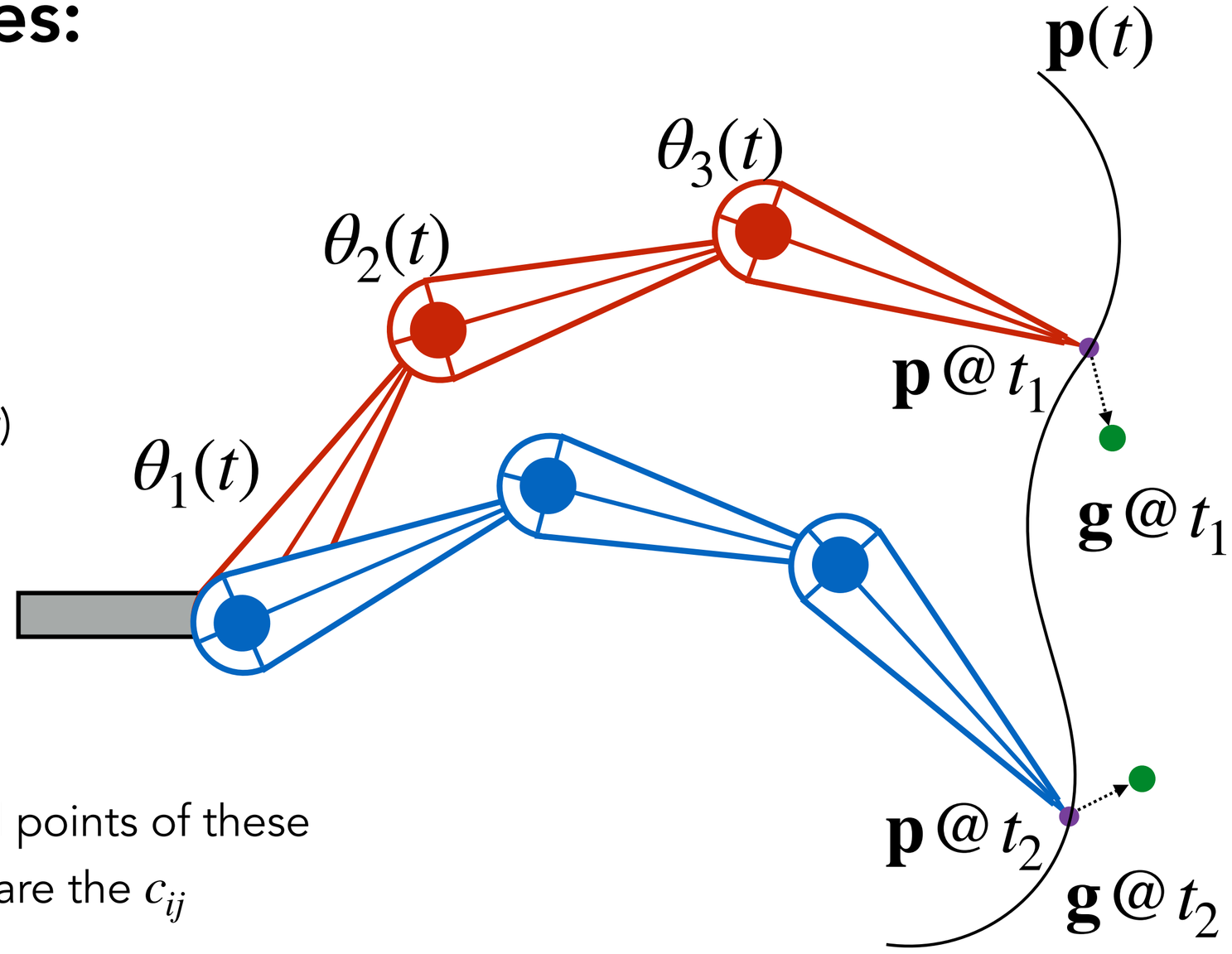
Basis functions (*e.g.* Bézier)

Joint angle is function of time (and control points)



Control points of these curves are the c_{ij}

Adjust c_{ij} to hit constraints.



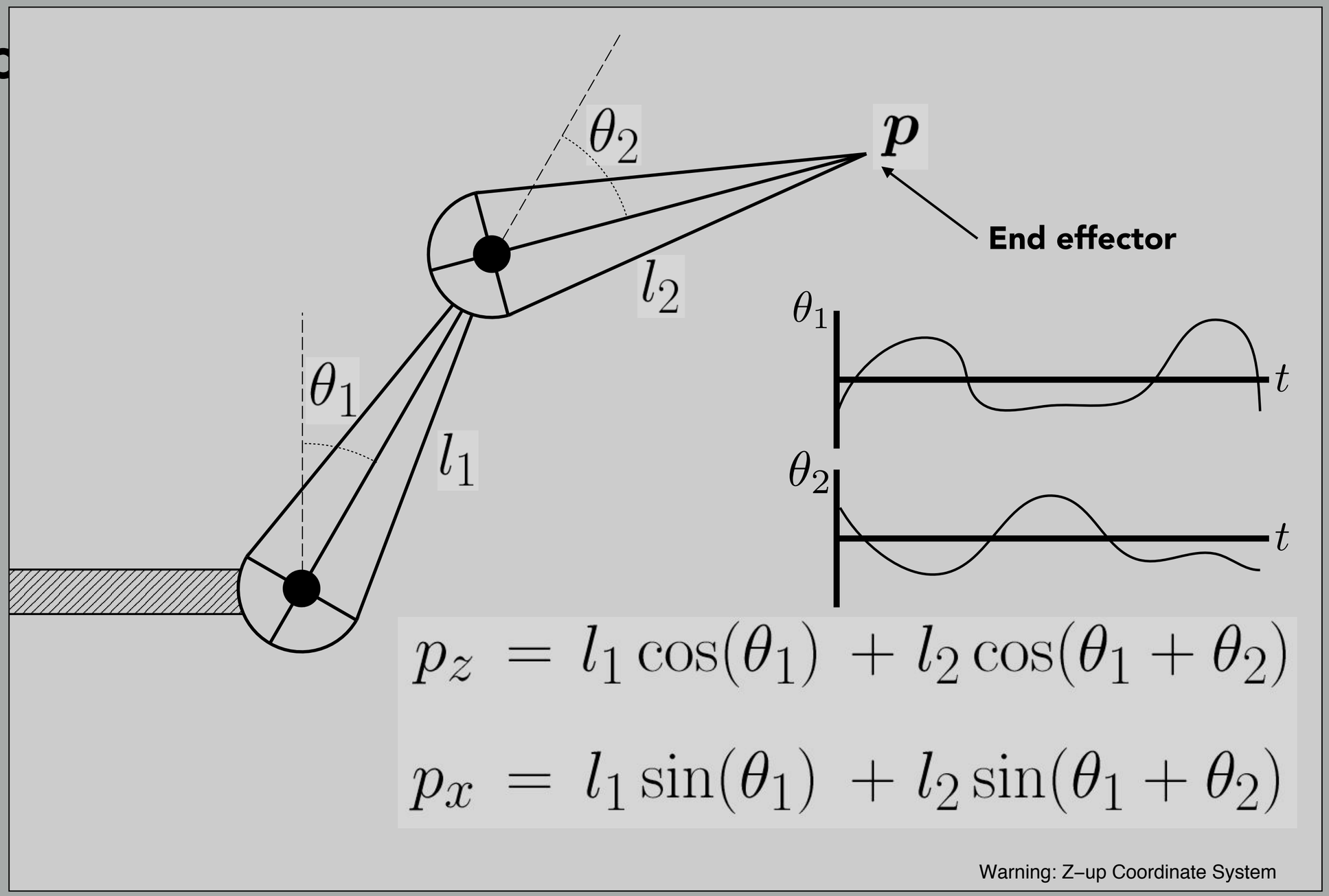
Inverse Kinematics

Recall simple two segment arm:

Stabilized Newton

Inverse Kinematics

Rec



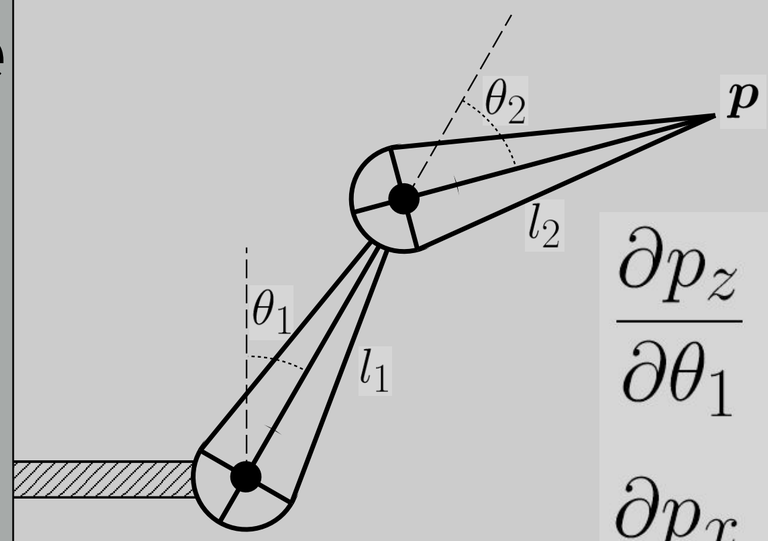
$$p_z = l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2)$$

$$p_x = l_1 \sin(\theta_1) + l_2 \sin(\theta_1 + \theta_2)$$

Warning: Z–up Coordinate System

Inverse Kinematics

We



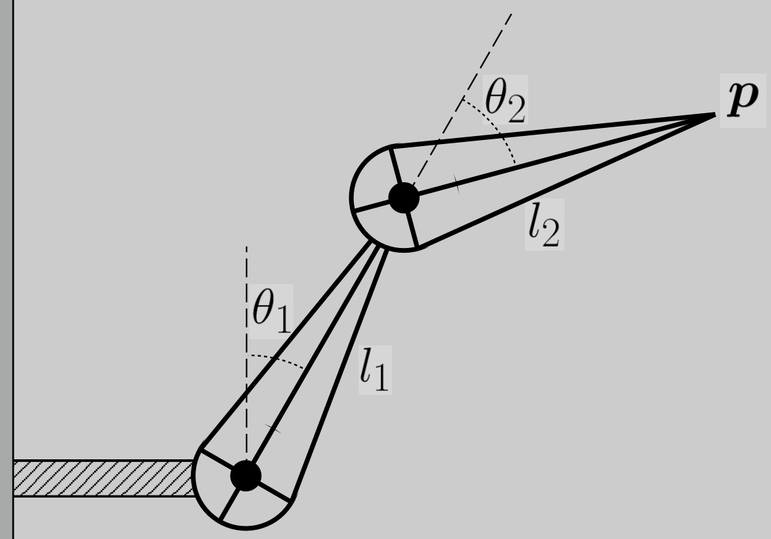
$$\frac{\partial p_z}{\partial \theta_1} = -l_1 \sin(\theta_1) - l_2 \sin(\theta_1 + \theta_2)$$

$$\frac{\partial p_x}{\partial \theta_1} = l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2)$$

$$\frac{\partial p_z}{\partial \theta_2} = - l_2 \sin(\theta_1 + \theta_2)$$

$$\frac{\partial p_x}{\partial \theta_2} = + l_2 \cos(\theta_1 + \theta_2)$$

Inverse Kinematics



Direction in Config. Space

$$\theta_1 = c_1\theta_*$$

$$\theta_2 = c_2\theta_*$$

$$\frac{\partial p_z}{\partial \theta_*} = c_1\frac{\partial p_z}{\partial \theta_1} + c_2\frac{\partial p_z}{\partial \theta_2}$$

Inverse Kinematics

The Jacobian (of p w.r.t. θ)

$$J_{ij} = \frac{\partial p_i}{\partial \theta_j}$$

Example for two segment arm

$$J = \begin{bmatrix} \frac{\partial p_z}{\partial \theta_1} & \frac{\partial p_z}{\partial \theta_2} \\ \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} \end{bmatrix}$$

Inverse Kinematics

The Jacobian (of p w.r.t. θ)

$$J = \begin{bmatrix} \frac{\partial p_z}{\partial \theta_1} & \frac{\partial p_z}{\partial \theta_2} \\ \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} \end{bmatrix}$$

$$\frac{\partial \boldsymbol{p}}{\partial \theta_*} = J \cdot \begin{bmatrix} \frac{\partial \theta_1}{\partial \theta_*} \\ \frac{\partial \theta_2}{\partial \theta_*} \end{bmatrix} = J \cdot \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}$$

Inverse Kinematics

Solving for c_1 and c_2

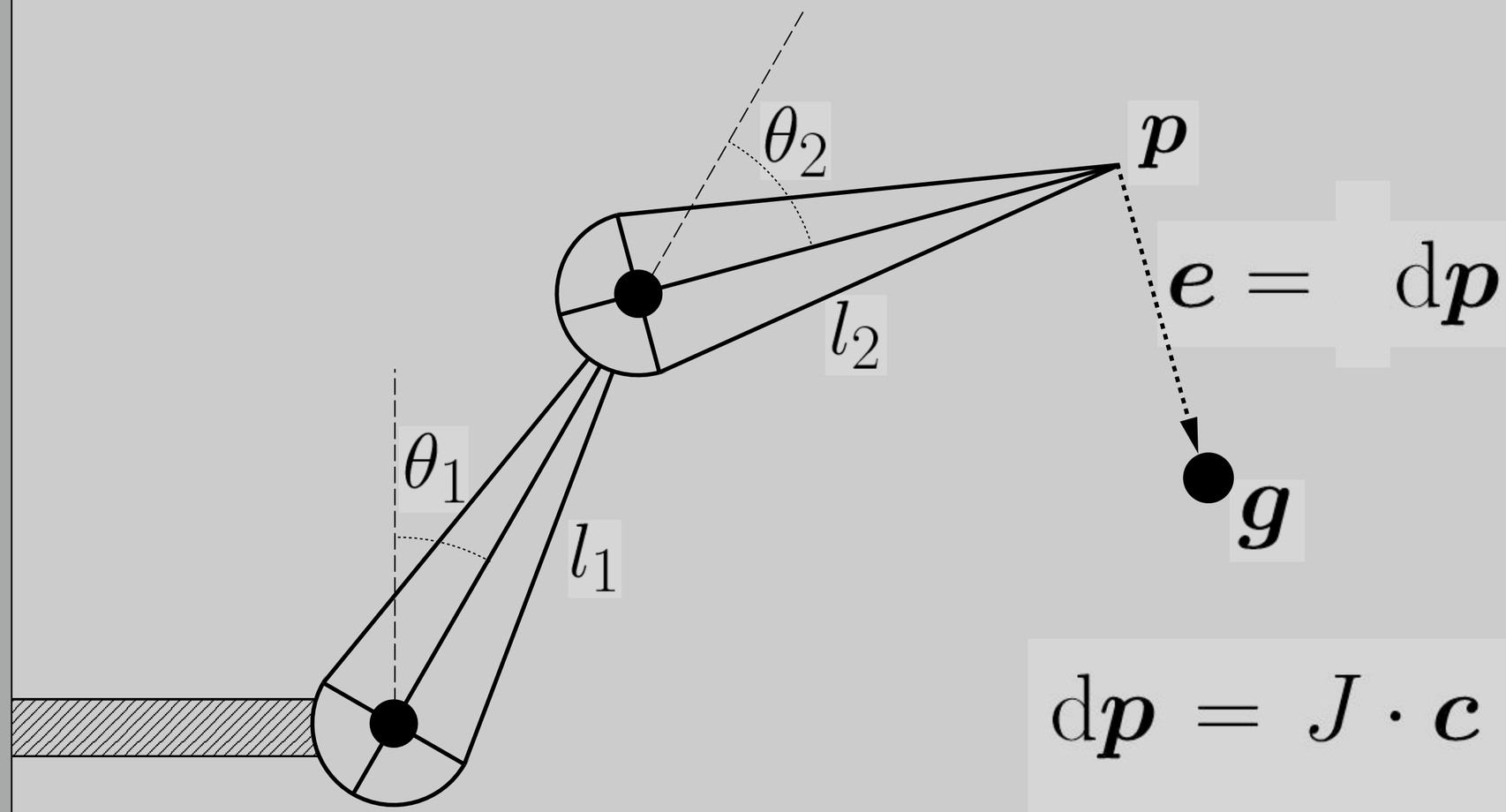
$$\boldsymbol{c} = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} \qquad \mathrm{d}\boldsymbol{p} = \begin{bmatrix} \mathrm{d}p_z \\ \mathrm{d}p_x \end{bmatrix}$$

$$\mathrm{d}\boldsymbol{p} = J \cdot \boldsymbol{c}$$

$$\boldsymbol{c} = J^{-1} \cdot \mathrm{d}\boldsymbol{p}$$

Inverse Kinematics

Solving for c_1 and c_2



$$\mathrm{d}\boldsymbol{p} = J \cdot \boldsymbol{c}$$

$$\boldsymbol{c} = J^{-1} \cdot \mathrm{d}\boldsymbol{p}$$

Is the Jacobian invertible?

Inverse Kinematics

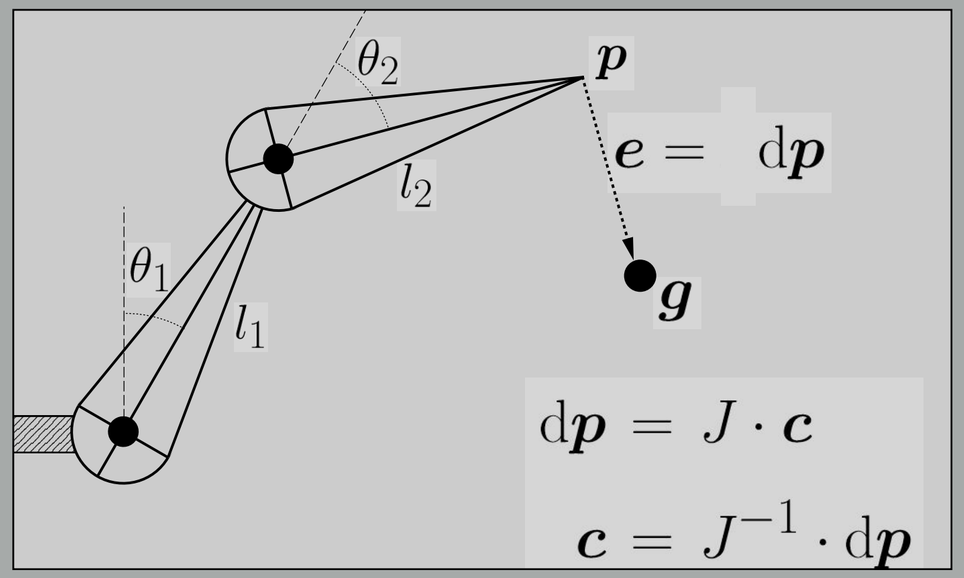
Problems

- Jacobian may (will!) not always be invertible
  - Use pseudo inverse (SVD)
  - Robust iterative method
- Jacobian is not constant

Nonlinear optimization, but problem is (mostly) well behaved

$$J = \begin{bmatrix} \frac{\partial p_z}{\partial \theta_1} & \frac{\partial p_z}{\partial \theta_2} \\ \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} \end{bmatrix} = J(\theta)$$

Inverse Kinematics



Jacobian is not always invertible

- Use pseudo inverse (SVD)

Computing a linear approximation

- End effector only locally moves

$$J = \begin{bmatrix} \frac{\partial p_z}{\partial \theta_1} & \frac{\partial p_z}{\partial \theta_2} \\ \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} \end{bmatrix} = J(\theta)$$

- So iterate (choosing proper step te Jacobian after each step
- Choosing step size requires line search at each step
  - Choose some step size (say 5 degrees) and compute how to update joint parameters
  - Calculate distance of end effector from goal
  - If distance decreased take step
  - Is distance did not decrease set parameters to be half the current change and try again

Inverse Kinematics

Extra material for 284A

More complex systems

- More complex joints (prism and ball)
- More links
- Other criteria (COM or height)
- Hard constraints (joint limits)
- Multiple criteria and multiple chains

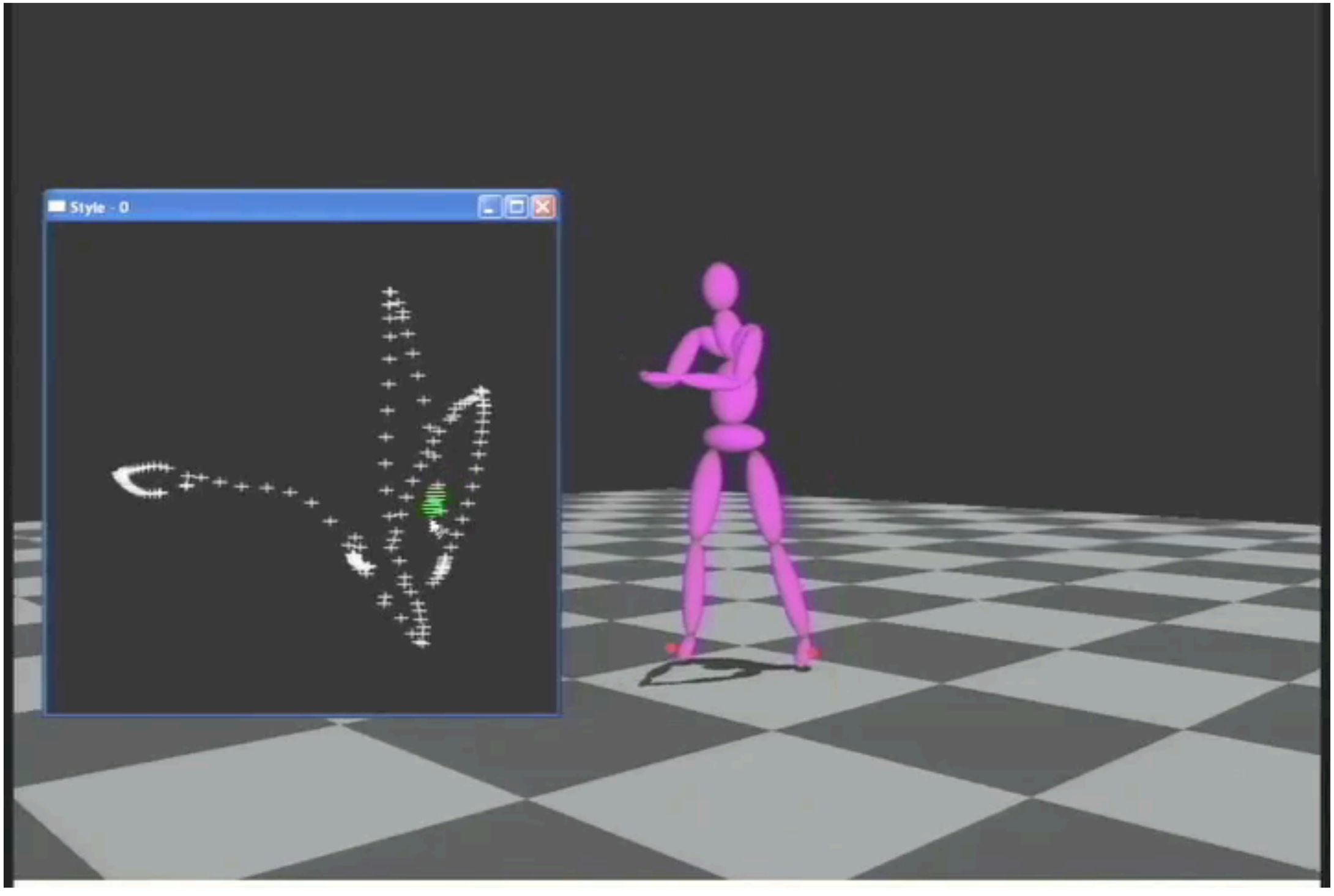
Inverse Kinematics

Some issues

- How to pick from multiple solutions?
- Robustness when no solutions
- Contradictory solutions
- Smooth interpolation
- Interpolation aware of constraints

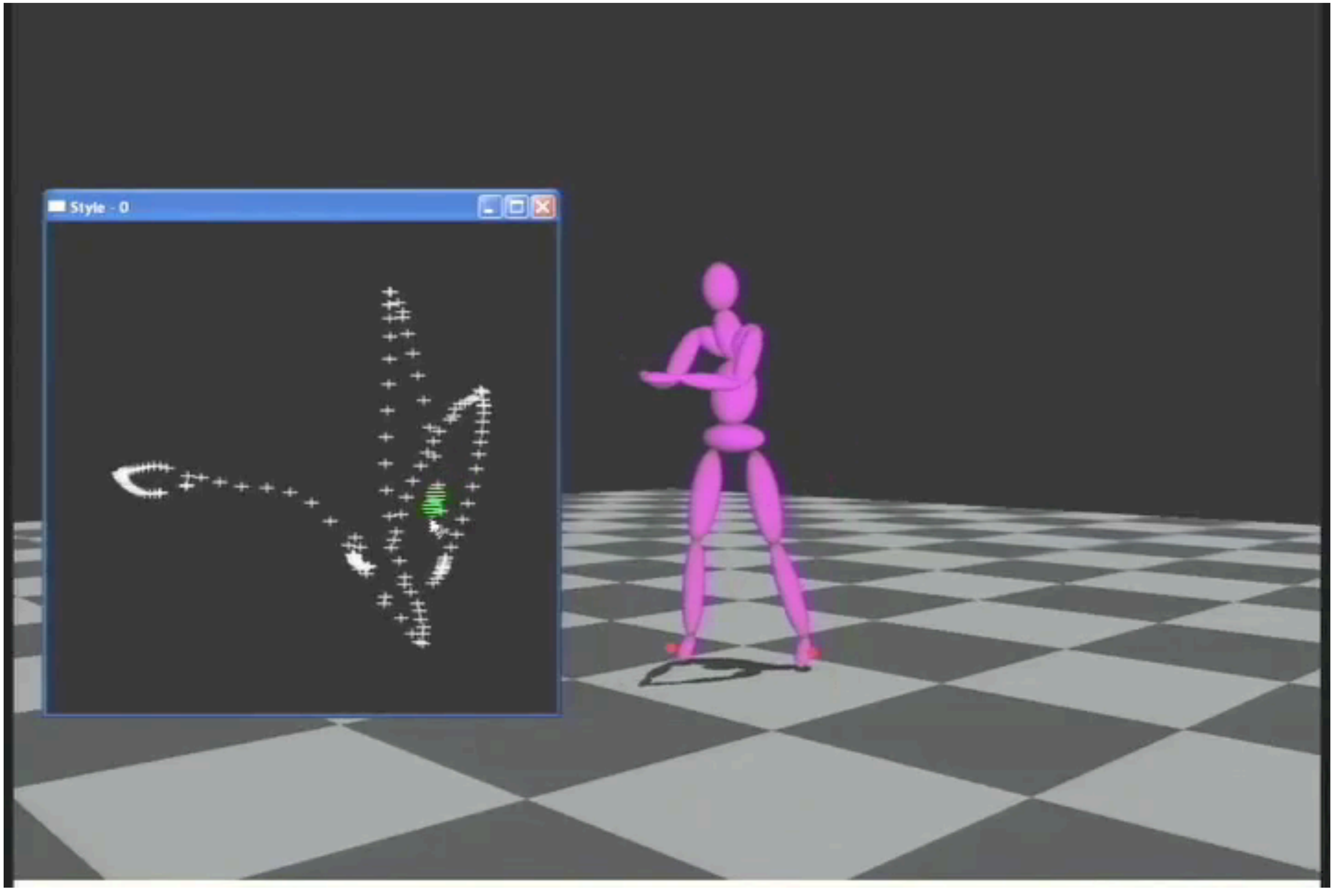
Numerical evaluation of Jacobian

Style-Based IK



Grochow et al., Style Based Inverse Kinematics

Style-Based IK



Grochow et al., Style Based Inverse Kinematics

Kinematic Animation Pros and Cons

Strengths

- Direct control is convenient
- Implementation is straightforward

Weaknesses

- Animation may be inconsistent with physics
- Time consuming for artists

IK is often used inside of other methods

- *e.g.* planner computes desired foot placement and uses IK to compute appropriate configuration

Acknowledgments

This slide set contain contributions from:

- Kayvon Fatahalian
- David Forsyth
- Pat Hanrahan
- Angjoo Kanazawa
- Steve Marschner
- Ren Ng
- James O’Brien
- Mark Pauly